A Concise Note on Theory of Computation (TOC)

Samena Bahleri

samenabahleri09@gmail.com

November 11, 2025

This page left intentionally blank

Contents

1 Introduction **7**
1.1 TOC Development History . . . 7
1.2 Central Concepts of TOC . . . 7

2 Regular Languages **8**
2.1 Regular Expressions . . . 9
2.2 Unix RegExp . . . 10
2.3 Identities of Regular Expressions . . . 10

3 Deterministic Finite Automata (DFA) **11**
3.1 Transition Diagram . . . 12
3.2 Transition Table . . . 12
3.3 Extended Transition Function . . . 12
3.4 Two-way Finite Automata (2DFA) . . . 14

4 Minimization **16**

5 Myhill-Nerode Theorem **19**

6 Non-deterministic Finite Automata (NFA) **21**
6.1 ε-NFA . . . 22
6.2 ε-Closure . . . 22

7 Moore and Mealy Machine **23**
7.1 Moore Machine . . . 23
7.2 Mealy Machine . . . 24

8 Arden's Theorem **25**

9 Conversion **26**
9.1 NFA to DFA . . . 26
9.2 ε-NFA to NFA . . . 29
9.3 NFA to RegExp . . . 30
9.4 DFA to RegExp . . . 32
9.5 RegExp to Finite Automaton FA . . . 34
9.6 Moore to Mealy . . . 35
9.7 Mealy to Moore . . . 37

10 Property of Regular Languages **39**
10.1 Decision Algorithms . . . 39
10.1.1 Emptiness . . . 39
10.1.2 Membership . . . 40
10.1.3 Finiteness . . . 41
10.1.4 Equivalence . . . 42
10.1.5 Subset . . . 43
10.2 Closure Properties . . . 44
10.2.1 Union . . . 44

10.2.2 Intersection 46
10.2.3 Complement 48
10.2.4 Difference 49
10.2.5 Concatenation 51
10.2.6 Kleene Star 53
10.2.7 Reversal 55
10.2.8 Homomorphism 56
10.2.9 Inverse Homomorphism 58
10.2.10 Substitution 59

11 Pumping Lemma **61**

12 Regular Grammar **61**

13 Context-Free Grammar and Language **62**
13.1 Recursive 63
13.2 Right and Left Linear Grammar 66
13.3 Pumping Lemma for Contex-Free Languages 66

14 Context-Sensitive Grammar (CSG) **67**

15 Derivation Tree **69**
15.1 Right and Left Derivation 69
15.2 Ambiguous Grammar 70

16 Breadth-First Search (BFS) **71**

17 Depth-First Search (DFS) **72**

18 Simplification of CFG **73**
18.1 Reduction of CFG 73
18.2 Removal Unit of Production 74
18.3 Removal of Null Productions 75

19 Chomsky Normal Form (CNF) **76**
19.1 CFG to CNF 76

20 Greibach Normal Form (GNF) **79**
20.1 CFG to GNF 80

21 Pushdown Automata (PDA) **83**
21.1 CFG to PDA 85
21.2 PDA to CFG 86

22 Palindromes **89**

23 Turing Machine **92**
23.1 Turing Machine Programming Techniques 99
23.1.1 Union 100
23.1.2 Intersection 101

23.1.3 Concatenation . . . 101
23.1.4 Comparing Two Strings . . . 102
23.2 Multi-tape Turing Machines . . . 104
23.3 Nondeterministic Turing Machine (NTM) . . . 105
23.4 The Equivalence of NTM and Deterministic Turing Machine (DTM) . . . 106
23.5 Turing Machine as a Problem Solver . . . 107
23.6 Turing Machine as an Enumerator . . . 108
23.7 The Universal Turing Machine . . . 109

24 Decidability and Undecidability **110**

25 Halting Problem **111**

26 The Diagonalization Method **112**

27 Reducibility **113**
27.1 Mapping Reducibility . . . 113

28 Post Correspondence Problem (PCP) **116**
28.1 Modified Post Correspondence Problem (MPCP) . . . 118

29 Linear Bounded Automaton (LBA) **120**

30 Recursion Theorem **122**

31 Time Complexity **123**
31.1 Growth rate . . . 125
31.2 Big O Notation . . . 126
31.3 Big Ω Notation . . . 127
31.4 Big Θ Notation . . . 128
31.5 Little o Notation . . . 129
31.6 Little ω Notation . . . 131
31.7 Standard Functions . . . 132

32 Analyzing Algorithms **135**
32.1 TIME Complexity Classes . . . 139

33 The Class P **140**

34 Nondeterministic Polynomial Time (NP) **142**
34.1 Hamiltonian Path . . . 142
34.2 Clique . . . 144
34.3 Vertext Cover . . . 146
34.4 Independent Set . . . 148

35 coNP **150**
35.1 coNP-complete . . . 151

36 NP-complete **151**
36.1 NP-Hard 151
36.2 Boolean Satisfiability (SAT) 151
36.3 3-SAT 152
36.4 Conjunctive Normal Form (CNF) 153

37 The P versus NP **157**

38 Polynomial Time Reducibility **157**

39 The Cook-Levin Theorem **157**

40 Space Complexity **164**
40.1 PSPACE and NPSPACE 164
40.2 LOGSPACE (L) and NLOGSPACE (NL) 165
40.3 L-Reduction 169
40.4 NL-complete 172
40.5 PSPACE-complete 173

41 Savitch's Theorem **173**

42 True Quantified Boolean Formula (TQBF) **174**

43 Time and Space Hierarchy Theorem **176**
43.1 Time Hierarchy Theorem 176
43.2 Space Hierarchy Theorem 177

1 Introduction

1.1 TOC Development History

1. **Alan Turing (1936)**

   Introduced the concept of the Turing Machine, providing a formal model of computation and establishing the foundation for decidability, computability, and the limits of what machines can compute.

2. **Warren McCulloch and Walter Pitts (1943)**

   Proposed a mathematical model of artificial neurons, linking logic, computation, and brain function, and laying groundwork for automata theory and neural networks.

3. **Stephen Kleene (1943–1956)**

   Developed the theory of regular expressions and recursive functions, formalizing the foundations of computability and influencing automata and language theory.

4. **Noam Chomsky (1956)**

   Introduced the Chomsky hierarchy of formal grammars, classifying languages based on their generative power and deeply connecting syntax with automata theory.

5. **John Backus (1959)**

   Created the Backus–Naur Form (BNF) for describing the syntax of programming languages, bridging formal language theory with practical computing.

6. **Stephen Cook (1971)**

   Defined the concept of NP-completeness, introducing the Cook–Levin theorem, which became central to computational complexity theory.

7. **Richard Karp (1972)**

   Expanded on Cook's work by identifying 21 NP-complete problems, solidifying the framework for studying computational intractability.

1.2 Central Concepts of TOC

1. Alphabets

   Example:

   (a) $\Sigma = \{0, 1\}$ (Binary Alphabet)

   (b) $\Sigma = \{a, b, c\}$ (Ternary Alphabet)

   (c) $\Sigma = \{a, b, c, \ldots, z\}$ (English Alphabet)

   (d) $\Sigma = \{0, 1, 2, \ldots, 9\}$ (Decimal Digits)

2. Strings

   Example:

(a) Over $\Sigma = \{0, 1\}$: "101", "0001", "1110"

(b) Over $\Sigma = \{a, b, c\}$: "abc", "aab", "ccba"

(c) Over $\Sigma = \{0, 1, 2, \ldots, 9\}$: "123", "4567", "890"

(d) Empty String: Denoted by ε, represents a string with no characters.

(e) length of string: For a string S, the length is denoted by $|S|$. For example, if $S =$ "11011", then $|S| = 5$.

3. Powers

   Example:

   (a) $\Sigma = \{0, 1\}$

      i. $\Sigma^0 = \{\varepsilon\}$

      ii. $\Sigma^1 = \{0, 1\}$

      iii. $\Sigma^2 = \{00, 01, 10, 11\}$

      iv. $\Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$

   (b) $\Sigma^* = \bigcup_{k \geq 0} \Sigma^k$

4. Concatenation

   Example:

   (a) Let $s_1 =$ "101" and $s_2 =$ "11" over $\Sigma = \{0, 1\}$. Then, $s_1 \cdot s_2 =$ "10111".

   (b) Let $s_1 =$ "ab" and $s_2 =$ "cde" over $\Sigma = \{a, b, c, d, e\}$. Then, $s_1 \cdot s_2 =$ "$abcde$".

5. Language

   Example:

   (a) $L = \{w \mid w$ has an even number of 1s$\}$

   (b) $L = \{w \mid w$ starts with 'a' and ends with 'b'$\}$

   (c) $L = \{0^i 1^j \mid 0 \geq i \geq j\}$, Hence:

      – L contains strings like " ", "01", "0011", "000111", etc.

      – L does not contain strings like "10", "1100", "0110", etc.

2 Regular Languages

Regular Languages are a class of formal languages that can be recognized by finite automata, specifically Deterministic Finite Automata (DFA) and Non-deterministic Finite Automata (NFA). They are called "regular" because they can be described using regular expressions, which provide a concise way to represent patterns in strings.

2.1 Regular Expressions

A Regular Expression (regex or regexp) is a sequence of characters that defines a search pattern, primarily used for string matching within texts. Regular expressions are built using a combination of symbols and operators to represent sets of strings.

1. Union

   Symbolic notation for union is:

   $$L + M = \{x \in L \cup y \in M\}$$

   Example:

   Let L, M be two languages over the alphabet $\Sigma = \{1, 0\}$ such that:

   (a) $L = \{1^n \mid n \geq 1\}$

   (b) $M = \{0^n \mid n \geq 1\}$

   Then, the union of L and M is:

   $$L + M = \{1^n \mid n \geq 1\} \cup \{0^n \mid n \geq 1\}$$

2. Concatenation

   Symbolic notation for concatenation is:

   $$L \cdot M = \{x \in L \times y \in M\}$$

   Example:

   Let L, M be two languages over the alphabet $\Sigma = \{1, 0\}$ such that:

   (a) $L = \{1^n \mid n \geq 1\}$

   (b) $M = \{0^n \mid n \geq 1\}$

   Then, the concatenation of L and M is:

   $$L \cdot M = \{1^n 0^m \mid n \geq 1, m \geq 1\}$$

   In detail:

   Suppose $L = \{1, 11\}$ and $M = \{0, 00\}$. Then,

   $$L \cdot M = \{10, 100, 110, 1100\}.$$

3. Kleene Star

   Symbolic notation for Kleene Star is:

   $$L^* = \bigcup_{k=0}^{\infty} L^k$$

where $L^0 = \{\varepsilon\}$ and $L^k = L \cdot L^{k-1}$ for $k \geq 1$.

Example 1:

Let L be a language over the alphabet $\Sigma = \{0, 1\}$, and suppose $L = \{0, 1\}$. Then

(a) $L^0 = \{\varepsilon\}$
(b) $L^1 = \{0, 1\}$
(c) $L^2 = L \cdot L = \{00, 01, 10, 11\}$
(d) $L^3 = L \cdot L^2 = \{000, 001, 010, 011, 100, 101, 110, 111\}$
(e) $L^4 = L \cdot L^3 = \{0000, 0001, 0010, 0011, 0100, 0101, \ldots\}$

Example 2:

Let L be a language over the alphabet $\Sigma = \{0, 1\}$ such that:

(a) $0^*1^* = \{\varepsilon, 1, 11, 111, \ldots, 0, 01, 001, 0001, \ldots, 00, 000, 0000, \ldots\}$
(b) $1^*0 = 1^* \cdot 0 = \{0, 10, 110, 1110, 11110, \ldots\}$
(c) $10^* = \{1, 10, 100, 1000, 10000, \ldots\}$
(d) $1 + 01^* = \{1\} \cup \{0, 01, 011, 0111, \ldots\}$
(e) $1^* \cup \{1^*0\} = \{\varepsilon, 1, 11, 111, \ldots\} \cup \{0, 10, 110, 1110, 11110, \ldots\}$

2.2 Unix RegExp

In Unix, regular expressions are used in various command-line tools for pattern matching and text processing.

Example:

1. `[bch]at` matches `bat`, `cat`, and `hat`.
2. `[10xyz]ab` matches `1ab`, `0ab`, `xab`, `yab`, and `zab`.

2.3 Identities of Regular Expressions

1. Basic identities
$$f + \varnothing = f$$
$$f \cdot \varepsilon = \varepsilon \cdot f = f$$
$$f \cdot \varnothing = \varnothing \cdot f = \varnothing$$
2. Idempotent law
$$f + f = f$$
3. Commutative law (union only)
$$f + g = g + f$$
4. Associative laws
$$(f + g) + f = f + (g + f)$$
$$(f \cdot g) \cdot f = f \cdot (g \cdot f)$$

5. Distributive laws

$$f \cdot (g + f) = f \cdot g + f \cdot f$$
$$(f + g) \cdot f = f \cdot f + g \cdot f$$

6. Kleene star identities

$$f^* = \varepsilon + ff^*$$
$$f^* = \varepsilon + f^*f$$
$$(f^*)^* = f^*$$
$$\varnothing^* = \varepsilon$$
$$\varepsilon^* = \varepsilon$$

7. Positive closure

$$f^+ = ff^* = f^*f$$

8. Absorption laws

$$f + ff^* = ff^*$$
$$f + f^*f = f^*f$$

9. Star concatenation collapse

$$f^*f^* = f^*$$

10. Star unfolding

$$\varepsilon + ff^* = \varepsilon + f^*f = f^*$$

11. Star shift (rotation law)

$$(fg)^*f = f(gf)^*$$

12. Union–star expansion

$$(f + g)^* = (f^*g^*)^*$$

3 Deterministic Finite Automata (DFA)

A DFA is a finite-state machine where every state has exactly one transition for each input symbol. Given an input and a current state, the next state is uniquely determined, DFA is formally defined as a 5-tuple $M = (W, \Sigma, V, S, F)$ where:

1. $W = \{w_1, w_2, w_3 \cdots w_n\}$, finite set of states.
2. $\Sigma = \{0, 1\}$, finite set of input symbols .
3. $V : W \times \Sigma \to W.E.g, V(x, 1) = y$, transition function that takes a state and an input symbol and returns the next state.
4. $S \in W$ is the start state where the computation begins.
5. $F \subseteq W$ is the set of accept (final) states.

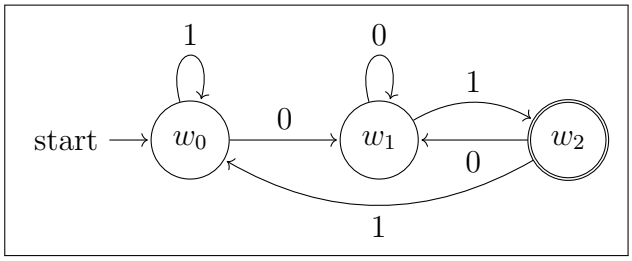
3.1 Transition Diagram

A graphical representation of the DFA, where states are represented as circles, and transitions are represented as arrows labeled with input symbols.

Example:

Let

$$L(M) = \{\, x \in \{0,1\}^* \mid x \text{ ends with } 01 \,\}$$



In this DFA:

1. States: $W = \{w_0, w_1, w_2\}$
2. Input Alphabet: $\Sigma = \{0, 1\}$
3. Transition Function V:

$$\begin{aligned} V(w_0, 0) &= w_1 & \qquad V(w_1, 1) &= w_2 \\ V(w_0, 1) &= w_0 & \qquad V(w_2, 0) &= w_1 \\ V(w_1, 0) &= w_1 & \qquad V(w_2, 1) &= w_0 \end{aligned}$$

4. Start State: $S = w_0$
5. Accept State: $F = w_2$

3.2 Transition Table

From that Transition Diagram we can construct a Transition Table as follow

W	**0**	**1**
w_0	w_1	w_0
w_1	w_1	w_2
w_2	w_1	w_0

3.3 Extended Transition Function

The extended transition function $\hat{V} : W \times \Sigma^* \to W$ is defined by

1. $\hat{V}(w, \varepsilon) = w$, $\forall w \in W$.
2. $\hat{V}(w, xy) = V\big(\hat{V}(w, x), y\big)$, $\forall w \in W$, $x \in \Sigma^*$, $y \in \Sigma$.

Hence, from the above DFA we compute (starting from the start state w_0):

1. $\hat{V}(w_0, \varepsilon) = w_0$
2. $\hat{V}(w_0, 0) = V(\hat{V}(w_0, \varepsilon), 0) = w_1$
3. $\hat{V}(w_0, 1) = V(\hat{V}(w_0, \varepsilon), 1) = w_0$
4. $\hat{V}(w_0, 01) = V(\hat{V}(w_0, 0), 1) = w_2$
5. $\hat{V}(w_0, 101) = V(\hat{V}(w_0, 10), 1) = w_2$.
6. $\hat{V}(w_0, 1101) = V(\hat{V}(w_0, 110), 1) = w_2$.
7. $\hat{V}(w_0, 100) = V(\hat{V}(w_0, 10), 0) = w_1$.

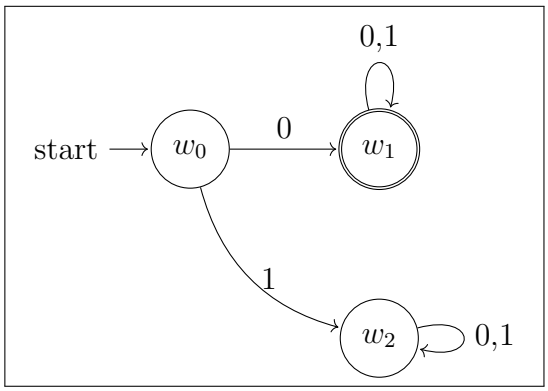
In general:

$$\hat{V}(w_0, x) = F \;\leftrightarrow\; x \text{ ends with } xy, \text{ so } L(M) = \{\, x \in \{0,1\}^* \mid \hat{V}(w_0, x) \in F \,\}.$$

Example 1:
Let

$$L(M_1) = \{0(0+1)^*\}$$

Hence



Specifically, it only accept the input strings that start with 0. Oterwise, it goes to the dead state w_2. Formally,

$$L(M_1) = \begin{cases} \{\, 0x \mid x \in \{0,1\}^* \,\}, & \text{if the first symbol is 0,} \\ \varnothing, & \text{otherwise.} \end{cases}$$

1. Transition Function V:

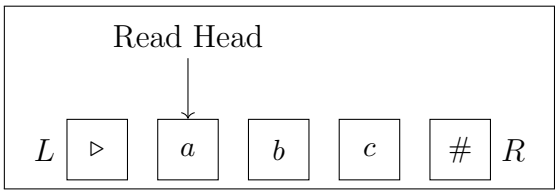
$$\begin{aligned} V(w_0, 0) &= w_1 & \qquad V(w_1, 1) &= w_1 \\ V(w_0, 1) &= w_2 & \qquad V(w_2, 0) &= w_2 \\ V(w_1, 0) &= w_1 & \qquad V(w_2, 1) &= w_2 \end{aligned}$$

2. Transition Table:

W	**0**	**1**
w_0	w_1	w_2
w_1	w_1	w_1
w_2	w_2	w_2

3.4 Two-way Finite Automata (2DFA)

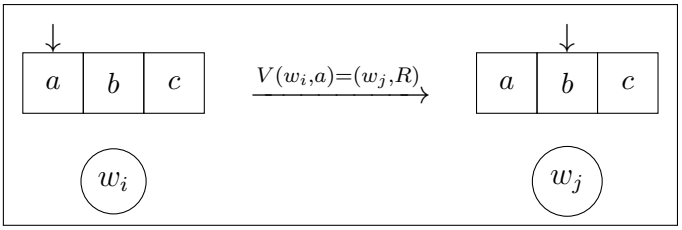
Formally, $V : W \times (\Sigma \cup \{\triangleright, \#\}) \to W \times \{L, R\}$ is the transition function, where # is the end-of-tape marker and $\triangleright$ is the start-of-tape marker.



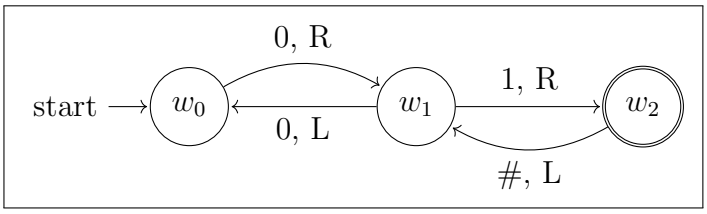
In general:

$$V(w_i, a) = (w_j, R)$$

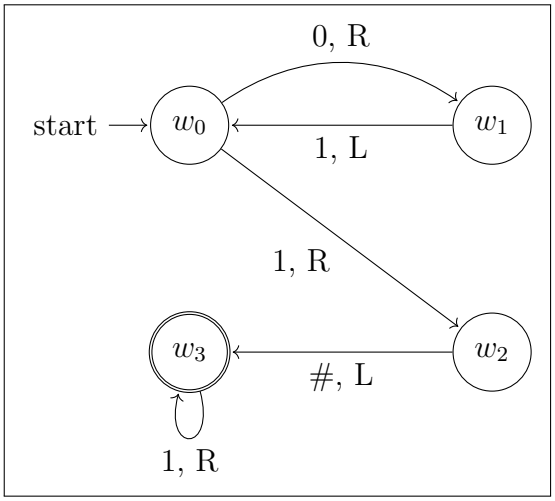
This means that when the 2DFA is in state w_i and reads the symbol a, it transitions to state w_j and moves the read head one cell to the right (R).



Intuitively, a 2DFA operates like a standard DFA



Example:
Let



This 2DFA accepts only the string 1. The transition function is defined as:

$$V(w_0, 0) = (w_1, R) \qquad V(w_2, \#) = (w_3, L)$$
$$V(w_0, 1) = (w_2, R) \qquad V(w_3, 1) = (w_3, R)$$
$$V(w_1, 1) = (w_0, L)$$

1. **Accepted:** Input 1
   (a) Start: w_0, $\boxed{\underline{1}}\boxed{\#}\boxed{\sqcup}\boxed{\sqcup}\boxed{\sqcup}\cdots$
   (b) Read 1, move R: w_2, $\boxed{1}\boxed{\underline{\#}}\boxed{\sqcup}\boxed{\sqcup}\boxed{\sqcup}\cdots$
   (c) Read #, move L: w_3, $\boxed{\underline{1}}\boxed{\#}\boxed{\sqcup}\boxed{\sqcup}\boxed{\sqcup}\cdots$
   (d) Read 1, move R: w_3, $\boxed{1}\boxed{\underline{\#}}\boxed{\sqcup}\boxed{\sqcup}\boxed{\sqcup}\cdots$
   (e) Accept (in state w_3 at end marker)
2. **Rejected (crash):** Input 0
   (a) Start: w_0, $\boxed{\underline{0}}\boxed{\#}\boxed{\sqcup}\boxed{\sqcup}\boxed{\sqcup}\cdots$
   (b) Read 0, move R: w_1, $\boxed{0}\boxed{\underline{1}}\boxed{\#}\boxed{\sqcup}\boxed{\sqcup}\cdots$
   (c) Read 1: No transition for $(w_1, 1)$, Crash
3. **Rejected (crash):** Input 11
   (a) Start: w_0, $\boxed{\underline{1}}\boxed{1}\boxed{\#}\boxed{\sqcup}\boxed{\sqcup}\cdots$
   (b) Read 1, move R: w_2, $\boxed{1}\boxed{\underline{1}}\boxed{\#}\boxed{\sqcup}\boxed{\sqcup}\cdots$
   (c) Read 1: No transition for $(w_2, 1)$, Crash
4. **Rejected (crash):** Input 10
   (a) Start: w_0, $\boxed{\underline{1}}\boxed{0}\boxed{\#}\boxed{\sqcup}\boxed{\sqcup}\cdots$
   (b) Read 1, move R: w_2, $\boxed{1}\boxed{\underline{0}}\boxed{\#}\boxed{\sqcup}\boxed{\sqcup}\cdots$
   (c) Read 0: No transition for $(w_2, 0)$, Crash
5. **Infinite loop:** Input 01
   (a) Start: w_0, $\boxed{\underline{0}}\boxed{1}\boxed{\#}\boxed{\sqcup}\boxed{\sqcup}\cdots$
   (b) Read 0, move R: w_1, $\boxed{0}\boxed{\underline{1}}\boxed{\#}\boxed{\sqcup}\boxed{\sqcup}\cdots$
   (c) Read 1, move L: w_0, $\boxed{\underline{0}}\boxed{1}\boxed{\#}\boxed{\sqcup}\boxed{\sqcup}\cdots$
   (d) Read 0, move R: w_1, $\boxed{0}\boxed{\underline{1}}\boxed{\#}\boxed{\sqcup}\boxed{\sqcup}\cdots$
   (e) Read 1, move L: w_0, $\boxed{\underline{0}}\boxed{1}\boxed{\#}\boxed{\sqcup}\boxed{\sqcup}\cdots$
   (f) Cycles indefinitely between steps 3–5, Infinite loop

6. **Infinite loop:** Input 010101

   (a) Start: w_0, $\boxed{\underline{0}}\boxed{1}\boxed{0}\boxed{1}\boxed{0}\boxed{1}\boxed{\#}\cdots$

   (b) Read 0, move R: w_1, $\boxed{0}\boxed{\underline{1}}\boxed{0}\boxed{1}\boxed{0}\boxed{1}\boxed{\#}\cdots$

   (c) Read 1, move L: w_0, $\boxed{\underline{0}}\boxed{1}\boxed{0}\boxed{1}\boxed{0}\boxed{1}\boxed{\#}\cdots$

   (d) Cycles between positions 0 and 1, Infinite loop

4 Minimization

A minimal DFA M' for a language L is a DFA such that

$$L(M') = L \ \wedge \ \nexists \text{ DFA } M'' \text{ with } L(M'') = L \text{ and } |W''| < |W'|.$$

For any DFA M with $L(M) = L$, the minimization process produces such an automaton M' by merging equivalent states, where $w \sim w'$ denotes that the states w and w' are equivalent.

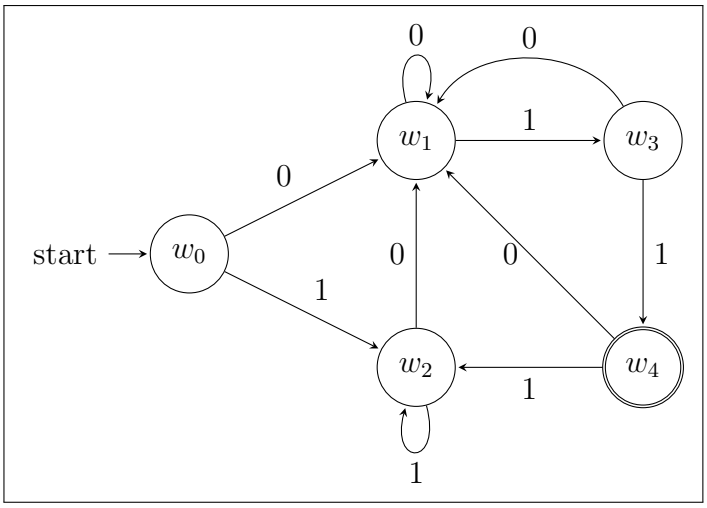
Example 1:

Let $M = (W, \Sigma, V, S, F)$, where

$$W = \{w_0, w_1, w_2, w_3, w_4\}, \quad \Sigma = \{0, 1\}, \quad S = w_0, \quad F = \{w_4\}.$$

The transition function is given by:

W	0	1
w_0	w_1	w_2
w_1	w_1	w_3
w_2	w_1	w_2
w_3	w_1	w_4
w_4	w_1	w_2



1. For a DFA M, the n-equivalence relation $\equiv_n$ on states is defined inductively as follows:

$$w \equiv_0 w' \ \leftrightarrow \ (w \in F \leftrightarrow w' \in F).$$

$$w \equiv_{n+1} w' \ \leftrightarrow \ \big(w \equiv_0 w' \ \wedge \ \forall a \in \Sigma, \ V(w, a) \equiv_n V(w', a)\big).$$

2. The equivalence relation used in DFA minimization is obtained when this sequence stabilizes:

$$\exists k \in \mathbb{N} \text{ such that } \equiv_k = \equiv_{k+1} .$$

3. The resulting equivalence relation can be characterized as:

$$w \sim w' \;\leftrightarrow\; \forall x \in \Sigma^*,\; \big(V^*(w,x) \in F \leftrightarrow V^*(w',x) \in F\big).$$

From the transition structure, we compute the successive partitions:

1. **0-Equivalence:** Partition states by acceptance.

$$\phi_0 = \big\{\{w_0, w_1, w_2, w_3\}, \{w_4\}\big\}$$

2. **1-Equivalence:** Refine partitions based on transitions.

   For $\{w_0, w_1, w_2, w_3\}$:

   (a) w_0: $0 \to w_1$, $1 \to w_2$
   (b) w_1: $0 \to w_1$, $1 \to w_3$
   (c) w_2: $0 \to w_1$, $1 \to w_2$
   (d) w_3: $0 \to w_1$, $1 \to w_4$

   Since w_3 transitions to an accepting state on input 1, we split:

$$\phi_1 = \big\{\{w_0, w_1, w_2\}, \{w_3\}, \{w_4\}\big\}$$

3. **2-Equivalence:** Refine $\{w_0, w_1, w_2\}$.

   (a) w_0: $1 \to w_2 \in \{w_0, w_2\}$
   (b) w_1: $1 \to w_3 \in \{w_3\}$
   (c) w_2: $1 \to w_2 \in \{w_0, w_2\}$

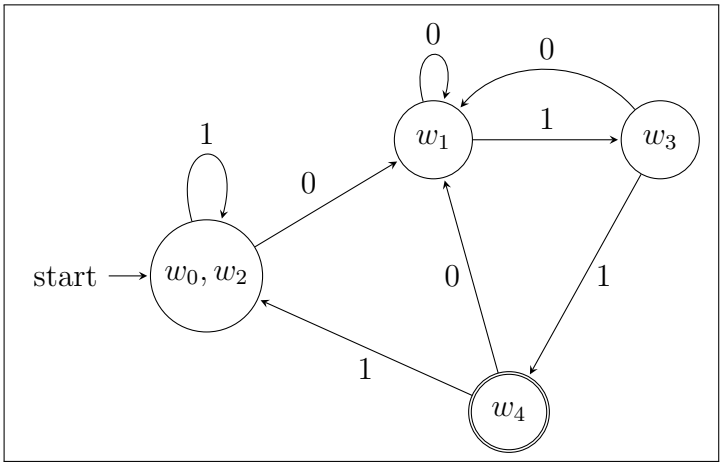
   Thus,

$$\phi_2 = \big\{\{w_0, w_2\}, \{w_1\}, \{w_3\}, \{w_4\}\big\}$$

4. **3-Equivalence:** The block $\{w_0, w_2\}$ cannot be further refined, so

$$\phi_3 = \phi_2.$$

Hence, the minimal DFA merges w_0 and w_2 into a single state. The minimized DFA has states $\{\{w_0, w_2\}, w_1, w_3, w_4\}$.



Example 2:
Let

$$M = (W, \Sigma, V, S, F)$$

where

$$W = \{w_0, w_1, w_2, w_3, w_4, w_5, w_6, w_7\}, \quad \Sigma = \{0, 1\}, \quad S = w_0, \quad F = \{w_2\}.$$

The transition function is:

W	0	1
w_0	w_1	w_5
w_1	w_6	w_2
w_2	w_0	w_2
w_3	w_2	w_6
w_4	w_7	w_5
w_5	w_2	w_6
w_6	w_6	w_4
w_7	w_6	w_2

1. **0-Equivalence**

   Partition states by acceptance:

$$f_0 = \big\{\{w_2\}, \{w_0, w_1, w_3, w_4, w_5, w_6, w_7\}\big\}$$

2. **1-Equivalence**

   Refine the non-accepting block by transitions:

W	0	1
w_0	w_1	w_5
w_1	w_6	w_2
w_3	w_2	w_6
w_4	w_7	w_5
w_5	w_2	w_6
w_6	w_6	w_4
w_7	w_6	w_2

   States transitioning to the accepting state w_2 are separated:

$$f_1 = \big\{\{w_2\}, \{w_1, w_3, w_5, w_7\}, \{w_0, w_4, w_6\}\big\}$$

3. **2-Equivalence**

   Refine each block further. For $\{w_1, w_3, w_5, w_7\}$:

W	0	1
w_1	w_6	w_2
w_3	w_2	w_6
w_5	w_2	w_6
w_7	w_6	w_2

This splits into:

$$\{w_1, w_7\}, \quad \{w_3, w_5\}$$

For $\{w_0, w_4, w_6\}$:

W	0	1
w_0	w_1	w_5
w_4	w_7	w_5
w_6	w_6	w_4

This splits into:

$$\{w_0, w_4\}, \quad \{w_6\}$$

Thus:

$$f_2 = \big\{\{w_2\}, \{w_1, w_7\}, \{w_3, w_5\}, \{w_0, w_4\}, \{w_6\}\big\}$$

4. **3-Equivalence**

   All blocks are stable under further refinement:

$$f_3 = f_2$$

   The equivalence classes of the minimized DFA are:

$$\{w_2\},\ \{w_1, w_7\},\ \{w_3, w_5\},\ \{w_0, w_4\},\ \{w_6\}$$

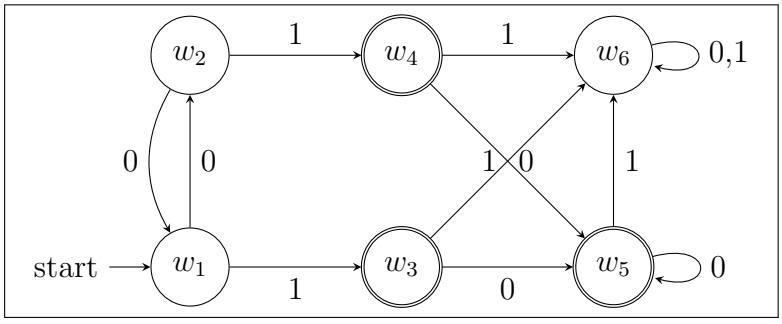
   Hence, the minimal DFA has 5 states

5 Myhill-Nerode Theorem

We can also minimize DFA with Myhill-Nerode Theorem by following these steps

Example:

Suppose we have a DFA M as below:



1. Step 1: Draw a table of all pair of statess (X,Y).

	w_1	w_2	w_3	w_4	w_5	w_6
w_1	–		✓	✓	✓	
w_2		–	✓	✓	✓	
w_3			–			✓
w_4				–		✓
w_5					–	✓
w_6						–

2. step 2: Mark all pairs where $(X \in F \wedge Y \notin F) \vee (X \notin F \wedge Y \in F)$

$$(w_1, w_2) \begin{cases} V(w_1, 0) = w_2, & V(w_2, 0) = w_1, \\ V(w_1, 1) = w_3, & V(w_2, 1) = w_4 \end{cases}$$

$$(w_3, w_4) \begin{cases} V(w_3, 0) = w_5, & V(w_4, 0) = w_5, \\ V(w_3, 1) = w_6, & V(w_4, 1) = w_6 \end{cases}$$

$$(w_3, w_5) \begin{cases} V(w_3, 0) = w_5, & V(w_5, 0) = w_5, \\ V(w_3, 1) = w_6, & V(w_5, 1) = w_6 \end{cases}$$

$$(w_4, w_5) \begin{cases} V(w_4, 0) = w_5, & V(w_5, 0) = w_5, \\ V(w_4, 1) = w_6, & V(w_5, 1) = w_6 \end{cases}$$

$$(w_1, w_6) \begin{cases} V(w_1, 0) = w_2, & V(w_6, 0) = w_6, \\ V(w_1, 1) = w_3, & V(w_6, 1) = w_6 \end{cases}$$

$$(w_2, w_6) \begin{cases} V(w_2, 0) = w_1, & V(w_6, 0) = w_6, \\ V(w_2, 1) = w_4, & V(w_6, 1) = w_6 \end{cases}$$

3. step 3: Iteratively mark pairs where transitions lead to already marked pairs until no new pairs can be marked.

$$(w_1, w_2) \begin{cases} V(w_1, 0) = w_2, & V(w_2, 0) = w_1, \\ V(w_1, 1) = w_3, & V(w_2, 1) = w_4 \end{cases}$$

$$(w_3, w_4) \begin{cases} V(w_3, 0) = w_5, & V(w_4, 0) = w_5, \\ V(w_3, 1) = w_6, & V(w_4, 1) = w_6 \end{cases}$$

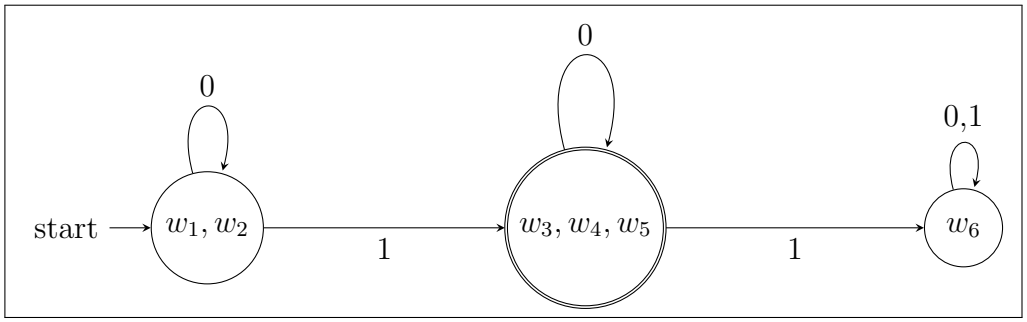
$$(w_3, w_5) \begin{cases} V(w_3, 0) = w_5, & V(w_5, 0) = w_5, \\ V(w_3, 1) = w_6, & V(w_5, 1) = w_6 \end{cases}$$

$$(w_4, w_5) \begin{cases} V(w_4, 0) = w_5, & V(w_5, 0) = w_5, \\ V(w_4, 1) = w_6, & V(w_5, 1) = w_6 \end{cases}$$

4. step 4: Combine all the unmarked pairs into equivalence classes.

$$\{w_1, w_2\},\ \{w_3, w_4, w_5\},\ \{w_6\}$$

Hence the final diagram is:



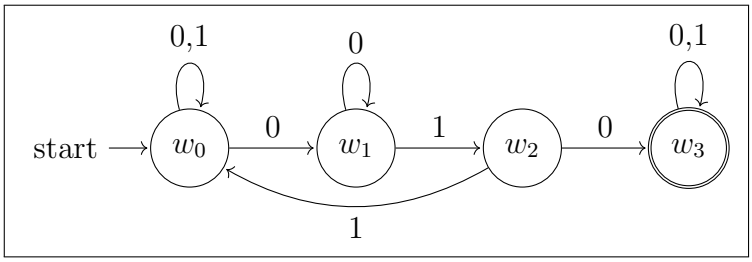
6 Non-deterministic Finite Automata (NFA)

An NFA is similar like DFA, but a state can have zero, one, or many possible transitions for the same input symbol, and it may even move without consuming input (ε-Transition). It's more flexible in structure, though it recognizes the same class of languages as a DFA. Formally, an NFA is defined as a 5-tuple $M = (W, \Sigma, V, S, F)$ where $V : W \times \Sigma \to 2^{|W|}$, and the language it recognizes is defined as: $L(M) = \{x \mid V(w, x) \cap F \neq \varnothing\}$.

Example:

Let

$$L(M) = \{\, x \in \{0,1\}^* \mid x \text{ contains the substring } 010 \,\}$$



In this NFA:

1. Transition Function V:

$$\begin{aligned} V(w_0, 0) &= \{w_0, w_1\} & V(w_2, 0) &= \{w_3\} \\ V(w_0, 1) &= \{w_0\} & V(w_2, 1) &= \{w_0\} \\ V(w_1, 0) &= \{w_1\} & V(w_3, 0) &= \{w_3\} \\ V(w_1, 1) &= \{w_2\} & V(w_3, 1) &= \{w_3\} \end{aligned}$$

2. Transition Table:

W	**0**	**1**
w_0	$\{w_0, w_1\}$	$\{w_0\}$
w_1	$\{w_1\}$	$\{w_2\}$
w_2	$\{w_3\}$	$\{w_0\}$
w_3	$\{w_3\}$	$\{w_3\}$

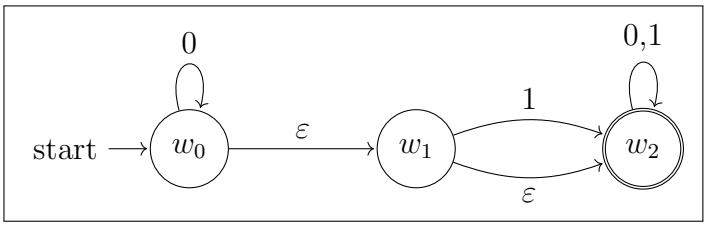
6.1 ε-NFA

An ε-NFA is an extension of the NFA that allows transitions without consuming any input symbols, known as ε-transitions. Formally, $V : W \times (\Sigma \cup \{\varepsilon\}) \to 2^{|W|}$.

Example:

Let

$$L(M) = \{\, x \in \{0,1\}^* \mid V^*(w_0, x) \cap \{w_2\} \neq \varnothing \,\}$$



In this ε-NFA:

$$\begin{aligned} V(w_0, \varepsilon) &= \{w_1\} & V(w_1, \varepsilon) &= \{w_2\} \\ V(w_0, 0) &= \{w_0\} & V(w_2, 0) &= \{w_2\} \\ V(w_1, 1) &= \{w_2\} & V(w_2, 1) &= \{w_2\} \end{aligned}$$

6.2 ε-Closure

The ε-closure of a state w is the set of all states reachable from w using only ε-transitions, including w itself. Formally,

$$\varepsilon\text{-closure}(w) = \{\, u \in W \mid \exists k \geq 0,\ w = w_0,\ w_k = u \text{ such that } (w_i, \varepsilon, w_{i+1}) \in V, \forall 0 \leq i < k \,\}.$$

For a set of states $S \subseteq W$,

$$\varepsilon\text{-closure}(S) = \bigcup_{w \in S} \varepsilon\text{-closure}(w).$$

In light of this, for the example ε-NFA above:

1. $\varepsilon(w_0) = \{w_0, w_1, w_2\}$ (since $w_0 \xrightarrow{\varepsilon} w_1 \xrightarrow{\varepsilon} w_2$)
2. $\varepsilon(w_1) = \{w_1, w_2\}$ (since $w_1 \xrightarrow{\varepsilon} w_2$)
3. $\varepsilon(w_2) = \{w_2\}$ (no ε-transitions from w_2)

Moreover, the extended Transition Function for ε-NFA is defined as:

1. $\hat{V}(w_0, \varepsilon) = \varepsilon(w_0) = \{w_0, w_1, w_2\}$
2. $\hat{V}(w_0, 0) = V(\varepsilon(w_0), 0) = V(\{w_0, w_1, w_2\}, 0) = \{w_0, w_2\}$
3. $\hat{V}(w_0, 1) = V(\varepsilon(w_0), 1) = V(\{w_0, w_1, w_2\}, 1) = \{w_2\}$

Thus, the ε-closures can be summarized in the following table:

W	**ε-Closure**
w_0	$\{w_0, w_1, w_2\}$
w_1	$\{w_1, w_2\}$
w_2	$\{w_2\}$

7 Moore and Mealy Machine

7.1 Moore Machine

Formally, a Moore machine is defined as a 6-tuple

$$M = (W, \Sigma, \Delta, V, \lambda, S)$$

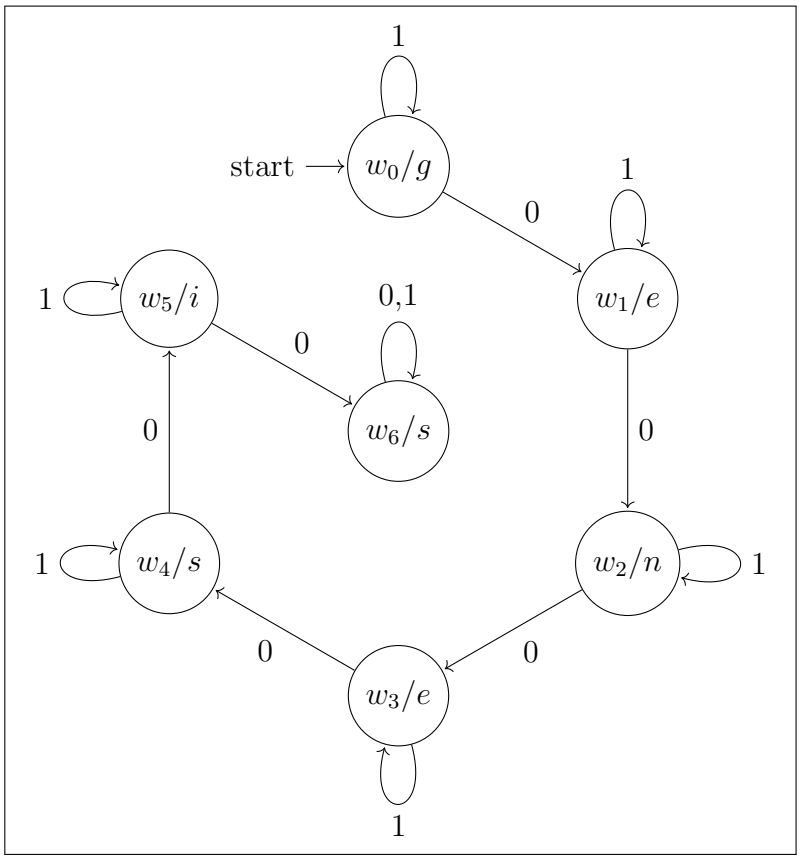
where

1. Δ is the output alphabet,
2. $\lambda : W \to \Delta$ is the output function that maps each state to an output symbol.

Example:
Let $W = \{w_0, w_1, w_2, w_3, w_4, w_5, w_6\}$, $\Sigma = \{0, 1\}$, $\Delta = \{g, e, n, s, i\}$, and $S = w_0$.

1. Transition Function V and Output Function λ:

W	**0**	**1**
w_0	w_1	w_0
w_1	w_2	w_1
w_2	w_3	w_2
w_3	w_4	w_3
w_4	w_5	w_4
w_5	w_6	w_5
w_6	w_6	w_6



Operation on input string 000000:

Input	**State**	**Output**
–	w_0	g
0	w_1	e
0	w_2	n
0	w_3	e
0	w_4	s
0	w_5	i
0	w_6	s

Hence, final output sequence: $\boxed{genesis}$

7.2 Mealy Machine

Bassically, a Mealy machine is defined as a 6-tuple, just like the Moore machine:

$$M = (W, \Sigma, \Delta, V, \lambda, S)$$

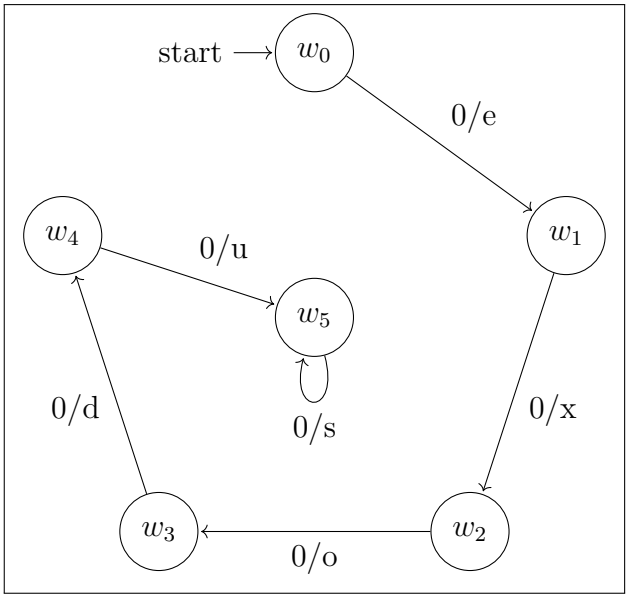
In this case, if Moore machine's output function maps states to outputs, Mealy machine's output function maps state-input pairs to outputs:

1. Δ is the output alphabet,
2. $\lambda : W \times \Sigma \to \Delta$ is the output function that maps each state-input pair to an output symbol,
3. The output is determined by the transition: $\lambda(w, x) = y$, where $w \in W$, $x \in \Sigma$, and $y \in \Delta$.

Example:
Let $W = \{w_0, w_1, w_2, w_3, w_4, w_5\}$, $\Sigma = \{0, 1\}$, $\Delta = \{e, x, o, d, u, s\}$, and $S = w_0$.

1. Transition Function V and Output Function λ:

$$\begin{aligned}
V(w_0, 0) &= w_1, & \lambda(w_0, 0) &= e, \\
V(w_1, 0) &= w_2, & \lambda(w_1, 0) &= x, \\
V(w_2, 0) &= w_3, & \lambda(w_2, 0) &= o, \\
V(w_3, 0) &= w_4, & \lambda(w_3, 0) &= d, \\
V(w_4, 0) &= w_5, & \lambda(w_4, 0) &= u, \\
V(w_5, 0) &= w_5, & \lambda(w_5, 0) &= s.
\end{aligned}$$



Operation on input string 000000:

Input	**State**	**Output**	**Transition**
–	w_0	–	–
0	w_1	e	$V(w_0, 0) = w_1, \lambda(w_0, 0) = e$
0	w_2	x	$V(w_1, 0) = w_2, \lambda(w_1, 0) = x$
0	w_3	o	$V(w_2, 0) = w_3, \lambda(w_2, 0) = o$
0	w_4	d	$V(w_3, 0) = w_4, \lambda(w_3, 0) = d$
0	w_5	u	$V(w_4, 0) = w_5, \lambda(w_4, 0) = u$
0	w_5	s	$V(w_5, 0) = w_5, \lambda(w_5, 0) = s$

Hence, final output sequence: $\boxed{exodus}$

8 Arden's Theorem

Let f, g, h be regular expressions over an alphabet Σ.

If

$$f = g + fh$$

and

$$\varepsilon \notin L(h),$$

then the equation has a *unique solution*, namely

$$f = gh^*.$$

Proof.
Assume that

$$f = g + fh$$

with $\varepsilon \notin L(h)$.
We first show that $f = gh^*$ is a solution.

$$\begin{aligned} gh^* &= g(\varepsilon + hh^*) \\ &= g + ghh^* \\ &= g + (gh^*)h. \end{aligned}$$

Thus,

$$f = gh^*$$

satisfies the equation $f = g + fh$.
Next, we prove uniqueness.
Let f be any solution of the equation. By repeated substitution,

$$\begin{aligned} f &= g + fh \\ &= g + (g + fh)h \\ &= g + gh + fh^2 \\ &= g + gh + gh^2 + \cdots \end{aligned}$$

Hence,

$$f = g(\varepsilon + h + h^2 + h^3 + \cdots) = gh^*.$$

Since $\varepsilon \notin L(h)$, the expression h^* is well-defined and the solution is unique.
Therefore, the unique solution of

$$f = g + fh$$

is

$$\boxed{f = gh^*}$$

QED

9 Conversion

9.1 NFA to DFA

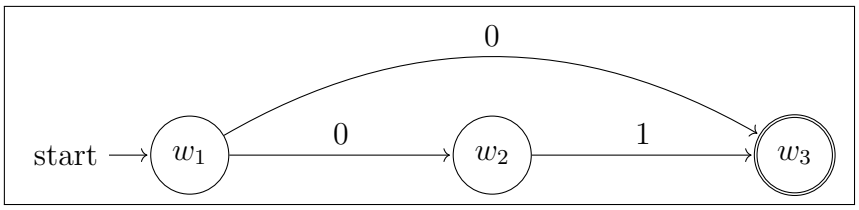
1. Given an NFA
$$N = (W, \Sigma, V, S, F)$$
2. the equivalent DFA
$$D = (W', \Sigma, V', S', F')$$

where

$$W' = 2^{|W|} \qquad V'(W, x) = \varepsilon\text{-closure}\left(\bigcup_{w \in W} V(w, x)\right)$$
$$S' = \varepsilon\text{-closure}(S) \qquad F' = \{\, W \in W' \mid W \cap F \neq \varnothing \,\}$$

In general

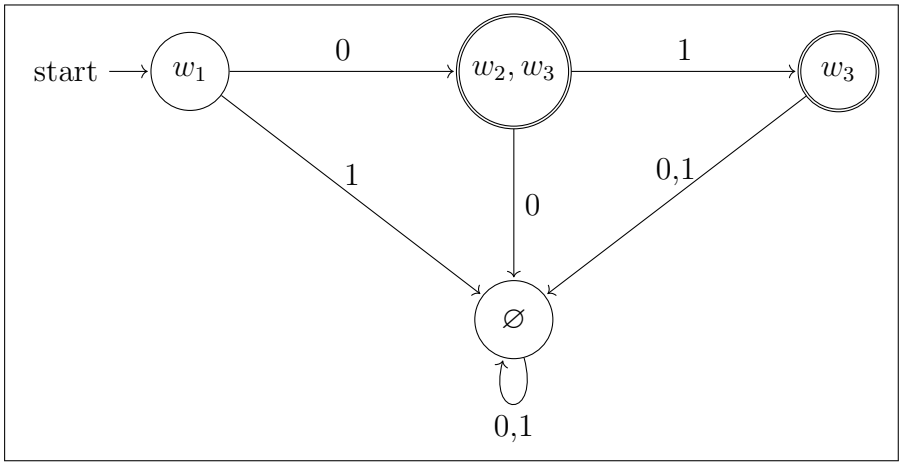
1. NFA



NFA Transition Table

W	**0**	**1**
w_1	$\{w_2, w_3\}$	$\varnothing$
w_2	$\varnothing$	$\{w_3\}$
w_3	$\varnothing$	$\varnothing$

2. DFA



DFA Transition Table

W'	**0**	**1**
w_1	$\{w_2, w_3\}$	$\varnothing$
$\{w_2, w_3\}$	$\varnothing$	$\{w_3\}$
$\{w_3\}$	$\varnothing$	$\varnothing$
$\varnothing$	$\varnothing$	$\varnothing$

1. **Example 1:**

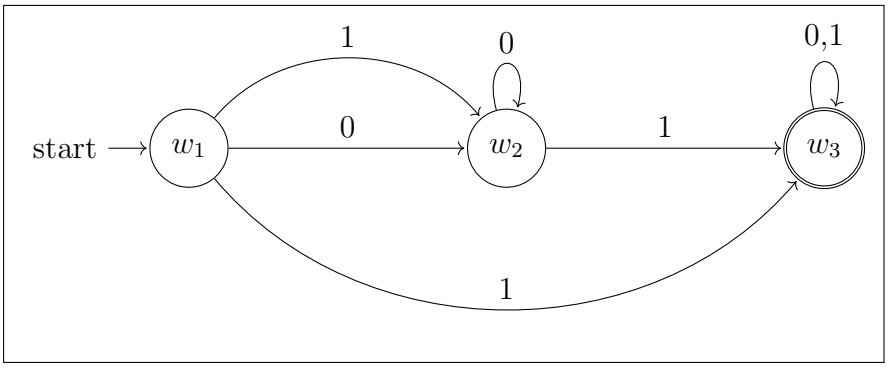
   Consider the NFA, where

$$W = \{w_1, w_2, w_3\}, \quad \Sigma = \{0, 1\}, \quad F = \{w_3\},$$

   and the transition function V is defined as:

$$\begin{aligned} V(w_1, 0) &= \{w_2\}, & V(w_2, 0) &= \{w_2\}, \\ V(w_1, 1) &= \{w_2, w_3\}, & V(w_3, 0) &= \{w_3\}, \\ V(w_2, 1) &= \{w_3\}, & V(w_3, 1) &= \{w_3\}. \end{aligned}$$

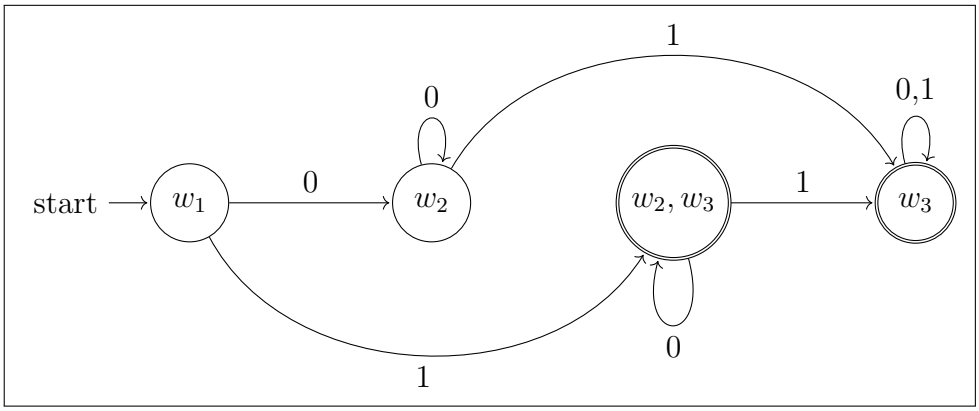
(a) NFA



Transition Table

W	**0**	**1**
w_1	w_2	$\{w_2, w_3\}$
w_2	w_2	w_3
w_3	w_3	w_3

(b) DFA



DFA Transition Table

W	**0**	**1**
w_1	w_2	$\{w_2, w_3\}$
w_2	w_2	w_3
w_3	w_3	w_3
$\{w_2, w_3\}$	$\{w_2, w_3\}$	w_3

2. **Example 2:**

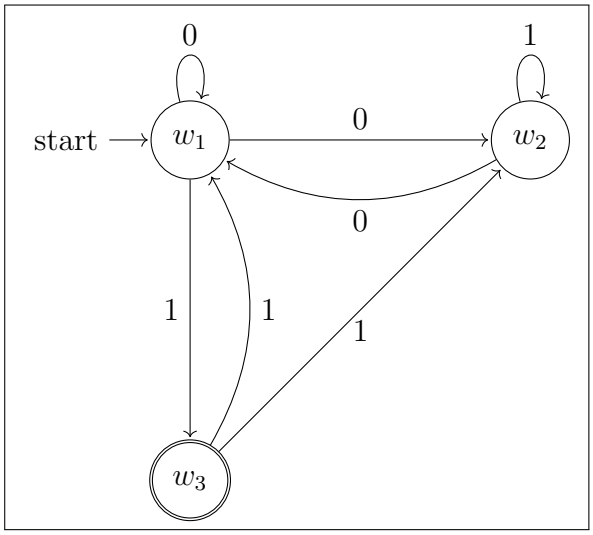
Consider the NFA:

$$W = \{w_1, w_2, w_3\}, \qquad \Sigma = \{0, 1\}, \qquad F = \{w_3\},$$

with transition function

$$V(w_1, 0) = \{w_1, w_2\}, \qquad V(w_2, 0) = \{w_1\}, \qquad V(w_3, 0) = \varnothing,$$
$$V(w_1, 1) = \{w_3\}, \qquad V(w_2, 1) = \{w_2\}, \qquad V(w_3, 1) = \{w_1, w_2\}.$$

(a) NFA



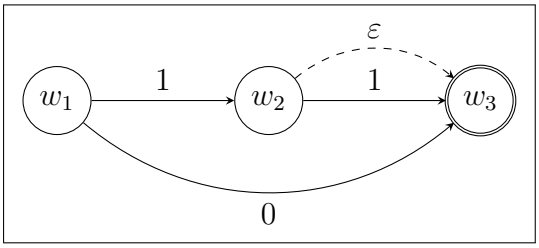
(b) DFA Transition Table

W	**0**	**1**
w_1	$\{w_1, w_2\}$	w_3
$\{w_1, w_2\}$	$\{w_1, w_2\}$	$\{w_2, w_3\}$
w_3	$\varnothing$	$\{w_1, w_2\}$
$\{w_2, w_3\}$	w_1	$\{w_1, w_2\}$

9.2 ε-NFA to NFA

Let

$$W = \{w_1, w_2, w_3\}, \quad \Sigma = \{0, 1\}, \quad F = \{w_3\}$$



1. Transition function of the ε-NFA:

W	**0**	**1**	ε
w_1	$\{w_3\}$	$\{w_2\}$	$\varnothing$
w_2	$\varnothing$	$\{w_3\}$	$\{w_3\}$
w_3	$\varnothing$	$\varnothing$	$\varnothing$

2. ε-closures:

$$\varepsilon(w_1) = \{w_1\}, \quad \varepsilon(w_2) = \{w_2, w_3\}, \quad \varepsilon(w_3) = \{w_3\}$$

3. Transition function of the equivalent NFA:

$$V_N(w, x) = \varepsilon\text{-closure}\Big(\bigcup_{w' \in \varepsilon(w)} V(w', x) \Big)$$

W	**0**	**1**
w_1	$\{w_3\}$	$\{w_2, w_3\}$
w_2	$\varnothing$	$\{w_3\}$
w_3	$\varnothing$	$\varnothing$

4. Accepting states of the NFA:

$$F_N = \{w \in W \mid \varepsilon(w) \cap F \neq \varnothing\} = \{w_2, w_3\}$$

9.3 NFA to RegExp

To convert an NFA to a regular expression, we follow these steps:

1. Represent each transition as a regular expression. If multiple transitions exist between the same states, combine them with + (union).
2. Identify the start and accepting states.
3. Iteratively eliminate intermediate states:
   - For a state w_k with incoming transitions from w_i and outgoing transitions to w_j, replace paths through w_k using the formula:

     $$R_{ij}^{\text{new}} = R_{ij}^{\text{old}} + R_{ik}(R_{kk})^* R_{kj}$$

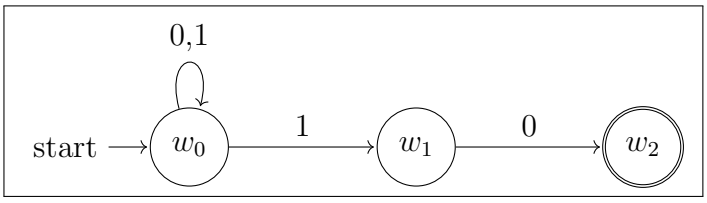
     where R_{ik} is the regex from w_i to w_k, R_{kk} is the loop at w_k, and R_{kj} is the regex from w_k to w_j.
4. After all intermediate states are removed, the resulting regex from the start to accepting state is the regular expression of the language.

Example:
Let the NFA be:

$$W = \{w_0, w_1, w_2\}, \quad \Sigma = \{0, 1\}, \quad S = \{w_0\}, \quad F = \{w_2\}$$

Transition function V:

$$\begin{aligned} &V(w_0, 0) = \{w_0\}, \quad V(w_0, 1) = \{w_0, w_1\} \\ &V(w_1, 0) = \{w_2\}, \quad V(w_1, 1) = \varnothing \\ &V(w_2, 0) = \varnothing, \qquad V(w_2, 1) = \varnothing \end{aligned}$$



1. Step 1: Initial Regular Expressions

   We label each transition with its corresponding regular expression:

$$
\begin{aligned}
&w_0 \xrightarrow{0+1} w_0 \quad \text{(self-loop)} \\
&w_0 \xrightarrow{1} w_1 \\
&w_1 \xrightarrow{0} w_2 \\
&w_2 \text{ has no outgoing transitions}
\end{aligned}
$$

   Initial edge labels:

$$
\begin{aligned}
R_{w_0,w_0} &= 0 + 1 \\
R_{w_0,w_1} &= 1 \\
R_{w_1,w_2} &= 0 \\
R_{w_1,w_1} &= \varnothing \quad \text{(no self-loop on } w_1\text{)}
\end{aligned}
$$

2. Step 2: Eliminate State w_1

   We eliminate the intermediate state w_1 using the formula:

$$
R_{ij}^{\text{new}} = R_{ij}^{\text{old}} + R_{ik}(R_{kk})^* R_{kj}
$$

   For the path from w_0 to w_2 through w_1:

$$
\begin{aligned}
R_{w_0,w_2}^{\text{new}} &= R_{w_0,w_2}^{\text{old}} + R_{w_0,w_1}(R_{w_1,w_1})^* R_{w_1,w_2} \\
&= \varnothing + 1 \cdot (\varnothing)^* \cdot 0 \\
&= 1 \cdot \varepsilon \cdot 0 \\
&= 10
\end{aligned}
$$

   After eliminating w_1, we have:

$$
\begin{aligned}
R_{w_0,w_0} &= 0 + 1 \\
R_{w_0,w_2} &= 10
\end{aligned}
$$

3. Step 3: Construct Final Regular Expression

   Starting from w_0 and reaching the accepting state w_2. We can loop at w_0 any number of times using $(0+1)^*$ Then take the direct path to w_2 using 10

   The final regular expression is:

$$
\boxed{(0+1)^*10}
$$

4. Step 4: Verification

   Examples of accepted strings:

(a) 10: $w_0 \xrightarrow{1} w_0, w_1 \xrightarrow{0} w_2$

(b) 010: $w_0 \xrightarrow{0} w_0 \xrightarrow{1} w_0, w_1 \xrightarrow{0} w_2$

(c) 11110: $w_0 \xrightarrow{1} w_0, w_1 \xrightarrow{1} w_0 \xrightarrow{1} w_0, w_1 \xrightarrow{1} w_0 \xrightarrow{1} w_0, w_1 \xrightarrow{0} w_2$

The language is: *all strings over* $\{0, 1\}$ *that end with* 10.

9.4 DFA to RegExp

To convert a DFA to a regular expression, we can use state elimination as well. The process is almost the same as for NFAs:

1. Represent each transition as a regular expression.
2. Identify the start and accepting states.
3. Iteratively eliminate intermediate states using:

$$R_{ij}^{\text{new}} = R_{ij}^{\text{old}} + R_{ik}(R_{kk})^* R_{kj}$$

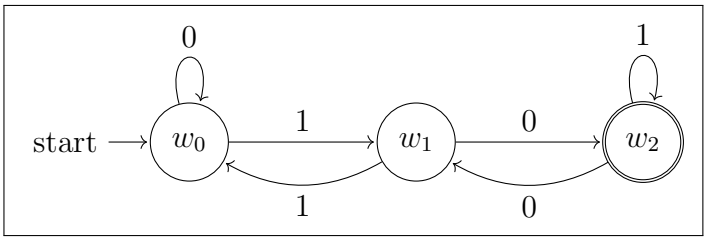
4. After eliminating all intermediate states, the resulting regex from the start to accepting state is the regular expression of the language.

Example:
Let the DFA be:

$$W = \{w_0, w_1, w_2\}, \quad \Sigma = \{0, 1\}, \quad S = \{w_0\}, \quad F = \{w_2\}$$

Transition function V:

$$\begin{aligned} V(w_0, 0) &= w_0, \quad V(w_0, 1) = w_1 \\ V(w_1, 0) &= w_2, \quad V(w_1, 1) = w_0 \\ V(w_2, 0) &= w_1, \quad V(w_2, 1) = w_2 \end{aligned}$$



1. Step 1:

   We label each transition with its corresponding regular expression:

$$w_0 \xrightarrow{0} w_0, \quad w_0 \xrightarrow{1} w_1, \quad w_1 \xrightarrow{0} w_2, \quad w_1 \xrightarrow{1} w_0, \quad w_2 \xrightarrow{0} w_1, \quad w_2 \xrightarrow{1} w_2$$

2. Step 2:

   We eliminate the intermediate state w_1 using the formula:

$$R_{ij}^{\text{new}} = R_{ij}^{\text{old}} + R_{ik}(R_{kk})^* R_{kj}$$

   First, we need to find R_{w_1,w_1}. Although w_1 has no direct self-loop, it can return to itself via w_0:

$$w_1 \xrightarrow{1} w_0 \xrightarrow{0^*} w_0 \xrightarrow{1} w_1$$

   Therefore:

$$R_{w_1,w_1} = 10^*1$$

   Path from w_0 to w_2 through w_1:

$$\begin{aligned} R_{w_0,w_2}^{\text{new}} &= R_{w_0,w_2}^{\text{old}} + R_{w_0,w_1}(R_{w_1,w_1})^* R_{w_1,w_2} \\ &= \varnothing + 1 \cdot (10^*1)^* \cdot 0 \\ &= 1(10^*1)^*0 \end{aligned}$$

   Self-loop on w_0:

$$\begin{aligned} R_{w_0,w_0}^{\text{new}} &= R_{w_0,w_0}^{\text{old}} + R_{w_0,w_1}(R_{w_1,w_1})^* R_{w_1,w_0} \\ &= 0 + 1 \cdot (10^*1)^* \cdot 1 \\ &= 0 + 1(10^*1)^*1 \end{aligned}$$

   Path from w_2 to w_2:

$$\begin{aligned} R_{w_2,w_2}^{\text{new}} &= R_{w_2,w_2}^{\text{old}} + R_{w_2,w_1}(R_{w_1,w_1})^* R_{w_1,w_2} \\ &= 1 + 0 \cdot (10^*1)^* \cdot 0 \\ &= 1 + 0(10^*1)^*0 \end{aligned}$$

   Then, the path from w_0 to w_2 is:

$$R_{w_0,w_2} = 1(10^*1)^*0$$

   We can loop at w_0 using $(0 + 1(10^*1)^*1)^*$, then go to w_2 and loop there using $(1 + 0(10^*1)^*0)^*$. The final regular expression for strings accepted by this DFA is:

$$\boxed{R = (0 + 1(10^*1)^*1)^* \cdot 1(10^*1)^*0 \cdot (1 + 0(10^*1)^*0)^*}$$

3. Step 3:

   Examples of accepted strings:

   (a) 10: $w_0 \xrightarrow{1} w_1 \xrightarrow{0} w_2$

(b) 110: $w_0 \xrightarrow{1} w_1 \xrightarrow{1} w_0 \xrightarrow{1} w_1 \xrightarrow{0} w_2$

(c) 101: $w_0 \xrightarrow{1} w_1 \xrightarrow{0} w_2 \xrightarrow{1} w_2$

(d) 0101: $w_0 \xrightarrow{0} w_0 \xrightarrow{1} w_1 \xrightarrow{0} w_2 \xrightarrow{1} w_2$

The language is: *all strings over* $\{0, 1\}$ *that contain* 10 *as a substring.*

9.5 RegExp to Finite Automaton FA

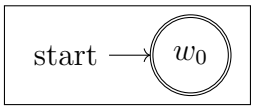
To convert a regular expression to a finite automaton (FA), we can use the following construction rules:

1. For the empty string ε, create an FA with a single state that is both the start and accepting state.

   Example:

   The regular expression ε is accepted by an FA consisting of one state w_0, where:

   $$S = \{w_0\}, \quad F = \{w_0\}$$

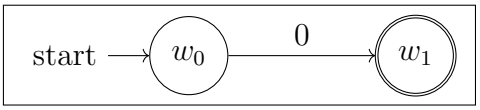
   No transitions are required. The empty string is accepted immediately.

2. For a single symbol $a \in \Sigma$, create an FA with two states: a start state and an accepting state, with a transition labeled a from the start to the accepting state.

   Example:

   For the regular expression 0, construct an FA with states w_0 and w_1 such that:

   $$S = \{w_0\}, \quad F = \{w_1\}, \quad V(w_0, 0) = \{w_1\}$$

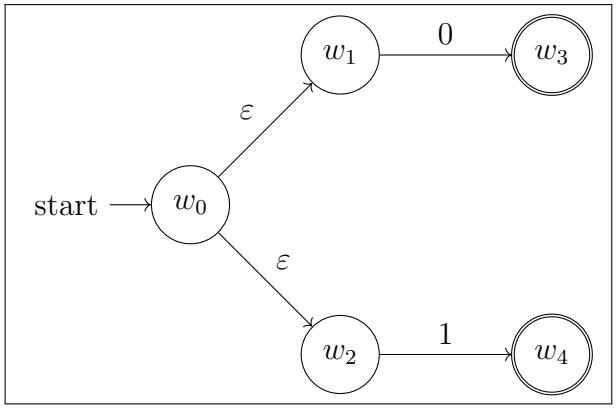
   

   The string 0 is accepted, while all other strings are rejected.

3. For union $R_1 + R_2$, create a new start state with ε-transitions to the start states of the FAs for R_1 and R_2. The accepting states of both FAs become accepting states of the new FA.

   Example:

   Let $R_1 = 0$ and $R_2 = 1$.

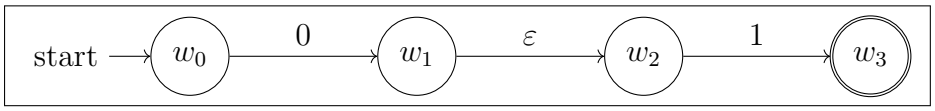


The resulting FA accepts either 0 or 1.

4. For concatenation R_1R_2, connect the accepting states of the FA for R_1 to the start state of the FA for R_2 using ε-transitions.

   Example:

   Let $R_1 = 0$ and $R_2 = 1$.

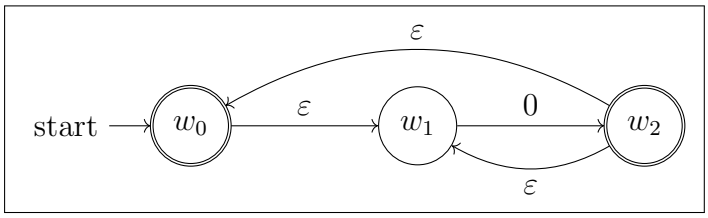


   The resulting FA accepts the string 01.

5. For Kleene star R^*, create a new start state that is also an accepting state. Add ε-transitions to allow repetition.

   Example:

   Let $R = 0$.



   This FA accepts $\varepsilon, 0, 00, 000$, and so on.

9.6 Moore to Mealy

To convert a Moore machine to an equivalent Mealy machine, we adjust the output function so that outputs are associated with transitions rather than states.

Given a Moore machine defined as:

$$M_o = (W, \Sigma, \Delta, V, \lambda, S),$$

we construct an equivalent Mealy machine:

$$M_e = (W, \Sigma, \Delta, V, \lambda', S),$$

where the new output function λ' is defined by:

$$\lambda'(w, x) = \lambda(V(w, x)) \quad \forall\, w \in W,\ \forall\, x \in \Sigma.$$

This ensures that the Mealy machine produces, during each transition, the same output that the Moore machine produces upon entering the next state.

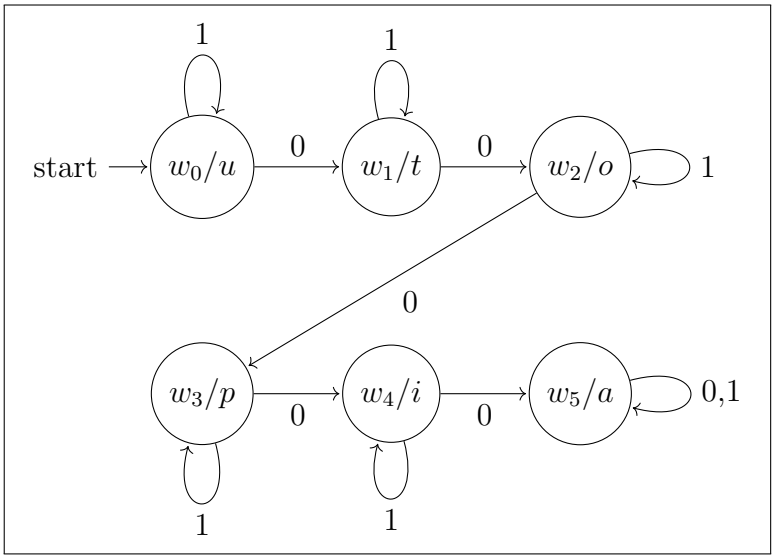
Example:

Let $W = \{w_0, w_1, w_2, w_3, w_4, w_5\}$, $\Sigma = \{0, 1\}$, $\Delta = \{u, t, o, p, i, a\}$, and $S = w_0$.

1. **Moore Machine**

   Transition Function V and Output Function λ:

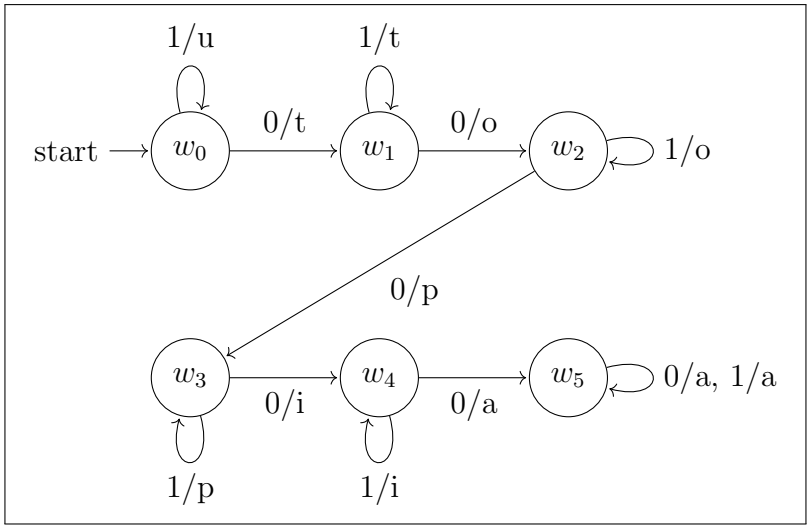
W	**0**	**1**	**Output**
w_0	w_1	w_0	u
w_1	w_2	w_1	t
w_2	w_3	w_2	o
w_3	w_4	w_3	p
w_4	w_5	w_4	i
w_5	w_5	w_5	a



2. **Mealy Machine**

   Using $\lambda'(w, x) = \lambda(V(w, x))$, we compute:

W	**0**	**1**
w_0	w_1/t	w_0/u
w_1	w_2/o	w_1/t
w_2	w_3/p	w_2/o
w_3	w_4/i	w_3/p
w_4	w_5/a	w_4/i
w_5	w_5/a	w_5/a



3. **Comparison on input string** 00000:

Input	**State**	**Moore Output**	**Mealy Output**
–	w_0	u	–
0	w_1	t	t
0	w_2	o	o
0	w_3	p	p
0	w_4	i	i
0	w_5	a	a

Hence, both machines produce the same output sequence: $\boxed{utopia}$

9.7 Mealy to Moore

To convert a Mealy machine to an equivalent Moore machine, we must modify the state space so that outputs are associated with states rather than transitions.

Let the Mealy machine be defined as

$$M_e = (W, \Sigma, \Delta, V, \lambda, S),$$

where $\lambda : W \times \Sigma \to \Delta$.

We construct an equivalent Moore machine

$$M_o = (W', \Sigma, \Delta, V', \lambda', S')$$

as follows.

1. State set

$$W' = \{\, (w, y) \mid w \in W,\ y \in \Delta,\ \exists x \in \Sigma \text{ such that } \lambda(w, x) = y \,\}$$

Each state (w, y) represents being in Mealy state w with output y.

2. Initial state

   Since a Mealy machine produces output only after reading an input symbol, we introduce a new symbol $s_\perp \notin W'$ to serve as the initial state.

$$S' = s_\perp$$

3. Output function

$$\lambda'(s_\perp) = \varepsilon$$
$$\lambda'((w, y)) = y \quad \forall (w, y) \in W'$$

4. Transition function

   From the initial symbol:

$$V'(s_\perp, x) = (V(S, x), \lambda(S, x)) \quad \forall x \in \Sigma$$

   For all other states:

$$V'((w, y), x) = (V(w, x), \lambda(w, x)) \quad \forall (w, y) \in W',\ \forall x \in \Sigma$$

The resulting Moore machine is equivalent to the original Mealy machine with respect to input/output behavior. The construction may introduce unreachable states; therefore, only states reachable from S' need to be retained to obtain a minimal equivalent Moore machine.

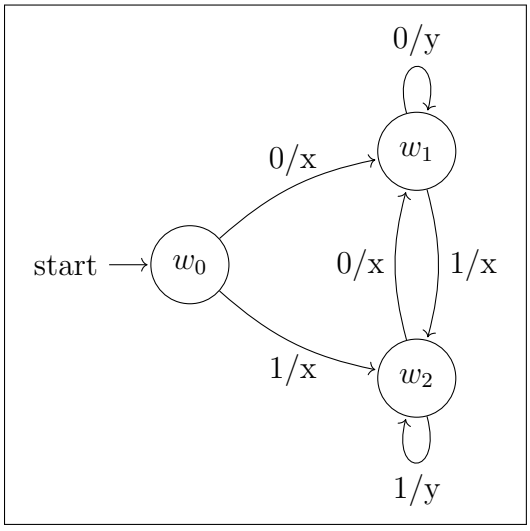
Example:

1. **Mealy Machine**

$$W = \{w_0, w_1, w_2\}, \quad \Sigma = \{0, 1\}, \quad \Delta = \{x, y\}, \quad S = w_0$$

   With this transition

W	0	1
w_0	w_1/x	w_2/x
w_1	w_1/y	w_2/x
w_2	w_1/x	w_2/y



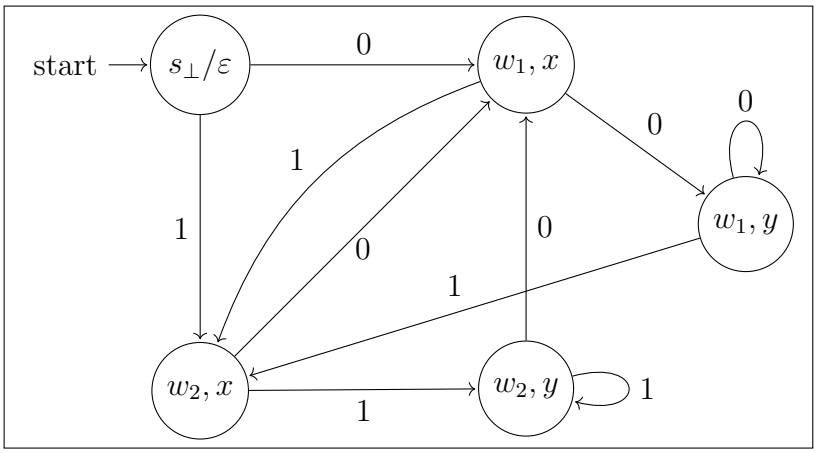
2. **Moore Machine**

   All reachable states:

$$W' = \{s_\perp \cup (w_1, x), (w_1, y), (w_2, x), (w_2, y)\}$$

   With this transition

W'	0	1
$s_\perp$	w_1, x	w_2, x
w_1, x	w_1, y	w_2, x
w_1, y	w_1, y	w_2, x
w_2, x	w_1, x	w_2, y
w_2, y	w_1, x	w_2, y



10 Property of Regular Languages

10.1 Decision Algorithms

A language is regular if there exists a finite automaton that recognizes it. The following decision problems can be solved for regular languages:

10.1.1 Emptiness

Formally,

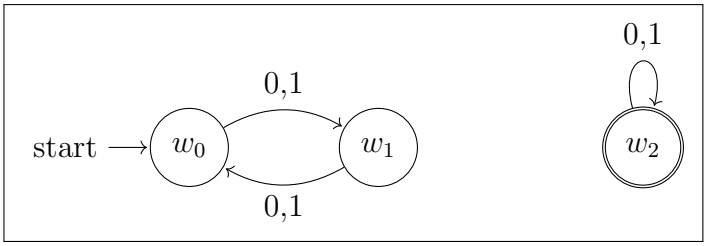
$$L(M) = \varnothing \leftrightarrow \forall f \in F,\ f \text{ is not reachable from } S$$

A DFA M has an empty language if no accepting state can be reached from the initial state.

Example:

Let $M = (W, \Sigma, V, S, F)$ where

$$W = \{w_0, w_1, w_2\}, \quad \Sigma = \{0, 1\}, \quad S = w_0, \quad F = \{w_2\}.$$



The accepting state w_2 is isolated and unreachable from the initial state w_0. Since there is no path from w_0 to w_2, no string can be accepted. Hence:

$$L(M) = \varnothing.$$

10.1.2 Membership

Formally,

$$x \in L(M) \leftrightarrow \hat{V}(S, x) \in F$$

where $\hat{V}$ is the extended transition function that processes the entire string x from the start state S.

Example:

Given DFA $M = (W, \Sigma, V, S, F)$ with

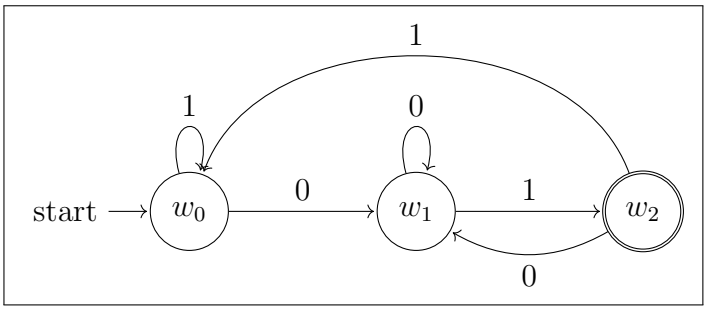
$$L(M) = \{x \in \{0,1\}^* \mid x \text{ ends in } 01\}$$

where:

$$W = \{w_0, w_1, w_2\}, \quad \Sigma = \{0,1\}, \quad S = w_0, \quad F = \{w_2\}.$$

Transition table:

V	0	1
w_0	w_1	w_0
w_1	w_1	w_2
w_2	w_1	w_0



Since $\hat{V}(w_0, 1001) = w_2 \in F$, the string $x = 1001$ is accepted. Therefore,

$$x \in L(M).$$

The string 1001 ends in 01, confirming membership in the language.

10.1.3 Finiteness

A language $L(M)$ is finite if it contains a finite number of strings, and infinite otherwise.

1. **Finite Language**

   Formally,

   $$L(M) \text{ is finite} \leftrightarrow \nexists w' \in W : S \xrightarrow{*} w' \xrightarrow{+} w' \xrightarrow{*} f,\ f \in F$$

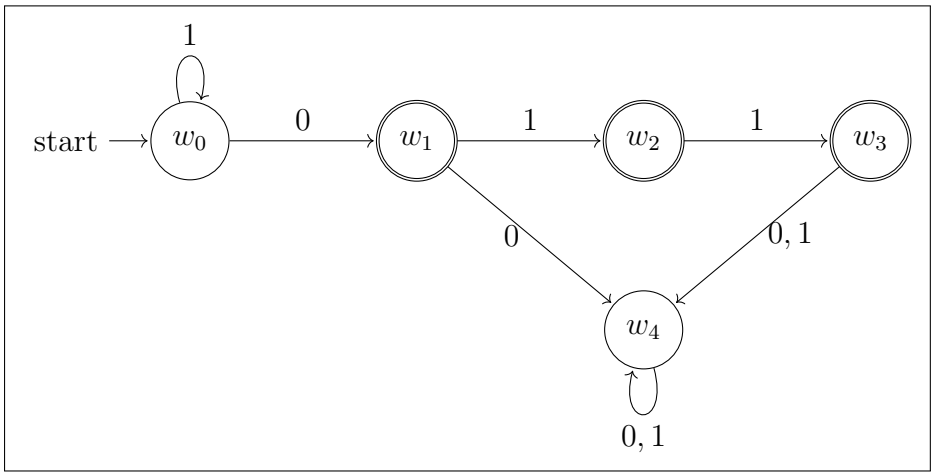
   A language is finite if there is no cycle on any path from the initial state to an accepting state.

   Example:

   DFA M_1 accepts $L = \{0, 01, 011\}$. Since $|L| = 3 < \infty$, the language is finite.

   Let $M_1 = (W, \Sigma, V, S, F)$ where:

   $$W = \{w_0, w_1, w_2, w_3, w_4\}, \quad \Sigma = \{0, 1\}, \quad S = w_0, \quad F = \{w_1, w_2, w_3\}.$$



   The language is finite because there are no cycles on any path from S to accepting states $\{w_1, w_2, w_3\}$.

2. **Infinite Language**

   Formally,

   $$L(M) \text{ is infinite} \leftrightarrow \exists w' \in W : S \xrightarrow{*} w' \xrightarrow{+} w' \xrightarrow{*} f,\ f \in F$$

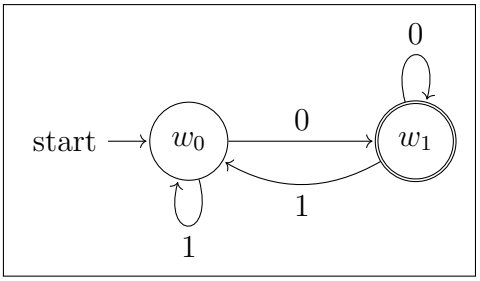
   A language is infinite if there exists a cycle on some path from the initial state to an accepting state.

   Example:

   DFA M_2 accepts $L = \{0^n \mid n \geq 1\}$, which is infinite.

   Let $M_2 = (W, \Sigma, V, S, F)$ where:

   $$W = \{w_0, w_1\}, \quad \Sigma = \{0, 1\}, \quad S = w_0, \quad F = \{w_1\}.$$



The language is infinite because state w_1 has a self-loop (cycle), is reachable from S, and is an accepting state, satisfying

$$S \xrightarrow{*} w_1 \xrightarrow{+} w_1.$$

10.1.4 Equivalence

Formally,

$$M_1 \equiv M_2 \leftrightarrow L(M_1) = L(M_2)$$

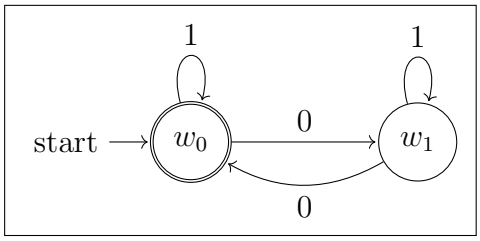
Two DFAs M_1 and M_2 are equivalent if and only if they accept exactly the same language, i.e., for all strings $x \in \Sigma^*$:

$$x \in L(M_1) \leftrightarrow x \in L(M_2)$$

Example:
Consider two DFAs that both accept strings with an even number of 0's over $\Sigma = \{0, 1\}$.
DFA M_1:
$M_1 = (W_1, \Sigma, V_1, S_1, F_1)$ where:

$$W_1 = \{w_0, w_1\}, \quad \Sigma = \{0, 1\}, \quad S_1 = w_0, \quad F_1 = \{w_0\}.$$

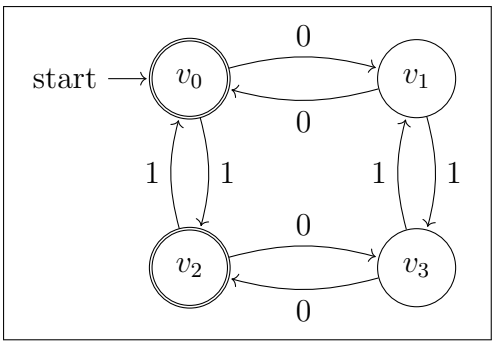
State	0	1
w_0	w_1	w_0
w_1	w_0	w_1



DFA M_2:
$M_2 = (W_2, \Sigma, V_2, S_2, F_2)$ where:

$$W_2 = \{v_0, v_1, v_2, v_3\}, \quad \Sigma = \{0, 1\}, \quad S_2 = v_0, \quad F_2 = \{v_0, v_2\}.$$

State	0	1
v_0	v_1	v_2
v_1	v_0	v_3
v_2	v_3	v_0
v_3	v_2	v_1



Let's test several strings to verify both DFAs accept the same language:

1. $x = \varepsilon$: M_1 ends at w_0, M_2 ends at v_0 (0 zeros, even)
2. $x = 00$: M_1 ends at w_0, M_2 ends at v_0 (2 zeros, even)
3. $x = 101$: M_1 ends at w_1, M_2 ends at v_3 (1 zero, odd)
4. $x = 0110$: M_1 ends at w_0, M_2 ends at v_2 (2 zeros, even)

Both DFAs accept exactly the same language:

$$L(M_1) = L(M_2) = \{x \in \{0,1\}^* \mid \text{number of 0's in } x \text{ is even}\}$$

Despite having different numbers of states and different structures, they recognize the same language. Therefore, $M_1 \equiv M_2$.

10.1.5 Subset

Formally,

$$L(M_1) \subseteq L(M_2) \;\leftrightarrow\; \forall x \in \Sigma^*,\; x \in L(M_1) \to x \in L(M_2)$$

Example:
Let

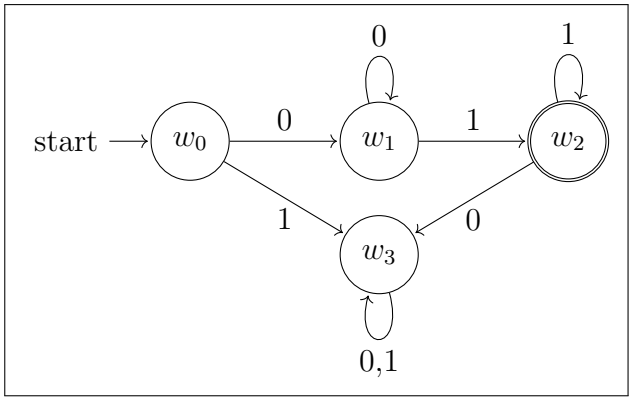
$$L(M_1) = \{0^n 1^m \mid n \geq 1,\; m \geq 1\}, \qquad L(M_2) = \{0^n 1^m \mid n \geq 0,\; m \geq 0\}.$$

DFA M_1:

$$M_1 = (W_1, \Sigma, V_1, S_1, F_1)$$

where

$$W_1 = \{w_0, w_1, w_2, w_3\}, \quad \Sigma = \{0,1\}, \quad S_1 = w_0, \quad F_1 = \{w_2\}.$$



This DFA accepts exactly the language

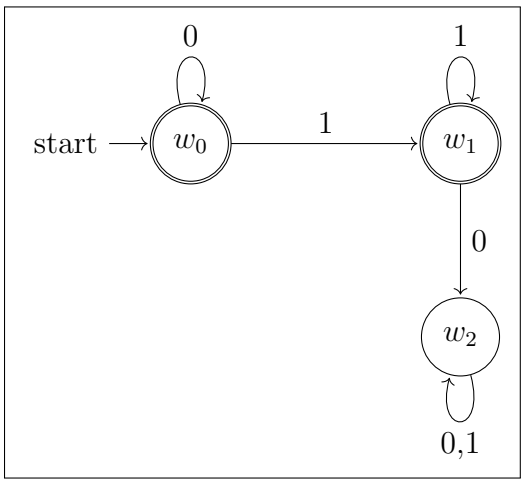
$$L(M_1) = \{0^n 1^m \mid n \geq 1,\ m \geq 1\}.$$

DFA M_2:

$$M_2 = (W_2, \Sigma, V_2, S_2, F_2)$$

where

$$W_2 = \{w_0, w_1, w_2\}, \quad \Sigma = \{0, 1\}, \quad S_2 = w_0, \quad F_2 = \{w_0, w_1\}.$$



This DFA accepts exactly

$$L(M_2) = \{0^n 1^m \mid n \geq 0,\ m \geq 0\} = 0^* 1^*.$$

Since every string with at least one 0 followed by at least one 1 is also a string of the form 0^*1^*, we conclude that

$$L(M_1) \subseteq L(M_2).$$

10.2 Closure Properties

10.2.1 Union

Formally,

$$L(M_1) \cup L(M_2) = \{x \in \Sigma^* \mid x \in L(M_1) \vee x \in L(M_2)\}$$

Given $M_1 = (W_1, \Sigma, V_1, S_1, F_1)$ and $M_2 = (W_2, \Sigma, V_2, S_2, F_2)$, the union is constructed as:

$$M = (W_1 \times W_2, \Sigma, V, (S_1, S_2), F)$$

where

$$F = \{(w_1, w_2) \mid w_1 \in F_1 \vee w_2 \in F_2\}$$

and

$$V((w_1, w_2), x) = (V_1(w_1, x), V_2(w_2, x))$$

Example:

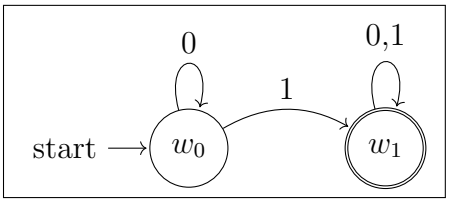
Let $L(M_1)$ be the set of strings over $\{0, 1\}$ containing at least one 1, and let $L(M_2)$ be the set of strings containing at least one 0.

The union language is:

$$L(M_1) \cup L(M_2) = \{x \in \{0, 1\}^* \mid x \text{ contains at least one 1 or one 0}\}.$$

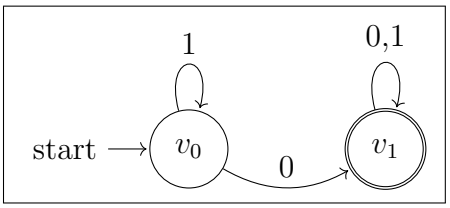
DFA M_1: Accepts strings with at least one 1.
$M_1 = (W_1, \Sigma, V_1, S_1, F_1)$ where

$$W_1 = \{w_0, w_1\}, \quad \Sigma = \{0, 1\}, \quad S_1 = w_0, \quad F_1 = \{w_1\}.$$



DFA M_2: Accepts strings with at least one 0.
$M_2 = (W_2, \Sigma, V_2, S_2, F_2)$ where

$$W_2 = \{v_0, v_1\}, \quad \Sigma = \{0, 1\}, \quad S_2 = v_0, \quad F_2 = \{v_1\}.$$



Union Construction:

$$M = M_1 \cup M_2 = (W_1 \times W_2,\ \Sigma,\ V,\ (w_0, v_0),\ F)$$

where

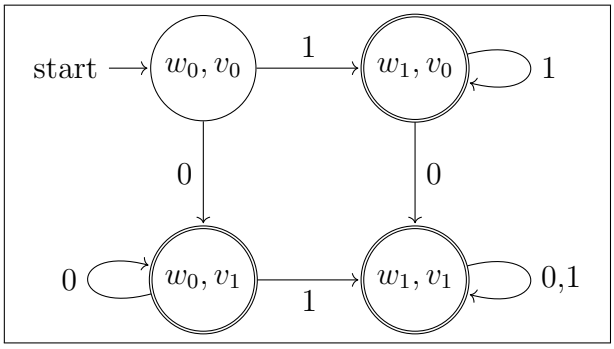
$$W_1 \times W_2 = \{(w_0, v_0), (w_0, v_1), (w_1, v_0), (w_1, v_1)\}$$

and

$$F = \{(x, y) \mid x \in F_1 \vee y \in F_2\} = \{(w_0, v_1), (w_1, v_0), (w_1, v_1)\}.$$

The transition function V is defined by:

$$V((w_i, v_j), a) = (V_1(w_i, a), V_2(v_j, a))$$



The product state (w_i, v_j) represents "M_1 in state w_i and M_2 in state v_j". A state is accepting if *either* component is in an accepting state (union semantics).

Thus,

$$L(M_1) \cup L(M_2) = L(M) = \{x \in \{0, 1\}^* \mid x \neq \varepsilon\}.$$

Verification:

1. ε: Neither M_1 nor M_2 accepts $\rightarrow$ rejected
2. 1: M_1 accepts $\rightarrow$ accepted
3. 0: M_2 accepts $\rightarrow$ accepted
4. 10: Both accept $\rightarrow$ accepted

10.2.2 Intersection

Formally,

$$L(M_1) \cap L(M_2) = \{x \in \Sigma^* \mid x \in L(M_1) \wedge x \in L(M_2)\}$$

Given $M_1 = (W_1, \Sigma, V_1, S_1, F_1)$ and $M_2 = (W_2, \Sigma, V_2, S_2, F_2)$, the intersection is constructed as:

$$M = (W_1 \times W_2, \Sigma, V, (S_1, S_2), F)$$

where

$$F = \{(w_1, w_2) \mid w_1 \in F_1 \wedge w_2 \in F_2\}$$

and

$$V((w_1, w_2), x) = (V_1(w_1, x), V_2(w_2, x))$$

Example:

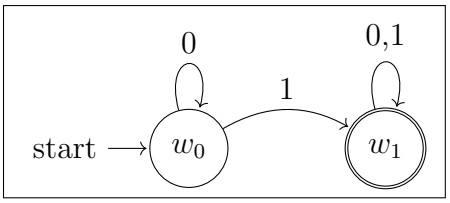
Let $L(M_1)$ be the set of strings over $\{0,1\}$ containing at least one 1, and let $L(M_2)$ be the set of strings containing at least one 0.

The intersection language is:

$$L(M_1) \cap L(M_2) = \{x \in \{0,1\}^* \mid x \text{ contains at least one 1 and at least one 0}\}.$$

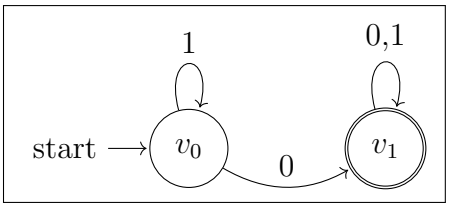
DFA M_1: Accepts strings with at least one 1.
$M_1 = (W_1, \Sigma, V_1, S_1, F_1)$ where

$$W_1 = \{w_0, w_1\}, \quad \Sigma = \{0,1\}, \quad S_1 = w_0, \quad F_1 = \{w_1\}.$$



DFA M_2: Accepts strings with at least one 0.
$M_2 = (W_2, \Sigma, V_2, S_2, F_2)$ where

$$W_2 = \{v_0, v_1\}, \quad \Sigma = \{0,1\}, \quad S_2 = v_0, \quad F_2 = \{v_1\}.$$



Intersection Construction:

$$M = M_1 \cap M_2 = (W_1 \times W_2,\ \Sigma,\ V,\ (w_0, v_0),\ F)$$

where

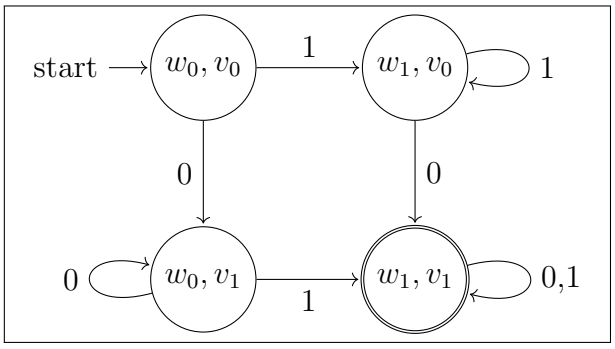
$$W_1 \times W_2 = \{(w_0, v_0), (w_0, v_1), (w_1, v_0), (w_1, v_1)\}$$

and

$$F = \{(x, y) \mid x \in F_1 \wedge y \in F_2\} = \{(w_1, v_1)\}.$$

The transition function V is defined by:

$$V((w_i, v_j), a) = (V_1(w_i, a), V_2(v_j, a))$$



The product state (w_i, v_j) represents "M_1 in state w_i and M_2 in state v_j". A state is accepting if *both* components are in accepting states (intersection semantics).

Thus,

$$L(M_1) \cap L(M_2) = L(M) = \{x \in \{0,1\}^* \mid x \text{ contains both 1 and 0}\}.$$

Verification:

1. ε: Neither contains 1 nor 0 $\rightarrow$ rejected
2. 1: Contains 1 but not 0 $\rightarrow$ rejected
3. 0: Contains 0 but not 1 $\rightarrow$ rejected
4. 10: Contains both 1 and 0 $\rightarrow$ accepted
5. 01: Contains both 1 and 0 $\rightarrow$ accepted
6. 1100: Contains both 1 and 0 $\rightarrow$ accepted

10.2.3 Complement

Formally,

$$L(M)^c = \Sigma^* \setminus L(M) = \{x \in \Sigma^* \mid x \notin L(M)\}$$

Given $M = (W, \Sigma, V, S, F)$, the complement is constructed as:

$$M^c = (W, \Sigma, V, S, F^c)$$

where

$$F^c = W \setminus F$$

The complement DFA has the same structure but with accepting and non-accepting states swapped.

Example:

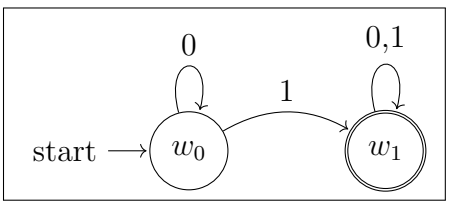
Let $L(M_1)$ be the set of strings over $\{0,1\}$ that contain at least one 1. The complement language is:

$$L(M_1)^c = \{x \in \{0,1\}^* \mid x \text{ contains no 1}\} = 0^*.$$

Original DFA M_1: Accepts strings with at least one 1.

$M_1 = (W, \Sigma, V, S, F)$ where

$$W = \{w_0, w_1\}, \quad \Sigma = \{0,1\}, \quad S = w_0, \quad F = \{w_1\}.$$

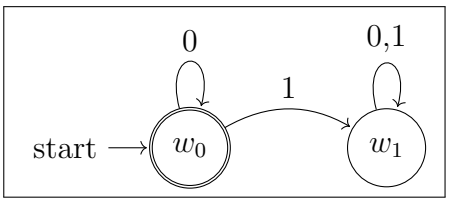


Complement Construction:

$$M_1^c = (W, \Sigma, V, S, F^c)$$

where

$$F^c = W \setminus F = \{w_0, w_1\} \setminus \{w_1\} = \{w_0\}.$$



The complement DFA accepts all strings that the original rejects, and vice versa. State w_0 (previously non-accepting) is now accepting, while state w_1 (previously accepting) is now non-accepting.

Thus,

$$L(M_1)^c = L(M_1^c) = 0^* = \{\varepsilon, 0, 00, 000, \ldots\}.$$

Verification:

1. ε: No 1 $\rightarrow$ accepted
2. 1: Contains 1 $\rightarrow$ rejected
3. 10: Contains 1 $\rightarrow$ rejected
4. 0: No 1 $\rightarrow$ accepted
5. 000: No 1 $\rightarrow$ accepted

10.2.4 Difference

Formally,

$$L(M_1) \setminus L(M_2) = \{x \in \Sigma^* \mid x \in L(M_1) \wedge x \notin L(M_2)\}$$

The difference is constructed using intersection with complement:

$$L(M_1) \setminus L(M_2) = L(M_1) \cap L(M_2)^c$$

Given $M_1 = (W_1, \Sigma, V_1, S_1, F_1)$ and $M_2 = (W_2, \Sigma, V_2, S_2, F_2)$:

1. First, construct $M_2^c = (W_2, \Sigma, V_2, S_2, F_2^c)$ where $F_2^c = W_2 \setminus F_2$
2. Then, construct the product: $M = (W_1 \times W_2, \Sigma, V, (S_1, S_2), F)$ where

$$F = \{(w_1, w_2) \mid w_1 \in F_1 \wedge w_2 \in F_2^c\}$$

3. The transition function $V : (W_1 \times W_2) \times \Sigma \to (W_1 \times W_2)$ is defined as:

$$V((w_i, v_j), x) = (V_1(w_i, x), V_2(v_j, x))$$

for all $(w_i, v_j) \in W_1 \times W_2$ and $x \in \Sigma$.

Example:

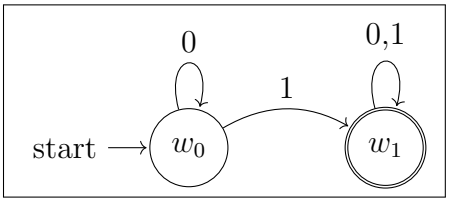
Let $L(M_1)$ be the set of strings over $\{0, 1\}$ that contain at least one 1, and let $L(M_2)$ be the set of strings that contain at least one 0.

The difference language is:

$$L(M_1) \setminus L(M_2) = \{x \in \{0,1\}^* \mid x \text{ contains at least one 1 and contains no 0}\} = 1^+.$$

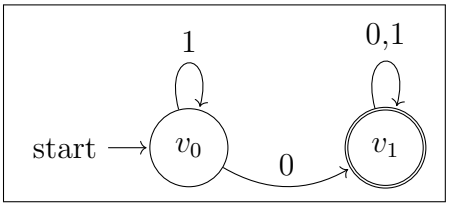
DFA M_1: Accepts strings with at least one 1.
$M_1 = (W_1, \Sigma, V_1, S_1, F_1)$ where

$$W_1 = \{w_0, w_1\}, \quad \Sigma = \{0, 1\}, \quad S_1 = w_0, \quad F_1 = \{w_1\}.$$



DFA M_2: Accepts strings with at least one 0.
$M_2 = (W_2, \Sigma, V_2, S_2, F_2)$ where

$$W_2 = \{v_0, v_1\}, \quad \Sigma = \{0, 1\}, \quad S_2 = v_0, \quad F_2 = \{v_1\}.$$



Difference Construction:
First, construct the complement of M_2:

$$M_2^c = (W_2, \Sigma, V_2, S_2, F_2^c) \quad \text{where} \quad F_2^c = W_2 \setminus F_2 = \{v_0, v_1\} \setminus \{v_1\} = \{v_0\}.$$

Then construct the product automaton:

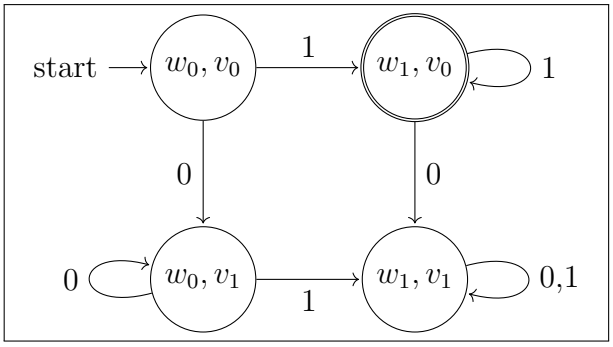
$$M = M_1 \setminus M_2 = (W_1 \times W_2,\ \Sigma,\ V,\ (w_0, v_0),\ F)$$

where

$$W = \{(w_0, v_0), (w_0, v_1), (w_1, v_0), (w_1, v_1)\}$$

and

$$F = \{(x, y) \mid x \in F_1 \wedge y \in F_2^c\} = \{(w_1, v_0)\}.$$



The product state (w_i, v_j) represents "M_1 in state w_i and M_2^c in state v_j". Only state (w_1, v_0) is accepting because it represents strings where M_1 accepts (contains 1) AND M_2^c accepts (contains no 0).

Thus,

$$L(M_1) \setminus L(M_2) = L(M) = 1^+ = \{1, 11, 111, \ldots\}.$$

Verification:

1. ε: No 1 $\rightarrow$ rejected
2. 1: Contains 1, no 0 $\rightarrow$ accepted
3. 0: Contains 0 $\rightarrow$ rejected
4. 10: Contains 0 $\rightarrow$ rejected
5. 111: Contains 1, no 0 $\rightarrow$ accepted

10.2.5 Concatenation

Formally,

$$L(M_1) \cdot L(M_2) = \{xy \mid x \in L(M_1) \wedge y \in L(M_2)\}$$

Given DFAs $M_1 = (W_1, \Sigma, V_1, S_1, F_1)$ and $M_2 = (W_2, \Sigma, V_2, S_2, F_2)$, concatenation requires constructing an NFA (with ε-transitions):

$$M = M_1 \cdot M_2 = (W_1 \cup W_2,\ \Sigma,\ V,\ S_1,\ F_2)$$

where the transition function V includes:

1. All transitions from M_1: $V(w, x) = V_1(w, x)$ for $w \in W_1$
2. All transitions from M_2: $V(w, x) = V_2(w, x)$ for $w \in W_2$
3. ε-transitions from every accepting state of M_1 to the start state of M_2:

$$V(f, \varepsilon) = S_2 \text{ for all } f \in F_1$$

Example:

Let $L(M_1)$ be the set of strings over $\{0,1\}$ that contain at least one 1, and let $L(M_2)$ be the set of strings over $\{0,1\}$ that contain at least one 0.

The concatenation language is:

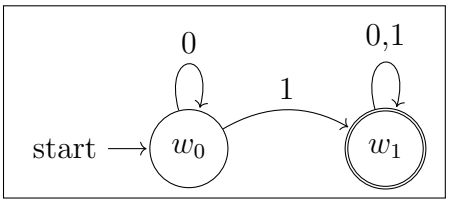
$$L(M_1) \cdot L(M_2) = \{uv \mid u \in L(M_1),\ v \in L(M_2)\}$$

This represents all strings that can be split into two parts: the first part contains at least one 1, and the second part contains at least one 0.

DFA M_1: Accepts strings with at least one 1.

$M_1 = (W_1, \Sigma, V_1, S_1, F_1)$ where

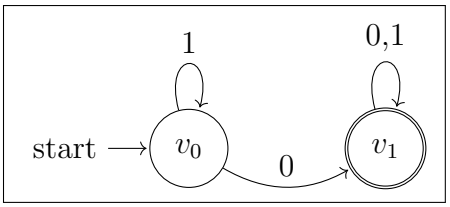
$$W_1 = \{w_0, w_1\}, \quad \Sigma = \{0,1\}, \quad S_1 = w_0, \quad F_1 = \{w_1\}.$$



DFA M_2: Accepts strings with at least one 0.

$M_2 = (W_2, \Sigma, V_2, S_2, F_2)$ where

$$W_2 = \{v_0, v_1\}, \quad \Sigma = \{0,1\}, \quad S_2 = v_0, \quad F_2 = \{v_1\}.$$

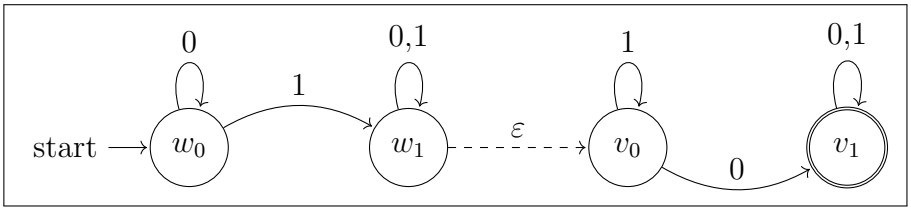


Concatenation Construction (NFA):

$$M = M_1 \cdot M_2 = (W_1 \cup W_2,\ \Sigma,\ V,\ w_0,\ F_2)$$

where $W_1 \cup W_2 = \{w_0, w_1, v_0, v_1\}$, start state is w_0, accepting states are $F = \{v_1\}$, and the ε-transition is:

$$w_1 \xrightarrow{\varepsilon} v_0$$



The ε-transition allows the NFA to non-deterministically "jump" from the accepting state of M_1 to the start state of M_2, effectively concatenating the two languages.

Thus,

$$L(M_1 \cdot M_2) = L(M_1)L(M_2).$$

Verification:

1. 10: First part $1 \in L(M_1)$, second part $0 \in L(M_2) \to$ accepted
2. 110: First part $11 \in L(M_1)$, second part $0 \in L(M_2) \to$ accepted
3. 100: First part $1 \in L(M_1)$, second part $00 \in L(M_2) \to$ accepted
4. 1100: Multiple valid splits $\to$ accepted
5. 1: No part with $0 \to$ rejected
6. 0: No part with $1 \to$ rejected

Unlike the previous closure properties (union, intersection, complement, difference), concatenation requires an NFA with ε-transitions. DFAs alone cannot directly construct concatenation, though the resulting NFA can be converted to a DFA using standard NFA-to-DFA conversion.

10.2.6 Kleene Star

Formally,

$$L(M)^* = \{w_1 w_2 \cdots w_k \mid k \geq 0,\ \forall i,\ w_i \in L(M)\}$$

Given a DFA $M = (W, \Sigma, V, S, F)$, the Kleene star requires constructing an NFA with ε-transitions:

$$M^* = (W \cup \{s\},\ \Sigma,\ V',\ s,\ F \cup \{s\})$$

where s is a new start state (also accepting to include ε), and the transition function V' includes:

1. All transitions from M: $V'(w, x) = V(w, x)$ for $w \in W$
2. ε-transition from the new start state to the original start state:
$$V'(s, \varepsilon) = S$$
3. ε-transitions from every accepting state back to the original start state:
$$V'(f, \varepsilon) = S \text{ for all } f \in F$$

Example:
Let $L(M_1)$ be the set of strings over $\{0, 1\}$ that contain at least one 1.
The Kleene star language is:

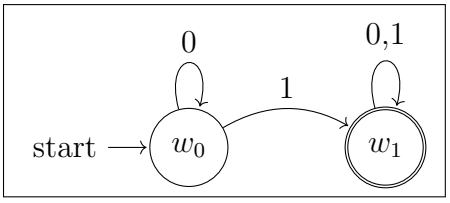
$$L(M_1)^* = \{w_1 w_2 \cdots w_k \mid k \geq 0,\ \forall i,\ w_i \in L(M_1)\}.$$

Since each w_i must contain at least one 1, any string in $L(M_1)^*$ is a concatenation of blocks each containing a 1.

$$L(M_1)^* = \{0, 1\}^* \setminus \{0^n \mid n \geq 1\}.$$

DFA M_1: Accepts strings with at least one 1.
$M_1 = (W_1, \Sigma, V_1, S_1, F_1)$ where

$$W_1 = \{w_0, w_1\}, \quad \Sigma = \{0, 1\}, \quad S_1 = w_0, \quad F_1 = \{w_1\}.$$

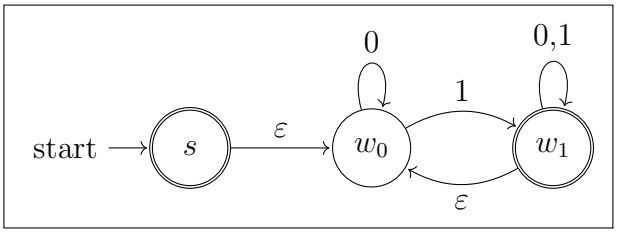


Kleene Star Construction (NFA):
To build M_1^*, we create a new start/accepting state s and add:

$$s \xrightarrow{\varepsilon} S_1, \qquad F_1 \xrightarrow{\varepsilon} S_1.$$

So the NFA is:

$$M_1^* = (W_1 \cup \{s\},\ \Sigma,\ V,\ s,\ F_1 \cup \{s\}).$$



The new state s is both initial and accepting (to accept ε). The ε-transition from s to w_0 allows entry into M_1, and the ε-transition from w_1 back to w_0 allows repeated concatenation of strings from $L(M_1)$.

Thus,

$$L(M_1)^* = L(M_1^*) = \{0,1\}^* \setminus \{0^n \mid n \geq 1\}.$$

Verification:

1. ε: Zero concatenations ($k = 0$) $\rightarrow$ accepted
2. 1: One block containing 1 $\rightarrow$ accepted
3. 11: One block 11 or two blocks $1 \cdot 1$ $\rightarrow$ accepted
4. 101: One block 101 or two blocks $1 \cdot 01$ $\rightarrow$ accepted
5. 0: Single 0 has no 1 $\rightarrow$ rejected
6. 00: String 00 cannot be split into blocks with 1 $\rightarrow$ rejected

Like concatenation, Kleene closure requires an NFA with ε-transitions. The resulting NFA can be converted to a DFA using standard NFA-to-DFA conversion.

10.2.7 Reversal

Formally,

$$L(M)^R = \{x^R \mid x \in L(M)\}$$

where x^R denotes the reverse of string x.
Given a DFA $M = (W, \Sigma, V, S, F)$, the reversal requires constructing an NFA:

$$M^R = (W,\ \Sigma,\ V^R,\ F,\ \{S\})$$

where the construction follows these steps:

1. Reverse all transitions: if $V(w_i, x) = w_j$ then $V^R(w_j, x) = w_i$
2. Make original final states into the new start states: start states are F
3. Make the original start state the only accepting state: $\{S\}$
4. If $|F| > 1$, add a new initial state with ε-transitions to all states in F

Example:
Let $L(M_1)$ be the set of strings over $\{0, 1\}$ that end with symbol 1.
The reversal language is:

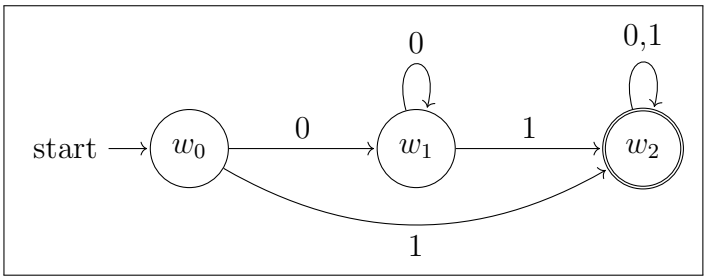
$$L(M_1)^R = \{x^R \mid x \in L(M_1)\}.$$

Since x ends with 1, the reversed string x^R must begin with 1.
Thus,

$$L(M_1)^R = \{1\{0, 1\}^*\}.$$

DFA M_1: Accepts strings ending with 1.
$M_1 = (W_1, \Sigma, V_1, S_1, F_1)$ where

$$W_1 = \{w_0, w_1, w_2\}, \quad \Sigma = \{0, 1\}, \quad S_1 = w_0, \quad F_1 = \{w_2\}.$$



Reversal Construction (NFA):
To construct M_1^R:

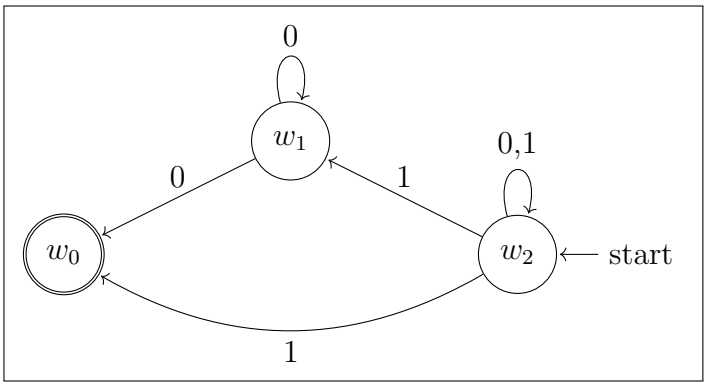
1. Reverse all transitions.
2. Make original final states into the new start states.
3. Make the original start state the only accepting state.

4. Because multiple ε-entries may arise, the construction yields an NFA.

Hence,

$$M_1^R = (W_1,\ \Sigma,\ V^R,\ F_1,\ \{S_1\}).$$

where V^R is the reverse of V_1.



The reversed automaton starts from the original accepting state w_2 and accepts at the original start state w_0. All transitions are reversed: edges that went from w_i to w_j on symbol a now go from w_j to w_i on symbol a.

Therefore,

$$L(M_1)^R = L(M_1^R) = \{1\{0,1\}^*\}.$$

Verification:

1. Original: $1 \in L(M_1)$ (ends with 1) $\rightarrow$ Reversed: $1 \in L(M_1)^R$ (begins with 1)
2. Original: $01 \in L(M_1) \rightarrow$ Reversed: $10 \in L(M_1)^R$
3. Original: $101 \in L(M_1) \rightarrow$ Reversed: $101 \in L(M_1)^R$
4. Original: $0 \notin L(M_1)$ (ends with 0) $\rightarrow$ Reversed: $0 \notin L(M_1)^R$ (begins with 0)
5. Original: $10 \notin L(M_1) \rightarrow$ Reversed: $01 \notin L(M_1)^R$

Like concatenation and Kleene closure, reversal may require an NFA construction, especially when the original DFA has multiple accepting states.

10.2.8 Homomorphism

Formally,

A homomorphism $h : \Sigma^* \rightarrow \Gamma^*$ is a function that maps strings over alphabet Σ to strings over alphabet Γ, defined by specifying $h(x)$ for each symbol $x \in \Sigma$ and extending to strings by concatenation:

$$h(\varepsilon) = \varepsilon, \quad h(xy) = h(x)h(y)$$

For a language $L \subseteq \Sigma^*$, the homomorphic image is:

$$h(L) = \{h(x) \mid x \in L\}$$

Given a DFA $M = (W, \Sigma, V, S, F)$ accepting L, we can construct a DFA M_h that accepts $h(L)$ by replacing each transition on symbol a with a path corresponding to $h(a)$.

Example:

Let $L(M_1)$ be the set of strings over $\{0, 1\}$ that contain at least one 1:

$$L(M_1) = \{x \in \{0, 1\}^* \mid x \text{ contains at least one } 1\}.$$

Define a homomorphism $h : \{0, 1\}^* \to \{0, 1\}^*$:

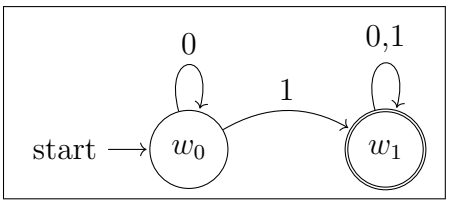
$$h(0) = 01, \quad h(1) = 1.$$

Then the homomorphic image of $L(M_1)$ is:

$$h(L(M_1)) = \{h(x) \mid x \in L(M_1)\}.$$

DFA M_1: Accepts strings with at least one 1.
$M_1 = (W_1, \Sigma, V_1, S_1, F_1)$ where

$$W_1 = \{w_0, w_1\}, \quad \Sigma = \{0, 1\}, \quad S_1 = w_0, \quad F_1 = \{w_1\}.$$



Homomorphism Construction:

To construct a DFA for $h(L(M_1))$, we replace each transition:

1. Transition on 0 becomes a path reading $h(0) = 01$
2. Transition on 1 becomes a path reading $h(1) = 1$

The resulting DFA M_h has additional intermediate states to read the multi-symbol strings $h(0)$ and $h(1)$.

Applying the Homomorphism:

The homomorphism h transforms each 0 into 01, and each 1 into 1. For example:

$$\begin{aligned} x &= 0101 \in L(M_1) \\ h(x) &= h(0) \cdot h(1) \cdot h(0) \cdot h(1) \\ &= 01 \cdot 1 \cdot 01 \cdot 1 \\ &= 011011 \in h(L(M_1)) \end{aligned}$$

Verification:

1. $x = 1 \in L(M_1) \to h(1) = 1 \in h(L(M_1))$
2. $x = 0 \notin L(M_1) \to h(0) = 01 \notin h(L(M_1))$
3. $x = 10 \in L(M_1) \to h(10) = h(1)h(0) = 1 \cdot 01 = 101 \in h(L(M_1))$

4. $x = 01 \in L(M_1) \to h(01) = h(0)h(1) = 01 \cdot 1 = 011 \in h(L(M_1))$
5. $x = 101 \in L(M_1) \to h(101) = 1 \cdot 01 \cdot 1 = 1011 \in h(L(M_1))$

Therefore,

$$h(L(M_1)) \text{ is regular.}$$

Regular languages are closed under homomorphism: if L is regular, then $h(L)$ is also regular for any homomorphism h.

10.2.9 Inverse Homomorphism

Formally,

Given a homomorphism $h : \Sigma^* \to \Gamma^*$ and a language $L \subseteq \Gamma^*$, the inverse homomorphic image is:

$$h^{-1}(L) = \{x \in \Sigma^* \mid h(x) \in L\}$$

Given a DFA $M = (W, \Gamma, V, S, F)$ accepting L, we can construct a DFA $M_{h^{-1}}$ that accepts $h^{-1}(L)$ by simulating M on $h(x)$ for each input symbol $x \in \Sigma$. The construction works as follows: for each state w in M and each symbol $x \in \Sigma$, we determine where M goes when reading the string $h(x)$, and use that as the transition in $M_{h^{-1}}$.

Example:

Let $L(M_2)$ be the set of strings over $\{0, 1\}$ that start with 1:

$$L(M_2) = \{x \in \{0, 1\}^* \mid x \text{ starts with } 1\}.$$

Define a homomorphism $h : \{0, 1\}^* \to \{0, 1\}^*$:

$$h(0) = 01, \quad h(1) = 1.$$

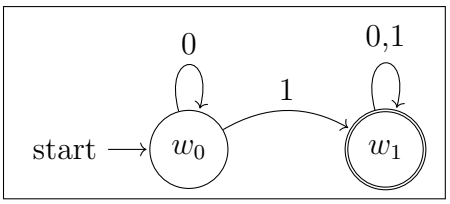
The inverse homomorphic image of $L(M_2)$ is:

$$h^{-1}(L(M_2)) = \{x \in \{0, 1\}^* \mid h(x) \in L(M_2)\}.$$

DFA M_2: Accepts strings starting with 1.

$M_2 = (W_2, \Sigma, V_2, S_2, F_2)$ where

$$W_2 = \{w_0, w_1\}, \quad \Sigma = \{0, 1\}, \quad S_2 = w_0, \quad F_2 = \{w_1\}.$$



Inverse Homomorphism Construction:

To construct a DFA for $h^{-1}(L(M_2))$, for each state w and input symbol x, we compute where M_2 goes when reading $h(x)$:

1. From w_0 on input 0: M_2 reads $h(0) = 01$, which goes $w_0 \xrightarrow{0} w_0 \xrightarrow{1} w_1$

2. From w_0 on input 1: M_2 reads $h(1) = 1$, which goes $w_0 \xrightarrow{1} w_1$

3. From w_1 on input 0: M_2 reads $h(0) = 01$, which goes $w_1 \xrightarrow{0} w_1 \xrightarrow{1} w_1$

4. From w_1 on input 1: M_2 reads $h(1) = 1$, which goes $w_1 \xrightarrow{1} w_1$

This gives us the transitions for $M_{h^{-1}}$.

Applying the Inverse Homomorphism:

The inverse homomorphism h^{-1} gives the set of strings x such that $h(x)$ starts with 1. For instance:

1. $x = 0 \to h(0) = 01$ (starts with 0) $\to 0 \notin h^{-1}(L(M_2))$

2. $x = 1 \to h(1) = 1$ (starts with 1) $\to 1 \in h^{-1}(L(M_2))$

3. $x = 00 \to h(00) = 01 \cdot 01 = 0101$ (starts with 0) $\to 00 \notin h^{-1}(L(M_2))$

4. $x = 10 \to h(10) = 1 \cdot 01 = 101$ (starts with 1) $\to 10 \in h^{-1}(L(M_2))$

5. $x = 01 \to h(01) = 01 \cdot 1 = 011$ (starts with 0) $\to 01 \notin h^{-1}(L(M_2))$

Verification:

Notice that $h^{-1}(L(M_2))$ accepts strings where:

1. The first symbol is 1 (since $h(1) = 1$ starts with 1), or

2. After any number of 0s, the first 1 appears (since reading 0s produces strings starting with 0, but eventually a 1 will make $h(x)$ start with 1)

Therefore,

$$h^{-1}(L(M_2)) = \{x \in \{0,1\}^* \mid x \text{ contains at least one } 1\}$$

which is regular.

Regular languages are closed under inverse homomorphism: if L is regular, then $h^{-1}(L)$ is also regular for any homomorphism h.

10.2.10 Substitution

Formally,

A substitution ψ is a function that maps each symbol $x \in \Sigma$ to a language $\psi(x) \subseteq \Gamma^*$. It extends to strings by:

$$\psi(\varepsilon) = \{\varepsilon\}, \quad \psi(wx) = \psi(w) \cdot \psi(x)$$

where $L_1 \cdot L_2 = \{xy \mid x \in L_1, y \in L_2\}$ is the concatenation of languages.

For a language $L \subseteq \Sigma^*$, the substitution is:

$$\psi(L) = \bigcup_{w \in L} \psi(w)$$

If L is regular and each $\psi(x)$ is regular, then $\psi(L)$ is also regular (regular languages are closed under substitution).

Example:
Let $L(M_3)$ be the set of strings over $\{0,1\}$ that contain exactly one 1:

$$L(M_3) = \{x \in \{0,1\}^* \mid x \text{ contains exactly one } 1\}.$$

Define a substitution $\psi : \{0,1\}^* \to \{0,1\}^*$:

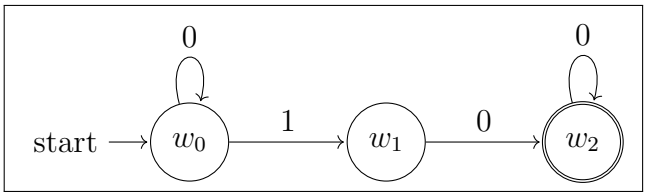
$$\psi(0) = \{0, 00\}, \quad \psi(1) = \{1, 11\}.$$

The substitution of $L(M_3)$ is:

$$\psi(L(M_3)) = \bigcup_{x \in L(M_3)} \psi(x),$$

where $\psi(x)$ replaces each symbol y in x by a string from $\psi(y)$.
DFA M_3: Accepts strings with exactly one 1.
$M_3 = (W_3, \Sigma, V_3, S_3, F_3)$ where

$$W_3 = \{w_0, w_1, w_2\}, \quad \Sigma = \{0,1\}, \quad S_3 = w_0, \quad F_3 = \{w_2\}.$$



Applying the Substitution:
The substitution allows each 0 to be replaced by 0 or 00, and each 1 by 1 or 11. For example:

1. $x = 1 \in L(M_3)$:
$$\psi(1) = \{1, 11\}$$
2. $x = 01 \in L(M_3)$:
$$\psi(01) = \psi(0) \cdot \psi(1) = \{0, 00\} \cdot \{1, 11\} = \{01, 011, 001, 0011\}$$
3. $x = 10 \in L(M_3)$:
$$\psi(10) = \psi(1) \cdot \psi(0) = \{1, 11\} \cdot \{0, 00\} = \{10, 100, 110, 1100\}$$
4. $x = 010 \in L(M_3)$:
$$\begin{aligned}\psi(010) &= \psi(0) \cdot \psi(1) \cdot \psi(0) = \{0, 00\} \cdot \{1, 11\} \cdot \{0, 00\} \\ &= \{010, 0100, 0110, 01100, 0010, 00100, 00110, 001100\}\end{aligned}$$

Therefore,

$$\psi(L(M_3)) = \bigcup_{w \in L(M_3)} \psi(w)$$

is the union of all possible ways to substitute symbols in strings from $L(M_3)$.
Since $L(M_3)$ is regular and both $\psi(0) = \{0, 00\}$ and $\psi(1) = \{1, 11\}$ are regular languages (both can be accepted by simple DFAs), we conclude:

$$\psi(L(M_3)) \text{ is regular.}$$

Regular languages are closed under substitution when each $\psi(a)$ is a regular language.

11 Pumping Lemma

For any regular language L, there exists a constant $p \geq 1$ (called the pumping length) such that for any string $\phi \in L$ with $|\phi| \geq p$, there exists a decomposition $\phi = xyz$ satisfying:

$$|y| > 0,\ |xy| \leq p,\ \forall i \geq 0,\ xy^i z \in L$$

Example:
Let

$$\aleph = \{0^n 1^n \mid n \geq 0\} \text{ is not regular}$$

Proof

Assume $\aleph$ is regular. Let p be the pumping length. Consider $\phi = 0^p 1^p \in \aleph$ with $|\phi| = 2p \geq p$. By the Pumping Lemma, $\phi = xyz$ where $|y| > 0$, $|xy| \leq p$, and $\forall i \geq 0, xy^i z \in \aleph$. Since $|xy| \leq p$, both x and y consist only of 0's. Let $y = 0^k$ where $k > 0$. Pump with $i = 2$: $xy^2z = 0^{p+k}1^p$. Since $k > 0$, we have $p + k > p$, so $xy^2z \notin \aleph$. Contradiction. Therefore, $\aleph$ is not regular.

QED

Intuitively:

Suppose $p = 7$. Consider $\phi = 0^7 1^7$ where $|xy| \leq 7$. There are three cases for where y can be located:

1. y is in the 0 part.

   Let $\phi = \underbrace{00}_{x}\underbrace{0000}_{y}\underbrace{01^7}_{z}$.

   Pumping with $i = 2$: $xy^2z = 0^{11}1^7 \notin \aleph$ since $11 \neq 7$.

2. y is in the 1 part.

   Let $\phi = \underbrace{0^7}_{x}\underbrace{11}_{y}\underbrace{1^5}_{z}$.

   Pumping with $i = 2$: $xy^2z = 0^7 1^9 \notin \aleph$ since $7 \neq 9$.

3. y spans both 0 and 1 parts.

   Let $\phi = \underbrace{0000}_{x}\underbrace{0001}_{y}\underbrace{1^6}_{z}$.

   Pumping with $i = 2$: $xy^2z = 0000000010011^6 \notin \aleph$ (wrong pattern for 0^n1^n).

Since $|xy| \leq p = 7$, all cases lead to contradiction. Therefore, $\aleph$ is not regular.

12 Regular Grammar

Formally

$$G = (N, T, S, R)$$

where:

1. N = a finite set of variables (nonterminal symbols),
2. T = a finite set of terminal symbols,
3. S = the start symbol ($S \in N$),
4. R = the set of production rules for terminals and nonterminals.

In general:
Let

$$G = (\{f, g, h\},\ \{0, 1\},\ f,\ \{f \to gh,\ g \to 0,\ h \to 1\})$$

where

$$\begin{aligned} N &= \{f, g, h\} & T &= \{0, 1\}, \\ S &= f & R &= \{\, f \to gh,\ g \to 0,\ h \to 1 \,\}. \end{aligned}$$

From this we can derive

$$\begin{aligned} f &\to gh \\ &\to 0h \\ &\to 01 \end{aligned}$$

Hence, we can coclude,

$$\boxed{\text{L(G)} = \{01\}}$$

13 Context-Free Grammar and Language

Formally, a context-free grammar is $G = (N, \Sigma, R, S)$ where

1. N is the set of nonterminals.
2. Σ is the set of terminals.
3. $S \in N$ is the start symbol.
4. $R \subseteq N \times (N \cup \Sigma)^*$ is the set of productions, each of the form $A \to \alpha$ with $A \in N$ and $\alpha \in (N \cup \Sigma)^*$.

To produce strings, a CFG uses the operations of concatenation and union.

1. Concatenation: If $A \to \alpha$ and $B \to \beta$, then writing $\alpha\beta$ denotes placing the two strings next to each other.
2. Union: A production such as $A \to xA \mid x$ means A may produce either xA or x.

Example:
Consider the grammar

$$G = (\{f, g, h\}, \{x, y\}, f,\ \{f \to gh,\ g \to xg \mid x,\ h \to yh \mid y\}).$$

This grammar generates strings consisting of one or more x's followed by one or more y's.

$$\begin{aligned} f &\to \underline{g}h && [f \to gh] \\ &\to xh && [g \to x] \\ &\to xy && [h \to y] \end{aligned}$$

$$\begin{aligned} f &\to g\underline{h} && [f \to gh] \\ &\to gy\underline{h} && [h \to yh] \\ &\to gyy && [h \to y] \\ &\to xyy && [g \to x] \end{aligned}$$

$$\begin{aligned} f &\to \underline{g}h && [f \to gh] \\ &\to x\underline{g}h && [g \to xg] \\ &\to xxh && [g \to x] \\ &\to xxy && [h \to y] \end{aligned}$$

$$\begin{aligned} f &\to \underline{g}h && [f \to gh] \\ &\to x\underline{g}h && [g \to xg] \\ &\to xx\underline{h} && [g \to x] \\ &\to xxy\underline{h} && [h \to yh] \\ &\to xxyy && [h \to y] \end{aligned}$$

Therefore,

$$\boxed{L(G) = \{xy, x^2y, xy^2, x^2y^2, \dots\}}$$

More compactly,

$$\boxed{L(G) = \{x^m y^n \mid m \geq 1,\ n \geq 1\}}.$$

13.1 Recursive

Formally, a nonterminal $A \in V$ is called *recursive* if there exists a derivation

$$A \to^+ \alpha A \beta$$

for some $\alpha, \beta \in (N \cup \Sigma)^*$.

1. **Example 1:**
   Let the grammar

$$G = (\{f\},\ \{0, 1\},\ f,\ \{\, f \to 01, \quad f \to 0f1 \,\})$$

   We can derive:

$$\begin{aligned} f &\to 0\underline{f}1 && [f \to 0f1] \\ &\to 0011 && [f \to 01] \end{aligned}$$

$$\begin{aligned} f &\to 0\underline{f}1 && [f \to 0f1] \\ &\to 00\underline{f}11 && [f \to 0f1] \\ &\to 000111 && [f \to 01] \end{aligned}$$

Thus the language is

$$\boxed{L(G) = \{\, 0^n 1^n \mid n \geq 1 \,\}}$$

2. **Example 2:**

   Let

$$G = (\{f, g\},\ \{0, 1\},\ f,\ \{\, f \to 0g1, \quad g \to 0g1, \quad g \to \varepsilon \,\})$$

   Then we can derive:

$$\begin{aligned} f &\to 0\underline{g}1 && [\text{by } f \to 0g1] \\ &\to 0\underline{0g1}1 && [\text{by } g \to 0g1] \\ &\to 0\underline{00g1}11 && [\text{by } g \to 0g1] \\ &\to 000\underline{g}111 && [\text{by } g \to \varepsilon] \\ &= 000111 \end{aligned}$$

$$\begin{aligned} f &\to 0\underline{g}1 && [\text{by } f \to 0g1] \\ &\to 01 && [\text{by } g \to \varepsilon] \\ &= 01 \end{aligned}$$

$$\begin{aligned} f &\to 0\underline{g}1 && [\text{by } f \to 0g1] \\ &\to 0\underline{0g1}1 && [\text{by } g \to 0g1] \\ &\to 00\underline{g}11 && [\text{by } g \to \varepsilon] \\ &= 0011 \end{aligned}$$

$$\begin{aligned} f &\to 0\underline{g}1 && [\text{by } f \to 0g1] \\ &\to 0\underline{0g1}1 && [\text{by } g \to 0g1] \\ &\to 00\underline{g1}11 && [\text{by } g \to 0g1] \\ &\to 0000\underline{g}1111 && [\text{by } g \to \varepsilon] \\ &= 00001111 \end{aligned}$$

   This grammar generates

$$\boxed{L(G) = \{0^n 1^n \mid n \geq 1\}}.$$

   Note that the grammar defines how to produce strings; the language is the complete set of strings producible from that grammar.

3. **Example 3:**

   Let

$$G = (\{f\},\ \{(,)\},\ f,\ \{\, f \to (f), \quad f \to ff, \quad f \to \varepsilon \,\})$$

   A sample derivation of ():

$$\begin{aligned} f &\to (\underline{f}) && [f \to (f)] \\ &\to ((\underline{f})) && [f \to (f)] \\ &\to (((\underline{f}))) && [f \to (f)] \\ &\to ((())) && [f \to \varepsilon] \end{aligned}$$

   A sample derivation of ()():

$$
\begin{aligned}
f &\to \underline{f}f && [f \to ff] \\
&\to (\underline{f})f && [f \to (f)] \\
&\to ()f && [f \to \varepsilon] \\
&\to ()(\underline{f}) && [f \to (f)] \\
&\to ()() && [f \to \varepsilon]
\end{aligned}
$$

A sample derivation of $((()()))$:

$$
\begin{aligned}
f &\to (\underline{f}) && [f \to (f)] \\
&\to ((\underline{f})) && [f \to (f)] \\
&\to ((\underline{f}f)) && [f \to ff] \\
&\to (((\,)\underline{f})) && [f \to (f)] \\
&\to ((()())) && [f \to (f),\ \text{then } f \to \varepsilon] \\
&\to ((()())) && [f \to \varepsilon]
\end{aligned}
$$

Therefore,

$$\boxed{L(G) = \text{all balanced parentheses strings}}.$$

4. **Example 4:**

   Let

$$G = (\{g\},\ \{+, *, (,), a\},\ g,\ \{\, g \to g + g, \quad g \to g * g, \quad g \to (g), \quad g \to a \,\})$$

   A sample derivation of $(a + a) * a$:

$$
\begin{aligned}
g &\to \underline{g} * g && [g \to g * g] \\
&\to (\underline{g}) * g && [g \to (g)] \\
&\to (\underline{g} + g) * g && [g \to g + g] \\
&\to (a + \underline{g}) * g && [g \to a] \\
&\to (a + a) * \underline{g} && [g \to a] \\
&\to (a + a) * a && [g \to a]
\end{aligned}
$$

   Hence,

$$\boxed{L(G) = \text{the set of all well-formed arithmetic expressions over } a}$$

   (with operations $+, *, ()$).

13.2 Right and Left Linear Grammar

1. Right Linear (nonterminal on the right):

$$g \to 1h \quad \text{(produces strings like } 111\ldots \text{ from left to right)}$$

Example:
Let

$$G_R = (\{f, g\},\ \{1, 0\},\ f,\ R_R)$$

with productions

$$R_R = \{\, f \to 1g,\ g \to 1g,\ g \to 0 \,\}.$$

This generates strings such as

$$f \to 1g \to 11g \to 111g \to 1110.$$

2. Left Linear (nonterminal on the left):

$$g \to h1 \quad \text{(produces strings like } \ldots 111 \text{ from right to left)}$$

Example:
Let

$$G_L = (\{f, g\},\ \{1, 0\},\ f,\ R_L)$$

with productions

$$R_L = \{\, f \to g1,\ g \to g1,\ g \to 0 \,\}.$$

This generates the same language, but the steps look reversed:

$$f \to g1 \to g11 \to g111 \to 0111.$$

13.3 Pumping Lemma for Contex-Free Languages

For any context-free language L, there exists a constant $p \geq 1$ such that for any string $\phi \in L$ with $|\phi| \geq p$, there exists a decomposition $\phi = uvxyz$ satisfying:

$$|vy| > 0,\ |vxy| \leq p,\ \forall i \geq 0,\ uv^ixy^iz \in L$$

Example:
Let

$$\aleph = \{f^n g^n h^n \mid n \geq 0\} \text{ is not context-free}$$

Proof

Assume $\aleph$ is context-free. Let p be the pumping length. Consider $\phi = f^p g^p h^p \in \aleph$ with $|\phi| = 3p \geq p$. By the Pumping Lemma, $\phi = uvxyz$ where $|vy| > 0$, $|vxy| \leq p$, and $\forall i \geq 0,\ uv^ixy^iz \in \aleph$. Since $|vxy| \leq p$ and ϕ contains p copies each of f, g, and h, the substring vxy can contain at most two distinct symbols. Therefore, v and y together can only pump at most two of the three symbols f, g, h. Pump with $i = 2$: uv^2xy^2z will have unequal counts of f's, g's, and h's. Thus $uv^2xy^2z \notin \aleph$. Contradiction. Therefore, $\aleph$ is not context-free.

QED

Intuitively:

Suppose $p = 4$. Consider $\phi = f^4g^4h^4$ where $|vxy| \leq 4$. There are several cases for where vxy can be located:

1. vxy is in the f part only.

   Let $\phi = \underbrace{}_{u}\underbrace{f}_{v}\underbrace{ff}_{x}\underbrace{f}_{y}\underbrace{g^4h^4}_{z}$.

   Pumping with $i = 2$: $uv^2xy^2z = f^6g^4h^4 \notin \aleph$ since $6 \neq 4 \neq 4$.

2. vxy spans the f and g parts.

   Let $\phi = \underbrace{ff}_{u}\underbrace{f}_{v}\underbrace{fg}_{x}\underbrace{g}_{y}\underbrace{g^2h^4}_{z}$.

   Pumping with $i = 2$: $uv^2xy^2z = f^5g^5h^4 \notin \aleph$ since $5 \neq 4$.

3. vxy is in the g part only.

   Let $\phi = \underbrace{f^4}_{u}\underbrace{g}_{v}\underbrace{gg}_{x}\underbrace{g}_{y}\underbrace{h^4}_{z}$.

   Pumping with $i = 2$: $uv^2xy^2z = f^4g^6h^4 \notin \aleph$ since $4 \neq 6 \neq 4$.

4. vxy spans the g and h parts.

   Let $\phi = \underbrace{f^4g^2}_{u}\underbrace{g}_{v}\underbrace{gh}_{x}\underbrace{h}_{y}\underbrace{h^2}_{z}$.

   Pumping with $i = 2$: $uv^2xy^2z = f^4g^5h^5 \notin \aleph$ since $4 \neq 5$.

5. vxy is in the h part only.

   Let $\phi = \underbrace{f^4g^4}_{u}\underbrace{h}_{v}\underbrace{hh}_{x}\underbrace{h}_{y}\underbrace{}_{z}$.

   Pumping with $i = 2$: $uv^2xy^2z = f^4g^4h^6 \notin \aleph$ since $4 \neq 6$.

Since $|vxy| \leq p = 4$, the substring vxy can span at most two of the three symbol types. In all cases, pumping causes unequal counts. Therefore, $\aleph$ is not context-free.

14 Context-Sensitive Grammar (CSG)

A *context-sensitive grammar* (CSG) is a 4-tuple $G = (V, \Sigma, R, S)$ where:

1. V is a finite set of variables (non-terminals),

2. Σ is a finite set of terminals, disjoint from V,
3. R is a finite set of production rules of the form $\alpha A\beta \to \alpha\gamma\beta$, where $A \in V$, $\alpha, \beta \in (V \cup \Sigma)^*$, and $\gamma \in (V \cup \Sigma)^+$,
4. $S \in V$ is the start variable.

Every rule $\alpha \to \beta \in R$ must satisfy $|\alpha| \leq |\beta|$ (non-contracting condition).
The language generated by G is:

$$L(G) = \{w \in \Sigma^* \mid S \xrightarrow{*} w\}$$

Example:
Consider the language

$$L = \{a^n b^n c^n \mid n \geq 1\}$$

Define $G = (V, \Sigma, R, S)$ where:

$$V = \{S, B, C\}, \qquad \Sigma = \{a, b, c\}, \qquad S \text{ is the start variable}$$

with production rules R:

$$\begin{aligned} S &\to aSBC \mid aBC \\ CB &\to BC \\ aB &\to ab \\ bB &\to bb \\ bC &\to bc \\ cC &\to cc \end{aligned}$$

Derivation of $aabbcc$:

$$\begin{aligned} S &\to aSBC \\ &\to a \cdot aBC \cdot BC \\ &\to aaBCBC \\ &\to aaBBCC \quad (CB \to BC) \\ &\to aabBCC \quad (aB \to ab) \\ &\to aabbCC \quad (bB \to bb) \\ &\to aabbcC \quad (bC \to bc) \\ &\to aabbcc \quad (cC \to cc) \end{aligned}$$

This language cannot be generated by any context-free grammar, since the pumping lemma for CFLs is violated. However, the CSG above generates it correctly by using context to ensure the counts of a, b, and c remain equal.

15 Derivation Tree

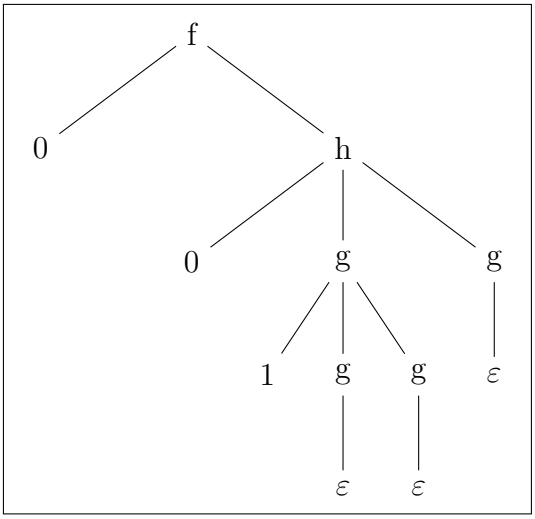
A Derivation Tree or Parse Tree is an ordered rooted tree that graphically represents the semantic information of strings derived from a Context-free Grammar.

Example:

Given the grammar:

$$G_T = \{f \to 0h, g \to 1gg \mid \varepsilon, h \to 0gg\}$$

Derivation tree for the string "001":



15.1 Right and Left Derivation

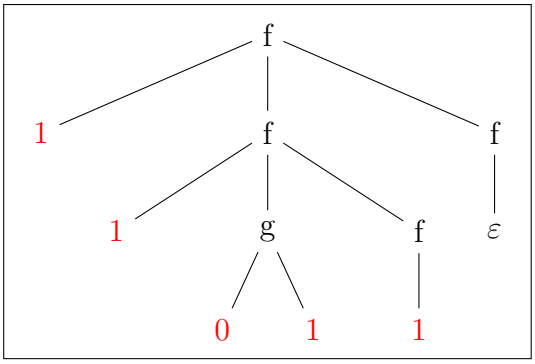
Example:

Given the grammar:

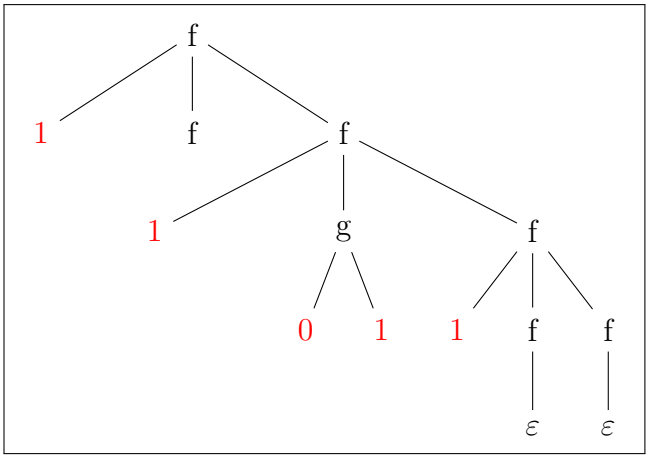
$$G_T = \{f \to 1gf \mid 1ff \mid \varepsilon, g \to f0g \mid 01\}$$

Generate the string "11011":

1. Left derivation



2. Right derivation:



15.2 Ambiguous Grammar

A grammar is said to be ambiguous if there exist two or more derivation tree for a string

Example:

Let

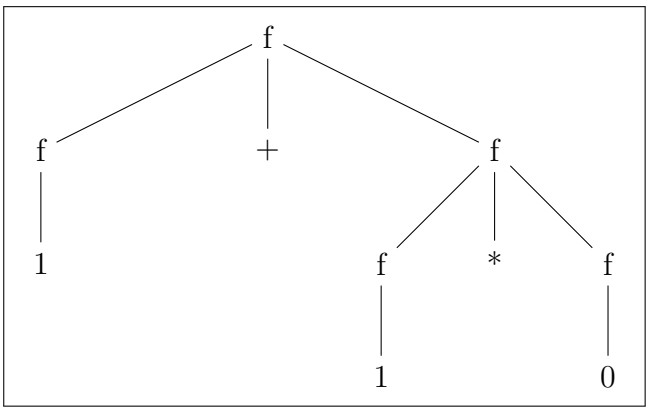
$$G = (\{f\}, \{1, 0, +, *\}, f, \{f \to f + f \mid f * f \mid 1 \mid 0\})$$

This grammar is ambiguous because we can derive the string $1+(1*0)$ in two different ways:

1. Derivation 1 (Addition at root)

$$\begin{aligned} f &\to f + f \\ &\to 1 + f \\ &\to 1 + f * f \\ &\to 1 + 1 * f \\ &\to 1 + 1 * 0 \end{aligned}$$

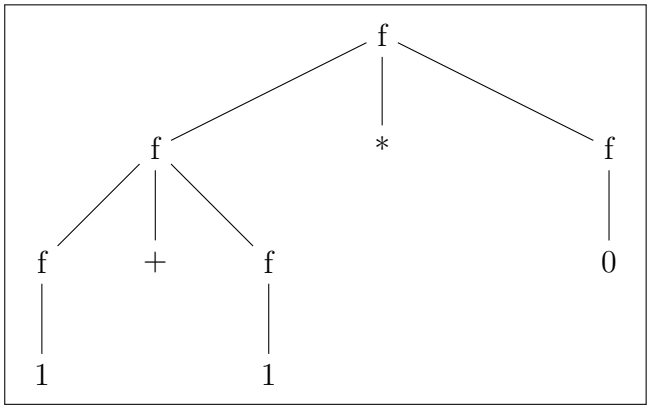
This corresponds to the parse tree: $1 + (1 * 0)$ where multiplication has higher precedence.



2. Derivation 2 (Multiplication at root)

$$\begin{aligned} f &\to f * f \\ &\to f + f * f \\ &\to 1 + f * f \\ &\to 1 + 1 * f \\ &\to 1 + 1 * 0 \end{aligned}$$

This corresponds to the parse tree: $(1+1)*0$ where addition has higher precedence.



Since the same string $1+(1*0)$ can be derived with two different parse trees (different structural interpretations), the grammar is ambiguous. This ambiguity arises because the grammar doesn't specify operator precedence or associativity rules.

16 Breadth-First Search (BFS)

Breadth-First Search is a graph and tree traversal algorithm that explores nodes level by level, visiting all nodes at the current depth before proceeding to the next. It uses a queue (First-In First-Out) to determine the order in which nodes are visited.

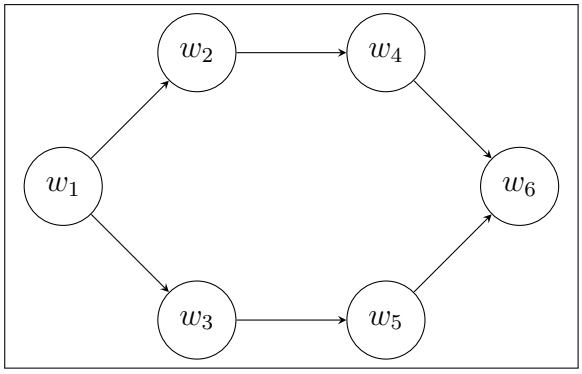
Formally,given a graph $G = (V, E)$ and a source node $s \in V$, BFS computes the shortest path from s to every reachable vertex. It maintains a queue Q and a visited set $\mathcal{V}$, and proceeds as follows:

1. Initialize $Q \leftarrow \{s\}$ and $\mathcal{V} \leftarrow \{s\}$.
2. While $Q \neq \varnothing$:
   (a) Dequeue the front node u from Q.
   (b) For each neighbor $v \in \mathrm{Adj}(u)$:
      i. If $v \notin \mathcal{V}$, add v to $\mathcal{V}$ and enqueue v.

The BFS tree rooted at s consists of all edges (u, v) traversed during the search. For any vertex v reachable from s, the path in the BFS tree is the shortest path in G.

Example:

Consider the following graph with source node w_1:



Starting from w_1, BFS proceeds as follows:

Step	Dequeued	Queue after step
1	w_1	$w_2,\ w_3$
2	w_2	$w_3,\ w_4$
3	w_3	$w_4,\ w_5$
4	w_4	$w_5,\ w_6$
5	w_5	w_6
6	w_6	$\varnothing$

The BFS traversal order is therefore:

$$w_1 \rightarrow w_2 \rightarrow w_3 \rightarrow w_4 \rightarrow w_5 \rightarrow w_6$$

All nodes at depth 1 (w_2, w_3) are visited before any node at depth 2 (w_4, w_5), which are visited before depth 3 (w_6). This level-by-level guarantee ensures that BFS always finds the shallowest solution first.

17 Depth-First Search (DFS)

Depth-First Search is a graph and tree traversal algorithm that explores nodes by going as deep as possible along each branch before backtracking. It uses a stack (Last-In First-Out), or equivalently, recursion to determine the order in which nodes are visited.

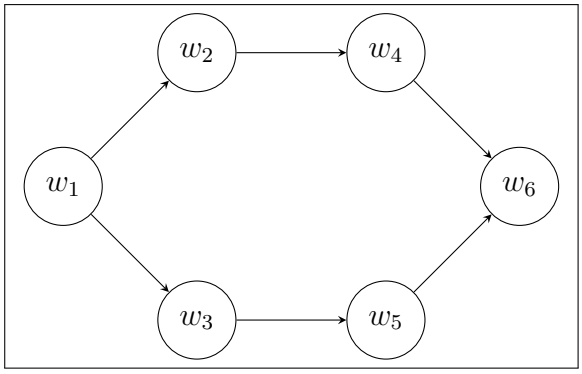
Formally, given a graph $G = (V, E)$ and a source node $s \in V$, DFS explores all vertices reachable from s. It maintains a stack S and a visited set $\mathcal{V}$, and proceeds as follows:

1. Initialize $S \leftarrow \{s\}$ and $\mathcal{V} \leftarrow \{s\}$.
2. While $S \neq \varnothing$:
   (a) Pop the top node u from S.
   (b) For each neighbor $v \in \mathrm{Adj}(u)$:
      i. If $v \notin \mathcal{V}$, add v to $\mathcal{V}$ and push v onto S.

The DFS tree rooted at s consists of all edges (u, v) traversed during the search. Edges in G are classified with respect to the DFS tree as *tree edges*, *back edges*, *forward edges*, or *cross edges*. A graph G is acyclic if and only if DFS produces no back edges.

Example:

Consider the following graph with source node w_1:



Starting from w_1, DFS proceeds as follows:

Step	Popped	Stack after step
1	w_1	$w_2,\ w_3$
2	w_3	$w_2,\ w_5$
3	w_5	$w_2,\ w_6$
4	w_6	w_2
5	w_2	w_4
6	w_4	$\varnothing$

The DFS traversal order is therefore:

$$w_1 \ \to\ w_3 \ \to\ w_5 \ \to\ w_6 \ \to\ w_2 \ \to\ w_4$$

DFS dives immediately from w_1 down the branch $w_3 \to w_5 \to w_6$ before backtracking to explore $w_2 \to w_4$. This depth-first guarantee means DFS will always fully explore one branch before moving to the next, in contrast to the level-by-level exploration of BFS.

18 Simplification of CFG

18.1 Reduction of CFG

A symbol $\psi \in (V \cup T)$ is useful if:

1. ψ is generating: $\psi \to^* w$ where $w \in T^*$
2. ψ is reachable: $S \to^* X\psi Y$ where $X, Y \in (V \cup T)^*$

Algorithm:

1. Eliminate non-generating symbols
2. Eliminate unreachable symbols

Example:
Consider the context-free grammar with production set

$$P = \{\, S \to XY \mid Z,\ X \to 1,\ Y \to 0 \mid ZY,\ E \to 1X \mid e \,\}.$$

1. Step 1: Eliminate non-generating symbols

   A nonterminal W is *generating* if $W \to^* w$ for some $w \in T^*$.

(a) X is generating since $X \to 1$.

(b) Y is generating since $Y \to 0$.

(c) E is generating since $E \to e$.

(d) S is generating since $S \to XY \to 10$.

(e) Z is non-generating, as it has no productions.

Thus, the set of generating nonterminals is $\{S, X, Y, E\}$. All productions containing the non-generating symbol Z are removed, yielding:

$$P_1 = \{\, S \to XY,\ X \to 1,\ Y \to 0,\ E \to 1X \mid e \,\}.$$

2. Step 2: Eliminate unreachable symbols

   Starting from the start symbol S:

   (a) The reachable symbols are S, X, Y.

   (b) The symbol E is unreachable from S.

   Removing all unreachable symbols and their productions gives:

$$P_2 = \{\, S \to XY,\ X \to 1,\ Y \to 0 \,\}.$$

Hence, the reduced grammar is:

$$\boxed{G' = (\{S, X, Y\}, \{1, 0\}, S, \{S \to XY,\ X \to 1,\ Y \to 0\})}$$

18.2 Removal Unit of Production

A production $\psi \to \chi$ is a unit production if $\psi, \chi \in V$ (both are variables).

Algorithm:

1. Find all unit productions

2. For each unit production $\psi \to \chi$, replace it with $\psi \to 1$ for all non-unit productions $\chi \to 1$

3. Remove all unit productions

Example:

Given

$$P = \{S \to XY, X \to 1, Y \to Z \mid 0, Z \to L, L \to T, T \to 1\}$$

1. Step 1: Identify unit productions

$$Y \to Z, \quad Z \to L, \quad L \to T$$

2. Step 2: Find unit closure for each variable

(a) Since $T \to 1$, we add $L \to 1$
(b) Since $L \to 1$, we add $Z \to 1$
(c) Since $Z \to 1$, we add $Y \to 1$

Then

$$P = \{S \to XY, X \to 1, Y \to 1 \mid 0, Z \to 1, L \to 1, T \to 1\}$$

Furthermore, we remove the unreachable symbols, that is L, T, and Z.

Therefore, the reduced grammar is:

$$\boxed{G' = (\{S, X, Y\}, \{1, 0\}, S, \{S \to XY, X \to 1, Y \to 1 \mid 0\})}$$

18.3 Removal of Null Productions

A production $\psi \to \varepsilon$ is called a *null production* (or ε-production).

Algorithm:

1. Identify all nullable variables (variables that can derive ε).
2. For each production, generate all possible productions obtained by deleting nullable variables in every possible combination (excluding the empty right-hand side unless the start symbol is nullable).
3. Remove all null productions.

Example

Consider the grammar with production set

$$P = \{\, S \to XYXZ,\ X \to 1X \mid \varepsilon,\ Y \to xY \mid \varepsilon,\ Z \to 0 \,\}.$$

1. Step 1: Identify nullable variables

   (a) $X \to \varepsilon$, hence X is nullable.
   (b) $Y \to \varepsilon$, hence Y is nullable.

   Thus, the set of nullable variables is $\{X, Y\}$.

2. Step 2: Generate new productions

   We consider all subsets of nullable variables $\{X, Y\}$ in each production.

   From $S \to XYXZ$, the possible right-hand sides are:

$$S \to XYXZ \mid YXZ \mid XXZ \mid XZ \mid YZ \mid Z$$

   From $X \to 1X$, removing the nullable variable X yields:

$$X \to 1X \mid 1$$

   From $Y \to xY$, removing the nullable variable Y yields:

$$Y \to xY \mid x$$

3. Step 3: Remove all null productions

   After removing the productions $X \to \varepsilon$ and $Y \to \varepsilon$, the resulting grammar has production set:

$$\boxed{\begin{gathered} P' = \{\, S \to XYXZ \mid YXZ \mid XXZ \mid XZ \mid YZ \mid Z, \\ X \to 1X \mid 1,\ Y \to xY \mid x,\ Z \to 0 \,\} \end{gathered}}$$

19 Chomsky Normal Form (CNF)

A CFG is in CNF if every production rule has one of these forms:

1. $\psi \to \phi\gamma$, where $\psi, \phi, \gamma \in V$ (non-terminals)
2. $\psi \to 1$, where $\psi \in V$ and $1 \in \Sigma$ (terminal)
3. $S \to \varepsilon$, where S is the start symbol and S does not appear on the right-hand side of any production

Example:
Consider the following grammar in CNF:

$$G = (V, \Sigma, R, S)$$

where:

$$V = \{f, g, h, j\}, \quad \Sigma = \{1, 0\}$$

Production Rules:

$$\begin{aligned}
f &\to gh, && \text{(Rule 1: two non-terminals)} \\
f &\to hj, && \text{(Rule 1: two non-terminals)} \\
g &\to 1, && \text{(Rule 2: single terminal)} \\
h &\to 0, && \text{(Rule 2: single terminal)} \\
j &\to fg, && \text{(Rule 1: two non-terminals)} \\
j &\to 1, && \text{(Rule 2: single terminal)}
\end{aligned}$$

19.1 CFG to CNF

There are 5 steps involved in the conversion of CFG to Chomsky Normal Form:

1. If the start symbol ψ occurs on the right-hand side of any production rule, create a new start symbol ψ' and add a new production rule $\psi' \to \psi$.
2. Remove null productions.
3. Remove unit productions.

4. Replace each production $\chi \to \eta_1\eta_2\ldots\eta_n$ where $n > 2$ with:

$$\begin{aligned}
\chi &\to \eta_1 L_1 \\
L_1 &\to \eta_2 L_2 \\
L_2 &\to \eta_3 L_3 \\
&\vdots \\
L_{n-2} &\to \eta_{n-1}\eta_n
\end{aligned}$$

   where $L_1, L_2, \ldots, L_{n-2}$ are new non-terminals.

5. Replace each production $\chi \to x$ where x contains terminals mixed with non-terminals with:

   (a) For each terminal a in x, create a new non-terminal T_a and production $T_a \to a$

   (b) Replace a with T_a in the original production

Example:
Given

$$P = \{f \to \chi f \chi \mid 1g,\ \chi \to g \mid f,\ g \to 0 \mid \varepsilon\}$$

1. Step 1: Since f is the start symbol and appears in RHS of $f \to \chi f \chi$, we add a new start symbol f' and add $f' \to f$

   Hence,

$$P = \{f' \to f,\ f \to \chi f \chi \mid 1g,\ \chi \to g \mid f,\ g \to 0 \mid \varepsilon\}$$

2. Step 2: Remove the null productions

   (a) g is nullable (has production $g \to \varepsilon$)

   (b) χ is nullable (has production $\chi \to g$, and g is nullable)

   (c) After removing $g \to \varepsilon$: for every production containing g, add a copy with g removed:

$$f' \to f,\ f \to \chi f \chi \mid 1g \mid 1,\ \chi \to g \mid f \mid \varepsilon,\ g \to 0$$

   (d) After removing $\chi \to \varepsilon$: for every production containing χ, add a copy with χ removed:

$$f' \to f,\ f \to \chi f \chi \mid f\chi \mid \chi f \mid f \mid 1g \mid 1,\ \chi \to g \mid f,\ g \to 0$$

3. Step 3: Remove unit productions: $f' \to f$, $f \to f$, $\chi \to g$, and $\chi \to f$

   (a) For $f' \to f$: Replace with all non-unit productions of f:

$$f' \to \chi f \chi \mid f\chi \mid \chi f \mid 1g \mid 1$$

(b) For $f \to f$: Remove it (self-loop unit production)

(c) For $\chi \to g$: Replace with all non-unit productions of g:

$$\chi \to 0$$

(d) For $\chi \to f$: Replace with all non-unit productions of f:

$$\chi \to \chi f \chi \mid f\chi \mid \chi f \mid 1g \mid 1$$

After removing all unit productions:

$$\begin{aligned} f' &\to \chi f \chi \mid f\chi \mid \chi f \mid 1g \mid 1 \\ f &\to \chi f \chi \mid f\chi \mid \chi f \mid 1g \mid 1 \\ \chi &\to 0 \mid \chi f \chi \mid f\chi \mid \chi f \mid 1g \mid 1 \\ g &\to 0 \end{aligned}$$

4. Step 4: Replace productions with more than 2 symbols on RHS

   For $f' \to \chi f \chi$, $f \to \chi f \chi$, and $\chi \to \chi f \chi$: introduce $L_1 \to f\chi$, so $\chi f \chi$ becomes χL_1:

$$\begin{aligned} f' &\to \chi L_1 \\ f &\to \chi L_1 \\ \chi &\to \chi L_1 \\ L_1 &\to f\chi \end{aligned}$$

   After replacing all such productions:

$$\begin{aligned} f' &\to \chi L_1 \mid f\chi \mid \chi f \mid 1g \mid 1 \\ f &\to \chi L_1 \mid f\chi \mid \chi f \mid 1g \mid 1 \\ \chi &\to 0 \mid \chi L_1 \mid f\chi \mid \chi f \mid 1g \mid 1 \\ g &\to 0 \\ L_1 &\to f\chi \end{aligned}$$

5. Step 5: Replace terminals mixed with non-terminals in productions

   The productions $f' \to 1g$, $f \to 1g$, and $\chi \to 1g$ contain terminal 1 mixed with non-terminal g. Introduce $T_1 \to 1$ and replace 1 with T_1 in these productions. Note that standalone terminal productions such as $f' \to 1$, $\chi \to 0$, and $g \to 0$ are already valid CNF and do not need to be changed.

   Therefore, the final CNF:

$$\boxed{\begin{aligned} f' &\to \chi L_1 \mid f\chi \mid \chi f \mid T_1 g \mid 1 \\ f &\to \chi L_1 \mid f\chi \mid \chi f \mid T_1 g \mid 1 \\ \chi &\to 0 \mid \chi L_1 \mid f\chi \mid \chi f \mid T_1 g \mid 1 \\ g &\to 0 \\ L_1 &\to f\chi \\ T_1 &\to 1 \end{aligned}}$$

A CFG is in CNF if every production rule has one of these forms:

1. $\psi \to \phi\gamma$, where $\psi, \phi, \gamma \in V$ (non-terminals)
2. $\psi \to 1$, where $\psi \in V$ and $1 \in \Sigma$ (terminal)
3. $S \to \varepsilon$, where S is the start symbol and S does not appear on the right-hand side of any production

Example:
Consider the following grammar in CNF:

$$G = (V, \Sigma, R, S)$$

where:

$$V = \{f, g, h, j\}, \quad \Sigma = \{1, 0\}$$

Production Rules:

$$\begin{aligned}
&f \to gh, && \text{(Rule 1: two non-terminals)}\\
&f \to hj, && \text{(Rule 1: two non-terminals)}\\
&g \to 1, && \text{(Rule 2: single terminal)}\\
&h \to 0, && \text{(Rule 2: single terminal)}\\
&j \to fg, && \text{(Rule 1: two non-terminals)}\\
&j \to 1, && \text{(Rule 2: single terminal)}
\end{aligned}$$

20 Greibach Normal Form (GNF)

A CFG is in GNF if the productions are in the following forms:

$$\begin{aligned}
&\psi \to 1\\
&\psi \to 1\phi_1\phi_2 \ldots \phi_n
\end{aligned}$$

where:

1. $\psi, \phi_1, \phi_2, \ldots, \phi_n$ are non-terminal symbols
2. 1 is a terminal symbol
3. $n \geq 1$ (one or more non-terminals may follow the terminal)

Additionally, the start symbol S may have the production $S \to \varepsilon$ if ε is in the language.
In General:
Consider the following CFG:

$$\begin{aligned}
&f \to gh \mid 1\\
&g \to 1g \mid 0\\
&h \to 0
\end{aligned}$$

This grammar is not in GNF because:

1. $f \to gh$ starts with a non-terminal (not a terminal)
2. $h \to 0$ is acceptable (starts with terminal 0)

The same language can be represented in GNF as:

$$\begin{aligned} f &\to 1gh \mid 1h \mid 0h \mid 1 \\ g &\to 1g \mid 1 \mid 0 \\ h &\to 0 \end{aligned}$$

20.1 CFG to GNF

There are 3 major steps involved in the conversion of CFG to Greibach Normal Form:

1. Remove Unit and Null Productions.
2. Convert to Chomsky Normal Form (CNF).
3. Convert CNF to GNF:

   a) Order the productions:
   Alter the rules so that the non-terminals are in ascending order. For any production of the form:

   $$\psi_i \to \psi_j T$$

   where $T \in (V \cup \Sigma)^*$, we must ensure that:

   $$i < j$$

   If $i \geq j$: Replace ψ_j with its production rules to eliminate the violation.
   If $\psi_j \to L_1 \mid L_2 \mid \cdots \mid L_k$, then substitute:

   $$\psi_i \to L_1 T \mid L_2 T \mid \cdots \mid L_k T$$

   b) Remove left recursion:
   If after substitution, we have direct left recursion of the form:

   $$\psi_i \to \psi_i T_1 \mid \psi_i T_2 \mid \cdots \mid \psi_i T_r \mid L_1 \mid L_2 \mid \cdots \mid L_s$$

   where L_j does not start with ψ_i, then eliminate it using:

   $$\begin{aligned} \psi_i &\to L_1 \mid L_2 \mid \cdots \mid L_s \mid L_1\psi_{n+1} \mid L_2\psi_{n+1} \mid \cdots \mid L_s\psi_{n+1} \\ \psi_{n+1} &\to T_1 \mid T_2 \mid \cdots \mid T_r \mid T_1\psi_{n+1} \mid T_2\psi_{n+1} \mid \cdots \mid T_r\psi_{n+1} \end{aligned}$$

   where ψ_{n+1} is a new non-terminal.

c) Convert to terminal-leading form:
For each production $\psi_i \to \psi_j T$ where $i < j$, substitute ψ_j with its productions until all productions are of the form:

$$\psi_i \to 1\gamma$$

where $1 \in \Sigma$ (terminal) and $\gamma \in V^*$ (string of non-terminals).

Example:
Given

$$\begin{aligned}
&S \to CA \mid BB \\
&B \to 0 \mid SB \\
&C \to 0 \\
&A \to 1
\end{aligned}$$

1. Step 1: Rename non-terminals to χ_i form

   We change the non-terminals into a numbered sequence: $S \to \chi_1$, $C \to \chi_2$, $A \to \chi_3$, $B \to \chi_4$

$$\begin{aligned}
&\chi_1 \to \chi_2\chi_3 \mid \chi_4\chi_4 \\
&\chi_2 \to 0 \\
&\chi_3 \to 1 \\
&\chi_4 \to 0 \mid \chi_1\chi_4
\end{aligned}$$

2. Step 2: Check ordering rule ($i < j$)

   Analyze each production $\chi_i \to \chi_j\alpha$:

   (a) $\chi_1 \to \chi_2\chi_3$: $1 < 2$
   (b) $\chi_1 \to \chi_4\chi_4$: $1 < 4$
   (c) $\chi_2 \to 0$: terminal
   (d) $\chi_3 \to 1$: terminal
   (e) $\chi_4 \to 0$: terminal
   (f) $\chi_4 \to \chi_1\chi_4$: $4 \not< 1$ (violates rule)

   The production $\chi_4 \to \chi_1\chi_4$ violates the ordering rule because $i = 4 \geq j = 1$.

3. Step 3: Substitute χ_1 into χ_4 to fix violation

   Since $\chi_1 \to \chi_2\chi_3 \mid \chi_4\chi_4$, substitute into $\chi_4 \to \chi_1\chi_4$:

$$\chi_4 \to 0 \mid \chi_2\chi_3\chi_4 \mid \chi_4\chi_4\chi_4$$

4. Step 4: Remove left recursion from χ_4

   The production $\chi_4 \to \chi_4\chi_4\chi_4$ has direct left recursion, introduce new variable $\chi_5 = \psi_{n+1}$:

$$\begin{aligned}
&\chi_4 \to 0 \mid \chi_2\chi_3\chi_4 \mid 0\chi_5 \mid \chi_2\chi_3\chi_4\chi_5 \\
&\chi_5 \to \chi_4\chi_4 \mid \chi_4\chi_4\chi_5
\end{aligned}$$

   Current grammar:

$$\begin{aligned}
&\chi_1 \to \chi_2\chi_3 \mid \chi_4\chi_4 \\
&\chi_2 \to 0 \\
&\chi_3 \to 1 \\
&\chi_4 \to 0 \mid \chi_2\chi_3\chi_4 \mid 0\chi_5 \mid \chi_2\chi_3\chi_4\chi_5 \\
&\chi_5 \to \chi_4\chi_4 \mid \chi_4\chi_4\chi_5
\end{aligned}$$

5. Step 5: Convert to terminal-leading form

   For each production $\psi_i \to \psi_j T$ where $i < j$, substitute ψ_j with its productions.

   (a) Substitute $\chi_2 \to 0$ wherever χ_2 appears:

$$\begin{aligned}
&\chi_1 \to 0\chi_3 \mid \chi_4\chi_4 \\
&\chi_4 \to 0 \mid 0\chi_3\chi_4 \mid 0\chi_5 \mid 0\chi_3\chi_4\chi_5
\end{aligned}$$

   Since in $\chi_1 \to \chi_4\chi_4$, χ_4 appears twice, we need to substitute χ_4 again.

   (b) Substitute χ_4 into productions starting with χ_4:

$$\chi_1 \to 0\chi_3 \mid 0\chi_4 \mid 0\chi_3\chi_4\chi_4 \mid 0\chi_5\chi_4 \mid 0\chi_3\chi_4\chi_5\chi_4$$

   (c) Substitute χ_4 into χ_5 productions:
   For $\chi_5 \to \chi_4\chi_4$, substitute first χ_4:

$$\chi_5 \to 0\chi_4 \mid 0\chi_3\chi_4\chi_4 \mid 0\chi_5\chi_4 \mid 0\chi_3\chi_4\chi_5\chi_4$$

   For $\chi_5 \to \chi_4\chi_4\chi_5$, substitute first χ_4:

$$\chi_5 \to 0\chi_4\chi_5 \mid 0\chi_3\chi_4\chi_4\chi_5 \mid 0\chi_5\chi_4\chi_5 \mid 0\chi_3\chi_4\chi_5\chi_4\chi_5$$

   Combining both cases:

$$\begin{aligned}
\chi_5 \to\ & 0\chi_4 \mid 0\chi_3\chi_4\chi_4 \mid 0\chi_5\chi_4 \mid 0\chi_3\chi_4\chi_5\chi_4 \\
& \mid 0\chi_4\chi_5 \mid 0\chi_3\chi_4\chi_4\chi_5 \mid 0\chi_5\chi_4\chi_5 \mid 0\chi_3\chi_4\chi_5\chi_4\chi_5
\end{aligned}$$

6. Step 6: All productions are now of the form $\psi_i \to 1\gamma$ where $1 \in \Sigma$ (terminal 0 or 1) and $\gamma \in V^*$ (string of non-terminals):

$$\boxed{\begin{aligned}
&\chi_1 \to 0\chi_3 \mid 0\chi_4 \mid 0\chi_3\chi_4\chi_4 \mid 0\chi_5\chi_4 \mid 0\chi_3\chi_4\chi_5\chi_4 \\
&\chi_2 \to 0 \\
&\chi_3 \to 1 \\
&\chi_4 \to 0 \mid 0\chi_3\chi_4 \mid 0\chi_5 \mid 0\chi_3\chi_4\chi_5 \\
&\chi_5 \to 0\chi_4 \mid 0\chi_3\chi_4\chi_4 \mid 0\chi_5\chi_4 \mid 0\chi_3\chi_4\chi_5\chi_4 \\
&\qquad \mid 0\chi_4\chi_5 \mid 0\chi_3\chi_4\chi_4\chi_5 \mid 0\chi_5\chi_4\chi_5 \mid 0\chi_3\chi_4\chi_5\chi_4\chi_5
\end{aligned}}$$

21 Pushdown Automata (PDA)

A Pushdown Automaton (PDA) is formally defined as a 7-tuple:

$$M = (W, \Sigma, \Gamma, V, S, Z_0, F)$$

where:

1. W is a finite set of states
2. Σ is a finite input alphabet
3. Γ is a finite stack alphabet
4. $V : W \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \to \mathcal{P}(W \times \Gamma^*)$ is the transition function
5. $S \in W$ is the initial state
6. $Z_0 \in \Gamma$ is the initial stack symbol
7. $F \subseteq W$ is the set of accepting states

The transition function $V(w, x, X)$ returns a set of pairs (w', γ) where:

1. w is the current state
2. $x \in \Sigma \cup \{\varepsilon\}$ is the input symbol (or ε for no input)
3. $X \in \Gamma$ is the top symbol of the stack
4. w' is the next state
5. $\gamma \in \Gamma^*$ is the string to replace X on the stack

Example:
Given

$$L = \{0^n 1^n \mid n \geq 0\}$$

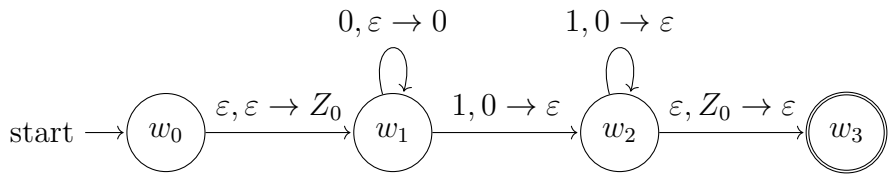
1. PDA Specification

$$W = \{w_0, w_1, w_2, w_3\} \qquad S = w_0$$
$$\Sigma = \{0, 1\} \qquad Z_0 = Z_0$$
$$\Gamma = \{Z_0, 0\} \qquad F = \{w_3\}$$

2. Transition Rules

$V(w_0, \varepsilon, \varepsilon) = \{(w_1, Z_0)\}$	Initialize stack with Z_0
$V(w_1, 0, \varepsilon) = \{(w_1, 0)\}$	Push 0 for each 0
$V(w_1, 1, 0) = \{(w_2, \varepsilon)\}$	Pop 0 for first 1
$V(w_2, 1, 0) = \{(w_2, \varepsilon)\}$	Pop 0 for each 1
$V(w_2, \varepsilon, Z_0) = \{(w_3, \varepsilon)\}$	Accept if all 0s matched

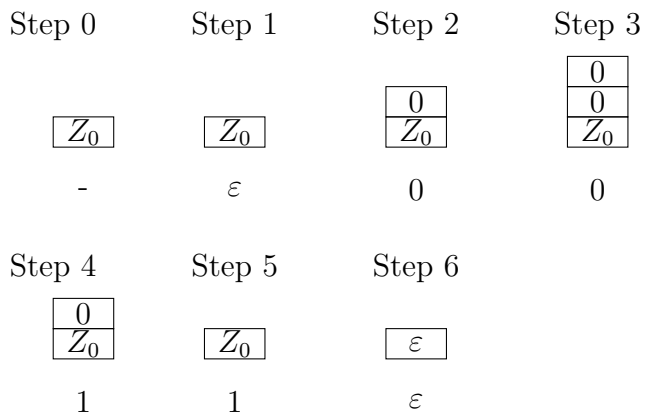
3. Transition Diagram



Execution for 0011

Step	**State**	**Input Symbol Read**	**Stack**	**Action**
0	w_0	-	Z_0	Initial configuration
1	w_1	ε	Z_0	ε-transition
2	w_1	0	$0Z_0$	Read 0, push 0
3	w_1	0	$00Z_0$	Read 0, push 0
4	w_2	1	$0Z_0$	Read 1, pop 0
5	w_2	1	Z_0	Read 1, pop 0
6	w_3	ε	ε	ε-transition, accept

4. Stack Visualization



The string 0011 is accepted because we end in an accepting state with the input fully consumed.

21.1 CFG to PDA

Given CFG $G = (N, \Sigma, R, S)$, construct PDA $M = (W, \Sigma, \Gamma, V, S, Z_0, F)$ where:

$$W = \{w\} \qquad \Gamma = N \cup \Sigma \qquad Z_0 = Z_0 \qquad F = \{w\}$$

Transition function V:

$$\begin{aligned}
V(w, \varepsilon, Z_0) &= \{(w, S)\} && \text{Initialize stack with start symbol} \\
V(w, \varepsilon, A) &= \{(w, \beta) \mid A \to \beta \in R\} && \text{Apply production rules for } A \in N \\
V(w, a, a) &= \{(w, \varepsilon)\} && \text{Match terminals for } a \in \Sigma
\end{aligned}$$

Example:
Given CFG

$$S \to 0S1 \mid \varepsilon$$

This generates $L = \{0^n 1^n \mid n \geq 0\}$

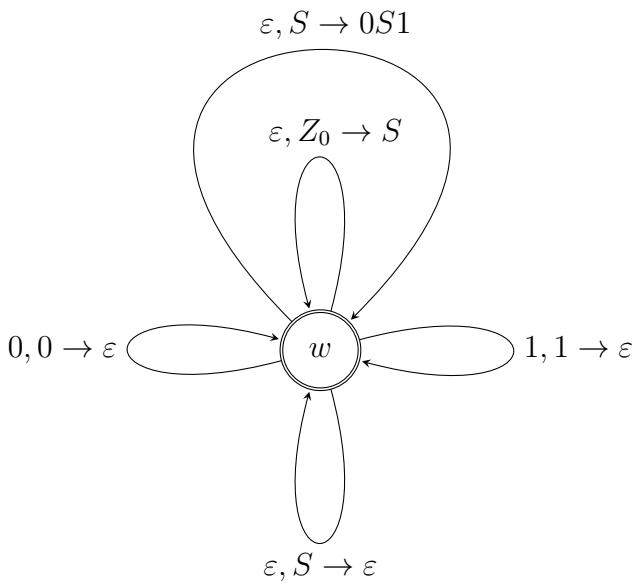
1. PDA Specification

$$\begin{aligned}
W &= \{w\} & S &= w \\
\Sigma &= \{0, 1\} & Z_0 &= Z_0 \\
\Gamma &= \{Z_0, S, 0, 1\} & F &= \{w\}
\end{aligned}$$

2. Transition Rules

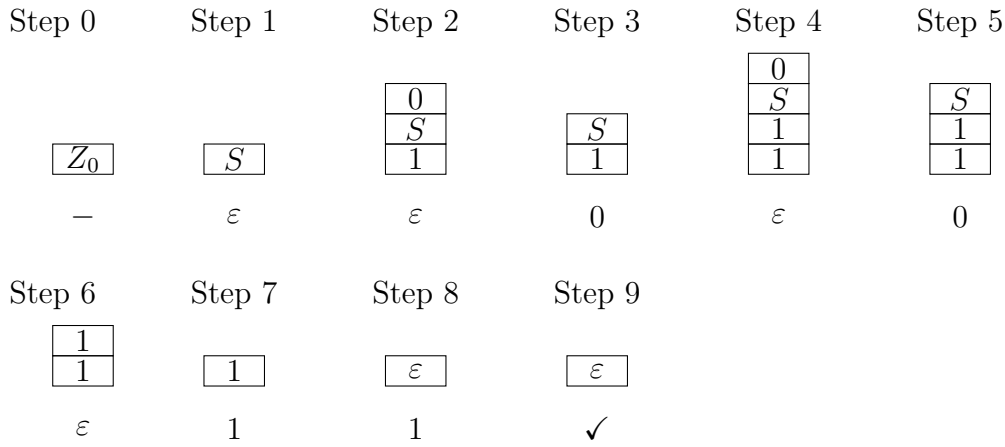
$$\begin{aligned}
V(w, \varepsilon, Z_0) &= \{(w, S)\} && \text{Initialize stack with } S \\
V(w, \varepsilon, S) &= \{(w, 0S1), (w, \varepsilon)\} && \text{Apply } S \to 0S1 \text{ or } S \to \varepsilon \\
V(w, 0, 0) &= \{(w, \varepsilon)\} && \text{Match and pop 0} \\
V(w, 1, 1) &= \{(w, \varepsilon)\} && \text{Match and pop 1}
\end{aligned}$$

3. Transition Diagram:



Execution for $x = 0011$

Step	**State**	**Input Symbol Read**	**Stack**	**Action**
0	w	-	Z_0	Initial configuration
1	w	ε	S	ε-transition, push S
2	w	ε	$0S1$	Apply $S \to 0S1$
3	w	0	$S1$	Read 0, pop 0
4	w	ε	$0S11$	Apply $S \to 0S1$
5	w	0	$S11$	Read 0, pop 0
6	w	ε	11	Apply $S \to \varepsilon$
7	w	1	1	Read 1, pop 1
8	w	1	ε	Read 1, pop 1
9	w	ε	ε	Accept



21.2 PDA to CFG

Given PDA $M = (W, \Sigma, \Gamma, V, w_0, Z_0, F)$, construct CFG $G = (N, \Sigma, R, S)$ where:

1. Non-terminals: $N = \{S\} \cup \{[w, \psi, w'] \mid w, w' \in W, \psi \in \Gamma\}$

   Each non-terminal $[w, \psi, w']$ represents: "starting from state w with ψ on top of stack, reach state w' with ψ popped"

2. Start symbol: S with productions:

$$S \to [w_0, Z_0, w_f] \quad \forall w_f \in F$$

3. Production rules:

   (a) For each transition $(w', x, \psi) \to (w'', \varepsilon)$ (pop ψ, read $x \in \Sigma \cup \{\varepsilon\}$):

$$[w', \psi, w''] \to x$$

   (b) For each transition $(w', x, \psi) \to (w'', \phi_1\phi_2 \cdots \phi_k)$ with $k \geq 1$ (replace ψ with $\phi_1\phi_2 \cdots \phi_k \in \Gamma^*$, read $x \in \Sigma \cup \{\varepsilon\}$):

$$[w', \psi, w_{k+1}] \to x\,[w'', \phi_1, w_1][w_1, \phi_2, w_2] \cdots [w_k, \phi_k, w_{k+1}]$$

   for all choices of intermediate states $w_1, w_2, \ldots, w_k \in W$, and all $w_{k+1} \in W$

Example:

1. PDA Specification

$$\begin{aligned} W &= \{w_0, w_1\} & S &= w_0 \\ \Sigma &= \{0, 1\} & Z_0 &= Z_0 \\ \Gamma &= \{Z_0, 0\} & F &= \{w_1\} \end{aligned}$$

2. Transition Rules

$$\begin{aligned} V(w_0, 0, Z_0) &= \{(w_0, 0Z_0)\} && \text{Push 0 on } Z_0 \\ V(w_0, 0, 0) &= \{(w_0, 00)\} && \text{Push 0 on 0} \\ V(w_0, 1, 0) &= \{(w_1, \varepsilon)\} && \text{Read first 1, pop 0, move to pop phase} \\ V(w_1, 1, 0) &= \{(w_1, \varepsilon)\} && \text{Pop 0 for each 1} \\ V(w_1, \varepsilon, Z_0) &= \{(w_1, \varepsilon)\} && \text{Pop } Z_0\text{, accept} \end{aligned}$$

   Execution for $x = 0011$:

3. Constructed CFG

   Non-terminals:

$$N = \{S\} \cup \{[w, \psi, w'] \mid w, w' \in W,\ \psi \in \Gamma\}$$

   Then, since $W = \{w_0, w_1\}$ and $\Gamma = \{Z_0, 0\}$, we have:

$$\begin{aligned} N = \{S,\ &[w_0, Z_0, w_0],\ [w_0, Z_0, w_1],\ [w_1, Z_0, w_0],\ [w_1, Z_0, w_1], \\ &[w_0, 0, w_0],\ [w_0, 0, w_1],\ [w_1, 0, w_0],\ [w_1, 0, w_1]\} \end{aligned}$$

Step	State	Input Symbol	Stack	Action
0	w_0	-	Z_0	Initial configuration
1	w_0	0	$0Z_0$	Read 0, push 0 on Z_0
2	w_0	0	$00Z_0$	Read 0, push 0 on 0
3	w_1	1	$0Z_0$	Read 1, pop 0, move to w_1
4	w_1	1	Z_0	Read 1, pop 0
5	w_1	ε	ε	Pop Z_0, accept

(a) Start production:

$$S \to [w_0, Z_0, w_1]$$

(b) Push transition $V(w_0, 0, Z_0) = (w_0, 0Z_0)$:

$$[w_0, Z_0, w_i] \to 0\,[w_0, 0, w_j][w_j, Z_0, w_i] \quad \forall\, w_i, w_j \in W$$

$$\begin{aligned}
[w_0, Z_0, w_0] &\to 0\,[w_0, 0, w_0][w_0, Z_0, w_0] \\
[w_0, Z_0, w_0] &\to 0\,[w_0, 0, w_1][w_1, Z_0, w_0] \\
[w_0, Z_0, w_1] &\to 0\,[w_0, 0, w_0][w_0, Z_0, w_1] \\
[w_0, Z_0, w_1] &\to 0\,[w_0, 0, w_1][w_1, Z_0, w_1]
\end{aligned}$$

(c) Push transition $V(w_0, 0, 0) = (w_0, 00)$:

$$[w_0, 0, w_i] \to 0\,[w_0, 0, w_j][w_j, 0, w_i] \quad \forall\, w_i, w_j \in W$$

$$\begin{aligned}
[w_0, 0, w_0] &\to 0\,[w_0, 0, w_0][w_0, 0, w_0] \\
[w_0, 0, w_0] &\to 0\,[w_0, 0, w_1][w_1, 0, w_0] \\
[w_0, 0, w_1] &\to 0\,[w_0, 0, w_0][w_0, 0, w_1] \\
[w_0, 0, w_1] &\to 0\,[w_0, 0, w_1][w_1, 0, w_1]
\end{aligned}$$

(d) Pop transition $V(w_0, 1, 0) = (w_1, \varepsilon)$:

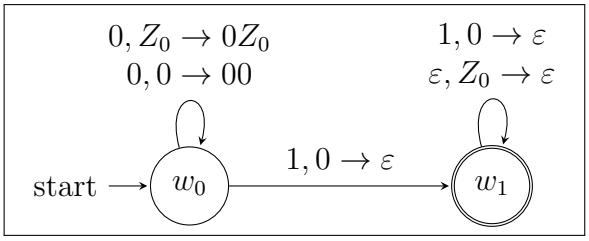
$$[w_0, 0, w_1] \to 1$$

(e) Pop transition $V(w_1, 1, 0) = (w_1, \varepsilon)$:

$$[w_1, 0, w_1] \to 1$$

(f) Accept transition $V(w_1, \varepsilon, Z_0) = (w_1, \varepsilon)$:

$$[w_1, Z_0, w_1] \to \varepsilon$$



Derivation for 0011:

$$
\begin{aligned}
S &\to [w_0, Z_0, w_1] \\
&\to 0\,[w_0, 0, w_1][w_1, Z_0, w_1] \\
&\to 0\,0\,[w_0, 0, w_1][w_1, 0, w_1][w_1, Z_0, w_1] \\
&\to 0\,0\,1\,[w_1, 0, w_1][w_1, Z_0, w_1] \\
&\to 0\,0\,1\,1\,[w_1, Z_0, w_1] \\
&\to 0011.
\end{aligned}
$$

The generated language: $L = \{0^n 1^n \mid n \geq 0\}$.

22 Palindromes

Formally

Let $\Sigma = \{0, 1\}$ be a binary alphabet and $x \in \Sigma^*$ be a string over Σ. We denote by x^R the reverse of x. Then x is a *palindrome* if and only if:

$$x = x^R$$

Equivalently, for a string $w = x_1 x_2 \cdots x_n$ where $x_i \in \Sigma$ and $n = |w|$, w is a palindrome if and only if:

$$x_i = x_{n-i+1} \quad \forall i \in \{1, 2, \ldots, n\}$$

In general:

Consider the string $w = 110011$ over $\Sigma = \{0, 1\}$.

We have:

$$w = 110011, \quad w^R = 110011$$

Since $w = w^R$, the string 110011 is a palindrome. Alternatively, checking each position:

$$
\begin{aligned}
x_1 &= x_6 = 1 \\
x_2 &= x_5 = 1 \\
x_3 &= x_4 = 0
\end{aligned}
$$

Therefore, 110011 is a palindrome.

Example:

Given

$$L = \{xx^R \mid x \in (0 + 1)^+\}$$

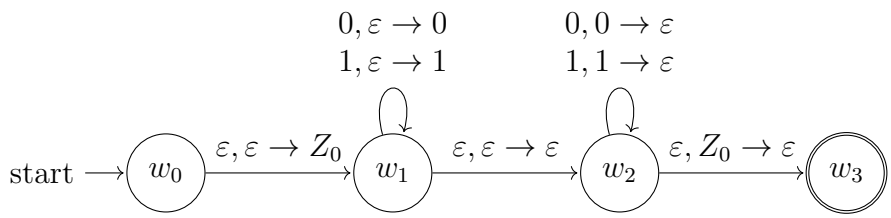
For input string $w = 0110$, we need to verify if it belongs to L.

1. PDA Specification

$$
\begin{aligned}
W &= \{w_0, w_1, w_2, w_3\} & S &= w_0 \\
\Sigma &= \{0, 1\} & Z_0 &= Z_0 \\
\Gamma &= \{Z_0, 0, 1\} & F &= \{w_3\}
\end{aligned}
$$

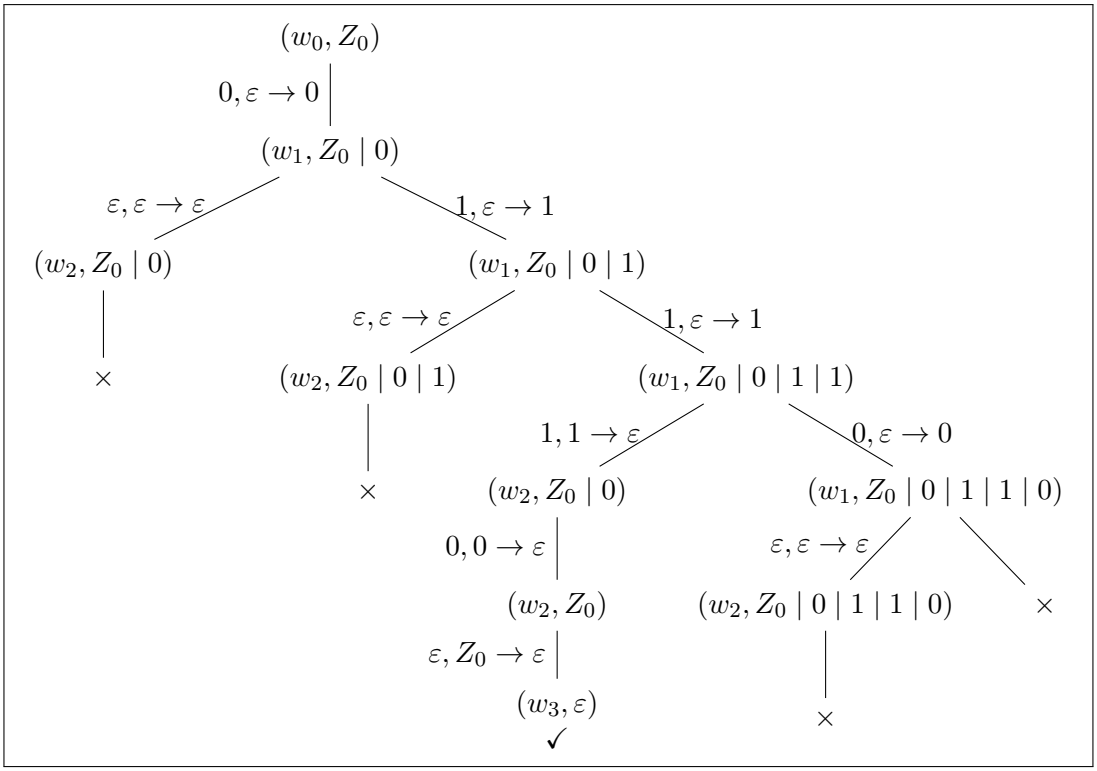
2. Transition Rules

$V(w_0, \varepsilon, \varepsilon) = \{(w_1, Z_0)\}$	Initialize stack with Z_0
$V(w_1, 0, \varepsilon) = \{(w_1, 0)\}$	Push 0 onto stack
$V(w_1, 1, \varepsilon) = \{(w_1, 1)\}$	Push 1 onto stack
$V(w_1, \varepsilon, \varepsilon) = \{(w_2, \varepsilon)\}$	Guess middle, transition to pop phase
$V(w_2, 0, 0) = \{(w_2, \varepsilon)\}$	Match and pop 0
$V(w_2, 1, 1) = \{(w_2, \varepsilon)\}$	Match and pop 1
$V(w_2, \varepsilon, Z_0) = \{(w_3, \varepsilon)\}$	Accept if stack is empty



3. Computation Tree for Input 0110

   Since the PDA is nondeterministic (can guess the middle at any point using ε-transition from w_1 to w_2), we explore all possible computation paths with this schema $[\varepsilon \mid 0 \mid \varepsilon \mid 1 \mid \varepsilon \mid 1 \mid \varepsilon \mid 0 \mid \varepsilon]$



(a) The PDA must nondeterministically guess where the middle of the string is

(b) The accepting path: Push 0, push 1, guess middle (via ε-transition), pop 1, pop 0, pop Z_0 to accept

(c) This corresponds to recognizing 01 | 10 where | marks the middle

(d) Other guesses of the middle position lead to rejection

Step	**State**	**Remaining Input**	**Stack (bottom→top)**	**Action**
0	w_0	0110	Z_0	Initial configuration
1	w_1	0110	Z_0	ε-transition to w_1
2	w_1	110	Z_0 0	Read 0, push 0
3	w_1	10	Z_0 0 1	Read 1, push 1
4	w_2	10	Z_0 0 1	ε-transition (guess middle)
5	w_2	0	Z_0 0	Read 1, match and pop 1
6	w_2	ε	Z_0	Read 0, match and pop 0
7	w_3	ε	ε	Pop Z_0, accept

Concisely,

For input

$$x = 0110 \in L = \{xx^R \mid x \in \{0,1\}^+\}$$

(a) String Decomposition

$$x = 0110 = \underbrace{01}_{x}\underbrace{10}_{x^R} \quad \text{where } |x| = 2$$

(b) Meeting Point Function

$$\text{MeetPoint}(x) = n = 2 \quad \text{such that } x[1..n] = x \text{ and } x[n+1..2n] = x^R$$

(c) Configuration at Middle (Step 4)
Using stack notation $Z_0\ a_1\ a_2\ \ldots\ a_k$ where Z_0 is leftmost (bottom) and elements grow rightward (top on right):

$$(w_1, Z_0 \mid 0 \mid 1) \xrightarrow{\varepsilon,\varepsilon\to\varepsilon} (w_2, Z_0 \mid 0 \mid 1)$$

This is the critical nondeterministic transition where the PDA "guesses" the middle.

(d) Invariant at Meeting Point

$$\boxed{\text{Stack}(w_2) = Z_0\ 01 \quad \wedge \quad \text{RemainingInput}(w_2) = 10 = x^R}$$

This captures the symmetry: what remains to read (10) equals the reverse of what has been stored (01).

(e) Push-Meet-Pop Schema

$$\underbrace{x[1..2] = 01}_{\text{read \& push}} \xrightarrow[\text{Steps 2–3}]{\text{push}} \underbrace{\text{Stack} = Z_0\ 01}_{\text{Step 4}} \xrightarrow[\text{Steps 5–6}]{\text{match \& pop}} \underbrace{x[3..4] = 10}_{\text{verify } x^R}$$

23 Turing Machine

A TM is formally defined as a 7-tuple:

$$M = (W, \Sigma, \Gamma, V, S, B, F)$$

where:

1. W is a finite set of states
2. Σ is the finite input alphabet not containing the blank symbol B
3. Γ is the finite tape alphabet, where $B \in \Gamma \wedge \Sigma \subseteq \Gamma$
4. $V : W \times \Gamma \to W \times \Gamma \times \{L, R\}$ is the transition function
5. $S \in W$ is the start state
6. $B \in \Gamma$ is the blank symbol
7. $F \subseteq W$ is the set of final (accepting) states

Example 1:
Given

$$L = \{0^n 1^n \mid n \geq 1\}$$

Let $M = (W, \Sigma, \Gamma, V, S, B, F)$ where:

$$W = \{w_0, w_1, w_2, w_3\} \qquad \Sigma = \{0, 1\} \qquad \Gamma = \{0, 1, X, Y, B\}$$

$$S = w_0 \qquad B = \triangle \qquad F = \{w_3\}$$

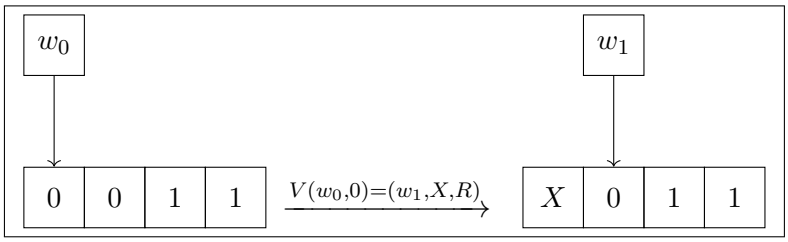
Transition Function V:

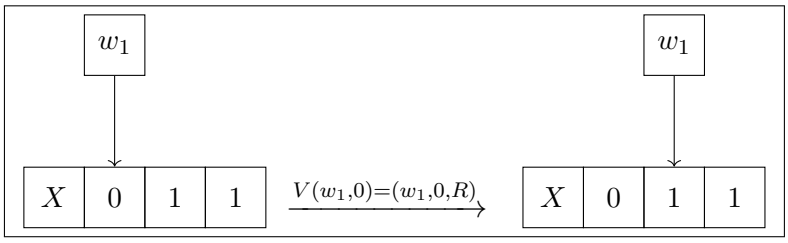
$V(w_0, 0) = (w_1, X, R)$	Mark first 0, move right
$V(w_0, Y) = (w_0, Y, R)$	Skip marked 1s
$V(w_0, \triangle) = (w_3, \triangle, R)$	Accept if tape is blank
$V(w_1, 0) = (w_1, 0, R)$	Scan right past 0s
$V(w_1, Y) = (w_1, Y, R)$	Scan right past marked 1s
$V(w_1, 1) = (w_2, Y, L)$	Mark first 1, move left
$V(w_2, 0) = (w_2, 0, L)$	Scan left past 0s
$V(w_2, Y) = (w_2, Y, L)$	Scan left past marked 1s
$V(w_2, X) = (w_0, X, R)$	Return to marked 0, repeat

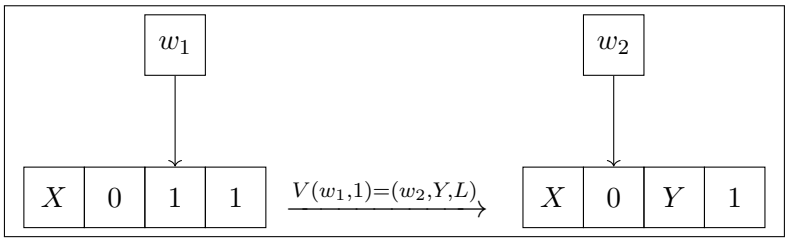
All other transitions are undefined (reject).

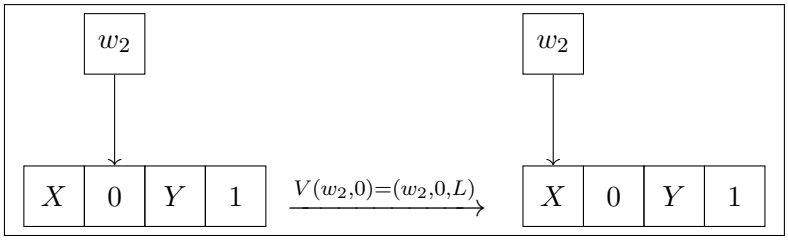
Example Execution on Input "0011":

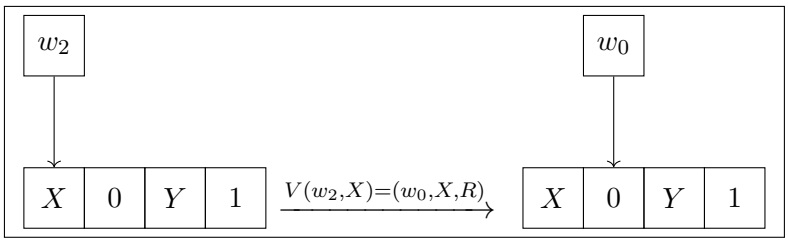
In state w_0 the machine reads 0, writes X, moves right, and enters state w_1.

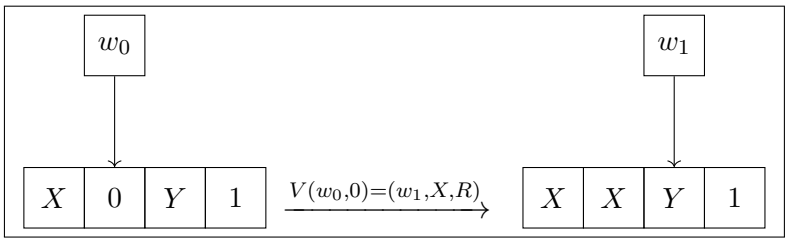


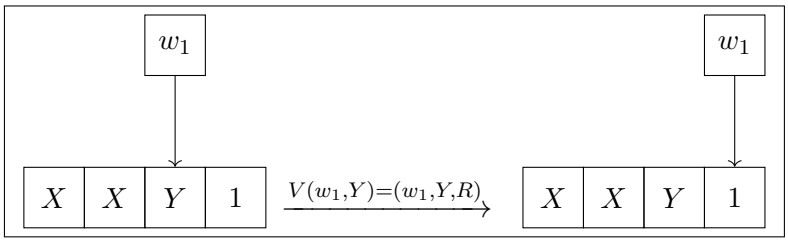


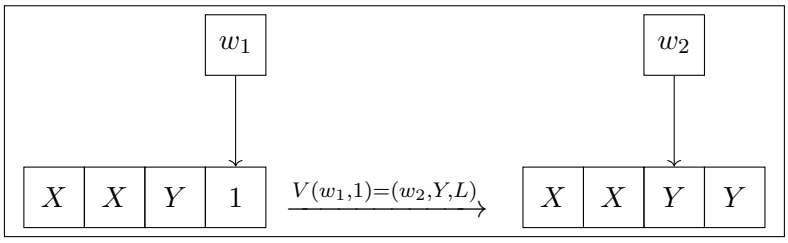


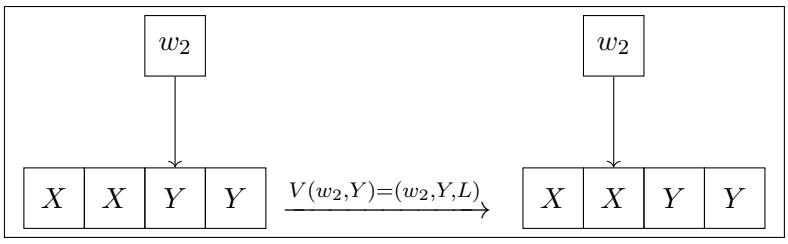

w_2
X 0 Y 1
$V(w_2,0)=(w_2,0,L)$
w_2
X 0 Y 1

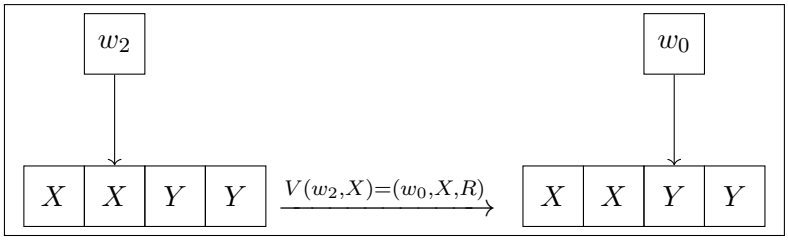

w_2
X 0 Y 1
$V(w_2,X)=(w_0,X,R)$
w_0
X 0 Y 1

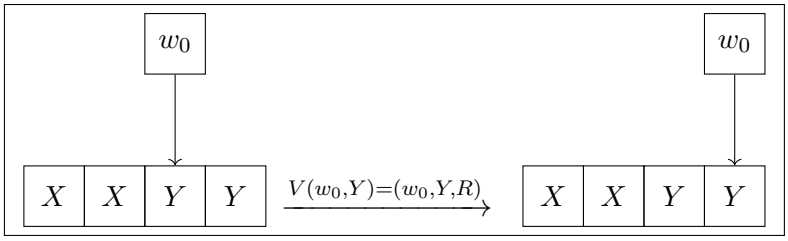

w_0
X 0 Y 1
$V(w_0,0)=(w_1,X,R)$
w_1
X X Y 1

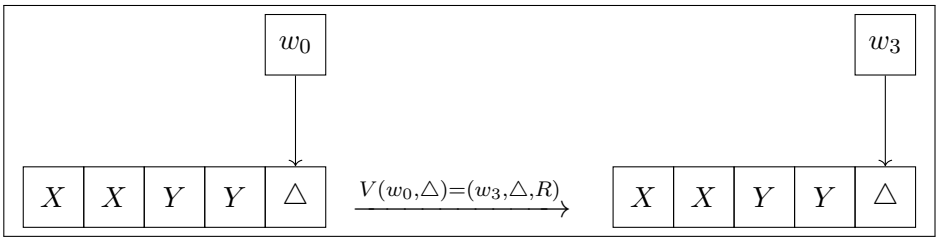

w_1
X X Y 1
$V(w_1,Y)=(w_1,Y,R)$
w_1
X X Y 1


w_1
X X Y 1
$V(w_1,1)=(w_2,Y,L)$
w_2
X X Y Y

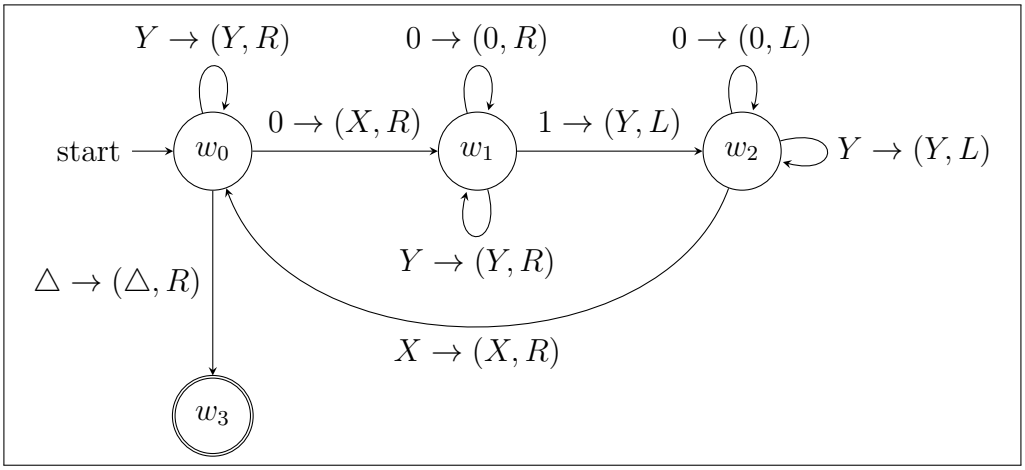

w_2
X X Y Y
$V(w_2,Y)=(w_2,Y,L)$
w_2
X X Y Y







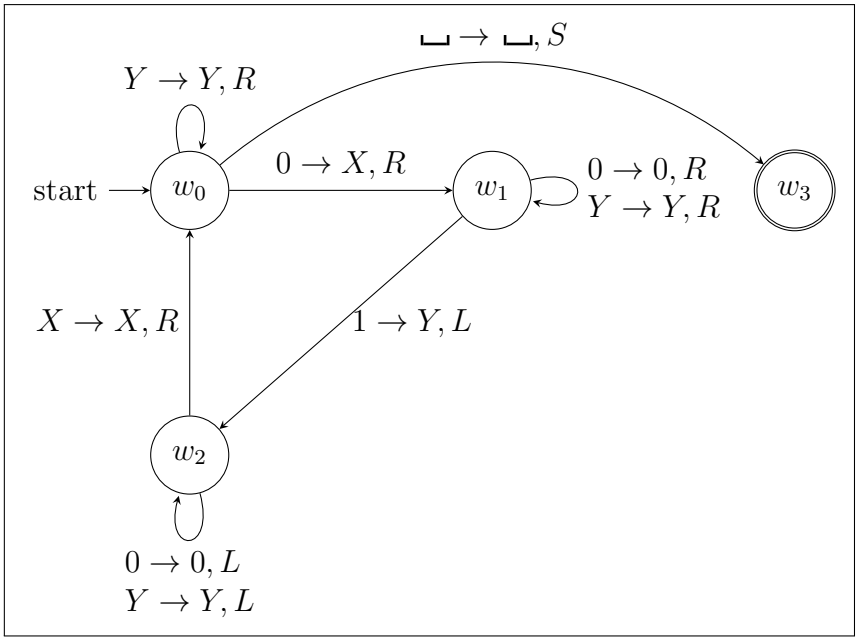
Equivalently,



Example 2:
Given

$$T = \{0^k 1^k \mid k \geq 1\}$$

with the TM defined as:



where the components are defined as:

$$W = \{w_0, w_1, w_2, w_3\} \qquad \Sigma = \{0, 1\} \qquad \Gamma = \{0, 1, X, Y, \sqcup\}$$

$$S = w_0 \qquad B = \sqcup \qquad F = \{w_3\}$$

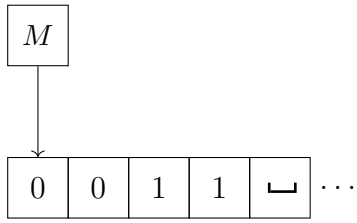
Execution Trace for Input "0011"

Step	**State**	**Tape Content**	**Head Position**	**Action**
0	w_0	$\boxed{0}$011	0	Read 0, write X, move R
1	w_1	X$\boxed{0}$11	1	Read 0, skip, move R
2	w_1	X0$\boxed{1}$1	2	Read 1, write Y, move L
3	w_2	X$\boxed{0}$Y1	1	Read 0, skip, move L
4	w_2	$\boxed{\text{X}}$0Y1	0	Read X, move R
5	w_0	X$\boxed{0}$Y1	1	Read 0, write X, move R
6	w_1	XX$\boxed{\text{Y}}$1	2	Read Y, skip, move R
7	w_1	XXY$\boxed{1}$	3	Read 1, write Y, move L
8	w_2	XX$\boxed{\text{Y}}$Y	2	Read Y, skip, move L
9	w_2	X$\boxed{\text{X}}$YY	1	Read X, move R
10	w_0	XX$\boxed{\text{Y}}$Y	2	Read Y, skip, move R
11	w_0	XXY$\boxed{\text{Y}}$	3	Read Y, skip, move R
12	w_0	XXYY$\boxed{\sqcup}$	4	Read $\sqcup$, accept
13	w_3	XXYY	4	**ACCEPT**

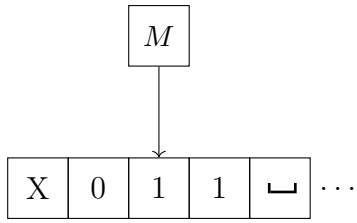
Transition Table

Current State	Read Symbol	Write Symbol	Next State, Direction
w_0	0	X	(w_1, R)
w_0	Y	Y	(w_0, R)
w_0	$\sqcup$	$\sqcup$	(w_3, S)
w_1	0	0	(w_1, R)
w_1	Y	Y	(w_1, R)
w_1	1	Y	(w_2, L)
w_2	0	0	(w_2, L)
w_2	Y	Y	(w_2, L)
w_2	X	X	(w_0, R)
w_3	-	-	Accept

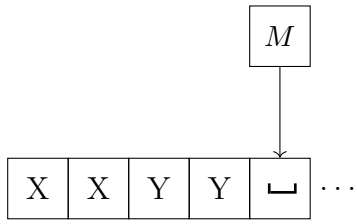
Illustratively,
Step 0:



Step 2:



Step 12:

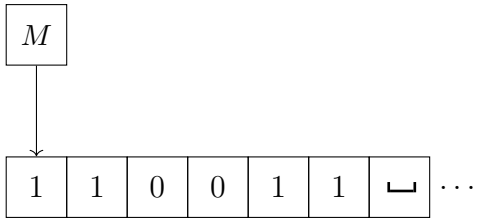


Example 3:
Given

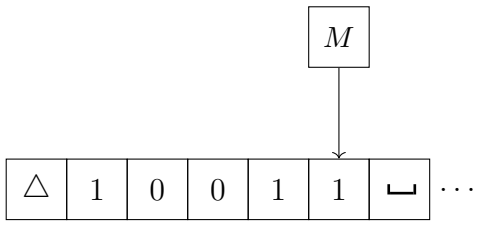
$$L = \{xx^R \mid x \in \{0,1\}^*\}$$

The input 110011 is accepted since $x = 110$ and $x^R = 011$. The TM works by repeatedly reading the leftmost symbol, remembering it, moving right to the rightmost unread symbol, checking it matches, then marking both as read. We use $\triangle$ to mark already-matched symbols (boundaries), and $\sqcup$ for blank.

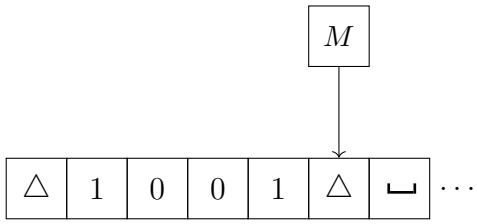
Initial tape:



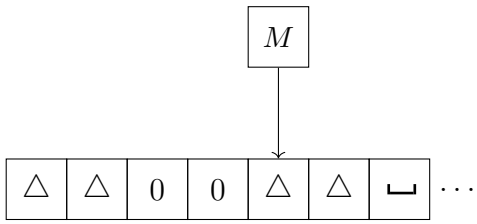
Step 1: Read leftmost 1, mark it with △, move right to find rightmost unread symbol:



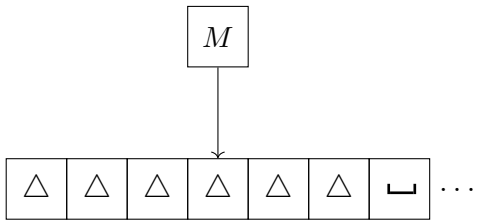
Step 2: Rightmost unread is 1, matches remembered symbol 1. Mark it with △:



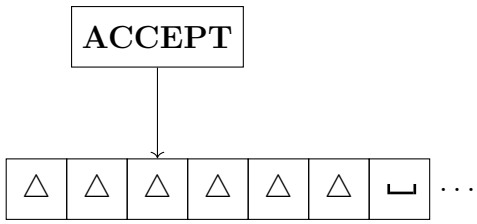
Step 3: Move back left to next unread symbol 1, mark it, move right to rightmost unread 1, match:



Step 4: Move back left to next unread symbol 0, mark it, move right to rightmost unread 0, match:



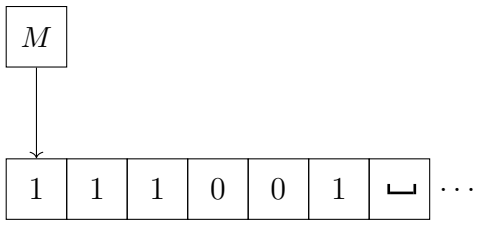
Step 5: All symbols matched. Head sees only △ and ␣. TM halts and accepts.



23.1 Turing Machine Programming Techniques

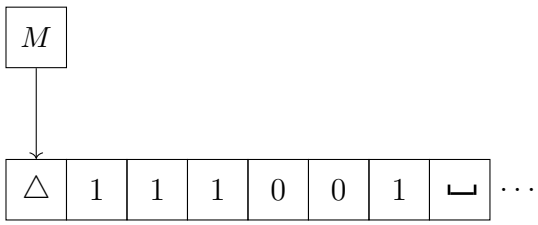
A standard TM cannot directly detect the left end of its tape. Hence, we place a special symbol $\triangle$ at the left end of the tape and shift the input one cell to the right.

Example:



1. Step 1: Shift input one cell to the right and place $\triangle$ at position 0

   The Turing machine reads the input tape and shifts all symbols one position to the right, then places the special symbol $\triangle$ at the leftmost position. This allows the machine to detect when it reaches the left boundary.

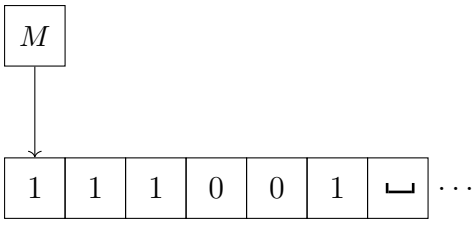


2. Step 2: Left boundary detection

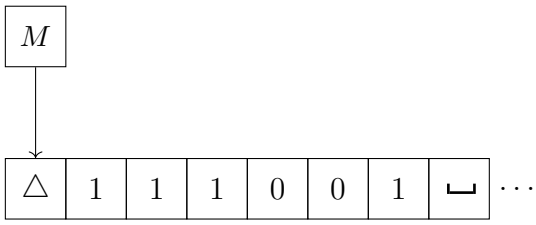
   When the Turing machine reads the $\triangle$ symbol, it knows it has reached the leftmost position and cannot move further left. This prevents the machine from falling off the tape.

   Example Execution:

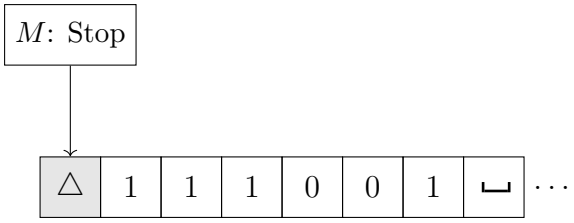
   (a) Initial tape, input `111001`:



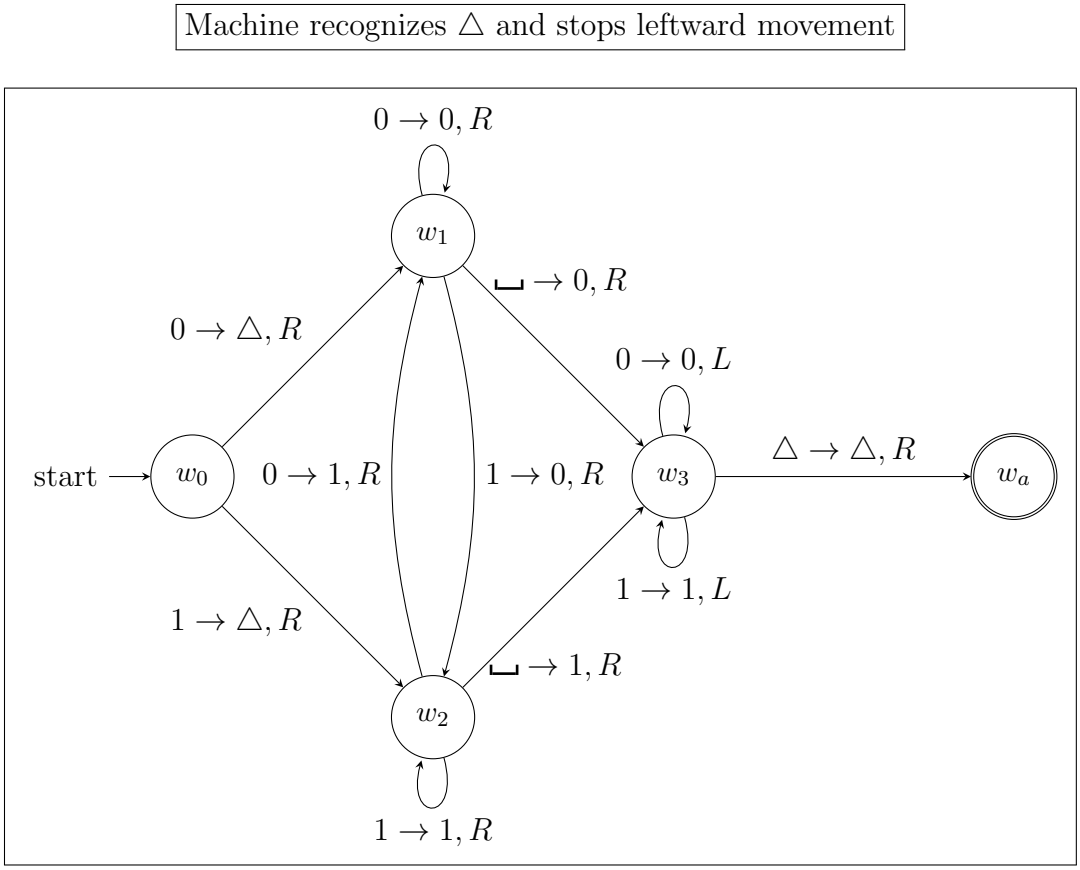
   (b) Shift input one cell to the right, place $\triangle$ at position 0:



   (c) Machine sweeps left, reaches $\triangle$:



Hence,

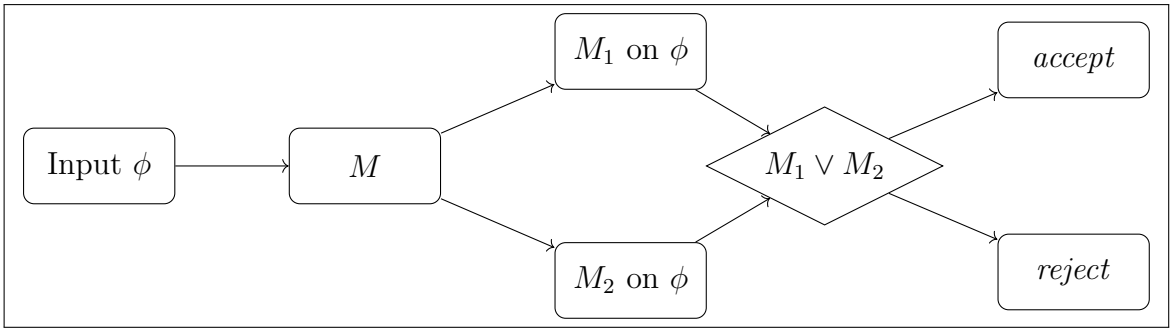


23.1.1 Union

The class of Turing-recognizable languages is closed under union. That is, if L_1 and L_2 are both Turing-recognizable, then $L_1 \cup L_2$ is also Turing-recognizable. We construct a new Turing machine M that recognizes $L_1 \cup L_2$ by simulating M_1 and M_2 in parallel on input w, alternating steps between the two machines. M accepts if M_1 accepts $\vee$ M_2 accepts. Note that sequential simulation is not sufficient, since M_1 might loop forever on inputs in L_2, preventing M_2 from ever running.

Example:

Let $L_1 = \{a^n b^n \mid n \geq 0\}$ and $L_2 = \{a^n b^n c^n \mid n \geq 0\}$, both Turing-recognizable languages. Then $L_1 \cup L_2 = \{x \mid x \in L_1 \vee x \in L_2\}$ is also Turing-recognizable. A TM M recognizing $L_1 \cup L_2$ simulates M_1 and M_2 in parallel on input x. For example, on input *aabbcc*, M_1 rejects but M_2 accepts ($M_1 \vee M_2$ is true), so M accepts. On input *aabb*, M_1 accepts ($M_1 \vee M_2$ is true), so M accepts immediately.

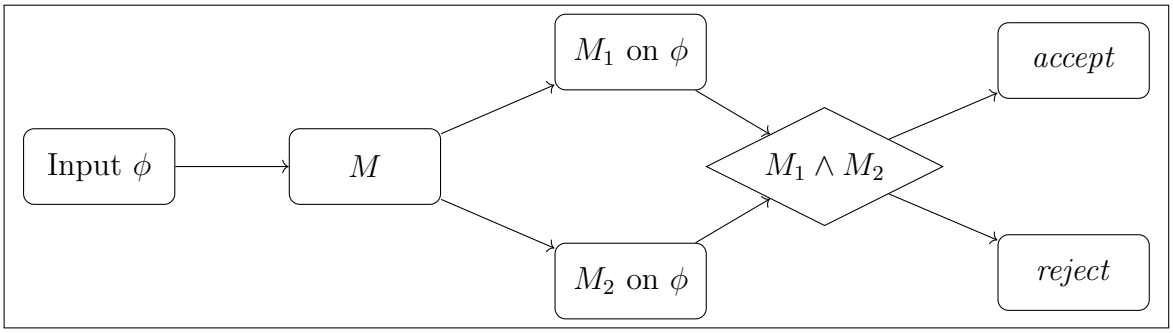


23.1.2 Intersection

The class of Turing-recognizable languages is closed under intersection. That is, if L_1 and L_2 are both Turing-recognizable, then $L_1 \cap L_2$ is also Turing-recognizable. We construct a new Turing machine M that recognizes $L_1 \cap L_2$ by simulating M_1 and M_2 in parallel on input w, alternating steps between the two machines. M accepts if M_1 accepts $\wedge$ M_2 accepts. Both machines must accept for M to accept.

Example:

Let $L_1 = \{a^n b^n \mid n \geq 0\}$ and $L_2 = \{a^n b^n c^* \mid n \geq 0\}$, both Turing-recognizable languages. Then $L_1 \cap L_2 = \{x \mid x \in L_1 \wedge x \in L_2\}$ is also Turing-recognizable. A TM M recognizing $L_1 \cap L_2$ simulates M_1 and M_2 in parallel on input x. For example, on input *aabb*, both M_1 and M_2 accept ($M_1 \wedge M_2$ is true), so M accepts. On input *aabbcc*, M_1 rejects ($M_1 \wedge M_2$ is false), so M rejects.

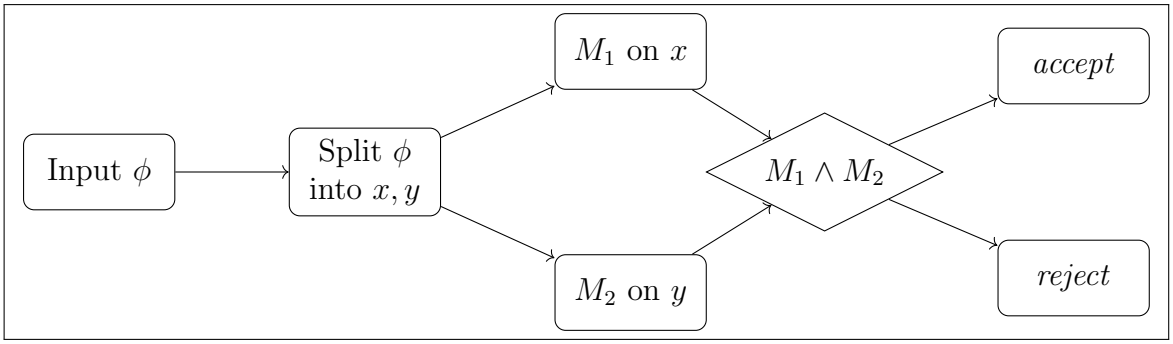


23.1.3 Concatenation

The class of Turing-recognizable languages is closed under concatenation. That is, if L_1 and L_2 are both Turing-recognizable, then $L_1 \cdot L_2 = \{xy \mid x \in L_1 \wedge y \in L_2\}$ is also Turing-recognizable. We construct a new Turing machine M that recognizes $L_1 \cdot L_2$ by nondeterministically splitting the input ϕ into two parts x and y, then simulating M_1 on x and M_2 on y in parallel. M accepts if M_1 accepts $x \wedge M_2$ accepts y.

Example:

Let $L_1 = \{a^n b^n \mid n \geq 0\}$ and $L_2 = \{c^n d^n \mid n \geq 0\}$, both Turing-recognizable languages. Then $L_1 \cdot L_2 = \{xy \mid x \in L_1 \wedge y \in L_2\}$ is also Turing-recognizable. A TM M recognizing $L_1 \cdot L_2$ nondeterministically splits input ϕ into x and y, then simulates M_1 on x and M_2 on y. For example, on input *aabbccdd*, M splits it into $x = aabb$ and $y = ccdd$. Since M_1 accepts $aabb \wedge M_2$ accepts *ccdd* ($M_1 \wedge M_2$ is true), so M accepts. On input *aabbcc*, no valid split exists such that both M_1 and M_2 accept, so M rejects.



23.1.4 Comparing Two Strings

To compare two strings separated by $\triangle$ using a Turing machine:

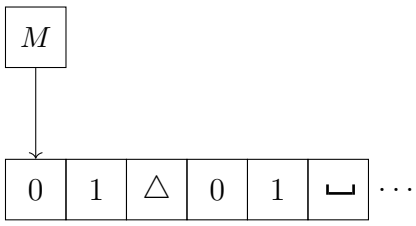
1. Mark leftmost unmarked symbol in first string ($0 \to X$, $1 \to Y$)
2. Scan right past $\triangle$ to leftmost unmarked symbol in second string
3. If symbols match, mark it; otherwise reject
4. Return to left end
5. Repeat until first string exhausted
6. Accept iff second string also exhausted; otherwise reject

Example:
Given

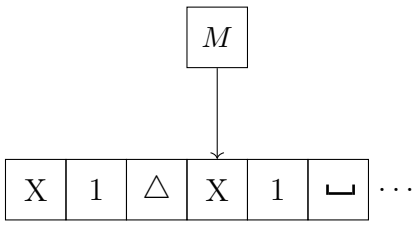
$$L = \{x\triangle x \mid x \in \{0,1\}^*\}$$

1. Accept 01$\triangle$01

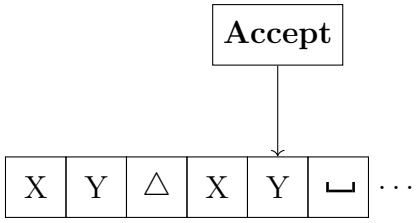
   Initial:



   Mark first 0, match across $\triangle$:

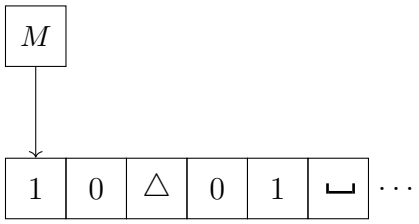


   Mark first 1, match across $\triangle$:

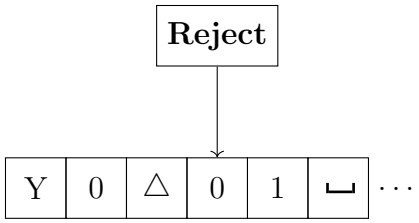


2. Reject 10△01

   Initial:

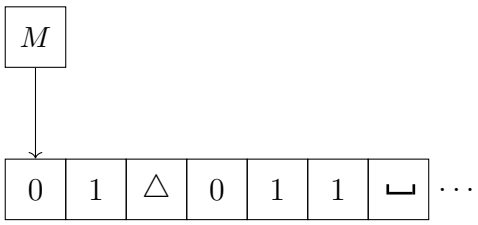


   Mark first symbol 1, compare with 0: mismatch:

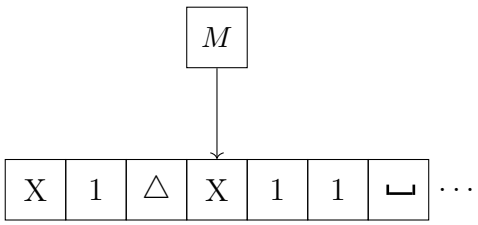


3. Reject 01△011 (unequal length)

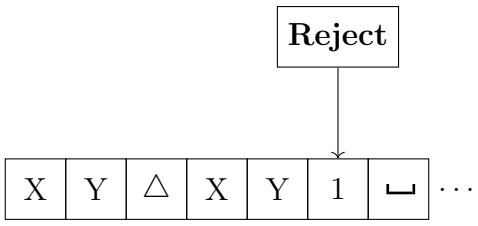
   Initial:



   Mark 0s:



   Mark 1s:



23.2 Multi-tape Turing Machines

A multi-tape Turing machine has multiple tapes, each with its own read/write head. The transition function is defined as:

$$V : W \times \Gamma^k \to W \times \Gamma^k \times \{L, R, S\}^k$$

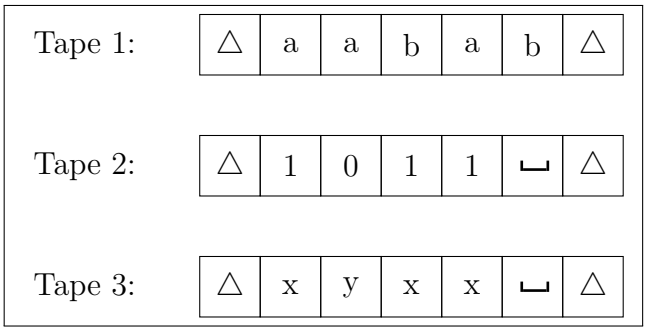
where W is the set of states, Γ is the tape alphabet, and L, R, S denote moving the head left, right, or staying in place, respectively.

Example:

Consider a 3-tape Turing machine simulating a finite state machine. The single tape contains the entire configuration encoded across three logical tracks:

1. Tape 1: Contains the input string (e.g., △ aabab △)
2. Tape 2: Contains working data (e.g., △1011 △)
3. Tape 3: Contains auxiliary symbols (e.g., △xyxx △)

The machine uses special markers (like △) to delimit sections and can simulate the FSM by coordinating operations across all three tapes simultaneously. Each tape has its own read/write head that can move independently according to the transition function V.



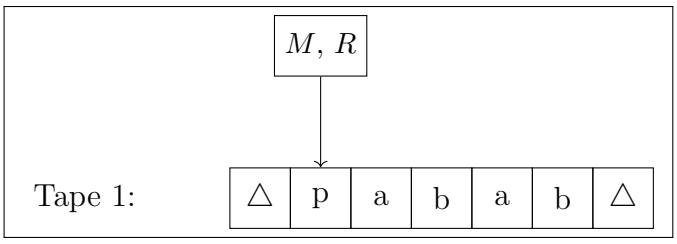
Suppose the TM is in state w_i and reads:

1. Tape 1: a
2. Tape 2: 1
3. Tape 3: x

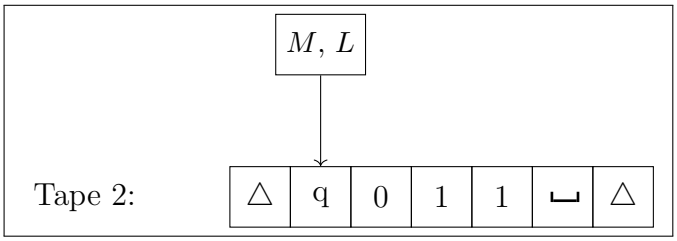
The transition function processes all three symbols simultaneously:

$$V(w_i, (a, 1, x)) = (w_j, (p, q, r), (R, L, R))$$

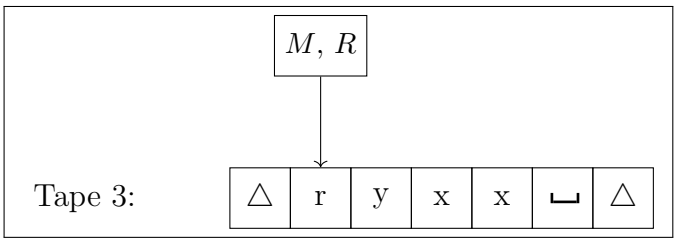
1. Write p on Tape 1, move Head to the right



2. Write q on Tape 2, move Head to the left

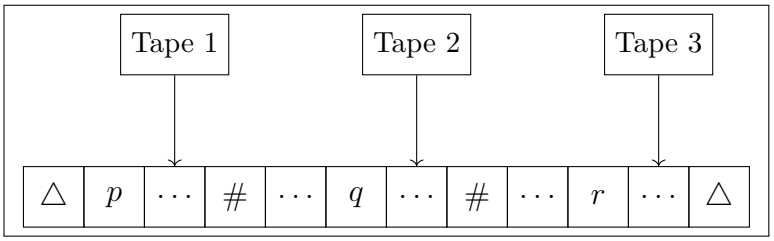


3. Write r on Tape 3, move Head to the right



After the multi-tape Turing machine has completed scanning all tapes, the final values written across the k tapes can be concatenated onto a single tape. This is done by encoding the contents of each tape sequentially, separated by a special delimiter symbol $\#$.

$$\triangle\, p \;\ldots\; \#\, q \;\ldots\; \#\, r \;\ldots\; \triangle$$



23.3 Nondeterministic Turing Machine (NTM)

For a nondeterministic Turing machine (NTM), the transition function is defined as:

$$V : W \times \Gamma \to \mathcal{P}(W \times \Gamma \times \{L, R\})$$

This means that for a given state and tape symbol, the NTM can transition to a set of possible states, write a new symbol, and move the head in either direction.

Example:

Let:

$$W = \{w_0, w_1, w_2\}, \quad \Gamma = \{0, 1, ␣\}, \quad w_1 \in F$$

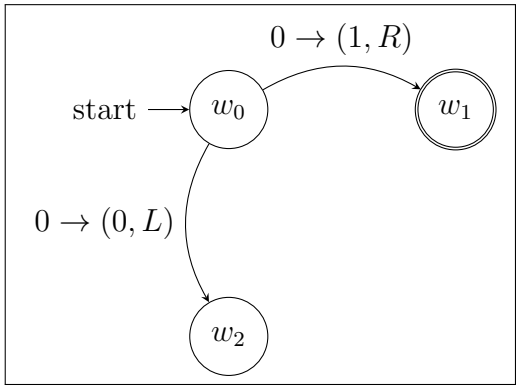
Suppose the machine is in state w_0 and reads the symbol 0. The transition function is defined as:

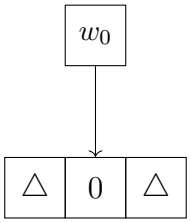
$$V(w_0, 0) = \{(w_1, 1, R),\ (w_2, 0, L)\}$$

This means that the NTM has two possible transitions:

1. Write 1, move the head to the right, and enter the accepting state w_1.
2. Leave the symbol unchanged, move the head to the left, and enter state w_2.

The machine simultaneously explores both transitions, and accepts if at least one computation path reaches an accepting state.





23.4 The Equivalence of NTM and Deterministic Turing Machine (DTM)

An NTM can be simulated by a DTM by systematically exploring all possible computation paths of the NTM. The deterministic machine performs a *breadth-first search* (BFS) over the computation tree of the NTM, ensuring that if any branch leads to an accepting state, the DTM will eventually find it.

Example:

Consider an NTM $M_N = (W, \Gamma, V, w_0, w_{\text{acc}}, w_{\text{rej}})$ defined as follows:

$$W = \{w_0, w_1, w_2, w_{\text{acc}}, w_{\text{rej}}\}, \quad \Gamma = \{0, 1, \sqcup\}$$

The transition function contains nondeterministic choices:

$$V(w_0, 0) = \{(w_1, 0, R),\ (w_2, 0, R)\}$$

$$V(w_1, \sqcup) = \{(w_{\text{acc}}, \sqcup, R)\}, \quad V(w_2, \sqcup) = \{(w_{\text{rej}}, \sqcup, R)\}$$

The NTM accepts an input string if at least one computation path reaches w_{acc}.

1. Computation Tree of the NTM

   For the input string 0, the NTM produces the following computation tree:

$$(w_0, 0) \ \to \ \begin{cases} (w_1, \sqcup) \ \to \ w_{\text{acc}} \\ (w_2, \sqcup) \ \to \ w_{\text{rej}} \end{cases}$$

   Since one branch leads to an accepting state, the NTM accepts the input.

2. DTM Simulation via Breadth-First Search

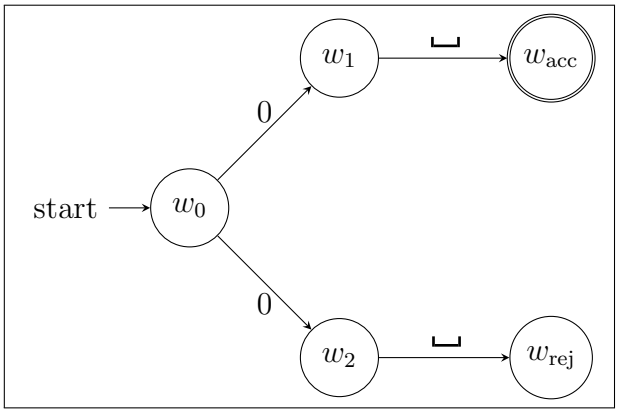
   A DTM M_D simulates M_N by enumerating all possible configurations of M_N level by level. At each step, M_D:

(a) Stores configurations of M_N on its tape.

(b) Expands all configurations at the current depth.

(c) Checks whether any configuration is accepting.

Because the simulation is breadth-first, M_D is guaranteed to discover an accepting configuration at finite depth if one exists.



23.5 Turing Machine as a Problem Solver

A TM can be designed to solve specific computational problems by defining its states, tape alphabet, and transition functions to manipulate input data according to the problem's requirements.

Example:

Let

$$A = \{\langle G \rangle \mid G \text{ is a connected graph}\}$$

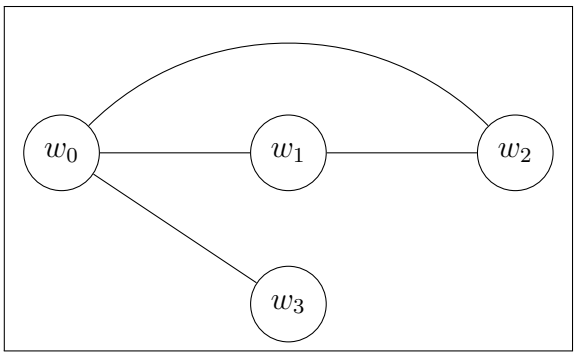
where the encoding of a graph G is given as a list of edges:

$$\langle G \rangle = (\{w_0, w_1\}, \{w_0, w_2\}, \{w_0, w_3\}, \{w_1, w_2\})$$

and the input alphabet is

$$\Sigma = \{(,), \{, \}, ,, x, 0, 1, 2, 3\}$$

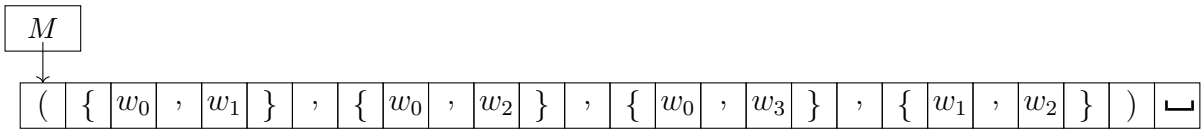
The graph G consists of vertices $\{w_0, w_1, w_2, w_3\}$ and edges connecting vertex pairs as specified above. For clarity, each vertex label w_i is treated as a two-symbol token (w, i) over Σ.



The input graph is encoded on the TM tape as a linear string using symbols from Σ. The encoding is:

$$(\{w_0, w_1\}, \{w_0, w_2\}, \{w_0, w_3\}, \{w_1, w_2\})$$

Initially, the tape contains the encoding followed by blank symbols, with the read/write head positioned at the first cell:



The TM processes the input to determine if the graph is connected by exploring the vertices and edges according to its transition functions. If it finds that all vertices are reachable from any starting vertex, it enters an accepting state; otherwise, it rejects the input.

23.6 Turing Machine as an Enumerator

A Turing machine enumerator is a TM with an attached printer. Instead of reading input and accepting or rejecting, it generates and prints all strings of a language one by one. The machine runs forever, and the language it enumerates is the set of all strings it eventually prints. Formally, a TM enumerator E is defined as:

$$E = (W, \Sigma, \Gamma, V, S, w_{print})$$

where the machine starts with a blank tape and uses its work tape to generate strings, printing each one to the output.

Example:

Let

$$L = \{01\}$$

The enumerator E generates the string 01 repeatedly, printing it to the output tape on each cycle.

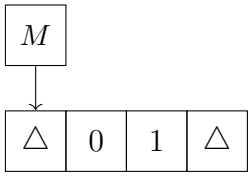
The states of E are:

$$W = \{w_0, w_1, w_2, w_{print}\}$$

where w_0 is the start state and w_{print} triggers the printer to output the current string. The transitions proceed as follows:

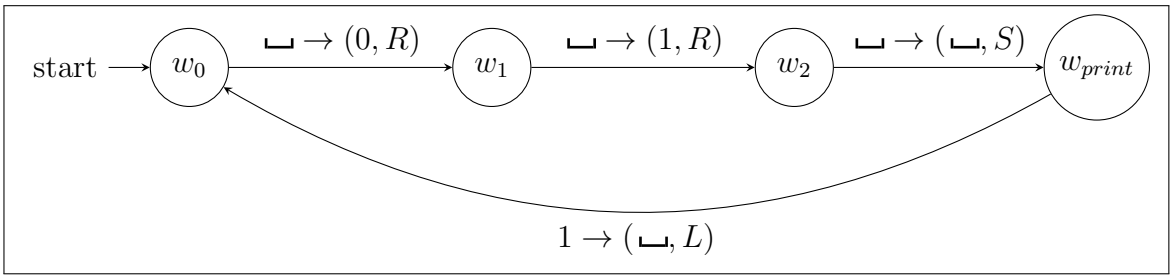
$$\begin{aligned} V(w_0, \sqcup) &= (w_1, 0, R) \\ V(w_1, \sqcup) &= (w_2, 1, R) \\ V(w_2, \sqcup) &= (w_{print}, \sqcup, S) \\ V(w_{print}, 1) &= (w_0, \sqcup, L) \end{aligned}$$

The work tape after one cycle is shown below:



The output tape accumulates the printed strings separated by a delimiter #:

$\triangle$	0	1	#	0	1	#	0	1	#	$\cdots$	$\triangle$



The enumerator never halts, it continues printing 01 indefinitely. A language L is Turing-recognizable if and only if some enumerator enumerates it.

When converting a TM M to an enumerator E, the order in which strings are printed depends on how E is constructed. If $L(M)$ is decidable, then E enumerates strings in lexicographic order, since the decider always halts on every input and E can safely test each string one by one. For example, if $L(M) = \{0^n 1^n \mid n \geq 1\}$, then E prints $01, 0011, 000111, \ldots$ in order, since membership can always be decided in finite time. If $L(M)$ is recognizable but not decidable, then E cannot enumerate in lexicographic order, since some strings may cause M to loop forever. For example, if $L(M) = \{\langle M \rangle \mid M \text{ accepts } \varepsilon\}$, then E must use dovetailing, running the simulation of each string for an increasing number of steps, and printing a string only when its simulation eventually accepts. As a result, strings are enumerated in the order their computations terminate, which is not necessarily lexicographic order.

23.7 The Universal Turing Machine

A Universal Turing Machine (UTM) is a theoretical construct that can simulate any other Turing machine. It takes as input an encoding $\langle M, x \rangle$ of a Turing machine M and an input string x, and simulates the behavior of M on x step-by-step.

Example:

Let M be a Turing machine that flips all bits in a binary string (replacing each 0 with 1 and each 1 with 0), defined as follows:

$$W = \{w_0, w_{\text{acc}}, w_{\text{rej}}\}, \quad \Sigma = \{0, 1\}, \quad \Gamma = \{0, 1, \sqcup\}$$

$$\begin{aligned} V(w_0, 0) &= (w_0, 1, R) \\ V(w_0, 1) &= (w_0, 0, R) \\ V(w_0, \sqcup) &= (w_{\text{acc}}, \sqcup, R) \end{aligned}$$

The UTM U receives the encoding $\langle M, 010 \rangle$ on its tape, where $\langle M \rangle$ is a formal encoding of the transition function of M, followed by the input string 010. U then simulates M on $x = 010$ step-by-step as follows:

$$\langle M, 010\rangle \longrightarrow \boxed{\text{UTM } U} \longrightarrow \text{Simulates } M \text{ on } 010$$

Step	State	Tape	Action
1	w_0	$[w_0]\ \underline{0}\ 1\ 0\ \sqcup$	Read 0, write 1, move Right
2	w_0	$1\ [w_0]\ \underline{1}\ 0\ \sqcup$	Read 1, write 0, move Right
3	w_0	$1\ 0\ [w_0]\ \underline{0}\ \sqcup$	Read 0, write 1, move Right
4	w_0	$1\ 0\ 1\ [w_0]\ \underline{\sqcup}$	Read $\sqcup$, move Right
5	w_{acc}	$1\ 0\ 1\ \sqcup\ [w_{\text{acc}}]$	**Accept**

After simulation, U halts and accepts, having transformed the tape contents from 010 to 101, correctly replicating the behavior of M.

24 Decidability and Undecidability

1. Decidable Languages

   A language L is decidable if it is a recursive language. All decidable languages are recursive and vice versa.

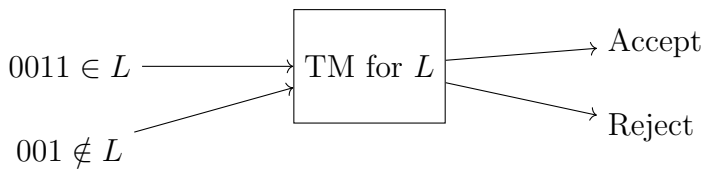
   Example:

   $$L = \{0^n 1^n \mid n \geq 0\}$$

   A TM can decide this language by:

   (a) Scanning left to right, marking each 0 and matching it with a 1
   (b) Accepting if all 0s and 1s are matched
   (c) Rejecting otherwise

   The TM always halts and gives a definite yes/no answer.

   Always halts with a definite yes/no answer.

2. Partially Decidable Languages

   A language L is partially decidable if there exists a TM that accepts all strings in L and either rejects or loops indefinitely on strings not in L. All partially decidable languages are recursively enumerable and vice versa.

   Example:

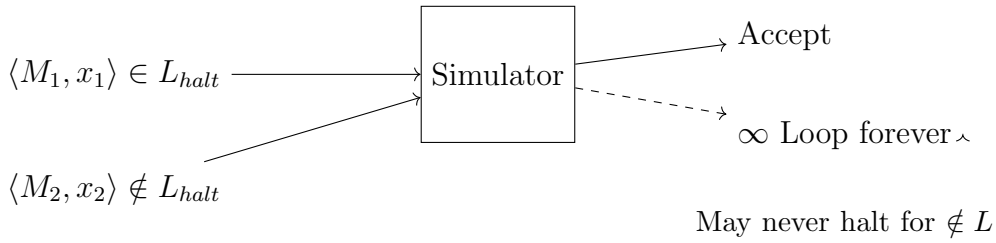
   $$L_{halt} = \{\langle M, x\rangle \mid \text{TM } M \text{ halts on input } x\}$$

   This is the halting problem. A TM can simulate M on x:

(a) If M halts on x, the simulator accepts (recognizes the string is in L_{halt})

(b) If M loops forever on x, the simulator also loops forever (never rejects)

We can recognize strings in the language but cannot decide the language.



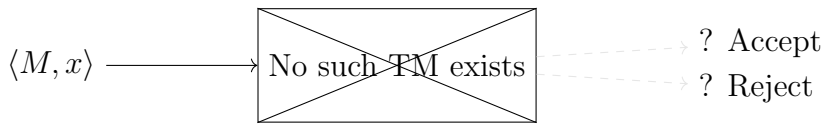
3. Undecidable Languages

   A language L is undecidable if there is no TM that can decide L. This means that no Turing machine can accept all strings in L and reject all strings not in L.

   Example:

$$\overline{L_{halt}} = \{\langle M, x \rangle \mid \text{TM } M \text{ does not halt on input } w\}$$

   This is the complement of the halting problem. Unlike L_{halt}, which is partially decidable, $\overline{L_{halt}}$ is not even partially decidable. If a TM M does not halt on w, no simulator can ever confirm this, since it would have to run forever waiting for M to halt. Therefore no TM can decide or even recognize $\overline{L_{halt}}$, making it strictly undecidable.



   Impossible to decide! No algorithm can solve this for all inputs.

25 Halting Problem

The Halting Problem is the decision problem of determining whether a given program halts on a given input. Formally, let P be a program and I an input to that program. The halting problem asks whether $P(I)$ terminates in a finite number of steps.

Turing proved that there exists no algorithm H such that

$$H(P, I) = \begin{cases} 1, & \text{if program } P \text{ halts on input } I, \\ 0, & \text{if program } P \text{ does not halt on input } I. \end{cases}$$

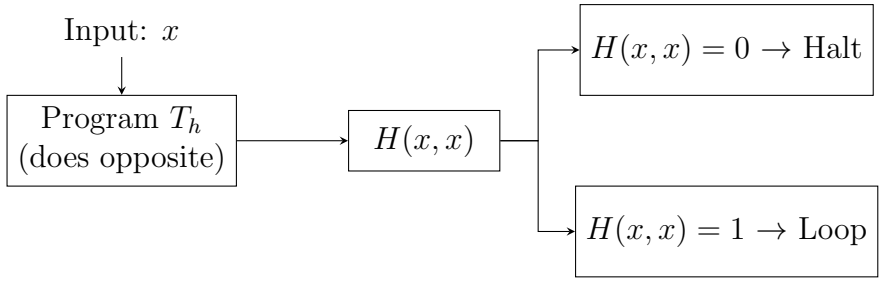
for all programs P and inputs I.

Example:

Define a program T_∞ such that

$$T_\infty(x) = \text{loop forever.}$$

Then $T_\infty(x)$ does not halt for any input x. Now define a program T_h as follows:



Running T_h on its own description leads to a contradiction:

1. If $H(T_h, T_h) = 1$, then by definition of T_h, it loops forever. This contradicts the assumption that H predicted it halts.
2. If $H(T_h, T_h) = 0$, then by definition of T_h, it halts. This contradicts the assumption that H predicted it does not halt.

Therefore, the Halting Problem is undecidable. No algorithm H can correctly decide whether all programs halt on all inputs.

26 The Diagonalization Method

One method to show that a language L is not decidable is by using the *diagonalization method*. The idea is to assume that there exists a Turing machine that decides L, and then construct a new Turing machine that differs from every machine in the assumed list, leading to a contradiction.

Example:

Let

$$L_\chi = \{\langle M, x\rangle \mid M \text{ is a TM and } M \text{ accepts } x\}.$$

Suppose, for contradiction, that L_χ is decidable. Then there exists a Turing machine H such that for all $\langle M, x\rangle$:

$$H(\langle M, x\rangle) = \begin{cases} 1, & \text{if } M \text{ accepts } x, \\ 0, & \text{otherwise.} \end{cases}$$

Since all Turing machines and inputs can be encoded as strings over $\{0, 1\}$, we may enumerate all Turing machines:

$$M_1, M_2, M_3, M_4, \ldots$$

and all possible inputs:

$$x_1, x_2, x_3, x_4, \ldots$$

We organize the computation outcomes into an infinite table, where row i corresponds to M_i and column j corresponds to input x_j:

	x_1	x_2	x_3	x_4	x_5	$\cdots$
M_1	$\boxed{0}$	1	0	1	1	$\cdots$
M_2	1	$\boxed{1}$	0	0	1	$\cdots$
M_3	0	1	$\boxed{0}$	1	0	$\cdots$
M_4	1	0	1	$\boxed{1}$	0	$\cdots$
M_5	0	1	0	1	$\boxed{0}$	$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

where the entry $L_{i,j}$ in row i and column j is defined as

$$L_{i,j} = \begin{cases} 1, & \text{if } M_i \text{ accepts } x_j, \\ 0, & \text{otherwise.} \end{cases}$$

The diagonal entries $L_{i,i}$ record whether M_i accepts its own index input x_i. We now define a new Turing machine $\bar{S}$ that *disagrees* with every M_i on input x_i by flipping the diagonal:

$$\bar{S}(x_i) = \begin{cases} 0, & \text{if } H(\langle M_i, x_i \rangle) = 1, \\ 1, & \text{if } H(\langle M_i, x_i \rangle) = 0. \end{cases}$$

Since H decides TM, the machine $\bar{S}$ is well-defined and always halts. By construction, for every i,

$$\bar{S}(x_i) \neq M_i(x_i).$$

1. If $\bar{S}(x_k)$ accepts $\rightarrow \bar{S} = M_k$ accepts x_k, but by definition of $\bar{S}$, M_k must reject x_k. Contradiction.
2. If $\bar{S}(x_k)$ rejects $\rightarrow \bar{S} = M_k$ rejects x_k, but by definition of $\bar{S}$, M_k must accept x_k. Contradiction.

Hence,

$$\boxed{L_\chi \text{ is undecidable.}}$$

27 Reducibility

In short, reducibility is a method where we solve problem X by reducing it to problem Y, such that any solution to Y can be used to solve X.

27.1 Mapping Reducibility

Formally, $X \leq_m Y$ means there exists a computable function f such that for all inputs x, $x \in X \leftrightarrow f(x) \in Y$.

Example 1:

Let

$$X = \{\langle M, x\rangle \mid M \text{ is a TM and } M \text{ halts on } x\}$$
$$Y = \{\langle M\rangle \mid M \text{ is a TM and } M \text{ halts on } \varepsilon\}$$

We show $X \leq_m Y$ by constructing a computable function f such that:

$$\langle M, x\rangle \in X \leftrightarrow f(\langle M, x\rangle) \in Y$$

Given $\langle M, x\rangle$, construct M' where:

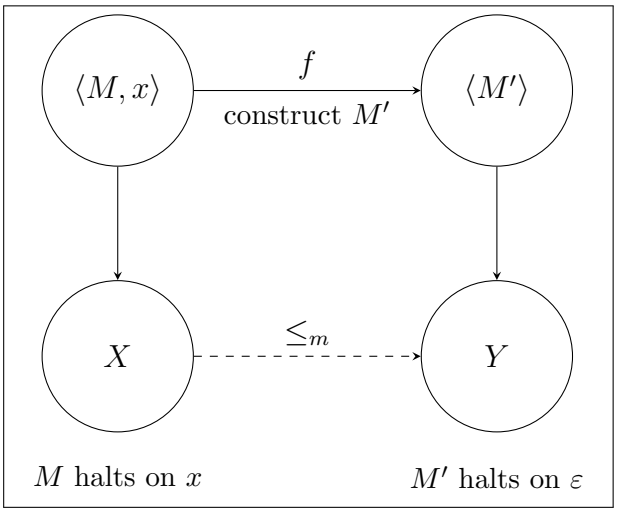
$$M'(y) = M(x) \quad \text{for all inputs } y$$

Then,

$$\langle M, x\rangle \in X \leftrightarrow M \text{ halts on } x \leftrightarrow M' \text{ halts on } \varepsilon \leftrightarrow \langle M'\rangle \in Y$$

Therefore,

$$X \leq_m Y$$



Theorem. Let $X \leq_m Y$. Then the following hold:

1. X decidable $\not\rightarrow Y$ decidable
2. X undecidable $\rightarrow Y$ undecidable
3. Y decidable $\rightarrow X$ decidable
4. Y undecidable $\not\rightarrow X$ undecidable

Example 2:

$$A_{TM} = \{\langle M, x\rangle \mid M \text{ is a TM and } M \text{ accepts } x\}$$
$$HALT_{TM} = \{\langle M, x\rangle \mid M \text{ is a TM and } M \text{ halts on } x\}$$

We show $A_{TM} \leq_m HALT_{TM}$. We construct a computable function f such that:

$$\langle M, x\rangle \in A_{TM} \leftrightarrow f(\langle M, x\rangle) \in HALT_{TM}$$

Given an instance $\langle M, x\rangle$ of A_{TM}, we construct a new Turing machine M' as follows:

M' = on input x:

1. Run M on x
2. If M accepts, accept
3. If M rejects, loop forever

The reduction maps $\langle M, x\rangle \to \langle M', x\rangle$. We verify the correctness:

($\to$) Suppose $\langle M, x\rangle \in A_{TM}$, i.e., M accepts x. Then M' simulates M on x, reaches the accept state, and halts. So $\langle M', x\rangle \in HALT_{TM}$.

($\leftarrow$) Suppose $\langle M', x\rangle \in HALT_{TM}$, i.e., M' halts on x. By construction, M' only halts if M accepts x (otherwise it loops). So M accepts x, meaning $\langle M, x\rangle \in A_{TM}$.

Therefore:

$$\langle M, x\rangle \in A_{TM} \leftrightarrow \langle M', x\rangle \in HALT_{TM}$$

Since $f : \langle M, x\rangle \to \langle M', x\rangle$ is computable, we conclude $A_{TM} \leq_m HALT_{TM}$.

Example 3:

$$A_{TM} = \{\langle M, x\rangle \mid M \text{ is a TM and } M \text{ accepts } x\}$$
$$\overline{A_{TM}} = \{\langle M, x\rangle \mid M \text{ is a TM and } M \text{ does not accept } x\}$$

We show $A_{TM} \leq_m \overline{A_{TM}}$. We construct a computable function f such that:

$$\langle M, x\rangle \in A_{TM} \leftrightarrow f(\langle M, x\rangle) \in \overline{A_{TM}}$$

Given an instance $\langle M, x\rangle$ of A_{TM}, we construct a new Turing machine M' as follows:

M' = on input x:

1. Run M on x
2. If M accepts, reject
3. If M rejects, accept

The reduction maps $\langle M, x\rangle \to \langle M', x\rangle$. We verify the correctness:

($\to$) Suppose $\langle M, x\rangle \in A_{TM}$, i.e., M accepts x. Then M' simulates M on x and rejects. So $\langle M', x\rangle \notin A_{TM}$, meaning $\langle M', x\rangle \in \overline{A_{TM}}$.

($\leftarrow$) Suppose $\langle M', x\rangle \in \overline{A_{TM}}$, i.e., M' does not accept x. By construction, M' rejects whenever M accepts. So M accepts x, meaning $\langle M, x\rangle \in A_{TM}$.

Therefore:

$$\langle M, x\rangle \in A_{TM} \leftrightarrow \langle M', x\rangle \in \overline{A_{TM}}$$

Since $f : \langle M, x\rangle \to \langle M', x\rangle$ is computable, we conclude $A_{TM} \leq_m \overline{A_{TM}}$.

Example 4:

$$HALT_{TM} = \{\langle M, x\rangle \mid M \text{ is a TM and } M \text{ halts on } x\}$$
$$E_{TM} = \{\langle M\rangle \mid M \text{ is a TM and } L(M) = \varnothing\}$$

We show $HALT_{TM} \leq_m E_{TM}$. We construct a computable function f such that:

$$\langle M, x\rangle \in HALT_{TM} \leftrightarrow f(\langle M, x\rangle) \in E_{TM}$$

Given an instance $\langle M, x\rangle$ of $HALT_{TM}$, we construct a new Turing machine M' as follows:

M' = on input y:
1. Run M on x for $\lvert y\rvert$ steps
2. If M halts, accept
3. Otherwise, reject

The reduction maps $\langle M, x\rangle \to \langle M'\rangle$. We verify the correctness:

($\to$) Suppose $\langle M, x\rangle \in HALT_{TM}$, i.e., M halts on x in k steps. Then for any y with $|y| \geq k$, M' accepts. So $L(M') \neq \varnothing$, meaning $\langle M'\rangle \notin E_{TM}$.

($\leftarrow$) Suppose $\langle M'\rangle \notin E_{TM}$, i.e., $L(M') \neq \varnothing$. Then there exists some y such that M' accepts y, meaning M halted on x within $|y|$ steps. So $\langle M, x\rangle \in HALT_{TM}$.

Therefore:

$$\langle M, x\rangle \in HALT_{TM} \leftrightarrow \langle M'\rangle \notin E_{TM}$$

Since $f : \langle M, x\rangle \to \langle M'\rangle$ is computable, we conclude $HALT_{TM} \leq_m E_{TM}$.

28 Post Correspondence Problem (PCP)

The Post Correspondence Problem (PCP) is a decision problem defined as follows.

A PCP instance consists of a finite set of dominoes

$$P = \left\{ \left[\frac{x_1}{y_1}\right], \left[\frac{x_2}{y_2}\right], \ldots, \left[\frac{x_n}{y_n}\right] \right\},$$

where each $x_i, y_i \in \Sigma^*$ for some finite alphabet Σ. The problem asks whether there exists a nonempty sequence of indices

$$i_1, i_2, \ldots, i_k \quad (k \geq 1)$$

such that

$$x_{i_1} x_{i_2} \cdots x_{i_k} = y_{i_1} y_{i_2} \cdots y_{i_k}.$$

Example 1

Consider the following set of dominoes:

$$P = \left\{ \left[\frac{1}{01}\right], \left[\frac{10}{1}\right], \left[\frac{0}{0}\right] \right\}.$$

We want to determine whether this PCP instance has a solution. One possible sequence of indices is $[2, 1, 3]$:

$$\left[\frac{10}{1}\right] \cdot \left[\frac{1}{01}\right] \cdot \left[\frac{0}{0}\right] = \frac{10 \cdot 1 \cdot 0}{1 \cdot 01 \cdot 0} = \frac{1010}{1010}$$

Since both concatenations are equal, this sequence is a solution, and the answer to this PCP instance is *yes*.

Example 2

Consider the following PCP instance:

$$P = \left\{ \left[\frac{10}{1}\right], \left[\frac{011}{10}\right], \left[\frac{01}{011}\right] \right\}.$$

We test several sequences of indices.

1. Sequence $[1, 2, 3]$:

$$\left[\frac{10}{1}\right] \cdot \left[\frac{011}{10}\right] \cdot \left[\frac{01}{011}\right] = \frac{10 \cdot 011 \cdot 01}{1 \cdot 10 \cdot 011} = \frac{1001101}{110011}$$

   These strings are not equal.

2. Sequence $[2, 1, 3]$:

$$\left[\frac{011}{10}\right] \cdot \left[\frac{10}{1}\right] \cdot \left[\frac{01}{011}\right] = \frac{011 \cdot 10 \cdot 01}{10 \cdot 1 \cdot 011} = \frac{0111001}{101011}$$

   These strings are not equal.

3. Sequence $[3, 3, 1, 2]$:

$$\left[\frac{01}{011}\right] \cdot \left[\frac{01}{011}\right] \cdot \left[\frac{10}{1}\right] \cdot \left[\frac{011}{10}\right] = \frac{01 \cdot 01 \cdot 10 \cdot 011}{011 \cdot 011 \cdot 1 \cdot 10} = \frac{01011011}{01101110}$$

   These strings are not equal.

Hence, no solution has been found for this PCP instance.

Example 3

Consider the PCP instance

$$P = \left\{ \left[\frac{0}{00}\right], \left[\frac{1}{11}\right] \right\}.$$

For any sequence of indices $[i_1, i_2, \ldots, i_k]$, suppose the sequence contains n occurrences of index 1 and m occurrences of index 2.

Then

$$\begin{aligned} x_{i_1} x_{i_2} \cdots x_{i_k} &= \text{a string of length } n + m, \\ y_{i_1} y_{i_2} \cdots y_{i_k} &= \text{a string of length } 2(n + m). \end{aligned}$$

Since the lengths of the two strings are different, no solution exists for this PCP instance.

28.1 Modified Post Correspondence Problem (MPCP)

The *Modified Post Correspondence Problem* (MPCP) is a constrained variant of PCP in which the first domino of any solution is fixed. Formally, given a finite list of domino pairs

$$P = \left\{ \left[\frac{u_1}{v_1}\right], \left[\frac{u_2}{v_2}\right], \ldots, \left[\frac{u_n}{v_n}\right] \right\}, \quad u_i, v_i \in \Sigma^*,$$

an *MPCP solution* is a sequence of indices $i_2, i_3, \ldots, i_k \in \{1, \ldots, n\}$ such that

$$u_1\, u_{i_2}\, u_{i_3} \cdots u_{i_k} \;=\; v_1\, v_{i_2}\, v_{i_3} \cdots v_{i_k}.$$

That is, the first domino (u_1, v_1) is *required* as the starting piece; the remaining indices $i_2, \ldots, i_k$ may be chosen freely from $\{1, \ldots, n\}$ with repetition. This is the only difference from ordinary PCP, where all indices may be chosen freely.

Example 1:

Consider the following MPCP instance over $\Sigma = \{0, 1\}$:

$$\left[\frac{u_1}{v_1}\right] = \left[\frac{10}{1}\right], \qquad \left[\frac{u_2}{v_2}\right] = \left[\frac{1}{11}\right], \qquad \left[\frac{u_3}{v_3}\right] = \left[\frac{01}{00}\right].$$

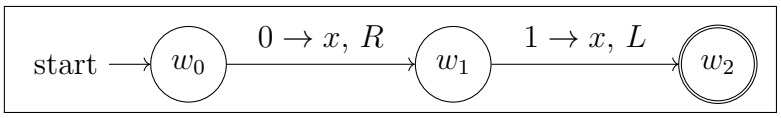
Since domino 1 is required to appear first, we try the sequence $1, 3, 2$:

$$\left[\frac{10}{1}\right] \cdot \left[\frac{01}{00}\right] \cdot \left[\frac{1}{11}\right] \;=\; \frac{10 \cdot 01 \cdot 1}{1 \cdot 00 \cdot 11} \;=\; \frac{10011}{10011}$$

The top and bottom concatenations are equal, so the sequence $1, 3, 2$ is a valid MPCP solution. Note that domino 1 appears first, as required by MPCP.

Example 2:

Let



where $\Gamma = \{0, 1, x, \triangle\}$ and the input string is $w = 010$.

1. Step 1: Start domino.

   We place the initial domino. The top starts with just #, while the bottom encodes the full start configuration $S_1 = \#\ w_0\ 0\ 1\ 0\ \#$. The bottom is always one configuration ahead of the top; this is the invariant that drives the whole construction.

$$\left[\frac{\#}{\#\ w_0\ 0\ 1\ 0\ \#}\right]$$

2. Step 2: Transition $V(w_0, 0) = (w_1, x, R)$.

   The machine reads 0 under state w_0, writes x, and moves right to state w_1, producing configuration $S_2 = \#\ x\ w_1\ 1\ 0\ \#$. We apply the right-move transition domino, which appends $w_0\ 0$ to the top and $x\ w_1$ to the bottom, encoding this transition. The copying dominoes then copy the remaining symbols 1, 0, # so the top catches up through the end of S_1 while the bottom finishes building S_2.

$$\left[\frac{w_0\ 0}{x\ w_1}\right] \qquad \left[\frac{1}{1}\right] \qquad \left[\frac{0}{0}\right] \qquad \left[\frac{\#}{\#}\right]$$

3. Step 3: Transition $V(w_1, 1) = (w_2, x, L)$.

   The machine reads 1 under state w_1, writes x, and moves left to state w_2, producing the accepting configuration $S_3 = \#\ w_2\ x\ x\ 0\ \#$. Since the head moves left, the domino must also carry the symbol y immediately to the left of the head; it cannot be ignored because it determines where w_2 lands. We therefore need one domino for every possible $y \in \Gamma$. In our run $y = x$, since cell 1 was written x in Step 2. Since $\Gamma = \{0, 1, x, \triangle\}$, the four left-move dominoes are:

$$\left[\frac{0\ w_1\ 1}{w_2\ 0\ x}\right] \qquad \left[\frac{1\ w_1\ 1}{w_2\ 1\ x}\right] \qquad \left[\frac{x\ w_1\ 1}{w_2\ x\ x}\right] \qquad \left[\frac{\triangle\ w_1\ 1}{w_2\ \triangle\ x}\right]$$

   The active one in this run is $\left[\frac{x\ w_1\ 1}{w_2\ x\ x}\right]$. The copying dominoes then copy 0 and # to complete S_3.

$$\left[\frac{0}{0}\right] \qquad \left[\frac{\#}{\#}\right]$$

4. Step 4: Copying dominoes.

   Any symbol not involved in a transition must be copied unchanged from S_i to S_{i+1}. These dominoes propagate untouched tape cells, keeping the configuration boundaries aligned.

$$\left[\frac{0}{0}\right] \qquad \left[\frac{1}{1}\right] \qquad \left[\frac{x}{x}\right] \qquad \left[\frac{\triangle}{\triangle}\right] \qquad \left[\frac{\#}{\#}\right]$$

5. Step 5: Acceptance dominoes.

   Once w_2 appears in S_3, the machine has accepted. We now erase all remaining tape symbols so that the top and bottom strings can become equal. The acceptance dominoes consume one symbol at a time from either side of w_2, replacing the pair with just w_2, effectively shrinking the configuration until only the separators remain.

   For symbols to the right of w_2:

$$\left[\frac{w_2\ 0}{w_2}\right] \qquad \left[\frac{w_2\ 1}{w_2}\right] \qquad \left[\frac{w_2\ x}{w_2}\right] \qquad \left[\frac{w_2\ \triangle}{w_2}\right]$$

   For symbols to the left of w_2:

$$\left[\frac{0\ w_2}{w_2}\right] \qquad \left[\frac{1\ w_2}{w_2}\right] \qquad \left[\frac{x\ w_2}{w_2}\right] \qquad \left[\frac{\triangle\ w_2}{w_2}\right]$$

   Once all tape symbols have been erased, only $\#\ w_2\ \#$ remains. The termination domino collapses this to a single #, making the top and bottom strings equal and completing the solution:

$$\left[\frac{\#\ w_2\ \#}{\#}\right]$$

6. Step 6: Complete solution.

   Putting it all together, the full sequence of dominoes is as follows.

   Round 1: top catches up through S_1, bottom builds S_2.

$$\left[\frac{\#}{\# \; w_0 \; 0 \; 1 \; 0 \; \#}\right] \left[\frac{w_0 \; 0}{x \; w_1}\right] \left[\frac{1}{1}\right] \left[\frac{0}{0}\right] \left[\frac{\#}{\#}\right]$$

   Round 2: top catches up through S_2, bottom builds S_3.

$$\left[\frac{\#}{\# \; x \; w_1 \; 1 \; 0 \; \#}\right] \left[\frac{x \; w_1 \; 1}{w_2 \; x \; x}\right] \left[\frac{0}{0}\right] \left[\frac{\#}{\#}\right]$$

   Round 3: erase first x from S_3.

$$\left[\frac{\#}{\# \; w_2 \; x \; x \; 0 \; \#}\right] \left[\frac{w_2 \; x}{w_2}\right] \left[\frac{x}{x}\right] \left[\frac{0}{0}\right] \left[\frac{\#}{\#}\right]$$

   Round 4: erase second x.

$$\left[\frac{\#}{\# \; w_2 \; x \; 0 \; \#}\right] \left[\frac{w_2 \; x}{w_2}\right] \left[\frac{0}{0}\right] \left[\frac{\#}{\#}\right]$$

   Round 5: erase 0.

$$\left[\frac{\#}{\# \; w_2 \; 0 \; \#}\right] \left[\frac{w_2 \; 0}{w_2}\right] \left[\frac{\#}{\#}\right]$$

   Round 6: termination.

$$\left[\frac{\# \; w_2 \; \#}{\#}\right]$$

All tape symbols have been erased. The termination domino fires, making the accumulated top and bottom strings identical. The MPCP solution is complete, witnessing that M accepts input 010.

29 Linear Bounded Automaton (LBA)

A *Linear Bounded Automaton* (LBA) is a restricted model of computation based on the Turing machine, in which the tape head is confined to the portion of the tape occupied by the input. Unlike a full Turing machine, which may use an unbounded amount of tape, an LBA may only read and write on the cells that initially contain the input string; it cannot move beyond the left or right endmarkers that delimit the input.

The tape head begins at the leftmost cell of the input. The machine operates like a Turing machine, except that the transition function must never move the head left of the left endmarker $\triangle$ or right of the right endmarker $\triangle$. Formally, if the head is scanning $\triangle$ it may not move left, and if it is scanning $\triangle$ it may not move right. The tape is therefore bounded by $|\,x\,|$ cells, where x is the input string.

Because the tape of an LBA is bounded by the length of the input, the total number of distinct configurations is finite. A configuration is a triple (w, α, i) where $w \in W$ is the current state, $\alpha \in \Gamma^{|x|}$ is the tape contents, and i is the head position. The number of configurations is at most

$$|W| \times |\Gamma|^{|x|} \times |x|,$$

which is finite for any fixed input x. This finiteness has direct decidability consequences.

Example:

Let

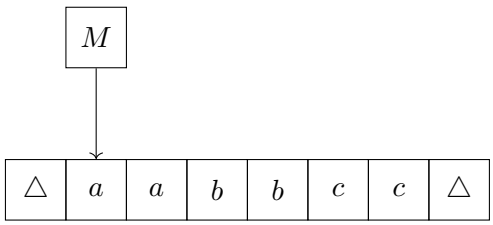
$$L = \{a^n b^n c^n \mid n \geq 1\}$$

Procedure:

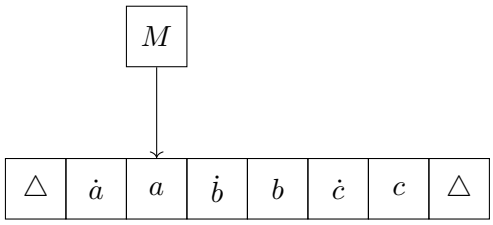
1. Scan left to right. If all symbols are marked ($\dot{a}$, $\dot{b}$, $\dot{c}$), accept.
2. Find the leftmost unmarked a. If none exists but unmarked b or c remain, reject.
3. Mark that a with $\dot{a}$. Continue scanning right to find the leftmost unmarked b.
4. If no unmarked b exists, reject. Otherwise mark it with $\dot{b}$. Continue scanning right to find the leftmost unmarked c.
5. If no unmarked c exists, reject. Otherwise mark it with $\dot{c}$.
6. Return the head to the left endmarker $\triangle$ and repeat from Step 1.

Trace on input *aabbcc*:

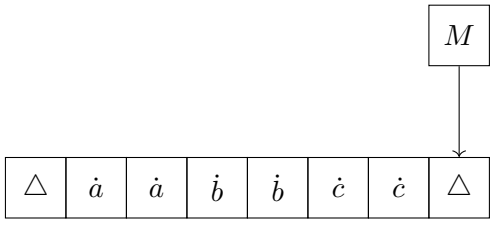
Initial tape:



Round 1: mark the first a, b, c.



Round 2: mark the second a, b, c.



Round 3: all symbols are marked. The machine accepts.

30 Recursion Theorem

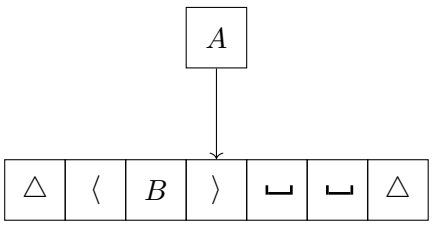
Formally, the Recursion Theorem states that for any Turing machine M, there exists a Turing machine R such that for every input x,

$$R(x) = M(\langle R \rangle, x)$$

Intuitively, R can obtain its own description $\langle R \rangle$ and pass it to M as part of the input.
Example 1:
We construct a Turing machine SELF that prints its own description $\langle \text{SELF} \rangle$.

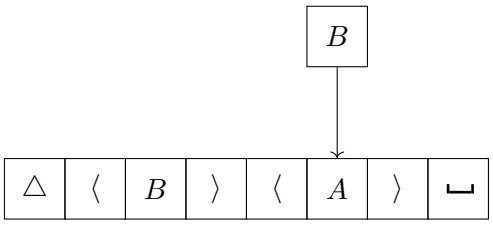
> SELF = on input x:
>
> 1. Obtain own description $\langle \text{SELF} \rangle = \langle AB \rangle$ using parts A and B:
>
> A: Write $\langle B \rangle$ onto the tape.
>
> B: Read $\langle B \rangle$ from the tape; compute $f(\langle B \rangle) = \langle A \rangle$.
>
> 2. Combine $\langle A \rangle$ and $\langle B \rangle$ to form $\langle AB \rangle = \langle \text{SELF} \rangle$.
>
> 3. Write $\langle \text{SELF} \rangle$ on the tape and halt.

1. Step 1: Part A runs. The tape is blank; A writes $\langle B \rangle$ onto the tape.



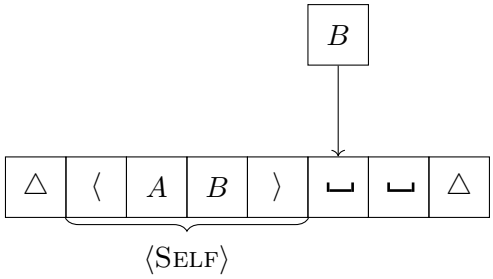
 A finishes; tape holds $\langle B \rangle$. Control passes to B.

2. Step 2: Part B reads $\langle B \rangle$ from the tape and computes $f(\langle B \rangle) = \langle A \rangle$.



 B has written $\langle A \rangle$ next to $\langle B \rangle$. Both descriptions are now on the tape.

3. Step 3: B combines $\langle A \rangle$ and $\langle B \rangle$ into $\langle AB \rangle = \langle \text{SELF} \rangle$.



B halts. The tape contains $\langle AB \rangle = \langle \textsc{Self} \rangle$: Self has printed its own description.

Example 2:
Suppose for contradiction that H is a TM that decides A_{TM}, i.e.,

$$H(\langle M, x \rangle) = \begin{cases} \textit{accept} & \text{if } M \text{ accepts } x \\ \textit{reject} & \text{if } M \text{ does not accept } x \end{cases}$$

We construct a TM B that takes input $\langle M \rangle$ and does the following:

> B = on input $\langle M \rangle$:
>
> 1. Obtain $\langle B \rangle$ via the Recursion Theorem
> 2. Run H on $\langle B, \langle M \rangle \rangle$
> 3. If H accepts, *reject*
> 4. If H rejects, *accept*

Now we run B on input $\langle B \rangle$:

$$H(\langle B, \langle B \rangle \rangle) = \textit{accept} \rightarrow B \text{ rejects } \langle B \rangle. \text{Contradiction.}$$
$$H(\langle B, \langle B \rangle \rangle) = \textit{reject} \rightarrow B \text{ accepts } \langle B \rangle. \text{Contradiction.}$$

In both cases we reach a contradiction. Therefore no such H exists, and A_{TM} is undecidable.

31 Time Complexity

Example 1:
Suppose we have an array like in the following example

3	7	1	9	4	6	2	8	5

and we want to sort it in ascending order. One simple algorithm is Selection Sort:

> Selection Sort = on input array A of length n:
>
> 1. For $i = 1$ to n:
> (a) Find the smallest element in $A[i \ldots n]$.
> (b) Swap it with $A[i]$.

As we can see that in order to sort the number in the array, we need to find the smallest element in the unsorted portion of the array and swap it with the first element of the unsorted portion. This process is repeated until the entire array is sorted. For that array then we sort it in the following way:

Pass 1: scan $A[1..9]$ to find minimum $= 1$, swap with $A[1]$:

1	7	3	9	4	6	2	8	5

Pass 2: scan $A[2..9]$ to find minimum $= 2$, swap with $A[2]$:

1	2	3	9	4	6	7	8	5

Pass 3: scan $A[3..9]$ to find minimum $= 3$, already in place:

1	2	3	9	4	6	7	8	5

Pass 4: scan $A[4..9]$ to find minimum $= 4$, swap with $A[4]$:

1	2	3	4	9	6	7	8	5

Pass 5: scan $A[5..9]$ to find minimum $= 5$, swap with $A[5]$:

1	2	3	4	5	6	7	8	9

Passes 6–8: elements $6, 7, 8$ are already in place; no swaps needed. The array is now fully sorted.

In total, selection sort performs 8 passes (i.e. $n - 1$ passes for $n = 9$). In each pass i, it scans $n - i$ elements to find the minimum, giving a total number of comparisons of:

$$(n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2} = \frac{9 \cdot 8}{2} = 36 \text{ comparisons.}$$

From this example we can examine that if an array have n elements, then selection sort will perform $\frac{n(n-1)}{2}$ comparisons in the worst case. This is a quadratic function of n, so we say that selection sort has a time complexity. Then, we can conclude that running time generally depends on the size of the input.

Example 2:

Suppose we have an array like in the following example

1	1	0	0	0

Since in the array the number of 1s and 0s are not equal, we want to insert another 1 to make them equal. The algorithm shifts all 0s one position to the right to make room, then inserts a 1 in the middle:

1	1	1	0	0	0	$\cdots$

To insert one element, we must shift 3 elements (all the 0s) one position to the right. In general, if the array has n elements and we insert at position i, we must shift $n - i$ elements. In the worst case, inserting at the beginning requires shifting all n elements.

Therefore, from the two examples above, we can see that the number of steps an algorithm takes depends on the size of the input n. We capture this relationship by defining $f(n)$ as the function that counts the number of steps the algorithm takes on an input of size n, and we call this the *time complexity* of the algorithm.

31.1 Growth rate

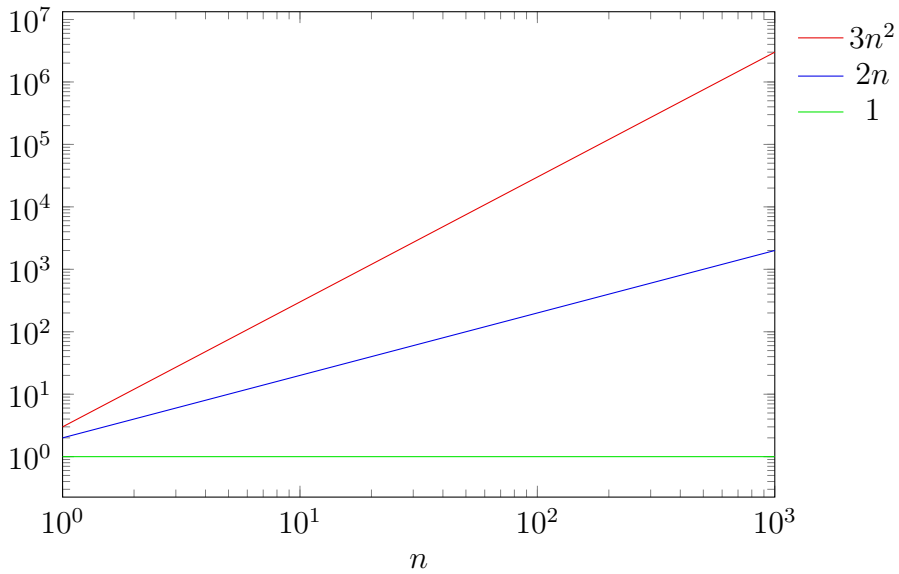
When analyzing algorithms, we are interested in how the running time grows as the input size n increases. Rather than counting the exact number of steps, we focus on the dominant term, the part that grows the fastest and eventually overshadows all other terms.

Example 1:

Suppose we have an algorithm that takes $f(n) = 3n^2 + 2n + 1$ steps to complete on an input of size n. As n grows larger, we can observe how each term contributes to the total:

n	$3n^2$	%	$2n$	%	1	$f(n)$
1	3	50%	2	33%	1	6
10	300	93%	20	6%	1	321
100	30000	99.3%	200	0.7%	1	30201
1000	3000000	99.9%	2000	0.1%	1	3002001
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

As n becomes large, the $3n^2$ term dominates the others. At $n = 1000$, it already accounts for 99.9% of the total running time, while $2n$ shrinks to 0.1% and the constant 1 becomes completely negligible. This suggests that for large n, the lower-order terms and constants have little effect on the overall growth of $f(n)$. What truly matters is the fastest growing term, which in this case is n^2.



31.2 Big O Notation

Big O notation provides a way to describe the *upper bound* of an algorithm's growth rate, focusing on the dominant term and ignoring constants and lower-order terms.

Formally,

$$f(n) = O(g(n)) \;\leftrightarrow\; \exists\, c > 0,\ \exists\, n_0 \text{ s.t. } \forall\, n \geq n_0,\ f(n) \leq c \cdot g(n).$$

Example:
Suppose

$$f(n) = 4n + 3 \quad \wedge \quad g(n) = n$$

To show that $f(n) = O(g(n))$, we need to find constants c and n_0 such that for all $n \geq n_0$, the inequality $f(n) \leq c \cdot g(n)$ holds.

Firstly, substituting the functions, we obtain

$$4n + 3 \leq c \cdot n.$$

Choose $c = 5$. Then we need

$$4n + 3 \leq 5n.$$

Subtracting $4n$ from both sides gives

$$3 \leq n.$$

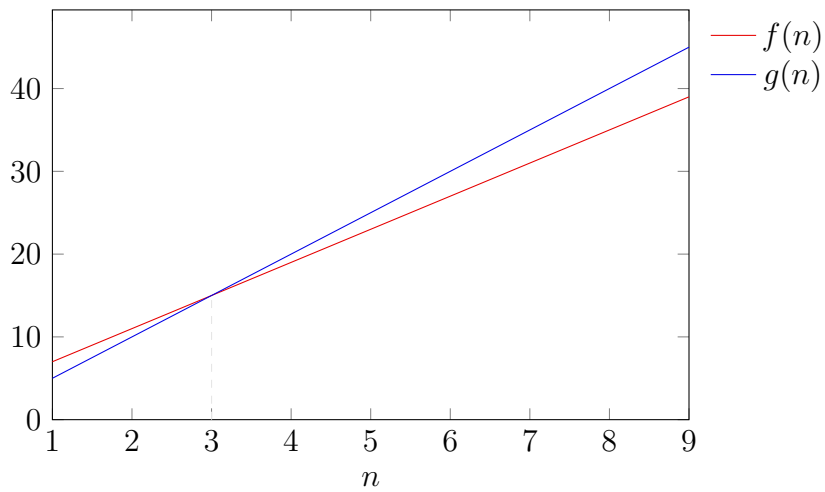
Thus, the inequality holds for all $n \geq 3$. Taking $c = 5$ and $n_0 = 3$, we conclude that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$. Therefore

$$\boxed{f(n) = O(g(n))}$$

Comparing $f(n) = 4n + 3$ against $c \cdot g(n) = 5n$ for $n = 1, \ldots, 9$, with $c = 5$:

n	$f(n)$	$g(n)$
1	7	5
2	11	10
3	15	15
4	19	20
5	23	25
6	27	30
7	31	35
8	35	40
9	39	45
$\vdots$	$\vdots$	$\vdots$

Intuitively,



We claim that both functions grow linearly, and that $f(n)$ is upper-bounded by $c \cdot g(n)$ for all sufficiently large n.

31.3 Big Ω Notation

Big Ω notation provides a way to describe the *lower bound* of an algorithm's growth rate, it captures the best-case or minimum rate at which a function grows.

Formally,

$$f(n) = \Omega(g(n)) \;\leftrightarrow\; \exists\, c > 0,\ \exists\, n_0 \text{ s.t. } \forall\, n \geq n_0,\ f(n) \geq c \cdot g(n).$$

Example:

Suppose

$$f(n) = 4n + 3 \quad \wedge \quad g(n) = n.$$

To show that $f(n) = \Omega(g(n))$, we need to find constants c and n_0 such that for all $n \geq n_0$, the inequality $f(n) \geq c \cdot g(n)$ holds.

Firstly, substituting the functions, we obtain

$$4n + 3 \geq c \cdot n.$$

Choose $c = 4$. Then we need

$$4n + 3 \geq 4n,$$

which simplifies to

$$3 \geq 0.$$

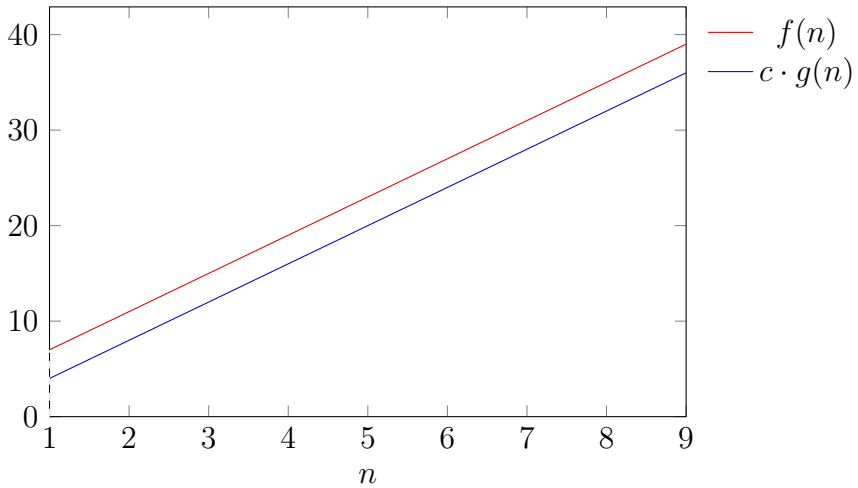
This holds for all $n \geq 1$. Taking $c = 4$ and $n_0 = 1$, we conclude that $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$. Therefore

$$\boxed{f(n) = \Omega(g(n))}$$

Comparing $f(n) = 4n + 3$ against $c \cdot g(n) = 4n$ for $n = 1, \ldots, 9$, with $c = 4$:

n	$f(n)$	$c \cdot g(n)$
1	7	4
2	11	8
3	15	12
4	19	16
5	23	20
6	27	24
7	31	28
8	35	32
9	39	36
$\vdots$	$\vdots$	$\vdots$

Intuitively,



We claim that both functions grow linearly, and that $f(n)$ is lower-bounded by $c \cdot g(n)$ for all sufficiently large n.

31.4 Big Θ Notation

Big Θ notation provides a *tight bound*, it says that a function grows neither faster nor slower than $g(n)$ up to constant factors.

Formally,

$$f(n) = \Theta(g(n)) \;\leftrightarrow\; \exists\, c_1 > 0,\ \exists\, c_2 > 0,\ \exists\, n_0 \text{ s.t. } \forall\, n \geq n_0,\ c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n).$$

Equivalently, $f(n) = \Theta(g(n))$ if and only if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

Example:

Suppose

$$f(n) = 4n + 3 \quad \wedge \quad g(n) = n.$$

To show that $f(n) = \Theta(g(n))$, we need constants c_1, c_2, and n_0 such that for all $n \geq n_0$:

$$c_1 \cdot n \leq 4n + 3 \leq c_2 \cdot n.$$

1. Upper bound: Choose $c_2 = 5$. Then $4n + 3 \leq 5n$ requires $3 \leq n$, so this holds for $n \geq 3$.

2. Lower bound: Choose $c_1 = 4$. Then $4n \leq 4n + 3$ holds trivially for all $n \geq 1$.

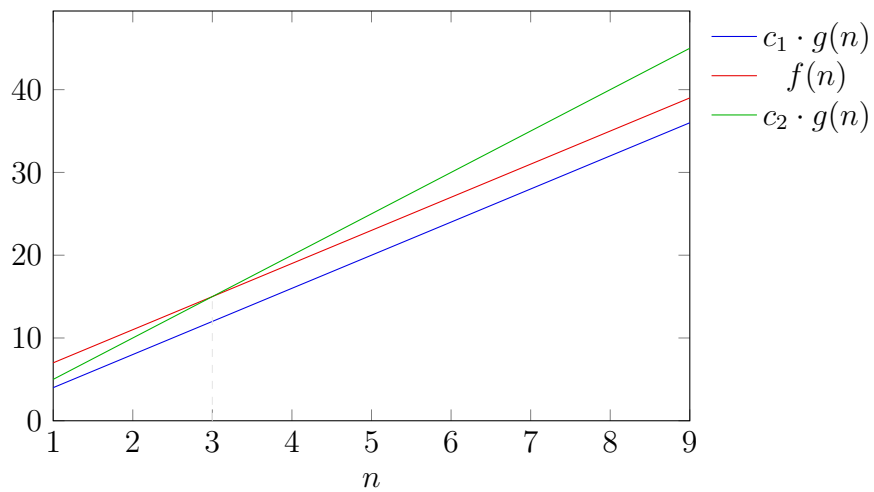
Taking $c_1 = 4$, $c_2 = 5$, and $n_0 = 3$, both inequalities hold for all $n \geq n_0$. Therefore

$$\boxed{f(n) = \Theta(g(n))}$$

Comparing $f(n) = 4n + 3$ against $c_1 \cdot g(n) = 4n$ and $c_2 \cdot g(n) = 5n$ for $n = 1, \ldots, 9$:

n	$c_1 \cdot g(n)$	$f(n)$	$c_2 \cdot g(n)$
1	4	7	5
2	8	11	10
3	12	15	15
4	16	19	20
5	20	23	25
6	24	27	30
7	28	31	35
8	32	35	40
9	36	39	45
$\vdots$	$\vdots$	$\vdots$	$\vdots$

Intuitively,



We claim that $f(n)$ is sandwiched between $c_1 \cdot g(n)$ and $c_2 \cdot g(n)$ for all sufficiently large n, confirming that all three functions share the same linear growth rate.

31.5 Little o Notation

Little o notation describes a *strict upper bound*, f grows *strictly* slower than g, meaning no positive constant c can make $c \cdot g(n)$ a tight bound for $f(n)$.

Formally,

$$f(n) = o(g(n)) \;\leftrightarrow\; \forall\, c > 0,\; \exists\, n_0 \text{ s.t. } \forall\, n \geq n_0,\; f(n) < c \cdot g(n).$$

The key distinction from Big O: in Big O, we only need *one* c to work. In little o, the inequality must hold for *every* $c > 0$, no matter how small.

Example:

Suppose

$$f(n) = n \quad \wedge \quad g(n) = n^2.$$

To show that $f(n) = o(g(n))$, we must verify that for *every* $c > 0$, the inequality $n < c \cdot n^2$ eventually holds.

Dividing both sides by n (valid for $n > 0$):

$$1 < c \cdot n,$$

which rearranges to

$$n > \frac{1}{c}.$$

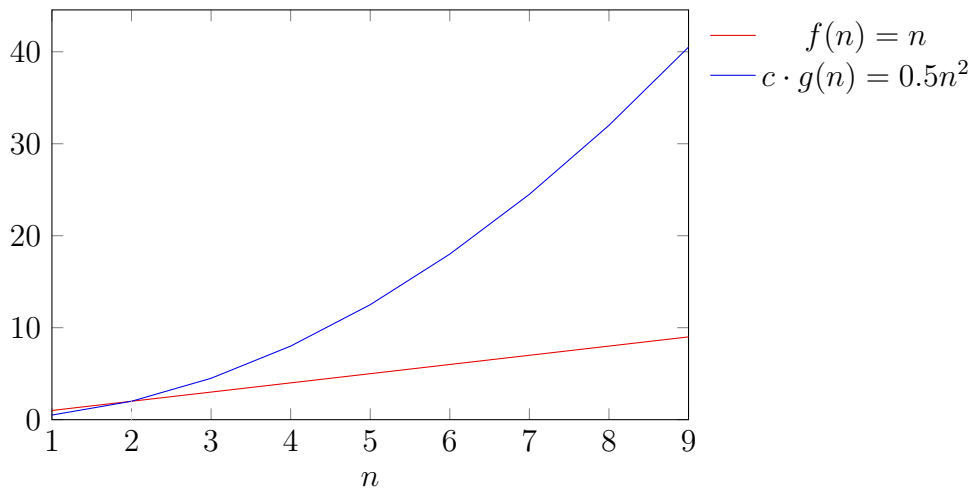
For any given $c > 0$, this holds for all $n \geq n_0 = \left\lfloor \frac{1}{c} \right\rfloor + 1$. Since we can always find such an n_0 regardless of how small c is, we conclude that $f(n) = o(g(n))$. Therefore

$$\boxed{f(n) = o(g(n))}$$

Comparing $f(n) = n$ against $c \cdot g(n) = c \cdot n^2$ for $n = 1, \ldots, 9$ with $c = 0.5$:

n	$f(n)$	$c \cdot g(n)$
1	1	0.5
2	2	2
3	3	4.5
4	4	8
5	5	12.5
6	6	18
7	7	24.5
8	8	32
9	9	40.5
$\vdots$	$\vdots$	$\vdots$

Intuitively,



We claim that $f(n) = n$ grows strictly slower than $g(n) = n^2$; the gap between $c \cdot g(n)$ and $f(n)$ widens without bound as n increases.

31.6 Little ω Notation

Little ω notation describes a *strict lower bound*, f grows *strictly* faster than g, and no constant c can make $c \cdot g(n)$ an upper bound for $f(n)$ eventually.

Formally,

$$f(n) = \omega(g(n)) \;\leftrightarrow\; \forall\, c > 0,\; \exists\, n_0 \text{ s.t. } \forall\, n \geq n_0,\; f(n) > c \cdot g(n).$$

This is the mirror image of little o: if $f(n) = o(g(n))$, then $g(n) = \omega(f(n))$.

Example:

Suppose

$$f(n) = n^2 \quad \wedge \quad g(n) = n.$$

To show that $f(n) = \omega(g(n))$, we must verify that for *every* $c > 0$, the inequality $n^2 > c \cdot n$ eventually holds.

Dividing both sides by n (valid for $n > 0$):

$$n > c.$$

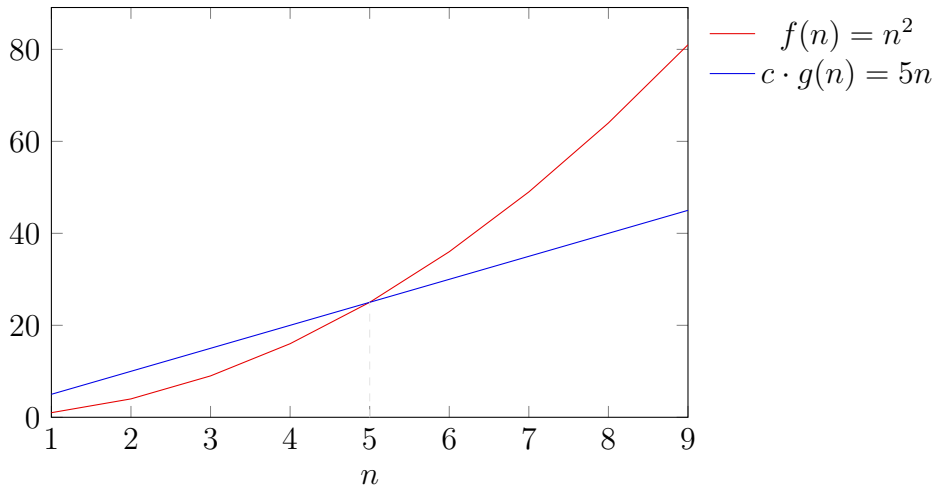
For any given $c > 0$, this holds for all $n \geq n_0 = \lfloor c \rfloor + 1$. Since we can always find such an n_0 regardless of how large c is, we conclude that $f(n) = \omega(g(n))$. Therefore

$$\boxed{f(n) = \omega(g(n))}$$

Comparing $f(n) = n^2$ against $c \cdot g(n) = c \cdot n$ for $n = 1, \ldots, 9$ with $c = 5$:

n	$f(n)$	$c \cdot g(n)$
1	1	5
2	4	10
3	9	15
4	16	20
5	25	25
6	36	30
7	49	35
8	64	40
9	81	45
$\vdots$	$\vdots$	$\vdots$

Intuitively,



We claim that $f(n) = n^2$ grows strictly faster than $g(n) = n$; the gap between $f(n)$ and $c \cdot g(n)$ widens without bound as n increases, regardless of the choice of c.

31.7 Standard Functions

1. Constant: $f(n) = c$

   Example:

   Suppose

   $$f(n) = 7 \quad \wedge \quad g(n) = 1$$

   We want to show that $f(n) = O(g(n))$.

   We need constants c_1 and n_0 such that

   $$7 \leq c_1 \cdot 1$$

   Choose $c_1 = 7$. The inequality holds for all $n \geq 1$.

   As a result,

$$\boxed{f(n) = O(1)}$$

2. Logarithmic: $f(n) = \log n$

   Example:

   Suppose

$$f(n) = 3\log n + 2 \quad \wedge \quad g(n) = \log n$$

   We need

$$3\log n + 2 \leq c\log n$$

   Choose $c = 5$. Then:

$$3\log n + 2 \leq 5\log n$$

   This holds for all $n \geq 2$.

   We conclude that,

$$\boxed{f(n) = O(\log n)}$$

3. Linear: $f(n) = n$

   Example:

   Suppose

$$f(n) = 4n + 3 \quad \wedge \quad g(n) = n$$

   We need

$$4n + 3 \leq cn$$

   Choose $c = 5$. Then:

$$4n + 3 \leq 5n$$

   This holds for all $n \geq 3$.

   Accordingly,

$$\boxed{f(n) = O(n)}$$

4. Linearithmic: $f(n) = n \log n$

   Example:

   Suppose

$$f(n) = 2n \log n + n \quad \wedge \quad g(n) = n \log n$$

   We need

$$2n \log n + n \leq cn \log n$$

   Choose $c = 3$. For $n \geq 2$, this holds.

   In conclusion,

$$\boxed{f(n) = O(n \log n)}$$

5. Quadratic: $f(n) = n^2$

   Example:

   Suppose

$$f(n) = 5n^2 + 2n + 1 \quad \wedge \quad g(n) = n^2$$

   We need

$$5n^2 + 2n + 1 \leq cn^2$$

   Choose $c = 8$. This holds for all $n \geq 1$.

   Hence,

$$\boxed{f(n) = O(n^2)}$$

6. Cubic: $f(n) = n^3$

   Example:

   Suppose

$$f(n) = n^3 + 10n^2 \quad \wedge \quad g(n) = n^3$$

   We need

$$n^3 + 10n^2 \leq cn^3$$

   Choose $c = 11$. This holds for all $n \geq 1$.

   Therefore,

$$\boxed{f(n) = O(n^3)}$$

7. Exponential: $f(n) = 2^n$

   Example:

   Suppose

$$f(n) = 3 \cdot 2^n + n \quad \wedge \quad g(n) = 2^n$$

   We need

$$3 \cdot 2^n + n \leq c \cdot 2^n$$

   Choose $c = 4$. For all $n \geq 1$, the inequality holds.

   Thus,

$$\boxed{f(n) = O(2^n)}$$

Specially, we have the following growth rates:

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n)$$

$g(n)$						
n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n
1	0	1	0	1	1	2
2	1	2	2	4	8	4
4	2	4	8	16	64	16
8	3	8	24	64	512	256
16	4	16	64	256	4096	65536
32	5	32	160	1024	32768	4294967296

32 Analyzing Algorithms

Example 1:

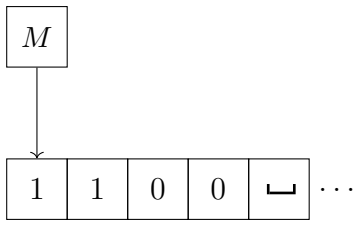
Suppose we have a Turing machine M that decides the language

$$A = \{\, 1^k 0^k \mid k \geq 1 \,\}.$$

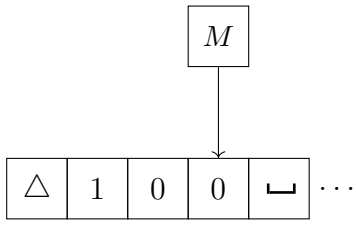
M on input w:

1. Scan the input to check whether $w \in 1^*0^*$. If not, *reject*.
2. Repeat until no unread symbols remain:
   (a) Scan from left to find the leftmost unread symbol and replace it with $\triangle$.
   (b) Scan from right to find the rightmost unread symbol and match it.
   (c) If the symbols do not match, *reject*.
   (d) Replace the matched symbol with $\triangle$.
3. If any unread symbols remain, *reject*; otherwise, *accept*.

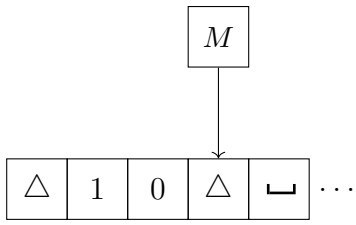
Initial tape:



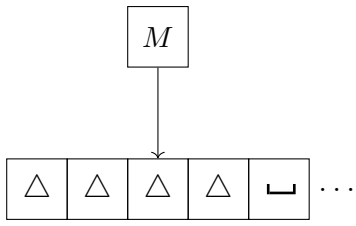
Step 1: Read leftmost 1, mark it with $\triangle$, move right to find rightmost unread symbol:



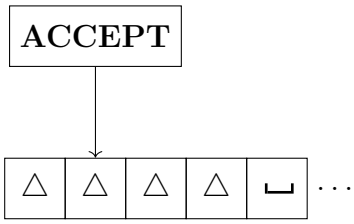
Step 2: Rightmost unread is 0, matches remembered symbol 0. Mark it with $\triangle$:



Step 3: Move back left to next unread symbol 1, mark it, move right to rightmost unread 0, match:



Step 4: All symbols matched. Head sees only $\triangle$ and $\sqcup$. TM halts and accepts.



From this algorithm, we can see that the running time is $O(n^2)$, where n is the length of the input string. In the worst case, the input is of the form 1^k0^k with $n = 2k$, so the algorithm performs k iterations, each requiring a full scan of the tape taking $O(n)$ time. Since $k = \frac{n}{2}$, we multiply by n to obtain the total number of steps across all iterations, as each iteration costs n steps:

$$k \cdot n = \frac{n}{2} \cdot n = \frac{n^2}{2} = O(n^2).$$

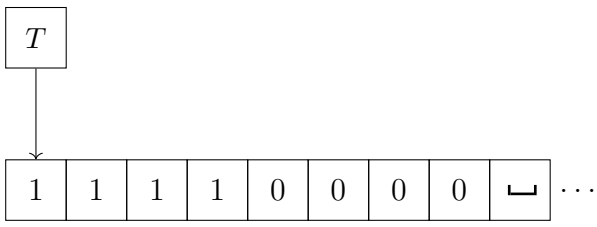
Example 2:
Suppose we have a Turing machine T that decides the language

$$A = \{\, 1^k0^k \mid k \geq 1 \,\}.$$

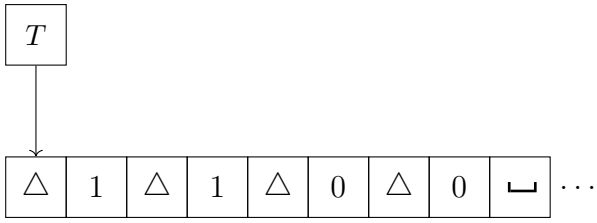
T on input w:

1. Scan the input to check whether $w \in 1^*0^*$. If not, *reject*.
2. Repeat until no unread symbols remain:
   (a) Scan left to right and mark every *other* unread 1 with $\triangle$.
   (b) Scan right to left and mark every *other* unread 0 with $\triangle$.
   (c) If the number of remaining 1s and 0s are unequal, *reject*.
3. If any unread symbols remain, *reject*; otherwise, *accept*.

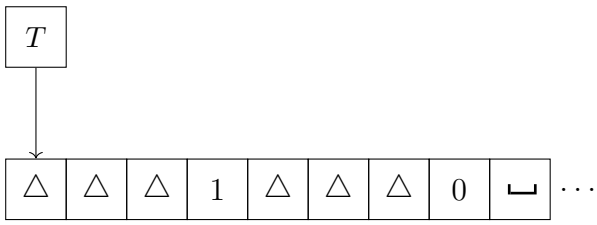
Initial tape (with $k = 4$, so $n = 8$):



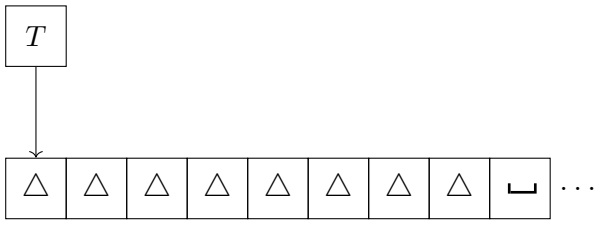
Step 1 (Iteration 1): Mark every other 1 and every other 0 with $\triangle$, halving the unread symbols:



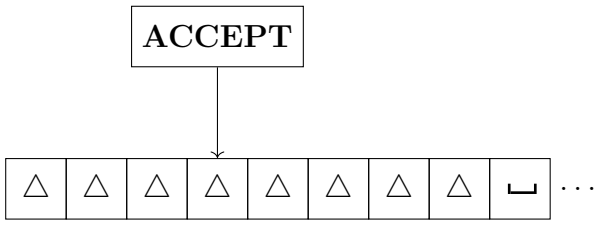
Step 2 (Iteration 2): Mark every other remaining 1 and 0, halving again:



Step 3 (Iteration 3): Mark the last remaining 1 and 0:



Step 4: All symbols matched. TM halts and accepts.



The running time of this algorithm is $O(n \log n)$, where n is the length of the input string. Consider the worst case where the input is of the form $1^k 0^k$ with $n = 2k$. Since each iteration marks half of the remaining symbols, the number of iterations is $O(\log n)$, and each iteration scans the full tape in $O(n)$ time. For example, with $n = 8$:

Iteration	**Remaining symbols**	**Marked this round**
1	8	$4 \to 4$ left
2	4	$2 \to 2$ left
3	2	$1 \to 1$ left

Since the symbols are halved each iteration, the total number of iterations is $\log_2 n$. Therefore, the total running time is:

$$O(\log n) \cdot O(n) = O(n \log n).$$

32.1 TIME Complexity Classes

Formally, let $t : \mathbb{N} \to \mathbb{N}$ be a function. The time complexity class $\mathbf{TIME}(t(n))$ is defined as the collection of all languages decidable by an $O(t(n))$ deterministic Turing machine:

$$\mathbf{TIME}(t(n)) = \{L \mid L \text{ is decided by an } O(t(n))\text{DTM}\}$$

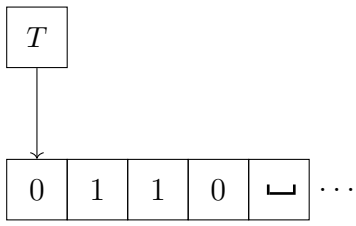
Example:
Suppose we have a Turing machine T that decides the language

$$L = \{\, x \mid x = x^{\mathcal{R}},\ x \in \{0,1\}^* \,\}.$$

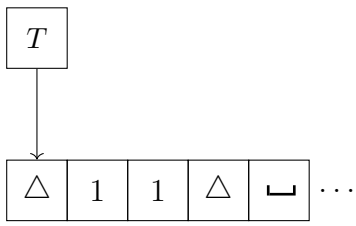
T on input x:

1. If the input is empty or a single symbol, *accept*.
2. Scan to the leftmost unread symbol, remember it, and mark it with $\triangle$.
3. Scan to the rightmost unread symbol and compare it to the remembered symbol. If they differ, *reject*; otherwise mark it with $\triangle$.
4. If any unread symbols remain, go to step 2; otherwise *accept*.

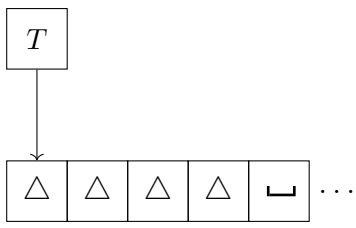
Initial tape (with $x = 0110$, so $n = 4$):



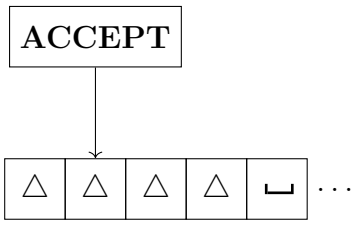
Step 1 (Iteration 1): Compare outermost symbols 0 and 0, they match, mark both with $\triangle$:



Step 2 (Iteration 2): Compare next outermost symbols 1 and 1, they match, mark both with $\triangle$:



Step 3: No unread symbols remain. TM halts and accepts.



Each iteration scans the full tape in $O(n)$ time to locate and compare the outermost unread symbols, marking one pair per iteration. Since the input has n symbols and we mark two per iteration, there are $n/2$ iterations in total. For example, with $n = 4$:

Iteration	Remaining symbols	Marked this round
1	4	1 pair $\to$ 2 left
2	2	1 pair $\to$ 0 left

Since one pair is marked per iteration, the total number of iterations is $n/2$. Therefore, the total running time is:

$$O(n) \cdot O(n/2) = O(n^2).$$

Hence $L \in \mathbf{TIME}(n^2)$.

33 The Class P

The class **P** consists of all languages decidable by a DTM in polynomial time:

$$\mathbf{P} = \bigcup_{k \geq 1} \mathrm{TIME}(n^k)$$

Example 1:
The language $T = \{x \mid x = x^R, x \in \{0,1\}^*\}$ is in **P**.

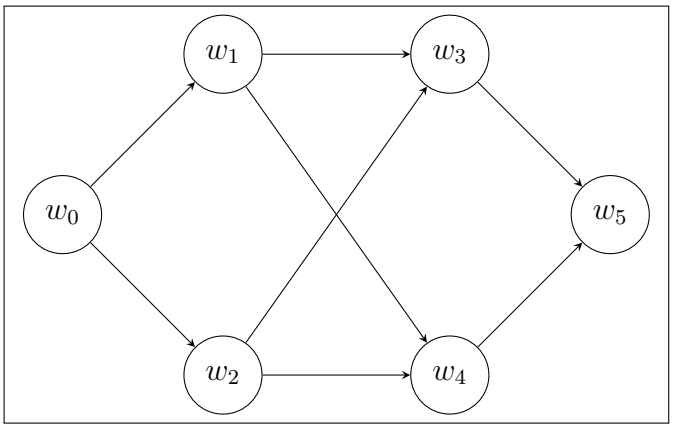
T on input x:

1. If the input is empty or a single symbol, *accept*.
2. Scan to the leftmost unread symbol, remember it, and mark it with $\triangle$.
3. Scan to the rightmost unread symbol and compare it to the remembered symbol. If they differ, *reject*; otherwise mark it with $\triangle$.
4. If any unread symbols remain, go to step 2; otherwise *accept*.

As shown in the previous example, this algorithm runs in $O(n^2)$ time, which is polynomial. Therefore, $T \in \mathbf{P}$.

Example 2:

$$\text{PATH} = \{\langle G, w_0, w_5\rangle \mid G \text{ is a directed graph with a path from } w_0 \text{ to } w_5\} \text{ is in } \mathbf{P}$$



where $G = (V, E)$ with:

$$V = \{w_0, w_1, w_2, w_3, w_4, w_5\}$$

$$E = \{(w_0, w_1),\ (w_0, w_2),\ (w_1, w_3),\ (w_1, w_4),\ (w_2, w_4),\ (w_2, w_3),\ (w_3, w_5),\ (w_4, w_5)\}$$

M on input $\langle G, w_0, w_5\rangle$:

1. Mark node w_0.
2. Scan all edges (u, v) in G: if u is marked and v is not, mark v.
3. If any new node was marked in step 2, repeat step 2.
4. If w_5 is marked, *accept*; otherwise *reject*.

We show PATH $\in \mathbf{P}$ by constructing a polynomial-time decider. A node v is marked if and only if there exists a directed path from w_0 to v. Hence M accepts if and only if $\langle G, w_0, w_5\rangle \in$ PATH.

Let $G = (V, E)$ where V is the set of nodes and E is the set of directed edges. Let $n = |V|$ and $m = |E|$. Each iteration of step 2 marks at least one new node, so the loop runs at most n times. Each iteration scans all m edges. Since each ordered pair of nodes admits at most one edge, we have $|E| \leq |V|^2$, i.e. $m \leq n^2$, and therefore:

$$n \cdot m \leq n \cdot n^2 = n^3$$

and thus:

$$O(n \cdot m) \subseteq O(n \cdot n^2) = O(n^3)$$

which is polynomial in $|\langle G, w_0, w_5\rangle|$. Therefore, M runs in polynomial time and PATH $\in \mathbf{P}$.

34 Nondeterministic Polynomial Time (NP)

Formally, let $t : \mathbb{N} \to \mathbb{N}$ be a function. The *nondeterministic time complexity class* $\mathbf{NTIME}(t(n))$ is defined as:

$$\mathbf{NTIME}(t(n)) = \{L \mid L \text{ is a language decided by a NTM in time } t(n)\}$$

Then **NP** is defined as the union over all polynomial time bounds:

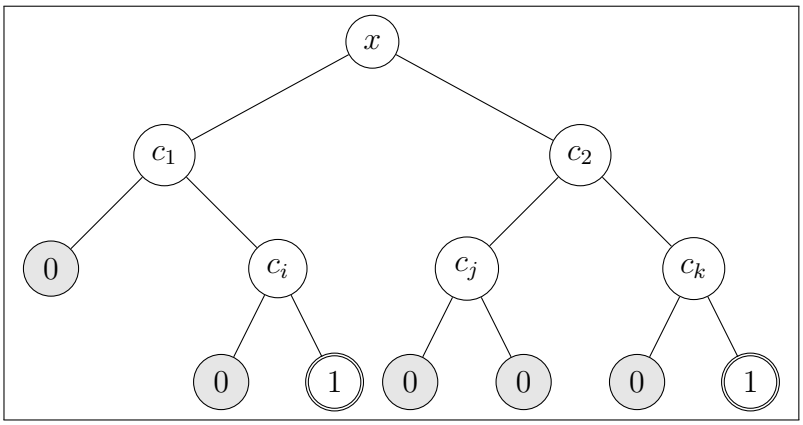
$$\mathbf{NP} = \bigcup_{k \in \mathbb{N}} \mathrm{NTIME}(n^k)$$

The class **NP** can equivalently be characterized using a *verifier*: a deterministic Turing machine that checks whether a proposed solution is correct. Formally, a language $L \subseteq \Sigma^*$ is in **NP** if and only if there exists a deterministic Turing machine V that runs in polynomial time and a polynomial $p : \mathbb{N} \to \mathbb{N}$ such that for all $x \in \Sigma^*$:

$$x \in L \leftrightarrow \exists c \in \Sigma^* \text{ with } |c| \leq p(|x|) \text{ such that } V(x, c) = 1$$

V is called the *verifier* for L, and c is called the *certificate* for x.

Intuitively, a nondeterministic machine can be thought of as *guessing* a certificate c and then verifying it.



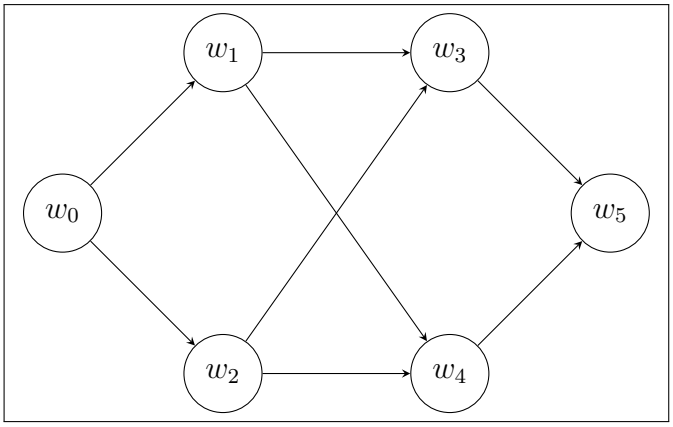
Each path from the root x to a leaf represents one possible guess of a certificate. The machine *accepts* if at least one branch reaches an accept state 1; it *rejects* if all branches reach a reject state 0. The depth of the tree is polynomial in $|x|$.

34.1 Hamiltonian Path

A *Hamiltonian path* from w_0 to w_5 is a directed path that visits every node in V exactly once.

Example:

$$\mathrm{HAMPATH} = \{\langle G, w_0, w_5 \rangle \mid G \text{ is a directed graph with a Hamiltonian path from } w_0 \text{ to } w_5\}$$



where $G = (V, E)$ with:

$$V = \{w_0, w_1, w_2, w_3, w_4, w_5\}$$

$$E = \{(w_0, w_1),\ (w_0, w_2),\ (w_1, w_3),\ (w_1, w_4),\ (w_2, w_4),\ (w_2, w_3),\ (w_3, w_5),\ (w_4, w_5)\}$$

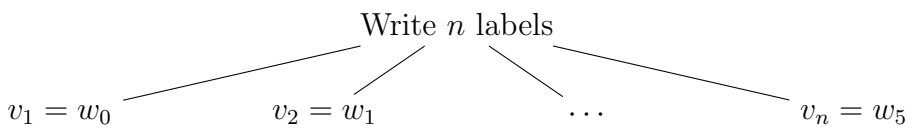
A *Hamiltonian path* from w_0 to w_5 is a directed path that visits every node in V exactly once. For this graph, the path $w_0 \to w_1 \to w_4 \to w_3 \to w_5$ visits only 5 of the 6 nodes and is therefore *not* Hamiltonian, whereas $w_0 \to w_2 \to w_3 \to w_5$ also fails. Indeed, one can check that no path visits all six nodes, so $\langle G, w_0, w_5 \rangle \notin$ HAMPATH.

N on input $\langle G, w_0, w_5 \rangle$:

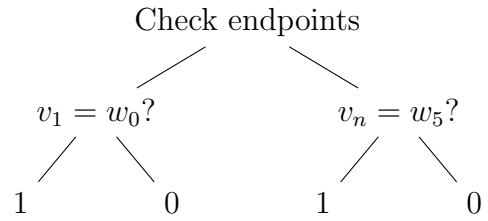
1. Nondeterministically select a permutation $\pi = (v_1, v_2, \ldots, v_n)$ of V.
2. Check that $v_1 = w_0$ and $v_n = w_5$.
3. For each $i \in \{1, \ldots, n-1\}$, check that $(v_i, v_{i+1}) \in E$.
4. If all checks pass, 1; otherwise 0.

We show HAMPATH $\in$ **NP** by constructing a polynomial-time nondeterministic Turing machine N. A permutation π passes all checks if and only if it is a Hamiltonian path from w_0 to w_5. Hence N accepts $\langle G, w_0, w_5 \rangle$ on some branch if and only if $\langle G, w_0, w_5 \rangle \in$ HAMPATH. Let $n = |V|$ and $m = |E|$. The running time of each step is:

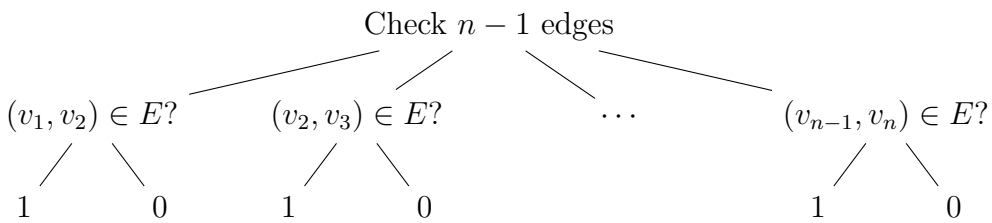
1. Step 1: Writes down n node labels in $O(n)$ time.



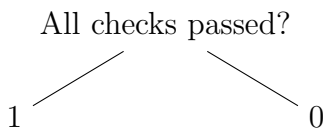
2. Step 2: Performs two comparisons in $O(1)$.



3. Step 3: Performs $n-1$ edge-membership checks; with an adjacency-matrix representation each check is $O(1)$, giving $O(n)$ total.



4. Step 4: Performs a single check in $O(1)$.



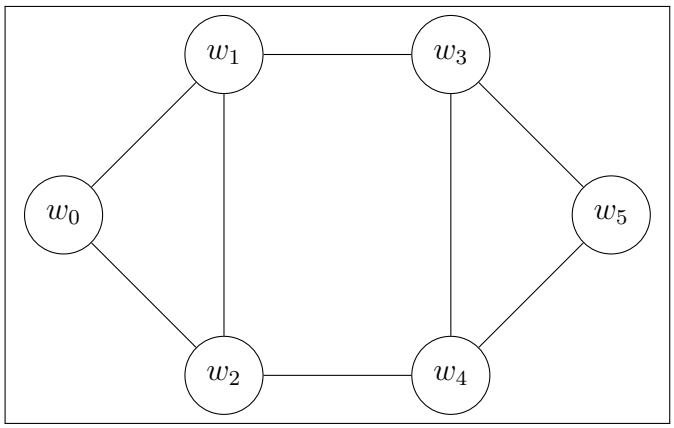
Thus the entire computation runs in $O(n)$ time, which is polynomial in $|\langle G, w_0, w_5\rangle|$. A *certificate* for membership is the sequence $(v_1, \ldots, v_n)$ itself, whose length is $O(n)$. A deterministic verifier checks the three conditions in Steps 2–3 in $O(n)$ time, confirming that HAMPATH $\in$ **NP**.

34.2 Clique

A *clique* is a subset $C \subseteq V$ such that every two nodes in C are connected by an edge, i.e.:

$$\forall u, v \in C, u \neq v : (u, v) \in E$$

Example:
Consider $G = (V, E)$ with:



$$V = \{w_0, w_1, w_2, w_3, w_4, w_5\}, \quad k = 3$$

$$E = \{(w_0, w_1),\ (w_0, w_2),\ (w_1, w_2),\ (w_1, w_3),\ (w_2, w_4),\ (w_3, w_4),\ (w_3, w_5),\ (w_4, w_5)\}$$

Taking $C = \{w_0, w_1, w_2\}$, we verify that every pair in C is connected:

$$\begin{array}{ll} (w_0, w_1) & (w_0, w_1) \in E \\ (w_0, w_2) & (w_0, w_2) \in E \\ (w_1, w_2) & (w_1, w_2) \in E \end{array}$$

All pairs are connected, so $C = \{w_0, w_1, w_2\}$ is a valid clique of size $k = 3$, and thus $\langle G, 3\rangle \in$ CLIQUE. Taking $C = \{w_3, w_4, w_5\}$, we verify:

$$\begin{array}{ll} (w_3, w_4) & (w_3, w_4) \in E \\ (w_3, w_5) & (w_3, w_5) \in E \\ (w_4, w_5) & (w_4, w_5) \in E \end{array}$$

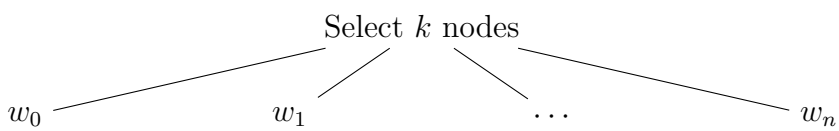
This is also a valid clique of size $k = 3$. However, no clique of size $k = 4$ exists, so $\langle G, 4\rangle \notin$ CLIQUE.

N on input $\langle G, k\rangle$:

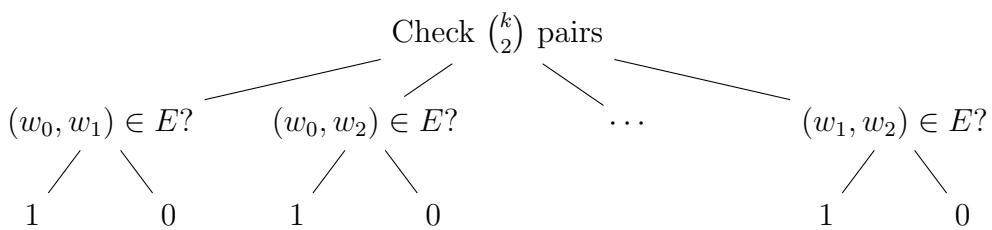
1. Nondeterministically select a subset $C \subseteq V$ with $|C| = k$.
2. For each pair $u, v \in C$ with $u \neq v$, check that $(u, v) \in E$.
3. If all checks pass, 1; otherwise 0.

We show CLIQUE $\in$ **NP** by constructing a polynomial-time nondeterministic Turing machine N. A subset C passes all checks if and only if it is a clique of size k. Hence N accepts $\langle G, k\rangle$ on some branch if and only if $\langle G, k\rangle \in$ CLIQUE. Let $n = |V|$ and $m = |E|$. The running time of each step is:

1. Step 1: Selects k nodes from V in $O(n)$ time.



2. Step 2: Performs $\binom{k}{2}$ pair checks; each membership check is $O(1)$, giving $O(k^2)$ total.



3. Step 3: Performs a single check in $O(1)$.

$$\begin{array}{ccc} & \text{All pairs connected?} & \\ 1 \diagup & & \diagdown 0 \end{array}$$

Since $k \leq n$, the number of pairs checked is $\binom{k}{2} \leq \binom{n}{2} = \frac{n(n-1)}{2}$, and therefore:

$$n \cdot \frac{n(n-1)}{2} = \frac{n^2(n-1)}{2}$$

and thus:

$$O\left(\frac{n^2(n-1)}{2}\right) = O(n^3)$$

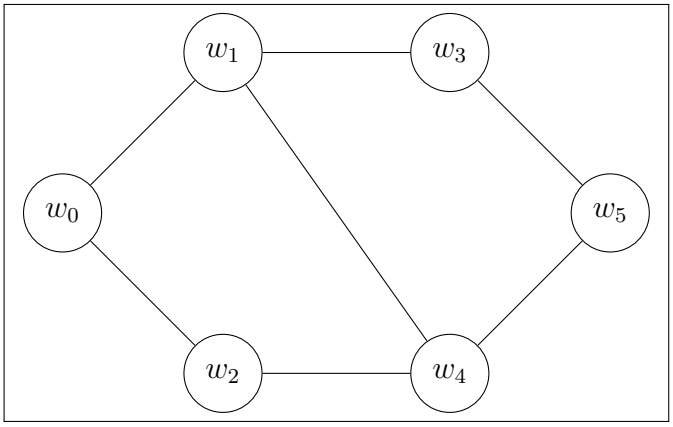
which is polynomial in $|\langle G, k\rangle|$. A *certificate* for membership is the subset C itself, whose length is $O(n)$. A deterministic verifier checks all pairs in $O(k^2) \subseteq O(n^2)$ time, confirming that CLIQUE $\in$ **NP**.

34.3 Vertext Cover

A *vertex cover* is a subset $S \subseteq V$ such that every edge in G has at least one endpoint in S.

Example:

Consider $G = (V, E)$ with:



$$V = \{w_0, w_1, w_2, w_3, w_4, w_5\}, \quad k = 3$$

$$E = \{(w_0, w_1),\ (w_0, w_2),\ (w_1, w_3),\ (w_1, w_4),\ (w_2, w_4),\ (w_3, w_5),\ (w_4, w_5)\}$$

Taking $S = \{w_1, w_2, w_4\}$, we verify that S covers every edge:

$$
\begin{array}{ll}
(w_0, w_1) & \text{covered by } w_1 \in S \\
(w_0, w_2) & \text{covered by } w_2 \in S \\
(w_1, w_3) & \text{covered by } w_1 \in S \\
(w_1, w_4) & \text{covered by } w_1 \in S \\
(w_2, w_4) & \text{covered by } w_2 \in S \\
(w_3, w_5) & \text{covered by } w_4 \notin S, \text{ not covered by } w_3 \notin S \\
(w_4, w_5) & \text{covered by } w_4 \in S
\end{array}
$$

Since (w_3, w_5) is not covered, $S = \{w_1, w_2, w_4\}$ is *not* a valid vertex cover. Taking instead $S = \{w_1, w_2, w_4\}$ with w_3 replaced, we try $S = \{w_1, w_3, w_4\}$:

$$
\begin{array}{ll}
(w_0, w_1) & \text{covered by } w_1 \in S \\
(w_0, w_2) & \text{not covered: } w_0 \notin S,\ w_2 \notin S
\end{array}
$$

Hence $\langle G, 3\rangle \notin$ VERTEX-COVER for these choices. Taking $S = \{w_0, w_1, w_4\}$:

$$
\begin{array}{ll}
(w_0, w_1) & \text{covered by } w_0 \in S \\
(w_0, w_2) & \text{covered by } w_0 \in S \\
(w_1, w_3) & \text{covered by } w_1 \in S \\
(w_1, w_4) & \text{covered by } w_1 \in S \\
(w_2, w_4) & \text{covered by } w_4 \in S \\
(w_3, w_5) & \text{not covered: } w_3 \notin S,\ w_5 \notin S \\
(w_4, w_5) & \text{covered by } w_4 \in S
\end{array}
$$

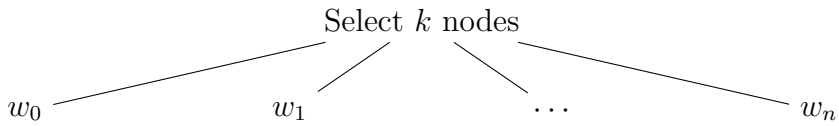
No subset of size $k = 3$ covers all edges, so $\langle G, 3\rangle \notin$ VERTEX-COVER. However, taking $k = 4$ with $S = \{w_0, w_1, w_4, w_3\}$ covers all edges, so $\langle G, 4\rangle \in$ VERTEX-COVER.

N on input $\langle G, k\rangle$:

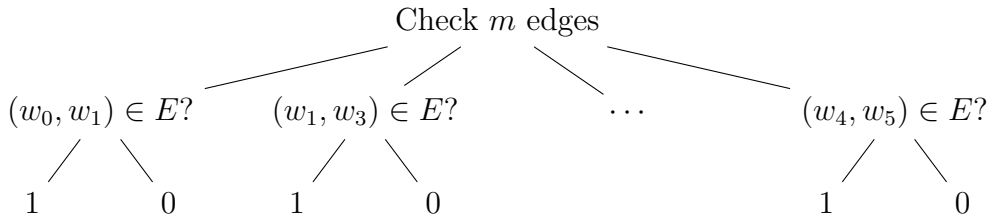
1. Nondeterministically select a subset $S \subseteq V$ with $|S| = k$.
2. For each edge $(u, v) \in E$, check that $u \in S$ or $v \in S$.
3. If all checks pass, 1; otherwise 0.

We show VERTEX-COVER $\in$ **NP** by constructing a polynomial-time nondeterministic Turing machine N. A subset S passes all checks if and only if it is a vertex cover of size k. Hence N accepts $\langle G, k\rangle$ on some branch if and only if $\langle G, k\rangle \in$ VERTEX-COVER. Let $n = |V|$ and $m = |E|$. The running time of each step is:

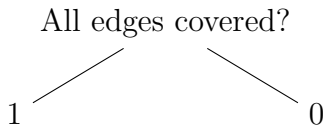
1. Step 1: Selects k nodes from V in $O(n)$ time.



2. Step 2: Performs m edge checks; each membership check is $O(1)$, giving $O(m)$ total.



3. Step 3: Performs a single check in $O(1)$.



Since each pair of nodes admits at most one edge, we have $|E| \leq \dfrac{n(n-1)}{2}$, and therefore:

$$n \cdot m \leq n \cdot \frac{n(n-1)}{2} = \frac{n^2(n-1)}{2}$$

and thus:

$$O(n \cdot m) \subseteq O\left(\frac{n^2(n-1)}{2}\right) = O(n^3)$$

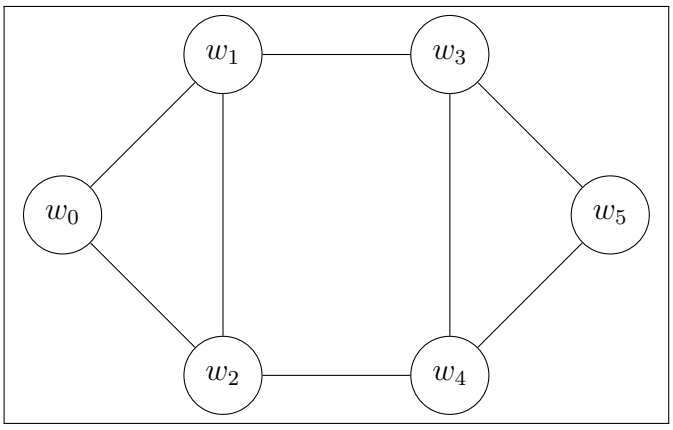
which is polynomial in $|\langle G, k \rangle|$. A *certificate* for membership is the subset S itself, whose length is $O(n)$. A deterministic verifier checks all edges in $O(m)$ time, confirming that VERTEX-COVER $\in$ **NP**.

34.4 Independent Set

A *independent set* is a subset $I \subseteq V$ such that no two nodes in I are connected by an edge, i.e.:

$$\forall u, v \in I, u \neq v : (u, v) \notin E$$

Example:
Consider $G = (V, E)$ with:



$$V = \{w_0, w_1, w_2, w_3, w_4, w_5\}, \quad k = 3$$

$$E = \{(w_0, w_1),\ (w_0, w_2),\ (w_1, w_2),\ (w_1, w_3),\ (w_2, w_4),\ (w_3, w_4),\ (w_3, w_5),\ (w_4, w_5)\}$$

Taking $I = \{w_0, w_3, w_4\}$, we verify that no two nodes in I are connected:

$$\begin{array}{lll} (w_0, w_3) & (w_0, w_3) \notin E & \text{no edge} \\ (w_0, w_4) & (w_0, w_4) \notin E & \text{no edge} \\ (w_3, w_4) & (w_3, w_4) \in E & \text{edge exists!} \end{array}$$

Since $(w_3, w_4) \in E$, $I = \{w_0, w_3, w_4\}$ is *not* a valid independent set. Taking instead $I = \{w_0, w_3, w_5\}$:

$$\begin{array}{lll} (w_0, w_3) & (w_0, w_3) \notin E & \text{no edge} \\ (w_0, w_5) & (w_0, w_5) \notin E & \text{no edge} \\ (w_3, w_5) & (w_3, w_5) \in E & \text{edge exists!} \end{array}$$

Since $(w_3, w_5) \in E$, $I = \{w_0, w_3, w_5\}$ is also *not* valid. Taking instead $I = \{w_0, w_4, w_5\}$:

$$\begin{array}{lll} (w_0, w_4) & (w_0, w_4) \notin E & \text{no edge} \\ (w_0, w_5) & (w_0, w_5) \notin E & \text{no edge} \\ (w_4, w_5) & (w_4, w_5) \in E & \text{edge exists!} \end{array}$$

Taking instead $I = \{w_0, w_3, w_4\}$ fails, so we try $I = \{w_0, w_5, w_2\}$:

$$\begin{array}{lll} (w_0, w_5) & (w_0, w_5) \notin E & \text{no edge} \\ (w_0, w_2) & (w_0, w_2) \in E & \text{edge exists!} \end{array}$$

Taking $I = \{w_0, w_5, w_3\}$ fails as shown. However, taking $I = \{w_0, w_4, w_3\}$ fails too. We find that $I = \{w_2, w_3, w_5\}$ is not valid either. Taking $I = \{w_0, w_5\}$ with $k = 2$:

$$(w_0, w_5) \quad (w_0, w_5) \notin E \quad \text{no edge}$$

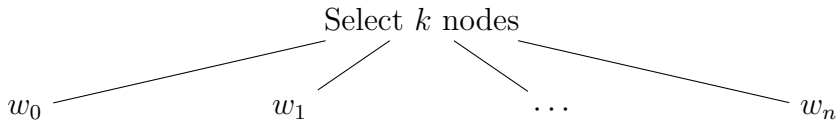
So $\langle G, 2\rangle \in$ INDEPENDENT-SET. No independent set of size $k = 3$ exists in this graph, so $\langle G, 3\rangle \notin$ INDEPENDENT-SET.

N on input $\langle G, k\rangle$:

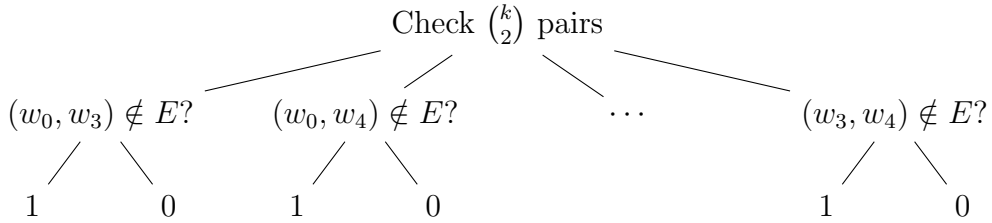
1. Nondeterministically select a subset $I \subseteq V$ with $|I| = k$.
2. For each pair $u, v \in I$ with $u \neq v$, check that $(u, v) \notin E$.
3. If all checks pass, 1; otherwise 0.

We show INDEPENDENT-SET $\in$ **NP** by constructing a polynomial-time nondeterministic Turing machine N. A subset I passes all checks if and only if it is an independent set of size k. Hence N accepts $\langle G, k\rangle$ on some branch if and only if $\langle G, k\rangle \in$ INDEPENDENT-SET. Let $n = |V|$ and $m = |E|$. The running time of each step is:

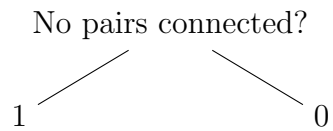
1. Step 1: Selects k nodes from V in $O(n)$ time.



2. Step 2: Performs $\binom{k}{2}$pair checks; each membership check is $O(1)$, giving $O(k^2)$ total.



3. Step 3: Performs a single check in $O(1)$.



Since $k \leq n$, the number of pairs checked is $\binom{k}{2} \leq \binom{n}{2}$, and therefore:

$$n \cdot \frac{n(n-1)}{2} = \frac{n^2(n-1)}{2}$$

and thus:

$$O\left(\frac{n^2(n-1)}{2}\right) = O(n^3)$$

which is polynomial in $|\langle G, k\rangle|$. A *certificate* for membership is the subset I itself, whose length is $O(n)$. A deterministic verifier checks all pairs in $O(k^2) \subseteq O(n^2)$ time, confirming that INDEPENDENT-SET $\in$ **NP**.

35 coNP

Formally:

$$\text{coNP} = \{L \mid \overline{L} \in \text{NP}\}$$

In other words, $L \in$ coNP if there exists a polynomial-time verifier that can verify a **no** instance, i.e. a certificate that $x \notin L$.

The difference between NP and coNP is:

NP	there exists a certificate that $x \in L$
coNP	there exists a certificate that $x \notin L$

For example:

SAT $\in$ NP	certificate is a satisfying assignment
UNSAT $\in$ coNP	certificate is a proof that no assignment satisfies ϕ

35.1 coNP-complete

A language L is **coNP-complete** if:

1. $L \in \text{coNP}$, and
2. $\forall X \in \text{coNP}$, $X \leq_p L$.

$$\text{coNP-complete} = \{L \in \text{coNP} \mid \forall X \in \text{coNP} : X \leq_p L\}$$

The canonical coNP-complete problem is **UNSAT**:

$$\text{UNSAT} = \{\langle\phi\rangle \mid \phi \text{ is an unsatisfiable Boolean formula}\}$$

Example:
The formula:

$$\phi = (x_1 + \bar{x}_1) \cdot (x_2 + \bar{x}_2)$$

is a tautology, it is true for every assignment, so $\overline{\phi} = 0$ is unsatisfiable, and $\langle\phi\rangle \in$ UNSAT. So since SAT is NP-complete, UNSAT is coNP-complete.

36 NP-complete

36.1 NP-Hard

A problem H is **NP-Hard** if every problem in NP reduces to it in polynomial time:

$$L \leq_p H \quad \text{for all } L \in \text{NP}$$

Note that NP-hard does not require $H \in \text{NP}$, H may be harder than any problem in NP. In other words, NP-complete = NP-hard $\cap$ NP.

36.2 Boolean Satisfiability (SAT)

A *Boolean formula* is an expression over Boolean variables $x_1, x_2, \ldots, x_n$ using the operators:

1. $\cdot$ (AND), true if both sides are true
2. $+$ (OR), true if at least one side is true
3. $\bar{x}$ (NOT), flips the value

A *literal* is either a variable or its negation, and a *clause* is a disjunction of literals. For example:

$$\underbrace{(x_1 + x_2)}_{\text{clause}} \cdot \underbrace{(\bar{x}_1 + x_3)}_{\text{clause}}$$

where $\underbrace{x_1}_{\text{literal}}$, $\underbrace{x_2}_{\text{literal}}$, $\underbrace{\bar{x}_1}_{\text{literal}}$, $\underbrace{x_3}_{\text{literal}}$ are the literals.

Importanly, a formula is *satisfiable* if there exists an assignment of values $\{0, 1\}$ to the variables that makes the formula evaluate to 1 (true). The SAT problem asks:

$$\text{SAT} = \{\langle\phi\rangle \mid \phi \text{ is a satisfiable Boolean formula}\}$$

Example:
Consider the formula:

$$\phi = (x_1 + x_2) \cdot (\bar{x_1} + x_3) \cdot (\bar{x_2} + \bar{x_3})$$

We try the assignment $x_1 = 1, x_2 = 0, x_3 = 1$:

$$\begin{aligned} (x_1 + x_2) &= (1 + 0) = 1 \\ (\bar{x_1} + x_3) &= (0 + 1) = 1 \\ (\bar{x_2} + \bar{x_3}) &= (1 + 0) = 1 \end{aligned}$$

Since all three clauses evaluate to 1, the entire formula $\phi = 1$. Therefore this assignment *satisfies* ϕ, and thus $\langle\phi\rangle \in \text{SAT}$.

36.3 3-SAT

3-SAT is a special case of SAT where every clause contains *exactly* 3 literals. A *literal* is either a variable x_i or its negation $\bar{x_i}$. Formally:

$$\text{3-SAT} = \{\langle\phi\rangle \mid \phi \text{ is a satisfiable Boolean formula where every clause has exactly 3 literals}\}$$

The difference between SAT and 3-SAT is:

SAT	clauses can have *any* number of literals
3-SAT	every clause has *exactly* 3 literals

Just like SAT, 3-SAT is NP-complete:

1. To verify a solution, given an assignment, check if it satisfies ϕ, is easy and takes polynomial time, so 3SAT $\in$ NP.
2. To find a solution, there are 2^n possible assignments for n variables, and no known algorithm does better than trying all of them in the worst case.

Therefore, if someone found a polynomial time algorithm for 3-SAT, it would imply $\mathbf{P} = \text{NP}$, since SAT $\leq_p$ 3SAT and SAT is NP-complete.
Example 1:
Consider the formula:

$$\phi = (x_1 + x_2 + x_3) \cdot (\bar{x_1} + \bar{x_2} + x_3) \cdot (x_1 + \bar{x_2} + \bar{x_3})$$

where each clause has exactly 3 literals. We try the assignment $x_1 = 1, x_2 = 0, x_3 = 1$:

$$\begin{aligned} (x_1 + x_2 + x_3) &= (1 + 0 + 1) = 1 \\ (\bar{x_1} + \bar{x_2} + x_3) &= (0 + 1 + 1) = 1 \\ (x_1 + \bar{x_2} + \bar{x_3}) &= (1 + 1 + 0) = 1 \end{aligned}$$

Since all three clauses evaluate to 1, the entire formula $\phi = 1$. Therefore this assignment *satisfies* ϕ, and thus $\langle\phi\rangle \in$ 3-SAT.

Now consider the formula:

$$\psi = (x_1 + x_2 + x_3) \cdot (\bar{x_1} + \bar{x_2} + \bar{x_3}) \cdot (x_1 + \bar{x_2} + x_3) \cdot (\bar{x_1} + x_2 + \bar{x_3})$$

One can verify that no assignment of $x_1, x_2, x_3 \in \{0, 1\}$ satisfies all four clauses simultaneously, so $\langle\psi\rangle \notin$ 3-SAT.

Example 2:

Consider the formula:

$$\psi = (x_1 + x_2 + x_3) \cdot (\bar{x_1} + \bar{x_2} + \bar{x_3}) \cdot (x_1 + \bar{x_2} + x_3) \cdot (\bar{x_1} + x_2 + \bar{x_3})$$

To determine if ψ is satisfiable, we must check all $2^3 = 8$ assignments:

x_1	x_2	x_3	C_1	C_2	C_3	C_4	ψ
0	0	0	0	1	0	1	0
0	0	1	1	1	1	0	0
0	1	0	1	1	0	1	0
0	1	1	1	1	1	1	1
1	0	0	1	1	1	0	0
1	0	1	1	0	1	1	0
1	1	0	1	0	1	1	0
1	1	1	1	0	1	0	0

We had to check all $2^3 = 8$ assignments before finding a satisfying one at row 4. In general, for n variables and k clauses, this brute force search takes:

$$O(2^n \cdot k) = O(2^n \cdot n^3) = O(2^n)$$

time, exponential in n. This confirms that finding a satisfying assignment is hard, even though verifying one takes only $O(k)$ time.

36.4 Conjunctive Normal Form (CNF)

Given sentence L with atomic sentences $x_1, \ldots, x_n$, we construct an equivalent sentence H in CNF.

Line	x_1	x_2	$\ldots$	x_n	L
1	1	1	$\ldots$	1	L_1
2	1	1	$\ldots$	0	L_2
3	1	0	$\ldots$	1	L_3
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
2^n	0	0	$\ldots$	0	L_{2^n}

1. Case 1:

   When L is true everywhere, it is a tautology. Any tautological formula in CNF serves as an equivalent, such as $(p + \bar{p})$ or any logically equivalent tautology.

2. Case 2:

   When L is false on some lines, for each line j where $L = 0$, construct clause ϕ_j:

$$\phi_j = \bigvee_{t=1}^{n} (\pm x_t)$$

   where

$$(\pm x_t) = \begin{cases} \bar{x}_t, & \text{if } x_t = 1 \text{ on line } j, \\ x_t, & \text{if } x_t = 0 \text{ on line } j. \end{cases}$$

   Each literal is the opposite of its truth value on line j, making ϕ_j false only on line j.

   Let $j_1, j_2, \ldots, j_s$ index all lines where $L = 0$. Define:

$$H = \phi_{j_1} \cdot \phi_{j_2} \cdot \cdots \cdot \phi_{j_s} = \bigwedge_{i=1}^{s} \left[\bigvee_{t=1}^{n} (\pm x_t) \right]$$

   H is in CNF. Since H is false exactly where L is false, $L \equiv H$.

Example 1:
Let

$$\Sigma = x_1 \to \bar{x}_2$$

First, we rewrite using the definition of implication:

$$\Sigma = (x_1 \to \bar{x}_2 \equiv \bar{x}_1 + \bar{x}_2)$$

Truth Table:

x_1	x_2	$\bar{x}_1$	$\bar{x}_2$	Σ
1	1	0	0	0
1	0	0	1	1
0	1	1	0	1
0	0	1	1	1

CNF Construction:

1. Line 1: $(\bar{x}_1 + \bar{x}_2)$

Thus,

$$\Sigma = (\bar{x}_1 + \bar{x}_2)$$

Example 2:
Let

$$\psi = \overline{(x_1 \equiv x_2)}$$

First, we rewrite using the definition of biconditional:

$$x_1 \leftrightarrow x_2 \equiv (x_1 \cdot x_2) + (\bar{x}_1 \cdot \bar{x}_2)$$

Then, applying negation:

$$\overline{(x_1 \leftrightarrow x_2)} \equiv \overline{[(x_1 \cdot x_2) + (\bar{x}_1 \cdot \bar{x}_2)]}$$

Using De Morgan's law:

$$\psi = (\bar{x}_1 + \bar{x}_2) \cdot (x_1 + x_2)$$

Truth Table:

x_1	x_2	$\bar{x}_1$	$\bar{x}_2$	$x_1 \equiv x_2$	ψ
1	1	0	0	1	0
1	0	0	1	0	1
0	1	1	0	0	1
0	0	1	1	1	0

CNF Construction:
$\psi = 0$ on lines 1 and 4.

1. Line 1: $(\bar{x}_1 + \bar{x}_2)$
2. Line 4: $(x_1 + x_2)$

Thus,

$$\psi = (\bar{x}_1 + \bar{x}_2) \cdot (x_1 + x_2)$$

Example 3:
Let

$$\delta = \bar{x}_1 \vee \overline{(x_2 \wedge x_3)}$$

First, we apply De Morgan's law to $\overline{(x_2 \wedge x_3)}$:

$$\overline{(x_2 \wedge x_3)} \equiv \bar{x}_2 + \bar{x}_3$$

Then, the expression becomes:

$$\delta = \bar{x}_1 + \bar{x}_2 + \bar{x}_3$$

Truth Table:

x_1	x_2	x_3	$\bar{x}_1$	$x_2 \wedge x_3$	$\overline{(x_2 \wedge x_3)}$	δ
1	1	1	0	1	0	0
1	1	0	0	0	1	1
1	0	1	0	0	1	1
1	0	0	0	0	1	1
0	1	1	1	1	0	1
0	1	0	1	0	1	1
0	0	1	1	0	1	1
0	0	0	1	0	1	1

CNF Construction:
$\delta = 0$ on line 1 only.

1. Line 1: $(\bar{x}_1 + \bar{x}_2 + \bar{x}_3)$

Thus,

$$\delta = \bar{x}_1 + \bar{x}_2 + \bar{x}_3$$

Example 4:
Let

$$\zeta = \overline{(x_1 \vee x_2)} \leftrightarrow ((\bar{x}_3 \wedge \bar{x}_1) \rightarrow \bar{x}_2)$$

First, we rewrite using the definition of implication and biconditional:

$$\overline{(x_1 \vee x_2)} \equiv \bar{x}_1 \cdot \bar{x}_2$$

$$(\bar{x}_3 \wedge \bar{x}_1) \rightarrow \bar{x}_2 \equiv \overline{(\bar{x}_3 \cdot \bar{x}_1)} + \bar{x}_2 \equiv (x_3 + x_1) + \bar{x}_2 \equiv x_3 + x_1 + \bar{x}_2$$

Truth Table:

x_1	x_2	x_3	$\overline{(x_1 \vee x_2)}$	$(\bar{x}_3 \wedge \bar{x}_1) \rightarrow \bar{x}_2$	ζ
1	1	1	0	1	0
1	1	0	0	1	0
1	0	1	0	1	0
1	0	0	0	1	0
0	1	1	0	1	0
0	1	0	0	0	1
0	0	1	1	1	1
0	0	0	1	1	1

CNF Construction:
$\zeta = 0$ on lines 1, 2, 3, 4, and 5.

1. Line 1: $(\bar{x}_1 + \bar{x}_2 + \bar{x}_3)$
2. Line 2: $(\bar{x}_1 + \bar{x}_2 + x_3)$
3. Line 3: $(\bar{x}_1 + x_2 + \bar{x}_3)$
4. Line 4: $(\bar{x}_1 + x_2 + x_3)$
5. Line 5: $(x_1 + \bar{x}_2 + \bar{x}_3)$

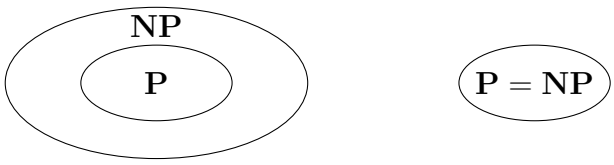
Simplified form:

$$\zeta = (\bar{x}_1) \cdot (\bar{x}_2 + \bar{x}_3)$$

37 The P versus NP

P	the class of languages for which membership can be *decided* quickly.
NP	the class of languages for which membership can be *verified* quickly.

Languages such as HAMPATH and CLIQUE are known to be in **NP**, yet no polynomial time algorithm has been found for either. This suggests that polynomial verifiability is strictly more powerful than polynomial decidability. However, we cannot rule out the possibility that $\mathbf{P} = \mathbf{NP}$, as no one has proven the existence of a single language in **NP** that is provably not in **P**. Whether $\mathbf{P} = \mathbf{NP}$ remains one of the deepest open problems in mathematics and computer science. A positive answer would mean that every problem whose solution can be checked quickly can also be solved quickly.



If $\mathbf{P} \neq \mathbf{NP}$ If $\mathbf{P} = \mathbf{NP}$

38 Polynomial Time Reducibility

A language X is *polynomial-time reducible* to a language Y, written $X \leq_P Y$, if there exists a polynomial-time computable function f such that:

$$x \in X \leftrightarrow f(x) \in Y$$

The function f is called a *polynomial-time reduction* from X to Y. This means:

1. f is computable in polynomial time.
2. If $x \in X$, then $f(x) \in Y$.
3. If $x \notin X$, then $f(x) \notin Y$.

Intuitively, $X \leq_P Y$ means that Y is *at least as hard* as X, if we can solve Y in polynomial time, then we can also solve X in polynomial time by first applying f to transform the input, then solving Y.

39 The Cook-Levin Theorem

The Cook-Levin Theorem states that SAT is NP-complete. That is:

1. SAT $\in$ **NP**, and
2. every $X \in \mathbf{NP}$ satisfies $X \leq_P$ SAT.

This was established independently by Stephen Cook in 1971 and Leonid Levin in 1973. The significance of this theorem is:

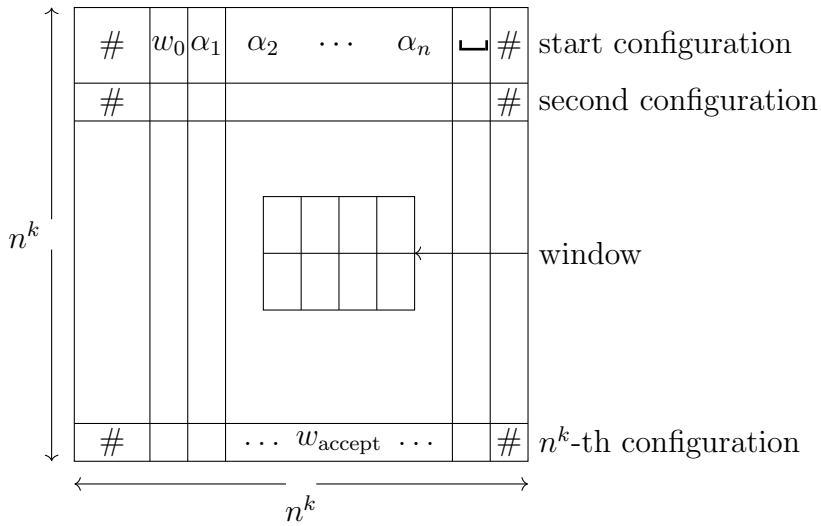
1. If SAT $\in \mathbf{P}$, then $\mathbf{P} = \mathbf{NP}$.
2. SAT serves as the *root* of all NP-completeness proofs, meaning to show any problem Y is NP-complete, it suffices to show SAT $\leq_P Y$.

This gives rise to the reduction chain:

$$\text{SAT} \leq_P \text{3-SAT} \leq_P \text{CLIQUE} \leq_P \text{VERTEX-COVER} \leq_P \text{INDEPENDENT-SET}$$

Let N be a NTM that decides some language $X \in \mathbf{NP}$ in time n^k. We construct a Boolean formula χ such that χ is satisfiable if and only if N accepts input α. A *tableau* for N on α is an $n^k \times n^k$ grid where:

1. Each *row* represents a configuration of N.
2. Each *column* represents a tape cell at a given time step.
3. Row 1 is the start configuration.
4. The last row is the n^k-th configuration.



1. For each cell (i, j) in the tableau and each symbol $s \in C$ where $C = W \cup \Gamma \cup \{\#\}$, we define a Boolean variable:

$$x_{i,j,s} = 1 \leftrightarrow \text{cell } (i, j) \text{ contains symbol } s$$

2. We construct χ as:

$$\chi = \chi_{\text{cell}} \cdot \chi_{\text{start}} \cdot \chi_{\text{accept}} \cdot \chi_{\text{move}}$$

where each component is defined as follows:

(a) χ_{cell}: Every cell contains exactly one symbol:

$$\chi_{\text{cell}} = \bigwedge_{1 \leq i,j \leq n^k} \left[\left(\bigvee_{s \in C} x_{i,j,s} \right) \cdot \left(\bigwedge_{\substack{s,t \in C \\ s \neq t}} \overline{x_{i,j,s} \cdot x_{i,j,t}} \right) \right]$$

(b) χ_{start}: The first row is the start configuration $\#w_0\,\alpha_1\,\alpha_2\cdots\alpha_n\,\sqcup\cdots\#$:

$$\chi_{\text{start}} = x_{1,1,\#}\cdot x_{1,2,w_0}\cdot x_{1,3,\alpha_1}\cdots x_{1,n+2,\alpha_n}\cdot x_{1,n^k,\#}$$

(c) χ_{accept}: Some row contains the accept state w_{accept}:

$$\chi_{\text{accept}} = \bigvee_{1\le i,j\le n^k} x_{i,j,w_{\text{accept}}}$$

(d) χ_{move}: Every 2×3 window in the tableau is legal, i.e. it corresponds to a valid transition of N:

$$\chi_{\text{move}} = \bigwedge_{1\le i,j\le n^k} (\text{the window at } (i,j) \text{ is legal})$$

A 2×3 window is *legal* if it corresponds to a valid transition of N. $V(w_0,\triangle) = \{(w_0,\square,R)\}$ and $V(w_0,\square) = \{(w_1,\Diamond,L),(w_1,\triangle,R)\}$.
Legal windows:

(a)

$\triangle$	w_0	$\square$
w_1	$\triangle$	$\Diamond$

(b)

$\triangle$	w_0	$\square$
$\triangle$	$\triangle$	w_1

(c)

$\#$	$\square$	$\triangle$
$\#$	$\square$	$\triangle$

(d)

$\triangle$	$\square$	$\triangle$
$\triangle$	$\square$	w_1

Illegal windows:

(a)

$\triangle$	$\triangle$	w_0
$\triangle$	$\triangle$	$\square$

(b)

$\square$	$\square$	$\square$
$\Diamond$	$\square$	$\square$

3. The tableau has an accepting computation if and only if χ is satisfiable:
   (a) $\chi_{\text{cell}} = 1$ ensures every cell has exactly one symbol.
   (b) $\chi_{\text{start}} = 1$ ensures the first row is the start configuration.
   (c) $\chi_{\text{accept}} = 1$ ensures some row reaches q_{accept}.
   (d) $\chi_{\text{move}} = 1$ ensures each row legally follows from the previous.
4. The tableau has n^{2k} cells, each with $|C|$ variables, so the total number of variables is $O(n^{2k})$. Each of the four components is polynomial in n^k, so $|\chi|$ is polynomial in n. Therefore the reduction runs in polynomial time.

Example 1:
Let

$$\text{SAT} \le_p \text{3SAT}$$

Given a SAT formula $\phi = \bigwedge_{m=1}^{k} S_m$ over variables $x_1,\ldots,x_n$, we construct an equisatisfiable 3SAT formula ϕ' by converting each clause S_m into a conjunction of 3-literal clauses as follows.

1. $S_m = (x_1)$ has exactly 1 literal. Introduce fresh variables y_1, y_2 and replace S_m with:
$$(x_1 + y_1 + y_2) \cdot (x_1 + y_1 + \bar{y}_2) \cdot (x_1 + \bar{y}_1 + y_2) \cdot (x_1 + \bar{y}_1 + \bar{y}_2)$$
This is satisfied if and only if $x_1 = 1$, regardless of y_1, y_2.

2. $S_m = (x_1 + x_2)$ has exactly 2 literals. Introduce fresh variable y and replace S_m with:
$$(x_1 + x_2 + y) \cdot (x_1 + x_2 + \bar{y})$$
This is satisfied if and only if $x_1 + x_2 = 1$, regardless of y.

3. $S_m = (x_1 + x_2 + x_3)$ has exactly 3 literals. Keep S_m unchanged.

4. $S_m = (x_1 + x_2 + \cdots + x_k)$ has $k > 3$ literals. Introduce fresh variables $y_1, \ldots, y_{k-3}$ and replace S_m with:
$$(x_1 + x_2 + y_1) \cdot (\bar{y}_1 + x_3 + y_2) \cdot (\bar{y}_2 + x_4 + y_3) \cdot \ \cdots \ \cdot (\bar{y}_{k-3} + x_{k-1} + x_k)$$

Cases 1–3 are straightforward by inspection. For Case 4, we show S_m is satisfiable iff the replacement clauses are satisfiable.

($\rightarrow$) Suppose $S_m = 1$, so some $x_i = 1$. Set $y_1 = \cdots = y_{i-2} = 1$ and $y_{i-1} = \cdots = y_{k-3} = 0$. Then every clause in the chain is satisfied.

($\leftarrow$) Suppose all replacement clauses are satisfied but $S_m = 0$, i.e., $x_i = 0$ for all i. Then from $(x_1 + x_2 + y_1)$ we get $y_1 = 1$. From $(\bar{y}_1 + x_3 + y_2)$ and $y_1 = 1$, $x_3 = 0$, we get $y_2 = 1$. Continuing this chain, we eventually reach $(\bar{y}_{k-3} + x_{k-1} + x_k)$ with $y_{k-3} = 1$ and $x_{k-1} = x_k = 0$, which is unsatisfied, a contradiction.

Each clause S_m of length k produces at most $k-2$ new clauses and $k-3$ fresh variables. Summing over all clauses, the output size is $O(|\phi|)$, so the reduction runs in polynomial time.

Therefore
$$\text{SAT} \leq_p \text{3SAT}$$
.

Example 2:
Let
$$\text{3SAT} \leq_p \text{CLIQUE}$$

Given a 3SAT formula $\phi = \bigwedge_{m=1}^{k} S_m$ where each $S_m = (x_1^m + x_2^m + x_3^m)$, we construct a graph $G = (V, E)$ such that ϕ is satisfiable iff G has a clique of size k.

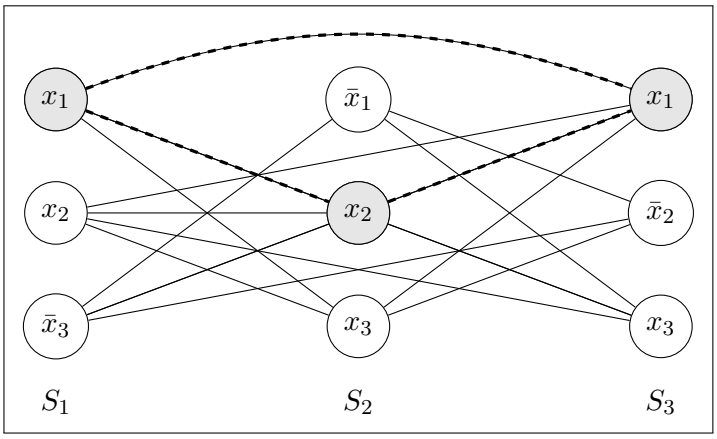
1. For each clause S_m and each literal x_i^m, create a node (i, m). This gives $|V| = 3k$ nodes, grouped into k triples.

2. Add an edge between (i, m) and (j, m') iff:
   (a) $m \neq m'$ (nodes are in different clauses), and
   (b) $x_i^m \neq \bar{x}_j^{m'}$ (the literals do not contradict each other).

let $k = 3$ and:

$$\phi = \underbrace{(x_1 + x_2 + \bar{x}_3)}_{S_1} \cdot \underbrace{(\bar{x}_1 + x_2 + x_3)}_{S_2} \cdot \underbrace{(x_1 + \bar{x}_2 + x_3)}_{S_3}$$

The constructed graph G is:



The dashed triangle is a clique of size $k = 3$, corresponding to the satisfying assignment $x_1 = 1, x_2 = 1$.

We show ϕ is satisfiable iff G has a clique of size k.

($\rightarrow$) Suppose ϕ is satisfiable. Then there exists an assignment that sets at least one literal $x_i^m = 1$ in each clause S_m. Pick one such literal per clause to get a set $C = \{(i_1, 1), (i_2, 2), \ldots, (i_k, k)\}$ of k nodes. Any two nodes (i_m, m) and $(i_{m'}, m')$ in C are from different clauses, and since both literals are set to 1, they cannot contradict each other. Thus every pair in C is connected, so C is a clique of size k.

($\leftarrow$) Suppose G has a clique C of size k. Since no two nodes in the same triple are connected, C contains exactly one node (i_m, m) per clause S_m. For each such node, set $x_{i_m}^m = 1$. This assignment is consistent since no two nodes in C contradict each other. This assignment satisfies every clause, so ϕ is satisfiable.

The graph has $3k$ nodes and at most $\binom{3k}{2}$ edges, both polynomial in $|\phi|$. The construction runs in polynomial time.

Thus

$$\text{3SAT} \leq_p \text{CLIQUE}$$

Example 3:

Let

$$\text{3SAT} \leq_p \text{VERTEX COVER}$$

Given a 3SAT formula $\phi = \bigwedge_{m=1}^{k} S_m$ where each $S_m = (x_1^m + x_2^m + x_3^m)$, we construct a graph $G = (V, E)$ and integer k' such that ϕ is satisfiable $\leftrightarrow$ G has a vertex cover of size k'.

1. For each variable x_i, create two nodes x_i and $\bar{x}_i$ connected by an edge. This gives $2n$ nodes and n edges, one per variable.

2. For each clause $S_m = (x_i^m + x_j^m + x_l^m)$, create a triangle of 3 nodes x_i^m, x_j^m, x_l^m connected to each other. This gives $3k$ more nodes and $3k$ more edges.

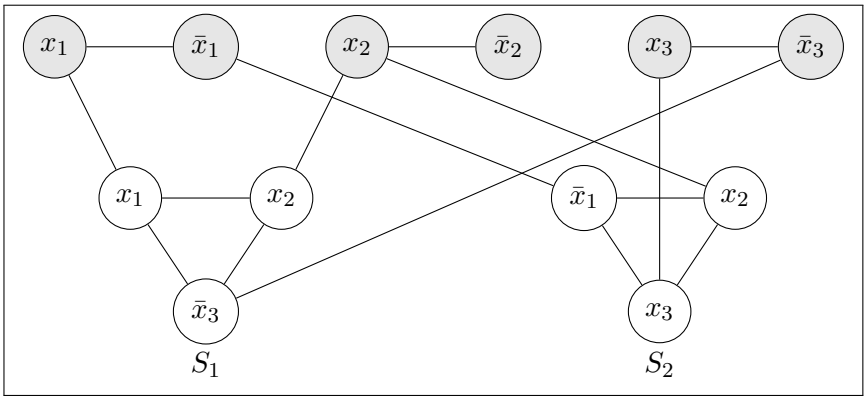
3. For each literal node in a clause triangle, add an edge connecting it to its corresponding variable node.

4. Set $k' = n + 2k$.

For example, let $n = 3$, $k = 2$ and:

$$\phi = \underbrace{(x_1 + x_2 + \bar{x}_3)}_{S_1} \cdot \underbrace{(\bar{x}_1 + x_2 + x_3)}_{S_2}$$

The constructed graph G is:



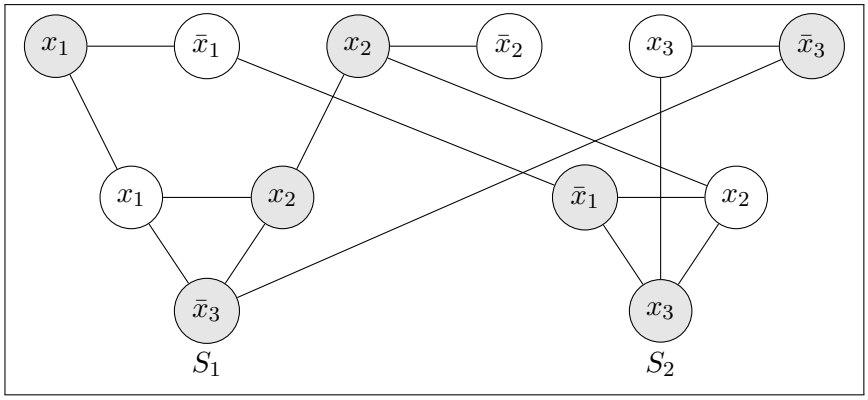
Variable gadgets (top) connected to clause triangles (bottom). A vertex cover of size $k' = n + 2k = 7$ exists iff ϕ is satisfiable.

We show ϕ is satisfiable iff G has a vertex cover of size $k' = n + 2k$.

($\rightarrow$) Suppose ϕ is satisfiable under some assignment. For each variable x_i, add the node set to 1 (either x_i or $\bar{x}_i$) to the cover, this covers all variable edges, contributing n nodes. For each clause triangle S_m, since at least one literal is 1 and already in the cover via its variable node, pick the remaining 2 nodes of the triangle, contributing $2k$ nodes. This gives a cover of size $n + 2k = k'$.

($\leftarrow$) Suppose G has a vertex cover C of size $k' = n + 2k$. Each variable edge $(x_i, \bar{x}_i)$ requires at least one endpoint in C, so at least n nodes come from variable gadgets. Each clause triangle requires at least 2 nodes in C to cover its 3 edges, so at least $2k$ nodes come from triangles. Since $|C| = n + 2k$, exactly one node per variable edge and exactly 2 nodes per triangle are in C. Set $x_i = 1$ if $x_i \in C$, else $x_i = 0$. The uncovered triangle node (not in C) must be connected to a variable node in C, so its literal is set to 1, satisfying every clause.

The graph has $2n + 3k$ nodes and $n + 6k$ edges, both polynomial in $|\phi|$. The construction runs in polynomial time.



Grey nodes form the vertex cover of size $k' = n + 2k = 7$, assignment $x_1 = 1, x_2 = 1, \bar{x}_3 = 1$.

Hence

$$\text{3SAT} \leq_p \text{VERTEX COVER}$$

Example 4:
Let

$$\text{3SAT} \leq_p \text{INDEPENDENT SET}$$

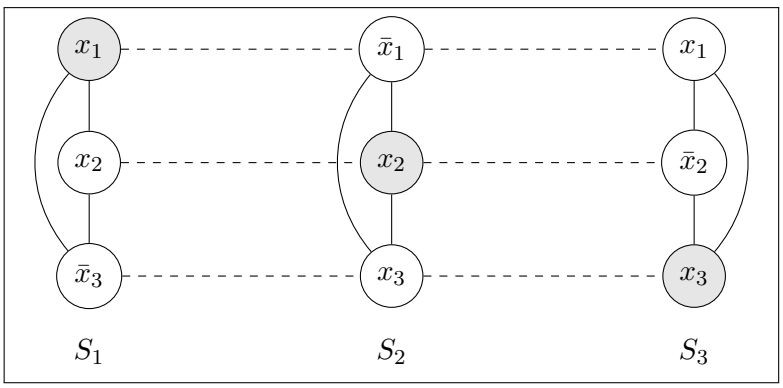
Given a 3SAT formula $\phi = \bigwedge_{m=1}^{k} S_m$ where each $S_m = (x_1^m + x_2^m + x_3^m)$, we construct a graph $G = (V, E)$ and integer k such that ϕ is satisfiable iff G has an independent set of size k.

1. For each clause $S_m = (x_i^m + x_j^m + x_l^m)$, create a triangle of 3 nodes x_i^m, x_j^m, x_l^m connected to each other. This gives $|V| = 3k$ nodes grouped into k triples.
2. For each pair of nodes $(x_i^m, x_j^{m'})$ from different clauses $m \neq m'$, add an edge if they contradict each other, i.e. $x_i^m = \bar{x}_j^{m'}$.
3. Set the target independent set size to k.

let $k = 3$ and:

$$\phi = \underbrace{(x_1 + x_2 + \bar{x}_3)}_{S_1} \cdot \underbrace{(\bar{x}_1 + x_2 + x_3)}_{S_2} \cdot \underbrace{(x_1 + \bar{x}_2 + x_3)}_{S_3}$$

The constructed graph G is:



Grey nodes form an independent set of size $k = 3$, corresponding to the satisfying assignment $x_1 = 1, x_2 = 1, x_3 = 1$.

In conclusion

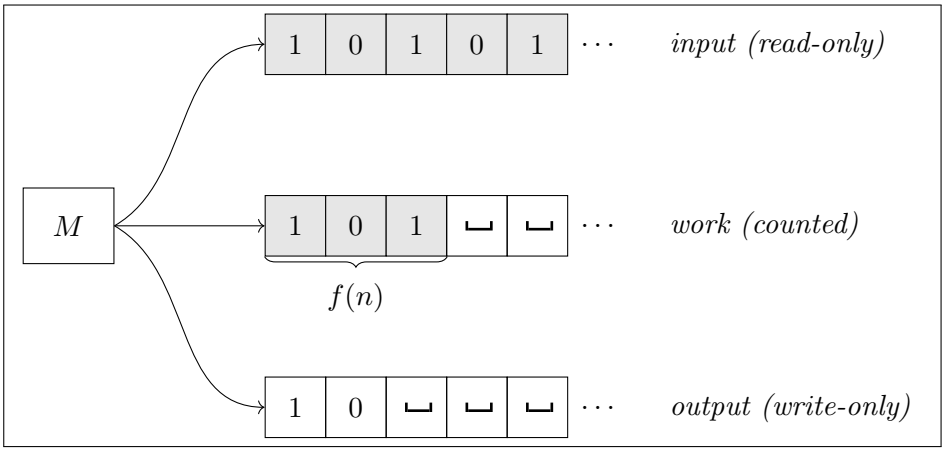
$$3\text{SAT} \leq_p \text{INDEPENDENT SET}$$

40 Space Complexity

Let $f : \mathbb{N} \to \mathbb{N}$ be a function. The space complexity classes $\mathbf{SPACE}(f(n))$ and $\mathbf{NSPACE}(f(n))$ are defined as:

$$\mathbf{SPACE}(f(n)) = \{L \mid L \text{ is a language decided by an } O(f(n)) \text{ space DTM}\}$$

$$\mathbf{NPSPACE}(f(n)) = \{L \mid L \text{ is a language decided by an } O(f(n)) \text{ space NTM}\}$$



Only the *work tape* cells count toward $f(n)$; the input and output tapes are excluded.

40.1 PSPACE and NPSPACE

PSPACE is the class of languages decidable by a DTM in polynomial space:

$$\mathbf{PSPACE} = \bigcup_{k \geq 1} \mathbf{SPACE}(n^k)$$

NPSPACE is the class of languages decidable by a NTM in polynomial space:

$$\mathbf{NPSPACE} = \bigcup_{k \geq 1} \mathbf{NPSPACE}(n^k)$$

Example:
Given

$$3\text{SAT} \in \mathbf{PSPACE}$$

Let

$$\phi = \bigwedge_{m=1}^{k} S_m$$

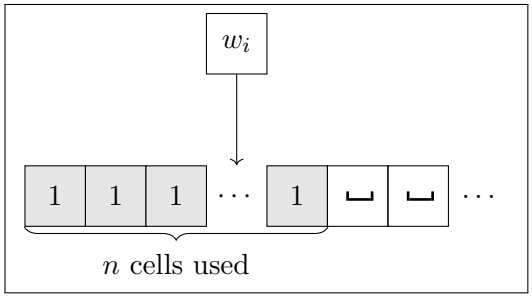
.

where $S_m = (x_i + x_j + \bar{x}_k), i, j, k \in \{1, \ldots, n\}$ The algorithm iterates over all 2^n assignments, storing only the current assignment on the work tape.

$$\phi = \begin{cases} 1 & \text{if } S_m = 1 \text{ for all } m \\ 0 & \text{otherwise} \end{cases}$$

x_1	x_2	x_3	$\cdots$	x_{i-1}	x_i	S_1	S_2	$\cdots$	S_k	ϕ
0	0	0	$\cdots$	0	0	0	1	$\cdots$	1	0
0	0	0	$\cdots$	0	1	0	1	$\cdots$	1	0
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
1	1	1	$\cdots$	1	1	1	1	$\cdots$	1	1

The work tape at the accepting step holds only the current assignment:



Work tape stores only the current assignment $x_1 = 1,\ x_2 = 1,\ \ldots,\ x_n = 1$, exactly n cells.

Since the work tape uses $O(n)$ space to store the current assignment, and $O(\log n)$ additional space to index the current clause $m \in \{1, \ldots, k\}$, the total space is $O(n)$, which is polynomial in the input length. Thus we conclude 3SAT $\in$ **PSPACE**.

40.2 LOGSPACE (L) and NLOGSPACE (NL)

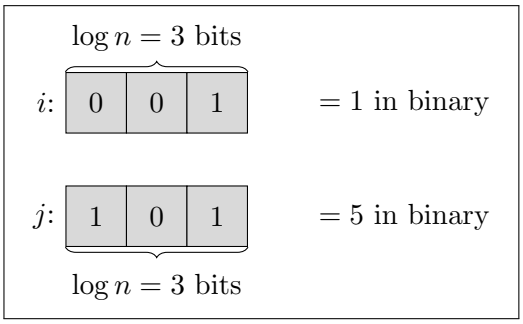
LOGSPACE is the class of languages decidable by a DTM in logarithmic space:

$$\mathbf{LOGSPACE} = \mathbf{SPACE}(\log n)$$

Example:

Checking if a string $x = x_1 x_2 \cdots x_n$ is a palindrome. The DTM uses two counters i and j on the work tape, where i starts at 1 and j starts at n. At each step, it reads x_i and x_j from the input tape and checks if they are equal, then increments i and decrements j. It accepts iff $x_i = x_j$ for all $i \leq j$. Since i and j are counters up to n, each uses $O(\log n)$ bits, giving total work tape space $O(\log n)$.

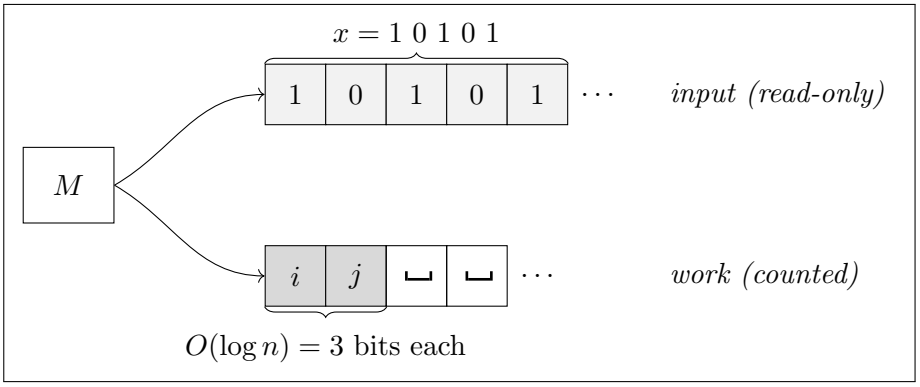
In detail, on input $x = 10101$ ($n = 5$), storing $i = 1$ and $j = 5$ requires only $\lfloor \log_2 n \rfloor + 1$ bits each.



Counters $i = 001_2 = 1$ and $j = 101_2 = 5$ stored in 3 bits each, total $O(\log n)$ space.

The algorithm proceeds as follows:

step	i	j	x_i	x_j
1	$001_2 = 1$	$101_2 = 5$	1	1
2	$010_2 = 2$	$100_2 = 4$	0	0
3	$011_2 = 3$	$011_2 = 3$	1	1



Work tape stores only two counters i and j, each $O(\log n)$ bits, to track positions from both ends of the input.

Since the work tape never exceeds $O(\log n)$ space, we conclude PALINDROME $\in$ **LOGSPACE**.

Theorem: $\mathbf{L} \subseteq \mathbf{P}$

Let M be a DTM deciding a language $X \in L$, using $O(\log n)$ space on input x of length n. We bound the number of distinct configurations of M.

A *configuration* of M is a tuple

$$(W,\, p_1,\, p_2,\, t)$$

where W is the current state, $p_1 \in \{1, \ldots, n\}$ is the input head position, p_2 is the work tape head position, and t is the work tape content. Since M uses $O(\log n)$ work tape space:

1. States: $|Q| = O(1)$ choices.
2. Input head position: n choices.
3. Work tape head position: $O(\log n)$ choices.
4. Work tape content: $2^{O(\log n)} = n^{O(1)}$ choices.

Hence the total number of distinct configurations is

$$O(1) \cdot n \cdot O(\log n) \cdot n^{O(1)} = n^{O(1)},$$

which is polynomial in n.

Since M decides X, it never loops and hence never repeats a configuration. Therefore M halts within $n^{O(1)}$ steps, so $X \in \mathbf{P}$. Since $X \in \mathbf{L}$ was arbitrary, $\mathbf{L} \subseteq \mathbf{P}$.

NLOGSPACE is the class of languages decidable by a NTM in logarithmic space:

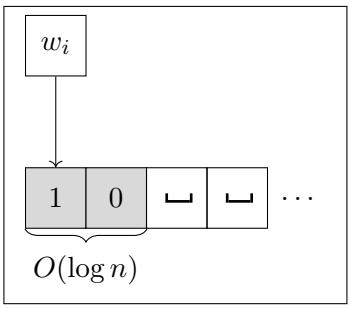
$$\mathbf{NLLOGSPACE} = \mathbf{NPSPACE}(\log n)$$

Example:

2SAT. Given a formula $\phi = \bigwedge_{m=1}^{k} S_m$ where each $S_m = (x_i + x_j)$, the NTM nondeterministically guesses a satisfying assignment one variable at a time, storing only the current variable index and its value on the work tape, using $O(\log n)$ space.

let:

$$\phi = \underbrace{(x_1 + x_2)}_{S_1} \cdot \underbrace{(\bar{x}_1 + x_3)}_{S_2} \cdot \underbrace{(\bar{x}_2 + \bar{x}_3)}_{S_3}$$



Work tape stores only the current variable index and value, e.g. $x_1 = 1, x_2 = 0$, using $O(\log n)$ space.

The assignment $x_1 = 1, x_2 = 0, x_3 = 0$ satisfies all clauses:

x_1	x_2	x_3	S_1	S_2	S_3	ϕ
1	0	0	1	0	1	0
1	0	1	1	1	1	1
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

At each step the NTM stores only $O(\log n)$ bits, the index of the current variable and its guessed value, never the full assignment, confirming 2SAT $\in$ **NLLOGSPACE**.

Theorem: NL $\subseteq$ SPACE$(\log^2 n)$

We use *Savitch's theorem*, which states that for $f(n) \geq \log n$:

$$\mathbf{NSPACE}(f(n)) \subseteq \mathbf{SPACE}(f(n)^2)$$

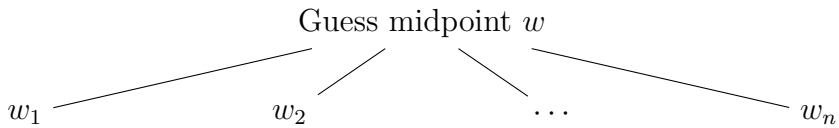
Since **NL** = **NSPACE**$(\log n)$, applying Savitch's theorem with $f(n) = \log n$ gives:

$$\mathbf{NL} = \mathbf{NSPACE}(\log n) \subseteq \mathbf{SPACE}(\log^2 n)$$

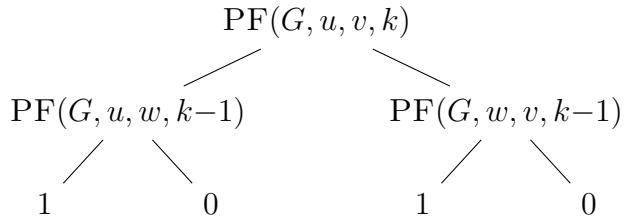
The key idea is the $\textsc{PathFind}(G, u, v, k)$ procedure, which determines whether there exists a path from u to v in at most 2^k steps, defined recursively as:

$$\mathrm{PF}(G, u, v, k) = \bigvee_{w \in V} \left[\mathrm{PF}(G, u, w, k-1) \wedge \mathrm{PF}(G, w, v, k-1)\right]$$

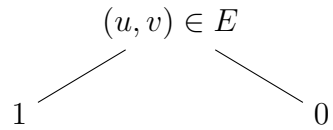
1. Step 1: Guess midpoint $w \in V$ at each recursion level.



2. Step 2: Recurse on both halves; each subproblem checks reachability in 2^{k-1} steps.



3. Step 3: Base case at depth $k = \lceil \log_2 n \rceil$, check direct edge in $O(1)$.



Each level stores one midpoint $w \in V$ requiring $O(\log n)$ bits, and the recursion has depth $k = \lceil \log_2 n \rceil$, giving total space:

$$O(\log n) \times O(\log n) = O(\log^2 n)$$

Since the DTM simulates M using $O(\log^2 n)$ space, we conclude $\mathbf{NL} \subseteq \mathbf{SPACE}(\log^2 n)$.

Theorem: NL $\subseteq$ P

Let $X \in \mathbf{NL}$, decided by an NTM M using $O(\log n)$ space on input x of length n. We construct the *configuration graph* G_M of M, where:

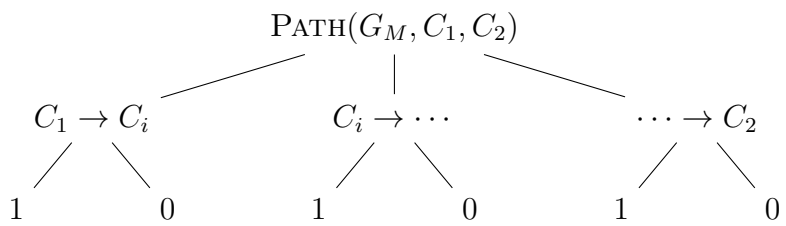
1. each node is a configuration $(W,\, p_1,\, p_2,\, t)$ of M, where W is the current state, p_1 is the input head position, p_2 is the work tape head position, and t is the work tape content

2. each edge (C_i, C_j) represents a single nondeterministic transition from C_i to C_j

Since M uses $O(\log n)$ space, the number of configurations is:

$$|W| \cdot n \cdot O(\log n) \cdot 2^{O(\log n)} = n^{O(1)}$$

so G_M has polynomially many nodes and edges and can be constructed in polynomial time. M accepts x iff there exists a path in G_M from the initial configuration C_1 to the accepting configuration C_2.



solvable by BFS or DFS in time $O(|V| + |E|) = n^{O(1)}$.
Hence $X \in \mathbf{P}$, and since $X \in \mathbf{NL}$ was arbitrary, $\mathbf{NL} \subseteq \mathbf{P}$.

40.3 L-Reduction

A *logspace reduction* from X to Y, written $X \leq_L Y$, is a function $f : \Sigma^* \to \Sigma^*$ computable in $O(\log n)$ work tape space such that $x \in X \leftrightarrow f(x) \in Y$. Formally,

$$X \leq_L Y \stackrel{\text{def}}{=} \exists\, f \text{ computable in } O(\log n) \text{ space}: \ \forall x,\ x \in X \leftrightarrow f(x) \in Y$$

We use logspace rather than polynomial-time reductions because within **NL**, polynomial-time reductions are too powerful and would trivialize hardness.

Logspace reductions are transitive, closed under composition, and strictly weaker than polynomial-time reductions: $X \leq_L Y \to X \leq_p Y$, but not vice versa.

Example:

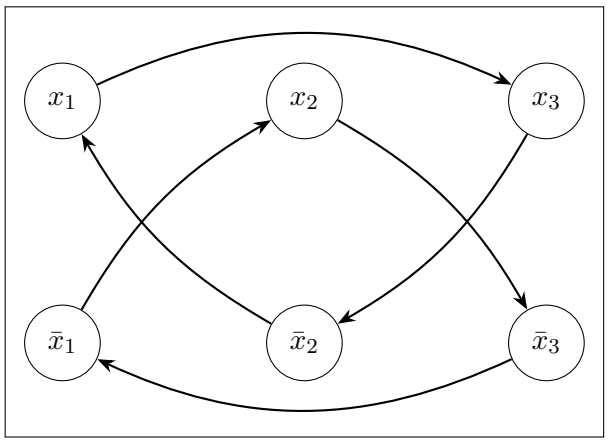
We show 2-SAT $\leq_L$ ST-CONNECTIVITY. Given a 2-SAT formula ϕ over variables $x_1, \ldots, x_n$, the reduction f builds the *implication graph* $G_\phi = (V, E)$:

1. For each variable x_i, include two vertices: x_i and $\bar{x}_i$.
2. For each clause $(a \vee b)$, include two directed edges: $\bar{a} \to b$ and $\bar{b} \to a$.

For example, the formula

$$\phi = (x_1 \vee x_2) \wedge (\bar{x}_1 \vee x_3) \wedge (\bar{x}_2 \vee \bar{x}_3)$$

produces the following implication graph:



The key property is that ϕ is unsatisfiable if and only if there exists a variable x_i such that x_i and $\bar{x}_i$ are mutually reachable in G_ϕ, i.e.,

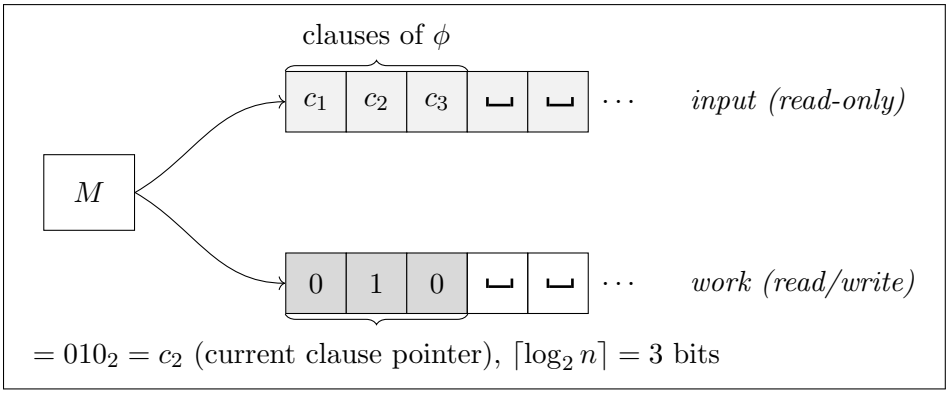
ϕ is unsatisfiable iff some variable x_i has mutual reachability in G_ϕ:

$$\phi \text{ is unsat } \leftrightarrow \exists\, i: \ \big(x_i \to \bar{x}_i \ \wedge\ \bar{x}_i \to x_i\big) \text{ in } G_\phi.$$

Checking each variable in the example yields no mutual reachability, so ϕ is *satisfiable*. The assignment $x_1 = 1, x_2 = 0, x_3 = 1$ witnesses this:

$$\begin{aligned}
(x_1 \vee x_2)&: \quad 1 \vee 0 = 1 \\
(\bar{x}_1 \vee x_3)&: \quad 0 \vee 1 = 1 \\
(\bar{x}_2 \vee \bar{x}_3)&: \quad 1 \vee 0 = 1
\end{aligned}$$

Define $f(\phi)$ to output G_ϕ along with all pairs $(x_i, \bar{x}_i)$ and $(\bar{x}_i, x_i)$ for STCON to query. The construction uses $O(\log n)$ space: a pointer to the current clause suffices to generate edges locally, and with m clauses this pointer needs only $\lceil \log_2 m \rceil$ bits.



Thus, 2-SAT $\leq_L$ STCON.

Theorem: NL = coNL, *(Immerman–Szelepcsényi, 1987–88)*

It suffices to show $\overline{\text{PATH}} \in \mathbf{NL}$, since PATH is **NL**-complete under logspace reductions.

Let $G = (V, E)$ be a directed graph with $|V| = n$, source s, and target t. Define

$$R_i = \{\, v \in V \mid \text{there is a path from } s \text{ to } v \text{ of length } \leq i \,\}, \qquad r_i = |R_i|.$$

Explicitly:

$$\begin{aligned} R_0 &= \{s\} \\ R_1 &= \{s\} \cup \{\text{direct neighbours of } s\} \\ R_2 &= R_1 \cup \{\text{vertices reachable in exactly 2 steps}\} \\ &\vdots \\ R_{n-1} &= \text{all vertices reachable from } s \end{aligned}$$

since any simple path in a graph of n vertices has length at most $n-1$, so R_{n-1} is exactly the reachable set from s.

We describe a nondeterministic logspace machine that computes r_i exactly for $i = 0, 1, \ldots, n-1$ using inductive counting, and then verifies $t \notin R_{n-1}$.

Base case: $r_0 = 1$, since only s is reachable in 0 steps. Inductive step: Assume r_i is known. We compute r_{i+1} as follows.

We iterate over all vertices $v \in V$, and for each v determine whether $v \in R_{i+1}$. A vertex v is in R_{i+1} iff either:

1. $v \in R_i$, or
2. there exists $u \in R_i$ such that $(u, v) \in E$.

To verify this in logspace, the machine performs a *global counting procedure*:

1. Initialize a counter $c = 0$.
2. For each vertex $v \in V$:
   (a) Nondeterministically guess whether $v \in R_{i+1}$.
   (b) Verify the guess:

i. If claiming $v \in R_i$, re-run the level-i counting procedure to verify membership in R_i.

ii. Otherwise, verify that there exists $u \in R_i$ with $(u, v) \in E$, by scanning all $u \in V$ and checking membership in R_i.

(c) If the verification succeeds, increment c.

3. After processing all vertices:

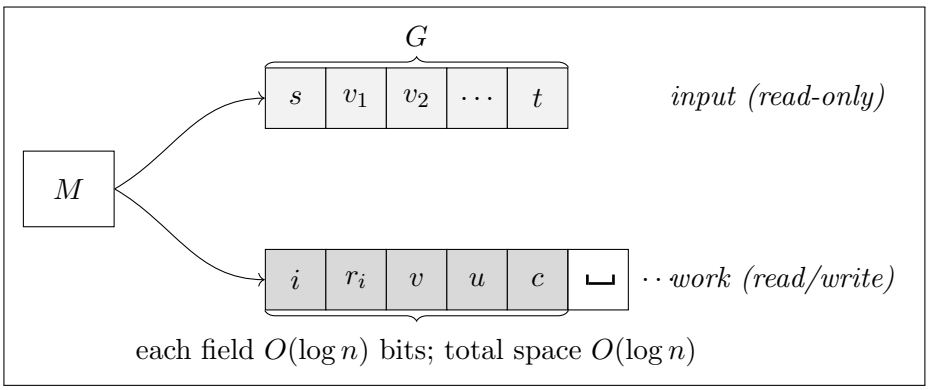
   (a) If $c = r_{i+1}$, accept this stage and proceed to $i + 1$.

   (b) If $c \neq r_{i+1}$, reject this computation path immediately.

Crucially, membership in R_i is always verified by a complete pass over all vertices, ensuring that exactly r_i vertices are accepted as being in R_i. This enforces *global consistency* of the computation.

The machine maintains only:

$$(i,\ r_i,\ v,\ u,\ c),$$



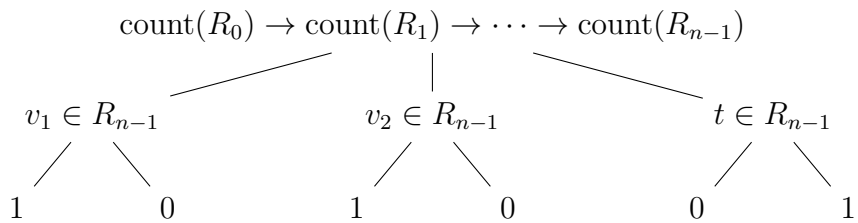
each requiring $O(\log n)$ bits. All iterations over V are performed by reusing these pointers, so the total space is $O(\log n)$.

After computing r_{n-1}, the machine performs one final pass:

1. Enumerate all $v \in V$ and verify membership in R_{n-1}.

2. Count exactly r_{n-1} such vertices; if the final count $\neq r_{n-1}$, reject this computation path immediately.

3. Reject if t is ever verified to be in R_{n-1}.



If $t \notin R_{n-1}$, there exists an accepting computation path that correctly counts all r_i, certifies $c = r_{i+1}$ at every stage, and excludes t in the final pass. If $t \in R_{n-1}$, then either some stage yields $c \neq r_{i+1}$ (forcing rejection of that path), or every consistent computation includes t in the final pass and rejects explicitly.

Thus $\overline{\text{PATH}} \in \mathbf{NL}$, implying $\mathbf{coNL} \subseteq \mathbf{NL}$. The reverse inclusion $\mathbf{NL} \subseteq \mathbf{coNL}$ follows by the same argument with accept and reject swapped, so $\mathbf{NL} = \mathbf{coNL}$.

40.4 NL-complete

A language L is **NL**-complete if:

1. $L \in \mathbf{NL}$, and
2. $\forall X \in \mathbf{NL} : X \leq_L L$

a function f computable by a DTM in $O(\log n)$ space such that $x \in X \leftrightarrow f(x) \in L$.

$$\textbf{NL-complete} = \{L \in \mathbf{NL} \mid \forall X \in \mathbf{NL} : X \leq_L L\}$$

In other words, L is **NL**-complete if it is in **NL** and is the *hardest* problem in **NL**, meaning every other problem in **NL** reduces to it via logspace reduction. The canonical **NL**-complete problem is Reachability

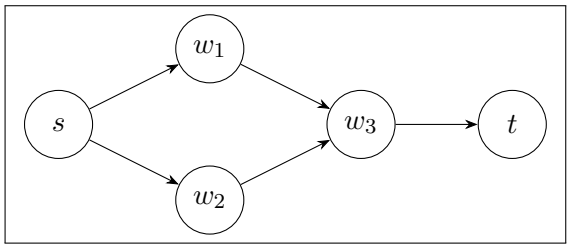
The relationship to other classes is:

$$\mathbf{L} \subseteq \textbf{NL-complete} \subseteq \mathbf{NL} \subseteq \mathbf{P}$$

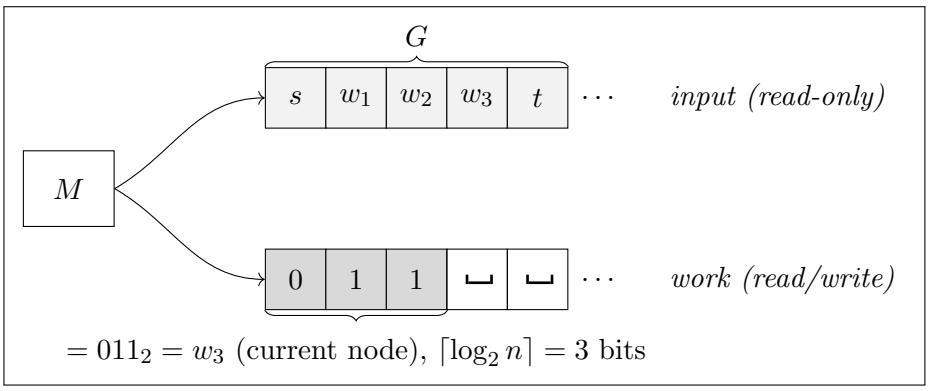
If any **NL**-complete problem is in **L**, then $\mathbf{L} = \mathbf{NL}$.

Example:

Consider the directed graph $G = (V, E)$ with $V = \{s, w_1, w_2, w_3, t\}$ and edges $E = \{(s, w_1), (s, w_2), (w_1, w_3), (w_2, w_3), (w_3, t)\}$. Is there a path from s to t?



The nodes are indexed in binary as: $s = 0 = 000_2$, $w_1 = 1 = 001_2$, $w_2 = 2 = 010_2$, $w_3 = 3 = 011_2$, $t = 4 = 100_2$. So storing the current node $w_3 = 3$ requires $\lceil \log_2 5 \rceil = 3$ bits, giving 011_2.



The path $s \to w_1 \to w_3 \to t$ exists, so the answer is 1. The NTM guesses each intermediate node in $O(\log n)$ bits, giving total space $O(\log n)$, confirming STCON $\in$ **NL**.

40.5 PSPACE-complete

Formally:

$$\textbf{PSPACE-complete} = \{L \in \textbf{PSPACE} \mid \forall X \in \textbf{PSPACE} : X \leq_p L\}$$

In other words, L is **PSPACE**-complete if it is in **PSPACE** and is the *hardest* problem in **PSPACE**, every other problem in **PSPACE** reduces to it in polynomial time.

The relationship to other classes is:

$$\textbf{NP} \subseteq \textbf{PSPACE-complete} \subseteq \textbf{PSPACE}$$

If any **PSPACE**-complete problem is in **P**, then **P** = **PSPACE**. Similarly, if any **PSPACE**-complete problem is in **NP**, then **NP** = **PSPACE**.

41 Savitch's Theorem

For any function $f : \mathbb{N} \to \mathbb{R}^+$, where $f(n) \geq \log n$:

$$\textbf{NPSPACE}(f(n)) \subseteq \textbf{SPACE}(f(n)^2)$$

In other words, any problem solvable by a NTM in $f(n)$ space is also solvable by a DTM in $f(n)^2$ space. The nondeterminism can be eliminated at the cost of *squaring* the space. The most important consequence is:

$$\textbf{NPSPACE} = \textbf{PSPACE}$$

Let M be a NTM using $f(n)$ space. A *configuration* of M is a complete description of its current state, head position, and work tape contents. Since the work tape has $O(f(n))$ cells, the total number of distinct configurations is at most:

$$|W| = 2^{O(f(n))}$$

M accepts input w iff there exists a sequence of configurations:

$$w_{\text{start}} \to w_1 \to w_2 \to \cdots \to w_{\text{accept}}$$

So the problem reduces to whether w_{start} can reach w_{accept} in the configuration graph of M. Define:

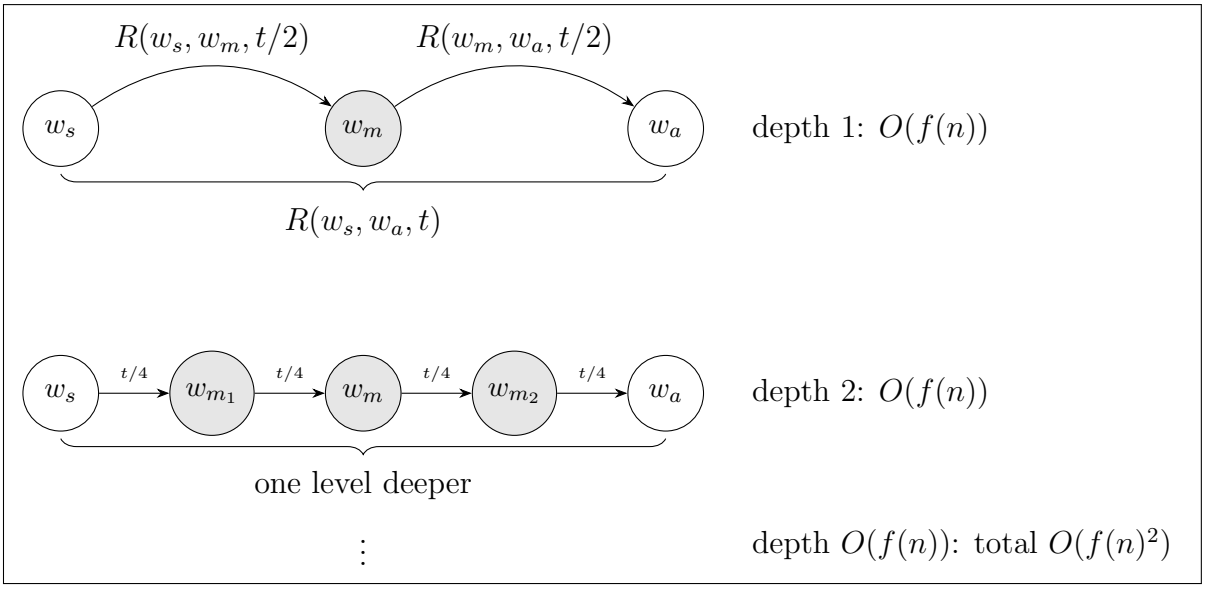
$$R(w, w', t) = \begin{cases} 1 & \text{if } w \text{ can reach } w' \text{ in at most } t \text{ steps} \\ 0 & \text{otherwise} \end{cases}$$

We compute $R(w, w', t)$ recursively by guessing a midpoint configuration w_m:

$$R(w, w', t) = \exists\, w_m : R(w, w_m, \lceil t/2 \rceil) \wedge R(w_m, w', \lfloor t/2 \rfloor)$$

with base case:

$$R(w, w', 1) = \begin{cases} 1 & \text{if } w = w' \text{ or } w \to w' \text{ in one step} \\ 0 & \text{otherwise} \end{cases}$$



The recursion splits $R(w_s, w_a, t)$ via midpoint w_m, halving t at each level.

The maximum number of steps is $t = 2^{O(f(n))}$, so the recursion depth is:

$$\log t = \log 2^{O(f(n))} = O(f(n))$$

At each level we store the current configurations w, w', w_m and the value of t, each using $O(f(n))$ space. With $O(f(n))$ levels, the total space is:

$$O(f(n)) \times O(f(n)) = O(f(n)^2)$$

The DTM therefore simulates M by computing $R(w_{\text{start}}, w_{\text{accept}}, 2^{O(f(n))})$ using $O(f(n)^2)$ space, giving $\mathbf{NPSPACE}(f(n)) \subseteq \mathbf{SPACE}(f(n)^2)$. Setting $f(n) = n^k$ for any $k \geq 1$ gives $f(n)^2 = n^{2k}$, so $\mathbf{NPSPACE} \subseteq \mathbf{PSPACE}$. Since trivially $\mathbf{PSPACE} \subseteq \mathbf{NPSPACE}$, we conclude:

$$\mathbf{PSPACE} = \mathbf{NPSPACE}$$

42 True Quantified Boolean Formula (TQBF)

A Quantified Boolean Formula (QBF) is a boolean formula in which every variable is bound by either $\exists$ or $\forall$ quantifier:

$$Q_1 x_1 \; Q_2 x_2 \; \cdots \; Q_n x_n \; \phi(x_1, x_2, \ldots, x_n), \quad Q_i \in \{\exists, \forall\}$$

where ϕ is a propositional formula over $x_1, \ldots, x_n$. Since every variable is quantified, the formula is *closed*, it evaluates to 1 or 0 with no assignment needed. The TQBF problem is defined as:

$$\text{TQBF} = \{ \; \langle \phi \rangle \mid \phi \text{ is a true fully QBF } \}$$

Example 1.
Given

$$\phi_1 = \forall x \; \exists y \; [(x + y) \cdot (\bar{x} + \bar{y})]$$

$\forall x$ checks every x, and for each x, $\exists y$ gets to respond:

$$\forall x = \begin{cases} x = 0,\ y = 1: & (0+1)\cdot(1+0) = 1\cdot 1 = 1 \\ x = 1,\ y = 0: & (1+0)\cdot(0+1) = 1\cdot 1 = 1 \end{cases}$$

Both pass, so $\phi_1 = 1$, hence $\phi_1 \in$ TQBF.
Example 2.
Given

$$\phi_2 = \exists y\ \forall x\ [(x+y)\cdot(\bar{x}+\bar{y})]$$

$\exists y$ must commit to one fixed y first, then $\forall x$ checks every x:

1. $y = 0$: check $\forall x$:
   (a) $x = 0$: $(0+0)\cdot(1+1) = 0\cdot 1 = 0$
2. $y = 1$: check $\forall x$:
   (a) $x = 1$: $(1+1)\cdot(0+0) = 1\cdot 0 = 0$

No choice of y survives all x, so $\phi_5 = 0$, hence $\phi_2 \notin$ TQBF.
Example 3:
Given

TQBF $\in$ **PSPACE**

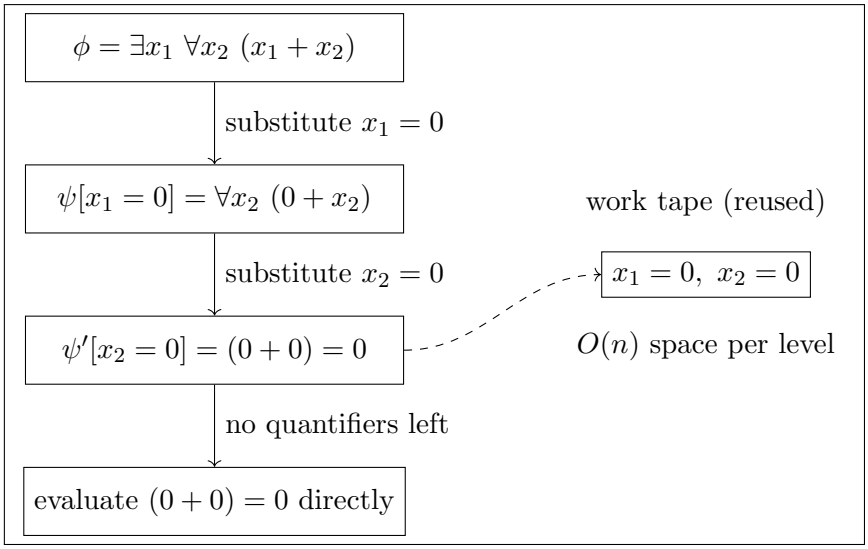
We give a recursive algorithm that evaluates a closed QBF ϕ using polynomial space.

1. If ϕ has no quantifiers, evaluate ϕ directly and return the result.
2. If $\phi = \exists x\ \psi$, recursively evaluate $\psi[x = 0]$ and $\psi[x = 1]$. Return 1 if either returns 1, otherwise return 0.
3. If $\phi = \forall x\ \psi$, recursively evaluate $\psi[x = 0]$ and $\psi[x = 1]$. Return 1 if both return 1, otherwise return 0.

The algorithm correctly evaluates ϕ by definition of $\exists$ and $\forall$. Let $n = |\phi|$ be the length of the input. At each recursive call, we substitute one variable with 0 or 1, so the recursion depth is at most n. At each level of recursion, we store only the current substitution, using $O(n)$ space. The two recursive calls are made *sequentially*, not simultaneously, so their space is reused. For example, given:

$$\phi = \exists x_1\ \underbrace{\forall x_2\ \underbrace{(x_1 + x_2)}_{\psi''}}_{\psi}$$

we substitute one variable at a time, peeling off one quantifier per level.



Therefore the total space used is:

$$O(n) \times n = O(n^2)$$

Since $O(n^2)$ is polynomial in n, we conclude TQBF $\in$ **PSPACE**.

43 Time and Space Hierarchy Theorem

43.1 Time Hierarchy Theorem

Let $f, g : \mathbb{N} \to \mathbb{N}$ be time-constructible functions such that

$$f(n) \log f(n) = o(g(n)).$$

Then

$$\mathbf{TIME}(f(n)) \subsetneq \mathbf{TIME}(g(n)).$$

Importanly, a function $f : \mathbb{N} \to \mathbb{N}$ is *time-constructible* if $f(n) \geq n$ and there exists a DTM M such that for every $n \in \mathbb{N}$, on input 1^n, the machine M halts after exactly $f(n)$ steps. Equivalently, M computes the function $1^n \to 1^{f(n)}$ in time $O(f(n))$.

Proof.

We construct a language $L \in \mathbf{TIME}(g(n)) \setminus \mathbf{TIME}(f(n))$ by diagonalization, defined explicitly as:

$$L = \left\{ \langle M, x' \rangle \;\middle|\; \begin{array}{l} M \text{ uses time } \leq f\big(|\langle M, x' \rangle|\big) \cdot \log f\big(|\langle M, x' \rangle|\big)\ , \\ M \text{ does not accept } \langle M, x' \rangle \end{array} \right\}.$$

Define the machine D as follows: on input x of length n,

1. Interpret $x = \langle M, x' \rangle$ as an encoding of a Turing machine M and input x'; if x does not have this form, reject.

2. Simulate M on x' for at most $f(n) \log f(n)$ steps.

3. If M accepts within the step bound, 0; otherwise, 1.

Since $f(n)\log f(n) = o(g(n))$, the simulation fits within $g(n)$ time, so $L = L(D) \in \mathbf{TIME}(g(n))$.

Suppose for contradiction $L \in \mathbf{TIME}(f(n))$, witnessed by some machine M^* running in time $f(n)$. Consider the input $x^* = \langle M^*, M^* \rangle$ of length n^*. Then:

$$x^* \in L \leftrightarrow D \text{ accepts } x^* \leftrightarrow M^* \text{ rejects } x^* \leftrightarrow x^* \notin L,$$

a contradiction. Hence $L \notin \mathbf{TIME}(f(n))$, giving $\mathbf{TIME}(f(n)) \subsetneq \mathbf{TIME}(g(n))$.

QED

Corollary.

$$\mathbf{P} \subsetneq \mathbf{EXP}, \qquad \mathbf{EXP} \subsetneq \mathbf{2EXP}, \qquad \ldots$$

43.2 Space Hierarchy Theorem

Let $f, g : \mathbb{N} \to \mathbb{N}$ be space-constructible functions such that

$$f(n) = o(g(n)).$$

Then

$$\mathbf{SPACE}(f(n)) \subsetneq \mathbf{SPACE}(g(n)).$$

Proof.

We construct a language $L \in \mathbf{SPACE}(g(n)) \setminus \mathbf{SPACE}(f(n))$ by diagonalization, defined explicitly as:

$$L = \left\{ \left(\langle M \rangle, 1^k\right) \;\middle|\; \begin{array}{l} M \text{ uses space } \leq f\left(\left|\langle M \rangle, 1^k\right|\right), \\ M \text{ uses time } \leq 2^{f\left(\left|\langle M \rangle, 1^k\right|\right)}, \\ M \text{ does not accept } \left(\langle M \rangle, 1^k\right) \end{array} \right\}.$$

Define the machine D as follows: on input x of length n,

1. Interpret $x = \langle M, x' \rangle$ as an encoding of a Turing machine M and input x'; if x does not have this form, reject.
2. Simulate M on x' using at most $f(n)$ work-tape cells, tracking cell usage with a $g(n)$-space counter to detect overflow.
3. If M accepts within the space bound, 0 otherwise, 1.

Since $f(n) = o(g(n))$, the simulation fits within $g(n)$ space (including the counter), so $L = L(D) \in \mathbf{SPACE}(g(n))$.

To handle infinite loops within the space bound, note that any machine using at most $f(n)$ space on an input of length n can visit at most

$$T(n) = |W| \cdot n \cdot |\Gamma|^{f(n)} \cdot f(n)$$

distinct configurations, where W is the state set and Γ the tape alphabet. D counts steps up to $T(n)$; if this bound is exceeded, the simulated machine is in a loop and D accepts.

Suppose for contradiction $L \in \mathbf{SPACE}(f(n))$, witnessed by some machine M^*. Consider $x^* = \langle M^*, M^* \rangle$ of length n^*. Then:

$$x^* \in L \leftrightarrow D \text{ accepts } x^* \leftrightarrow M^* \text{ rejects } x^* \leftrightarrow x^* \notin L,$$

a contradiction. Hence $L \notin \mathbf{SPACE}(f(n))$, giving $\mathbf{SPACE}(f(n)) \subsetneq \mathbf{SPACE}(g(n))$.

The above proof holds for the case of **PSPACE**, but some changes need to be made for the case of **NPSPACE**. The crucial point is that while on a deterministic TM, acceptance and rejection can be inverted (crucial for step 3), this is not possible on a nondeterministic machine. For the case of **NPSPACE**, L needs to be redefined first:

$$L = \Big\{ \big(\langle M \rangle, 1^k\big) \mid M \text{ uses space } \leq f\big(\big|\langle M \rangle, 1^k\big|\big) \text{ and } M \text{ accepts } \big(\langle M \rangle, 1^k\big) \Big\}.$$

The algorithm is changed to accept L by modifying step 3 to: if M accepted x during this simulation, 0; otherwise, 1.

L cannot be decided by any TM using $o(f(n))$ cells. Assuming L can be decided by some TM M using $o(f(n))$ cells, and following from the Immerman–Szelepcsényi theorem, $\overline{L}$ can also be determined by a TM (called $\overline{M}$) using $o(f(n))$ cells. Here lies the contradiction, therefore the assumption must be false:

1. If $x = \big(\langle \overline{M} \rangle, 1^k\big)$ (for some large enough k) is *not* in $\overline{L}$, then M will accept it, therefore $\overline{M}$ rejects x, therefore $x \in \overline{L}$ (contradiction).
2. If $x = \big(\langle \overline{M} \rangle, 1^k\big)$ (for some large enough k) is in $\overline{L}$, then M will reject it, therefore $\overline{M}$ accepts x, therefore $x \notin \overline{L}$ (contradiction).

QED

Corollary.

$$\mathbf{L} \subsetneq \mathbf{PSPACE}, \qquad \mathbf{PSPACE} \subsetneq \mathbf{EXPSPACE}, \qquad \ldots$$

The gap condition $f(n) \log f(n) = o(g(n))$ in the time hierarchy arises from the overhead of simulating M on a universal Turing machine, which incurs a logarithmic slowdown. The space hierarchy requires no such gap since simulation in space incurs no overhead beyond a constant factor.